

## **4. PREUVES D'INVARIANCE**

## **4. PREUVES D'INVARIANCE**

### **4.1 RELATIONS ENTRE SEMANTIQUES CONSERVANT L'INVARIANCE**

**4.1.1 CONSERVATION DE PROPRIETES D'INVARIANCE POUR DES SEMANTIQUES CONCORDANTES A DES RELATIONS ENTRE ETATS PRES**

**4.1.2 CONSERVATION DE PROPRIETES D'INVARIANCE APRES REDUCTION DES ETATS INOBSERVABLES**

**4.1.3 CONSERVATION DE PROPRIETES D'INVARIANCE PAR RETRACTION DE LA SEMANTIQUE PAR TRANSITIONS**

### **4.2 PRINCIPES D'INDUCTION**

**4.2.1 PRINCIPES D'INDUCTION POUR LES SEMANTIQUES CLOSES**

**4.2.1.1 Principe d'induction de base pour l'invariance**

**4.2.1.2 Transformations de principes d'induction**

**4.2.1.2.1 Transformation par distinction/confusion des états initiaux ou finaux**

**4.2.1.2.2 Transformation par déduction/prédiction**

**4.2.1.2.3 Transformation par inversion**

**4.2.1.2.4 Transformation contrapositive**

**4.2.1.2.5 Transformation relation/assertion**

**4.2.1.3 Principes d'induction dérivés par transformations**

**4.2.1.4 Equivalence forte des principes d'induction dérivés**

#### 4.2.2 PRINCIPES D'INDUCTION POUR UNE SEMANTIQUE NON CLOSE FERMEE PAR FUSIONS

#### 4.2.3 PRINCIPES D'INDUCTION POUR UNE SEMANTIQUE (NON CLOSE ET) NON FERMEE PAR FUSIONS

4.2.3.1 Principes d'induction pour une sémantique non fermée par fusions définie par une condition sur les préfixes des traces engendrées par un système de transition

4.2.3.2 Principes d'induction pour une sémantique non fermée par fusions définie par concordance avec une sémantique close

4.2.3.3 Equivalence forte des deux principes d'induction  $(\uparrow\downarrow)$  et  $(\downarrow\#)$

### 4.3 CONSTRUCTION D'UNE METHODE DE PREUVE D'INVARIANCE A PARTIR D'UNE SEMANTIQUE OPERATIONNELLE ET D'UN PRINCIPE D'INDUCTION PAR DECOMPOSITION DE L'INVARIANT GLOBAL EN INVARIANTS LOCAUX

#### 4.3.1 FORMALISATION DE LA CONSTRUCTION

4.3.1.1 Définition de la sémantique opérationnelle

4.3.1.2 Définition de la propriété invariante à démontrer

4.3.1.3 Choix d'un principe d'induction

4.3.1.4 Choix d'un langage pour exprimer les invariants locaux

4.3.1.5 Définition de la sémantique du langage exprimant les invariants locaux

4.3.1.6 Propriétés du langage exprimant les invariants locaux et sa sémantique

4.3.1.6.1 Treillis complets des invariants locaux

4.3.1.6.2 Correspondance entre invariants locaux et globaux

4.3.1.6.2.1 Correspondance monotone

4.3.1.6.2.2 Demi-correspondance de Galois

- 4.3.1.6.2.3 Quasi-correspondance de Galois
- 4.3.1.6.2.4 Correspondance de Galois
- 4.3.1.6.2.5 Correspondance de Galois surjective
- 4.3.1.6.2.6 Correspondance de Galois injective
- 4.3.1.6.2.7 Isomorphisme complet

#### **4.3.1.7 Dérivation de conditions de vérification correctes**

#### **4.3.1.8 Vérification de la complétude sémantique**

### **4.3.2 EXEMPLES DE CONSTRUCTIONS**

#### **4.3.2.1 Construction d'une méthode de preuve de non-terminaison, d'absence d'erreurs à l'exécution et d'invariance globale par l'absurde pour les programmes séquentiels**

- 4.3.2.1.1 La non-terminaison est une propriété d'invariance
- 4.3.2.1.2 Choix d'un principe d'induction
- 4.3.2.1.3 Choix d'un langage pour exprimer les invariants locaux
- 4.3.2.1.4 Dérivation de conditions de vérification correctes
  - 4.3.2.1.4.1 Base
  - 4.3.2.1.4.2 Induction
  - 4.3.2.1.4.3 Contradiction
- 4.3.2.1.5 Résumé informel des conditions de vérification
- 4.3.2.1.6 Exemple de preuve avec cette méthode
- 4.3.2.1.7 Vérification de la complétude sémantique
- 4.3.2.1.8 Preuve d'absence d'erreurs à l'exécution, par l'absurde, pour des programmes séquentiels
- 4.3.2.1.9 Preuve d'invariance globale, par l'absurde, pour des programmes séquentiels

#### **4.3.2.2 Extension de la méthode de Morris-Wegbreit dite "Subgoal induction" aux programmes parallèles et généralisation à d'autres propriétés d'invariance**

- 4.3.2.2.1 Programmes séquentiels
  - 4.3.2.2.1.1 Choix d'un langage pour exprimer les invariants locaux et sa sémantique
  - 4.3.2.2.1.2 Dérivation de conditions de vérification correctes
  - 4.3.2.2.1.3 Vérification de la complétude sémantique
  - 4.3.2.2.1.4 Résumé des conditions de vérification pour la preuve de correction partielle de programmes séquentiels par induction en arrière

- 4.3.2.2.1.5 Preuves d'autres propriétés d'invariance de programmes séquentiels par induction en arrière
- 4.3.2.2.2 Programmes parallèles asynchrones
  - 4.3.2.2.2.1 Choix d'un langage pour exprimer les invariants locaux et sa sémantique
  - 4.3.2.2.2.2 Construction de conditions de vérification correctes
  - 4.3.2.2.2.3 Vérification de la complétude sémantique
  - 4.3.2.2.2.4 Résumé des conditions de vérification pour la preuve de correction partielle de programmes parallèles asynchrones par induction en arrière
  - 4.3.2.2.2.5 Exemples
- 4.3.2.2.3 Construction d'une méthode d'absence d'interblocages dans les programmes parallèles asynchrones par induction en arrière
- 4.3.2.2.4 Construction d'une méthode de preuve d'exclusion mutuelle dans les programmes parallèles asynchrones par induction en arrière
- 4.3.2.2.5 Construction d'une méthode de preuve de non-terminaison de programmes parallèles par induction en arrière
- 4.3.2.2.6 Conclusion sur la preuve de propriétés d'invariance de programmes par induction en arrière
- 4.3.2.3 Construction d'une méthode de preuve pour les programmes parallèles communicants
- 4.3.2.4 Comparaison des méthodes de preuve pour les programmes parallèles connues dans la littérature
  - 4.3.2.4.1 Utilisation d'un seul invariant global
  - 4.3.2.4.2 Utilisation d'invariants sur les variables associés à chaque état de contrôle
  - 4.3.2.4.3 Utilisation d'invariants sur les variables associés à chaque point de contrôle du programme
  - 4.3.2.4.4 Utilisation d'invariants sur l'état de contrôle et les variables associés à chaque point de contrôle du programme
  - 4.3.2.4.5 Utilisation d'invariants sur les variables et des variables auxiliaires associés à chaque point de contrôle du programme
    - 4.3.2.4.5.1 Correction de la méthode

- 4.3.2.4.5.2 Complétude sémantique de la méthode
- 4.3.2.4.6 Utilisation d'invariants sur l'état de contrôle et les variables associés à chaque processus du programme
- 4.3.2.4.7 Utilisation d'un invariant global et d'invariants locaux
- 4.3.2.4.8 Classification des méthodes de preuve d'invariance selon la finesse de la décomposition de l'invariant global en invariants locaux
- 4.3.2.5 Analyse sémantique des programmes
  - 4.3.2.5.1 Analyse d'invariance "en avant"
  - 4.3.2.5.2 Analyse d'invariance "en arrière"
  - 4.3.2.5.3 Analyse d'invariance "avant-arrière"

## 4.4 REFERENCES



## 4. PREUVES D'INVARIANCE

Floyd [67] et Naur [66] sont souvent cités comme étant à l'origine des preuves de correction partielle des programmes déterministes séquentiels (bien que l'idée remonte certainement aux origines de la programmation (Turing, Von Neumann). Hoare [68] introduisit la présentation des preuves par induction sur la syntaxe des programmes. La méthode de Morris-Wegbreit [77] (dite "subgoal induction") a montré beaucoup plus tard qu'il n'y a pas qu'une seule manière de faire des preuves d'invariance. La généralisation au cas des programmes parallèles conduisit à une profusion de méthodes qu'il est bien difficile de comparer ne serait-ce que parce que les langages de programmation sont différents (Ashcroft [75], Ashcroft-Manna [70], Hoare [75], Howard [76], Keller [76], Lamport [77], Mazurkiewicz [77], Newton [75], Owicki-Gries [76a], [76b], etc.).

Le but de notre travail est de présenter un modèle abstrait pour étudier les méthodes de preuve d'invariance. Il s'agit d'en formaliser l'essence à l'aide de principes d'induction, d'en étudier la correction et la complétude relativement à une sémantique, de les comparer notamment du point de vue de l'équivalence forte et de proposer une méthode de construction systématique d'une méthode de preuve d'invariance à partir d'une définition de la sémantique opérationnelle.

## 4.1 RELATIONS ENTRE SEMANTIQUES CONSERVANT L'INVARIANCE

Pour démontrer une propriété d'un programme relativement à une sémantique, on cherche souvent à se ramener à une sémantique plus simple conservant la propriété à démontrer.

La relation entre ces deux sémantiques est souvent exprimée indirectement, par exemple elle est induite par une transformation du programme :

- Un exemple très courant est celui de la compilation. Pour éviter d'avoir à prendre en compte la compilation dans une preuve d'invariance, on ne raisonne jamais sur la sémantique du code objet mais toujours sur une sémantique du code source. Cette démarche est implicitement basée sur une hypothèse de correction du compilateur que nous pouvons formuler en disant qu'après réduction des états inobservables, les sémantiques source et objet sont concordantes à une relation entre états et actions près.

- Un autre exemple est fourni par l'utilisation que font Owicki-Gries [76a] de variables auxiliaires dans les preuves de programmes : une preuve de correction d'un programme  $P$  se fait en raisonnant sur un programme transformé  $P'$  qui utilise un ensemble  $VA$  de variables dites auxiliaires qui n'apparaissent que dans des commandes d'affectation  $x := E$  telles que  $x \in VA$ , et tel que  $P$  s'obtient à partir de  $P'$  en enlevant toutes les commandes d'affectation à ces variables auxiliaires. En définissant les états inobservables comme ceux dont l'état de contrôle désigne une commande d'affectation à une variable auxiliaire, on observe qu'après réduction des états inobservables, les sémantiques de  $P'$  et  $P$  sont concordantes à une fonction des états près qui consiste à éliminer les variables auxiliaires de l'état mémoire.

#### 4.1.1 CONSERVATION DE PROPRIETES D'INVARIANCE POUR DES SEMANTIQUES CONCORDANTES A DES RELATIONS ENTRE ETATS PRES

Théorème 4.1.1v1

Si  $\simeq \langle \pi_0, \pi_1 \rangle (\langle S, A, Z \rangle, \langle S', A', Z' \rangle)$  alors  
 $[\psi \text{ est invariante pour } \langle S, A, Z \rangle] \Leftrightarrow [\pi_0^{-1} \circ \psi \circ \pi_1 \text{ est invariante pour } \langle S', A', Z' \rangle]$

Démonstration

( $\Rightarrow$ ) Si  $\psi$  est invariante pour  $\langle S, A, Z \rangle$ ,  $p' \in Z'$  et  $i \in |p'|$  alors il existe  $p \in Z$  tel que  $|p| = |p'|$ ,  $\pi_0(p_0, p'_0)$  et  $\pi_1(p_i, p'_i)$ . Par conséquent  $\psi(p_0, p_i)$  entraîne  $\pi_0^{-1} \circ \psi \circ \pi_1(p'_0, p'_i)$ .

( $\Leftarrow$ ) Choisissons  $S = \{0, 1\}$ ,  $A = \emptyset$ ,  $Z = \{0, 1\}$ ,  $S' = \{0'\}$ ,  $Z' = \{0'\}$ ,  $\pi_0(0, 0')$ ,  $\pi_1(1, 0')$ ,  $\psi(0, 0)$ ,  $\neg \psi(1, 1)$ . Nous avons  $\pi_0^{-1} \circ \psi \circ \pi_1(0', 0')$  et donc  $\pi_0^{-1} \circ \psi \circ \pi_1$  est invariante pour  $Z'$  mais  $\psi$  n'est pas invariante pour  $Z$ .

□

La réciproque est vraie si nous ajoutons une condition supplémentaire :

Théorème 4.1.1v2

Si

$$\simeq \langle \pi_0, \pi_1 \rangle (\langle S, A, Z \rangle, \langle S', A', Z' \rangle) \quad (1)$$

$$\wedge (\pi_0^{-1} \circ \psi \circ \pi_1(\Delta'_1, \Delta'_2) \wedge \pi_0^{-1}(\Delta'_1, \Delta_1) \wedge \pi_1(\Delta_2, \Delta'_2)) \Rightarrow \psi(\Delta_1, \Delta_2) \quad (2)$$

alors

$$[\psi \text{ est invariante pour } \langle S, A, Z \rangle] \Leftrightarrow [\pi_0^{-1} \circ \psi \circ \pi_1 \text{ est invariante pour } \langle S', A', Z' \rangle]$$

Démonstration

( $\Rightarrow$ ) La même que pour 4.1.1v1. ( $\Leftarrow$ ) Si  $\kappa_0^{-1} \circ \psi \circ \kappa_0$  est invariante pour  $\langle s', A', \Sigma' \rangle$ ,  $p \in \Sigma$  et  $i \in |p|$  alors il existe  $p' \in \Sigma'$  tel que  $\kappa_0(p_0, p'_0)$  et  $\kappa_0(p_i, p'_i)$  - d'après (1). Par conséquent,  $\kappa_0^{-1} \circ \psi \circ \kappa_0(p'_0, p'_i)$  entraîne d'après (2),  $\psi(p_0, p_i)$ .

□

Observons que les théorèmes 4.1.1v1 et 4.1.1v2 ne sont pas vrais pour l'invariance conditionnelle. Un contre-exemple (pour 4.1.1v1) est donné par  $S = \{0, 1, 2\}$ ,  $A = \{a\}$ ,  $\Sigma = \{0 \xrightarrow{a} 1\}$ ,  $S' = \{0', 1'\}$ ,  $A' = \{a'\}$ ,  $\Sigma' = \{0' \xrightarrow{a'} 1'\}$ ,  $\neg \phi(0, 0)$ ,  $\psi(0, 0)$ ,  $\neg \psi(0, 1)$  (de sorte que  $\psi$  est invariant sous condition  $\phi$  pour  $\Sigma$ ),  $\kappa_0(0, 0')$ ,  $\kappa_0(1, 1')$ ,  $\kappa_0(a, a')$  (de sorte que  $\simeq \langle \kappa_0, \kappa_0 \rangle(\Sigma, \Sigma')$ ) et  $\phi(2, 2)$ ,  $\kappa_0(2, 0')$  et  $\neg \psi(2, 1)$  (de sorte que  $\kappa_0^{-1} \circ \psi \circ \kappa_0(0', 0')$  est vrai tandis que  $\kappa_0^{-1} \circ \psi \circ \kappa_0(0', 1')$  est faux). Toutefois sous des conditions plus restrictives, nous obtenons les théorèmes 4.1.1v3 et 4.1.1v4 suivants (dont les théorèmes 4.1.1v1 et 4.1.1v2 sont des cas particuliers respectifs) :

## Théorème 4.1.1v3

Si

$$\simeq \langle \kappa_0, \kappa_0 \rangle(\langle S, A, \Sigma \rangle, \langle S', A', \Sigma' \rangle) \quad (1)$$

$$\wedge (\phi'(A'_1, A'_2) \wedge \kappa_0^{-1}(A'_1, A_1) \wedge \kappa_0(A_2, A'_2)) \Rightarrow \phi(A_1, A_2) \quad (2)$$

$$\wedge (\psi(A_1, A_2) \wedge \kappa_0(A_1, A'_1) \wedge \kappa_0^{-1}(A'_2, A_2)) \Rightarrow \psi'(A'_1, A'_2) \quad (3)$$

alors

$$[\psi \text{ est invariante sous } \phi \text{ pour } \langle S, A, \Sigma \rangle]$$

 $\Leftrightarrow$ 

$$[\psi' \text{ est invariante sous } \phi' \text{ pour } \langle S', A', \Sigma' \rangle]$$

Démonstration

( $\Rightarrow$ ) Si  $p' \in \Sigma'$ ,  $i \in |p'|$  et  $\forall j \in i. \phi'(p'_0, p'_j)$  alors d'après (1),  $\exists p \in \Sigma$  tel que  $|p| = |p'|$  et  $\forall j \in i. \kappa_0(p_j, p'_j)$ . Par conséquent d'après (2) nous avons  $\forall j \in i. \phi(p_0, p_j)$  ce qui implique  $\psi(p_0, p_i)$  et donc  $\psi'(p'_0, p'_i)$  d'après (3).

( $\Leftarrow$ ) Même choix que dans la démonstration de 4.1.1v1 avec  $\phi'(s, s') = tt$ .

□

## Théorème 4.1.1v4

Si

$$\simeq \langle \mu_S, \mu_A \rangle (\langle S, A, \Sigma \rangle, \langle S', A', \Sigma' \rangle) \quad (1)$$

$$\wedge (\phi'(\Delta'_1, \Delta'_2) \wedge \mu_A^{-1}(\Delta'_1, \Delta_1) \wedge \mu_S(\Delta_2, \Delta'_2)) \Rightarrow \phi(\Delta_1, \Delta_2) \quad (2)$$

$$\wedge (\psi(\Delta_1, \Delta_2) \wedge \mu_S(\Delta_1, \Delta'_1) \wedge \mu_A^{-1}(\Delta'_2, \Delta_2)) \Rightarrow \psi'(\Delta'_1, \Delta'_2) \quad (3)$$

$$\wedge (\phi(\Delta_1, \Delta_2) \wedge \mu_S(\Delta_1, \Delta'_1) \wedge \mu_A^{-1}(\Delta'_2, \Delta_2)) \Rightarrow \phi'(\Delta'_1, \Delta'_2) \quad (4)$$

$$\wedge (\psi'(\Delta'_1, \Delta'_2) \wedge \mu_A^{-1}(\Delta'_1, \Delta_1) \wedge \mu_S(\Delta_2, \Delta'_2)) \Rightarrow \psi(\Delta_1, \Delta_2) \quad (5)$$

alors

$$[\psi \text{ est invariante sous } \phi \text{ pour } \langle S, A, \Sigma \rangle]$$

 $\Leftrightarrow$ 

$$[\psi' \text{ est invariante sous } \phi' \text{ pour } \langle S', A', \Sigma' \rangle]$$

Démonstration

( $\Rightarrow$ ) La même que pour 4.1.1v3. ( $\Leftarrow$ ) Si  $p \in \Sigma$ ,  $i \in |p|$  et  $\forall j \in i. \phi(p_0, p_j)$  alors d'après (1),  $\exists p' \in \Sigma'$  tel que  $|p'| = |p|$  et  $\forall j \in i. \mu_S(p_j, p'_j)$ . Par conséquent d'après (4) nous avons  $\forall j \in i. \phi'(p'_0, p'_j)$  ce qui implique  $\psi'(p'_0, p'_i)$  et donc  $\psi(p_0, p_i)$  d'après (5).

□

Dans le cas particulier d'une concordance entre sémantiques à une fonction entre états près, nous obtenons le corollaire suivant :

## Corollaire 4.1.1v5

Si

$$\langle S', A', \Sigma' \rangle = \simeq \langle f_S \rangle (\langle S, A, \Sigma \rangle)$$

$$\wedge \phi(\Delta_1, \Delta_2) = \phi'(f_S(\Delta_1), f_S(\Delta_2))$$

$$\wedge \psi(\Delta_1, \Delta_2) = \psi'(f_S(\Delta_1), f_S(\Delta_2))$$

alors

$$[\psi \text{ est invariante sous condition } \phi \text{ pour } \langle S, A, \Sigma \rangle]$$

 $\Leftrightarrow$ 

$$[\psi' \text{ est invariante sous condition } \phi' \text{ pour } \langle S', A', \Sigma' \rangle]$$

### 4.1.2 CONSERVATION DE PROPRIETES D'INVARIANCE APRES REDUCTION DES ETATS INOBSERVABLES

La conservation d'une propriété d'invariance après réduction des états inobservables est décrite (dans un cas simplifié mais dont la généralisation est aisée) par :

Théorème 4.1.2v1

Si

$$\langle s', A', \Sigma' \rangle = \text{Redei} \langle s' \rangle (\langle s, A, \Sigma \rangle) \quad \wedge \quad \forall p \in \Sigma. p_0 \in s'$$

alors

$$[\psi' \text{ est invariante sous condition } \phi' \text{ pour } \langle s', A', \Sigma' \rangle]$$

$\Leftrightarrow$

$$[\psi(\Delta, \Delta') = [\Delta \in s' \Rightarrow \psi'(\Delta, \Delta')] \text{ est invariante sous condition } \phi(\Delta, \Delta') = [\Delta \in s \Rightarrow \phi'(\Delta, \Delta')] \text{ pour } \langle s, A, \Sigma \rangle]$$

Démonstration

( $\Rightarrow$ ) Soit  $p$  une trace de  $\langle s, A, \Sigma \rangle$ ,  $i \in |p|$  tel que  $\forall j \in i. \phi(p_0, p_j)$ . Si  $p_i \notin s$  alors de manière évidente nous avons  $\psi(p_0, p_i)$ . Si  $p_i \in s$ , il faut montrer que  $\psi'(p_0, p_i)$  est vrai. Il suffit que  $\phi'(p_0, p_{\kappa(k)})$  soit vrai pour tout  $k \geq 0$  tel que  $\kappa(k) < i$  (où  $\kappa(k)$  a été défini en 2.5.4.1). Si  $k=0$  alors  $p_0 \in s'$  implique  $\kappa(k)=0$ . Si  $i > 0$  alors  $p_0 \in s'$  et  $\phi(p_0, p_0)$  impliquent  $\phi'(p_0, p_{\kappa(0)})$ . Si  $k > 0$  et  $\kappa(k) < i$  alors  $p_{\kappa(k)} \in s'$  et donc à nouveau  $\phi(p_0, p_{\kappa(k)})$  implique  $\phi'(p_0, p_{\kappa(k)})$ .

( $\Leftarrow$ ) Soit  $p'$  une trace de  $\langle s', A', \Sigma' \rangle$ ,  $i' \in |p'|$  tel que  $\forall j \in i'. \phi'(p'_0, p'_j)$ . Ceci entraîne que pour tout  $k \geq 0$  tel que  $\kappa(k) < \kappa(i')$  nous avons  $\phi(p_0, p_{\kappa(k)})$ . Soit maintenant  $j \in \kappa(i')$  tel que  $p_j \notin s'$ . Ceci entraîne de manière évidente  $\phi(p_0, p_j)$ . Finalement nous avons  $\forall j \in \kappa(i'). \phi(p_0, p_j)$ , d'où nous déduisons  $\psi(p_0, p_{\kappa(i')})$  et donc  $\psi'(p'_0, p'_i)$ .

□

### 4.1.3 CONSERVATION DE PROPRIETES D'INVARIANCE PAR RETRACTION DE LA SEMANTIQUE PAR TRANSITIONS

De manière générale, on fait des preuves d'invariance par induction sur la longueur des calculs. On cherche donc à utiliser un système de transition. Cette démarche qui consiste à remplacer une preuve d'invariance relative à une sémantique par une preuve relative à la rétraction de cette sémantique par transitions est justifiée par les résultats suivants :

Théorème 4.1.3 v1

$$\Rightarrow \begin{array}{l} [\psi \text{ est invariante sous condition } \phi \text{ pour } \langle S, A, \Sigma \rangle \wedge \langle S', A', \Sigma' \rangle \in \langle S, A, \Sigma \rangle] \\ [\psi \text{ est invariante sous condition } \phi \text{ pour } \langle S', A', \Sigma' \rangle] \end{array}$$

Théorème 4.1.3 v2

$$\Leftrightarrow \begin{array}{l} [\psi \text{ est invariante sous condition } \phi \text{ pour } \langle S, A, \Sigma \rangle] \\ [\psi \text{ est invariante sous condition } \phi \text{ pour } \text{Pref}^{<\omega}(\langle S, A, \Sigma \rangle)] \end{array}$$

Démonstration

( $\Rightarrow$ ) Supposons  $p \in \Sigma^{<\omega} \langle S, A \rangle$ ,  $q \in \Sigma$ ,  $p \leftrightarrow q$ ,  $\forall r \in \Sigma$ ,  $R \in |r|$ .  $[\forall j \in R. \phi(r_0, r_j)] \Rightarrow \psi(r_0, r_R)$  et  $i \in |p|$ . Alors  $\forall k \in |p|$ ,  $p_k = q_k$  et donc  $[\forall j \in i. \phi(p_0, p_j)] \Rightarrow [\forall j \in i. \phi(q_0, q_j)] \Rightarrow \psi(q_0, q_i) \Rightarrow \psi(p_0, p_i)$ .

( $\Leftarrow$ ) Si  $p \in \Sigma$ ,  $i \in |p|$  et  $\forall j \in i. \phi(p_0, p_j)$  alors  $p^{<i}$  est un préfixe fini de  $p$  et donc  $\psi(p_0, p_i)$  est vrai.

□

Nous en déduisons que pour faire une preuve d'invariance de  $\psi$  sous condition  $\phi$  pour une sémantique  $\langle S, A, \Sigma \rangle$ , il est toujours correct de faire la preuve relativement à  $\underline{Rtran}(\langle S, A, \Sigma \rangle)$  c'est-à-dire en raisonnant sur le système de transition qu'elle engendre. Cette démarche n'est pas toujours complète. Plus précisément :

Corollaire 4.1.3 v3

$[\psi \text{ est invariante sous condition } \phi \text{ pour } \langle S, A, \Sigma \rangle]$

- (1)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Pref}(\langle S, A, \Sigma \rangle)]$
- (2)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Suff}(\langle S, A, \Sigma \rangle)]$
- (3)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Redeqa}(\langle S, A, \Sigma \rangle)]$
- (4)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{E_{fus}}(\langle S, A, \Sigma \rangle)]$
- (5)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Flim}(\langle S, A, \Sigma \rangle)]$
- (6)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Retra}(\langle S, A, \Sigma \rangle)]$
- (7)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{Rtran}(\langle S, A, \Sigma \rangle)]$

Si  $\underline{card}(\alpha) < \omega$  alors

- (8)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{W_{fin}}(\alpha)(\langle S, A, \Sigma \rangle)]$
- (9)  $\Leftrightarrow [\psi \text{ est invariante sous condition } \phi \text{ pour } \underline{S_{fin}}(\alpha)(\langle S, A, \Sigma \rangle)]$

### Démonstration

- (1)  $(\Leftarrow)$   $\underline{Pref}$  est extensive et 4.1.3 v1.  $(\Rightarrow)$  2.6.1 v2 et 4.1.3 v2.
- (2)  $(\Leftarrow)$   $\underline{Suff}$  est extensive et 4.1.3 v1.  $(\Rightarrow)$   $\psi(\delta, \delta') = [\delta' = 1]$  est invariante sous condition  $\phi(\delta, \delta') = [\delta' \in \{0, 2\}]$  pour la trace  $0 \xrightarrow{a} 1 \xrightarrow{a} 2 \xrightarrow{a} 3$  mais pas pour son suffixe  $2 \xrightarrow{a} 3$ .
- (3) Trivial.
- (4)  $(\Leftarrow)$   $\underline{E_{fus}}$  est extensive et 4.1.3 v1.  $(\Rightarrow)$  Si  $\psi(0,0), \psi(0,1), \psi(1,1), \psi(1,2)$  et  $\phi(\delta, \delta')$  sont vrais alors  $\psi$  est invariant sous condition  $\phi$  pour les traces  $0 \xrightarrow{a} 1$  et  $1 \xrightarrow{a} 2$  mais pas pour la fusion  $0 \xrightarrow{a} 1 \xrightarrow{a} 2$ .

(5) 4.1.3 v2 et 2.6.7 v3-(1).

(6)  $(\Rightarrow)$  Relps est réductive et 4.1.3 v1.  $(\Leftarrow)$   $\psi(s, s') = \text{ff}$  est invariante sous condition  $\phi(s, s') = \text{tt}$  pour  $\langle \{0\}, \{a\}, \phi \rangle = \text{Relps}(\langle \{0\}, \{a\}, \Sigma \rangle)$  avec  $\Sigma = \{0, 0 \xrightarrow{a} 0, \dots, 0 \xrightarrow{a} 0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0, \dots\}$  mais pas pour  $\langle \{0\}, \{a\}, \Sigma \rangle$ .

(7)  $(\Leftarrow)$   $\psi$  est invariant sous condition  $\phi$  pour Rtran $(\langle s, A, \Sigma \rangle)$  si et seulement si d'après 4.1.3 v3.1  $\psi$  est invariant sous condition  $\phi$  pour Pref $\circ$ Rtran $(\langle s, A, \Sigma \rangle)$  ce qui entraîne d'après 2.6.8 v1 et 4.1.3 v1 que  $\psi$  est invariant sous condition  $\phi$  pour  $\langle s, A, \Sigma \rangle$ .  $(\Leftarrow)$  Même contre-exemple que pour (4).

(8), (9) 4.1.3 v2 et 2.6.4 v1.

□

Observons que la démarche qui consiste à remplacer une preuve d'invariance relative à une sémantique  $\langle s, A, \Sigma \rangle$  par une preuve relative à Rtran $(\langle s, A, \Sigma \rangle)$  est toujours correcte mais pas toujours complète (cf. 4.1.3 v3).

Toutefois la complétude est obtenue si la sémantique  $\langle s, A, \Sigma \rangle$  est fermée par fusions :

Théorème 4.1.3 v4

$\Rightarrow$  [ $\psi$  est invariante sous condition  $\phi$  pour  $\langle s, A, \Sigma \rangle \wedge \text{Efus}(\langle s, A, \Sigma \rangle) = \langle s, A, \Sigma \rangle$ ]  
 $\Rightarrow$  [ $\psi$  est invariante sous condition  $\phi$  pour Rtran $(\langle s, A, \Sigma \rangle)$ ]

Démonstration

$\psi$  est invariante sous condition  $\phi$  pour  $\langle s, A, \Sigma \rangle$  si et seulement si  $\psi$  est invariante sous condition  $\phi$  pour Pref <sup>$\omega$</sup>  $(\langle s, A, \Sigma \rangle)$  d'après 4.1.3 v2. Ceci entraîne d'après 2.6.8 v1 et 4.1.3 v1 que  $\psi$  est invariante sous condition  $\phi$  pour Pref <sup>$\omega$</sup>  $\circ$ Rtran $(\langle s, A, \Sigma \rangle)$  qui d'après 4.1.3 v2 implique que  $\psi$  est invariante sous condition  $\phi$  pour Rtran $(\langle s, A, \Sigma \rangle)$ .

□

Par conséquent, d'après 4.1.3v3.8, 4.1.3v3.9 et 4.1.3v4, les preuves d'invariance pour le langage que nous avons considéré en 3.8 peuvent toujours se faire en raisonnant sur un système de transition.

Dans le cas d'une sémantique non fermée par fusions, il est tout de même possible de se ramener à un raisonnement sur un système de transition, par exemple en spécifiant cette sémantique par concordance avec une sémantique close (cf. 2.7.2.2).

## 4.2 PRINCIPES D'INDUCTION

Un principe d'induction est l'essence d'une méthode de preuve.

### 4.2.1 PRINCIPES D'INDUCTION POUR LES SEMANTIQUES CLOSES

Nous commençons par considérer le cas des sémantiques engendrées par un système de transition.

#### 4.2.1.1 Principe d'induction de base pour l'invariance

Le principe d'induction de base est l'essence de la méthode de Floyd-Naur. Toutes les autres méthodes de preuve de propriétés d'invariance en dérivent. Ce principe est déduit de l'

##### Exemple 4.2.1.1-1

Pour démontrer la correction partielle du programme suivant qui calcule le quotient et le reste de deux entiers  $x$  et  $y$  :

```

1:    $q := 0;$ 
2:   while  $x \geq y$  do
3:        $q := q + 1;$ 
4:        $x := x - y;$ 
5:   od;
6:

```

il faut établir la relation  $x = \bar{q} \times y + \bar{x} \wedge \bar{x} < y$  entre les valeurs initiales  $\underline{x}, \underline{y}, \underline{q}$  et finales  $\bar{x}, \bar{y}, \bar{q}$  des variables  $x, y$  et  $q$ .

Comme la méthode de Floyd-Naur n'utilise que des assertions  $P(x, y, q)$  sur les valeurs des variables  $x, y$  et  $q$ , il faut introduire une variable auxiliaire  $x_1$  à laquelle est affectée la valeur initiale de  $x$  en début de programme et qui n'est plus modifiée par la suite :

$x_1 := x; q := 0; \text{ while } x \geq y \text{ do } q := q + 1; x := x - y; \text{ od}$

de sorte qu'à la terminaison nous pouvons prouver que  $[x_1 = q \times y + x \wedge x < y]$ .

Cette transformation peut être évitée en utilisant la méthode de Manna[71] qui est tout à fait similaire à la méthode de Floyd-Naur mais qui consiste à utiliser des relations entre valeurs initiales et courantes des variables plutôt que des assertions sur les valeurs courantes. La méthode consiste à associer un invariant local  $P_i$  à chaque point  $i, i=1, \dots, 6$  du programme. L'invariant local  $P_i$  associé au point  $i$  du programme est une relation entre les valeurs initiales  $\underline{x}, \underline{y}$  (nous omettons  $\underline{q}$  qui est inutile) des variables  $x, y$  et les valeurs courantes  $x, y, q$  de ces variables  $x, y, q$  qui est vrai quand le contrôle atteint ce point  $i$  :

$$P_1(\underline{x}, \underline{y}, x, y, q) = [x = \underline{x} \wedge y = \underline{y}]$$

$$P_2(\underline{x}, \underline{y}, x, y, q) = [x = \underline{x} \wedge y = \underline{y} \wedge q = 0]$$

$$P_3(\underline{x}, \underline{y}, x, y, q) = [y = \underline{y} \wedge x = q \times y + x]$$

$$P_4(\underline{x}, \underline{y}, x, y, q) = [y = \underline{y} \wedge x = (q-1) \times y + x]$$

$$P_5(\underline{x}, \underline{y}, x, y, q) = [y = \underline{y} \wedge x = q \times y + x]$$

$$P_6(\underline{x}, \underline{y}, x, y, q) = [x = q \times y + x \wedge x < y \wedge y = \underline{y}]$$

Les conditions de vérification sont similaires à celles obtenues par la méthode de Floyd-Naur, excepté pour la condition d'entrée :

$$P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [x = \underline{x} \wedge y = \underline{y}]$$

$$P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [\exists q'. P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q') \wedge q = 0]$$

$$P_3(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [(P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \vee P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)) \wedge x \geq y]$$

$$P_4(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [\exists q'. P_3(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q') \wedge q = q' + 1]$$

$$P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [\exists x'. P_4(\underline{x}, \underline{y}, x', \underline{y}, q) \wedge x = x' - y]$$

$$P_6(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \Leftarrow [(P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \vee P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)) \wedge x < y]$$

$$[x = q \times y + z \wedge x < y \wedge y = \underline{y}] \Leftarrow P_6(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

Ces conditions de vérification expriment que

- $P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$  est vrai quand l'exécution commence avec des valeurs  $\underline{x}, \underline{y}, q$  de  $x, y, q$
- Si l'exécution commencée avec les valeurs  $\underline{x}, \underline{y}, q$  de  $x, y, q$  atteint le point  $i$  du programme qui est immédiatement suivi par le point  $j$  du programme, alors l'hypothèse que  $P_i(\underline{x}, \underline{y}, x', y', q')$  est vrai en  $i$  pour les valeurs courantes  $x', y', q'$  des variables  $x, y, q$  implique que  $P_j(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$  est vrai quand le contrôle est en  $j$  avec les valeurs  $\underline{x}, \underline{y}, q$  des variables  $x, y, q$ .

Par induction sur la longueur des calculs, on en déduit que si l'exécution commence avec  $x = \underline{x}$  et  $y = \underline{y}$  et atteint le point  $i$  avec  $x = \underline{x}$ ,  $y = \underline{y}$  et  $q = q$  alors  $P_i(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$  est vrai. En particulier, si  $i = 6$  la dernière condition de vérification permet de conclure que le programme est partiellement correct.

Pour dégager l'essence de cette preuve nous considérons la sémantique du programme (comme elle a été définie en 2.8.1.2) :

Les états  $\langle L, M \rangle$  de ce programme consistent en un état de contrôle  $L$  (i.e. un point du programme) et un état mémoire (i.e. une fonction  $M$  qui définit les valeurs  $M(x), M(y), M(q)$  des variables  $x, y, q$ ) :

$$\mathbb{L} = \{1, 2, 3, 4, 5, 6\}$$

$$\mathcal{V} = \{x, y, q\}$$

$$\mathcal{M} = (\mathcal{V} \rightarrow \mathbb{Z})$$

$$S = \mathbb{L} \times \mathcal{M}$$

Les états initiaux  $\Delta$  du programme correspondent au point 1 du programme avec des valeurs arbitraires des variables :

$$\varepsilon(\Delta) = [\exists M \in (\mathcal{V} \rightarrow \mathbb{Z}). \Delta = \langle 1, M \rangle]$$

La relation de transition est définie par cas en écrivant  $\langle L, M \rangle \xrightarrow{t} \langle L', M' \rangle$  quand  $t(\langle L, M \rangle, \langle L', M' \rangle)$  est vraie (et en omettant l'action unique, cf. 2.8.1.2.2).  
En plus  $(x, y, q)$  dénote la fonction  $M$  lorsque  $M(x) = x$ ,  $M(y) = y$  et  $M(q) = q$  et ces valeurs des variables  $x, y, q$  du programme sont implicitement universellement quantifiées sur  $\mathbb{Z}$  :

$$\langle 1, (x, y, q) \rangle \xrightarrow{t} \langle 2, (x, y, 0) \rangle$$

$$\langle 2, (x, y, q) \rangle \xrightarrow{t} \langle 3, (x, y, q) \rangle \quad \text{si et seulement si } x \geq y$$

$$\langle 2, (x, y, q) \rangle \xrightarrow{t} \langle 6, (x, y, q) \rangle \quad \text{si et seulement si } x < y$$

$$\langle 3, (x, y, q) \rangle \xrightarrow{t} \langle 4, (x, y, q+1) \rangle$$

$$\langle 4, (x, y, q) \rangle \xrightarrow{t} \langle 5, (x-y, y, q) \rangle$$

$$\langle 5, (x, y, q) \rangle \xrightarrow{t} \langle 3, (x, y, q) \rangle \quad \text{si et seulement si } x \geq y$$

$$\langle 5, (x, y, q) \rangle \xrightarrow{t} \langle 6, (x, y, q) \rangle \quad \text{si et seulement si } x < y$$

Définissons

$$\bar{\psi}(x, y, q, \bar{x}, \bar{y}, \bar{q}) = [x = \bar{q} \times y + \bar{x} \wedge \bar{x} < y \wedge \bar{y} = y]$$

$$\psi(\Delta, \bar{\Delta}) = [\exists x, y, q, \bar{x}, \bar{y}, \bar{q} \in \mathbb{Z}. \Delta = \langle 1, (x, y, q) \rangle \wedge \bar{\Delta} = \langle 6, (\bar{x}, \bar{y}, \bar{q}) \rangle \wedge \bar{\psi}(x, y, q, \bar{x}, \bar{y}, \bar{q})]$$

alors lorsque nous disons que le programme est partiellement correct ceci signifie :

$$\forall p \in \Sigma \langle S, A, E, \varepsilon \rangle, i \in |p|. \quad \psi(p_0, p_i)$$

Nous pouvons maintenant dériver le principe d'induction de l'exemple par abstractions successives :

Notre première abstraction consiste à noter que les invariants locaux  $P_i$  associés aux points  $i, i=1, \dots, 6$  du programme peuvent être compris comme une relation  $I$  sur des états. Nous avons

$$P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{x} = \underline{x} \wedge \underline{y} = \underline{y}]$$

$$P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{x} = \underline{x} \wedge \underline{y} = \underline{y} \wedge q = 0]$$

$$P_3(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{y} = \underline{y} \wedge \underline{x} = q \times \underline{y} + \underline{x}]$$

$$P_4(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{y} = \underline{y} \wedge \underline{x} = (q-1) \times \underline{y} + \underline{x}]$$

$$P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{y} = \underline{y} \wedge \underline{x} = q \times \underline{y} + \underline{x}]$$

$$P_6(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) = [\underline{x} = q \times \underline{y} + \underline{x} \wedge \underline{x} < \underline{y} \wedge \underline{y} = \underline{y}]$$

de sorte que

$$I \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$$

$$I(\underline{\Delta}, \underline{\Delta}) = [\exists j \in \mathbb{N}, \underline{x}, \underline{y}, q, \underline{x}, \underline{y}, q \in \mathbb{Z}. \underline{\Delta} = \langle 1, (\underline{x}, \underline{y}, q) \rangle \wedge \underline{\Delta} = \langle j, (\underline{x}, \underline{y}, q) \rangle \wedge P_j(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)]$$

Notre seconde abstraction consiste à comprendre les conditions de vérification sur les  $P_i, i=1, \dots, 6$  en termes de conditions de vérification équivalentes faisant intervenir  $I$  et nos définitions abstraites du programme et de sa correction partielle (c'est à dire en termes de  $s, t, e, s$  et  $\psi$ ) :

- La première condition de vérification était

$$[\underline{x} = \underline{x} \wedge \underline{y} = \underline{y}] \Rightarrow P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

c'est à dire  $P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$  qui est équivalent à  $\forall \underline{\Delta}, \varepsilon(\underline{\Delta}) \Rightarrow I(\underline{\Delta}, \underline{\Delta})$  puisque  $[\exists \underline{x}, \underline{y}, q \in \mathbb{Z}. \underline{\Delta} = \langle 1, (\underline{x}, \underline{y}, q) \rangle] \Rightarrow I(\underline{\Delta}, \underline{\Delta})$  est équivalent à  $I(\langle 1, (\underline{x}, \underline{y}, q) \rangle, \langle 1, (\underline{x}, \underline{y}, q) \rangle)$  soit  $[\exists j \in \mathbb{N}. j=1 \wedge P_j(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)] = P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$ .

- Les conditions de vérification 2 à 6

$$[\exists q'. P_1(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q') \wedge q=0] \Rightarrow P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

$$[(P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \vee P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)) \wedge \underline{x} \geq \underline{y}] \Rightarrow P_3(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

$$[\exists q'. P_3(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q') \wedge q=q'+1] \Rightarrow P_4(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

$$[\exists \underline{x}'. P_4(\underline{x}, \underline{y}, \underline{x}', \underline{y}, q) \wedge \underline{x} = \underline{x}' - \underline{y}] \Rightarrow P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

$$[(P_2(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q) \vee P_5(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)) \wedge \underline{x} < \underline{y}] \Rightarrow P_6(\underline{x}, \underline{y}, \underline{x}, \underline{y}, q)$$

sont équivalentes à :

$$[\exists x', y', q'. P_1(\underline{x}, \underline{y}, x', y', q') \wedge x = x' \wedge y = y' \wedge q = 0] \Rightarrow P_2(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_2(\underline{x}, \underline{y}, x', y', q') \wedge x > y' \wedge x = x' \wedge y = y' \wedge q = q'] \Rightarrow P_3(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_5(\underline{x}, \underline{y}, x', y', q') \wedge x > y' \wedge x = x' \wedge y = y' \wedge q = q'] \Rightarrow P_3(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_3(\underline{x}, \underline{y}, x', y', q') \wedge x = x' \wedge y = y' \wedge q = q' + 1] \Rightarrow P_4(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_4(\underline{x}, \underline{y}, x', y', q') \wedge x = x' - y' \wedge y = y' \wedge q = q'] \Rightarrow P_5(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_2(\underline{x}, \underline{y}, x', y', q') \wedge x < y' \wedge x = x' \wedge y = y' \wedge q = q'] \Rightarrow P_6(\underline{x}, \underline{y}, x, y, q)$$

$$[\exists x', y', q'. P_5(\underline{x}, \underline{y}, x', y', q') \wedge x < y' \wedge x = x' \wedge y = y' \wedge q = q'] \Rightarrow P_6(\underline{x}, \underline{y}, x, y, q)$$

Utilisant  $I$ ,  $t$ ,  $\underline{M} = (\underline{x}, \underline{y}, q)$   $M' = (x', y', q')$  et  $M = (x, y, q)$ , elles peuvent s'écrire :

$$[I(\langle 1, \underline{M} \rangle, \langle 1, M' \rangle) \wedge t(\langle 1, M' \rangle, \langle 2, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 2, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 2, M' \rangle) \wedge t(\langle 2, M' \rangle, \langle 3, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 3, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 5, M' \rangle) \wedge t(\langle 5, M' \rangle, \langle 3, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 3, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 3, M' \rangle) \wedge t(\langle 3, M' \rangle, \langle 4, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 4, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 4, M' \rangle) \wedge t(\langle 4, M' \rangle, \langle 5, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 5, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 2, M' \rangle) \wedge t(\langle 2, M' \rangle, \langle 6, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 6, M \rangle)$$

$$[I(\langle 1, \underline{M} \rangle, \langle 5, M' \rangle) \wedge t(\langle 5, M' \rangle, \langle 6, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle 6, M \rangle)$$

c'est-à-dire

$$\forall L, L \in \mathcal{L}. [I(\langle 1, \underline{M} \rangle, \langle L, M' \rangle) \wedge t(\langle L, M' \rangle, \langle L, M \rangle)] \Rightarrow I(\langle 1, \underline{M} \rangle, \langle L, M \rangle)$$

qui est équivalent à :

$$\forall \underline{\Delta}, \Delta', \Delta. [\varepsilon(\underline{\Delta}) \wedge I(\underline{\Delta}, \Delta') \wedge t(\Delta', \Delta)] \Rightarrow I(\underline{\Delta}, \Delta)$$

La dernière condition de vérification est

$$P_6(\underline{x}, \underline{y}, \bar{x}, \bar{y}, \bar{q}) \Rightarrow [\underline{x} = \bar{q} \times \underline{y} + \bar{x} \wedge \bar{x} < \underline{y} \wedge \bar{y} = \underline{y}]$$

c'est-à-dire

$$I(\langle 1, \underline{M} \rangle, \langle 6, \bar{M} \rangle) \Rightarrow \Psi(\langle 1, \underline{M} \rangle, \langle 6, \bar{M} \rangle)$$

soit

$$\forall \underline{\Delta}, \bar{\Delta}. [\varepsilon(\underline{\Delta}) \wedge I(\underline{\Delta}, \bar{\Delta})] \Rightarrow \Psi(\underline{\Delta}, \bar{\Delta})$$

Ainsi, nous avons montré que cette méthode de preuve consiste essentiellement à découvrir un invariant  $I$  et à prouver que :

$$\begin{aligned} & [ \quad \forall \underline{a} \in S. \varepsilon(\underline{a}) \Rightarrow I(\underline{a}, \underline{a}) \\ & \quad \wedge \quad (\forall \underline{a}, \underline{a}', \underline{a} \in S. [\varepsilon(\underline{a}) \wedge I(\underline{a}, \underline{a}') \wedge t(\underline{a}', \underline{a})] \Rightarrow I(\underline{a}, \underline{a})) \\ & \quad \wedge \quad (\forall \underline{a}, \underline{\bar{a}} \in S. [\varepsilon(\underline{a}) \wedge I(\underline{a}, \underline{\bar{a}})] \Rightarrow \psi(\underline{a}, \underline{\bar{a}})) ] \end{aligned}$$

pour en déduire

$$\forall p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |p|. \psi(p_0, p_i)$$

□

Le principe d'induction de base est

Théorème 4.2.1.1 v1

$$\begin{aligned} & [ \exists I \in (S^2 \rightarrow \{\text{tt}, \text{ff}\}). \forall \underline{a}, \underline{a}, \underline{\bar{a}} \in S, a \in A. \\ & \quad \begin{array}{ll} (-\exists.\varepsilon) & \varepsilon(\underline{a}) \Rightarrow I(\underline{a}, \underline{a}) \\ & \wedge \\ (-\exists.i) & [ \exists \underline{a}' \in S. \varepsilon(\underline{a}) \wedge I(\underline{a}, \underline{a}') \wedge t_a(\underline{a}', \underline{a}) ] \Rightarrow I(\underline{a}, \underline{a}) \\ & \wedge \\ (-\exists.s) & [ \varepsilon(\underline{a}) \wedge I(\underline{a}, \underline{\bar{a}}) ] \Rightarrow \psi(\underline{a}, \underline{\bar{a}}) ] \end{array} & (-\exists) \\ & \quad \Leftrightarrow \\ & \quad [ \forall p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |p|. \psi(p_0, p_i) ] \end{aligned}$$

### Démonstration

( $\Rightarrow$ ) Pour la preuve de correction, soit  $p \in \Sigma \langle S, A, t, \varepsilon \rangle$ . Démontrons par récurrence sur  $i \in |p|$  que  $I(p_0, p_i)$ . Pour  $i=0$ , nous avons  $\varepsilon(p_0)$  et donc  $I(p_0, p_0)$  d'après  $(-\exists.\varepsilon)$ . Si  $I(p_0, p_i)$  et  $i+1 \in |p|$  alors  $t_{p_i}(p_i, p_{i+1})$  et  $(-\exists.i)$  impliquent  $I(p_0, p_{i+1})$ . Nous déduisons de  $(-\exists.s)$  que  $\forall i \in |p|. \psi(p_0, p_i)$ .

( $\Leftarrow$ ) La preuve de complétude sémantique est également très simple en choisissant  $I(\underline{a}, \underline{a}) = [ \exists p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |p|. (p_0 = \underline{a} \wedge p_i = \underline{a}) ]$ .  $(-\exists.\varepsilon)$  et  $(-\exists.i)$  découlent

alors de la définition de  $\Sigma\langle S, A, t, E \rangle$ . (-Y.5) dérive de l'hypothèse  
 $\forall p \in \Sigma\langle S, A, t, E \rangle, i \in |p|. \psi(p_0, p_i)$ .

□

Remarque 4.2.1.1-2 (Invariance conditionnelle)

Par souci de simplicité, nous donnons uniquement les principes d'induction pour l'invariance car la généralisation à l'invariance conditionnelle est triviale. Par exemple le principe d'induction (-Y) se généralise immédiatement en :

$$[\exists I \in (S \times S \rightarrow \{t, ff\}) . \forall \underline{a}, \underline{a}', \bar{a} \in S, a \in A .$$

$$E(\underline{a}) \Rightarrow I(\underline{a}, \underline{a})$$

$$\wedge [\exists \underline{a}' \in S . E(\underline{a}) \wedge I(\underline{a}, \underline{a}') \wedge \phi(\underline{a}, \underline{a}') \wedge t_a(\underline{a}', \underline{a})] \Rightarrow I(\underline{a}, \underline{a})$$

$$\wedge [E(\underline{a}) \wedge I(\underline{a}, \bar{a})] \Rightarrow \psi(\underline{a}, \bar{a})]$$

$$\Leftrightarrow$$

$$[\forall p \in \Sigma\langle S, A, t, E \rangle, i \in |p|. [\forall j \in i. \phi(p_0, p_j)] \Rightarrow \psi(p_0, p_i)]$$

La preuve de correction consiste essentiellement à démontrer que pour tout  $p \in \Sigma\langle S, A, t, E \rangle$  nous avons par récurrence sur  $i \in |p|$ ,  $[\forall j \in i. \phi(p_0, p_j)] \Rightarrow I(p_0, p_i)$ .  
 Pour la preuve de complétude nous choisissons  $I(\underline{a}, \underline{a}) =$   
 $[\exists p \in \Sigma\langle S, A, t, E \rangle, i \in |p|. (p_0 = \underline{a} \wedge \forall j \in i. \phi(p_0, p_j) \wedge p_i = \underline{a})]$ .

□

### 4.2.1.2 Transformations de principes d'induction

Par transformation du principe d'induction de base (Y), nous obtenons un certain nombre de principes d'induction dérivés qui permettent de rendre compte des méthodes de preuve d'invariance existantes mais également d'en découvrir de nouvelles.

#### 4.2.1.2.1 Transformation par distinction/confusion des états initiaux ou finaux

Observons que la définition de l'invariance

$$\forall p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |p|. \psi(p_0, p_i)$$

est équivalente à

$$\forall p \in \Sigma \langle S, A, t, tt \rangle, i \in |p|. (\varepsilon(p_0) \Rightarrow \psi(p_0, p_i))$$

en posant  $tt(a) = tt$ .

Par conséquent, nous obtenons un principe d'induction (Y) équivalent à (Y) en substituant  $tt$  à  $\varepsilon$  et  $\psi'(s, s') = (\varepsilon(s) \Rightarrow \psi(s, s'))$  à  $\psi(s, s')$  dans (Y) :

$$\begin{array}{lcl}
 & & [\exists I \in (S^2 \rightarrow \{tt, ff\}) . \forall \underline{a}, \bar{a} \in S, a \in A . \\
 (Y.\varepsilon) & & I(\underline{a}, \underline{a}) \\
 (Y.i) & \wedge & [\exists \bar{a}' \in S. I(\underline{a}, \bar{a}') \wedge t_a(\bar{a}', \underline{a})] \Rightarrow I(\underline{a}, \underline{a}) \\
 (Y.\varepsilon) & \wedge & I(\underline{a}, \bar{a}) \Rightarrow \psi(\underline{a}, \bar{a}) ] \\
 & \iff & \\
 & & [\forall p \in \Sigma \langle S, A, t, tt \rangle, i \in |p|. \psi(p_0, p_i)]
 \end{array}
 \quad (Y)$$

Par symétrie avec les états initiaux, on peut être amené à distinguer des états finaux, en écrivant la définition de l'invariance sous la forme

$$\forall p \in Z\langle S, A, t, e \rangle, i \in |p|. (\phi(p_i) \Rightarrow \psi(p_0, p_i))$$

Si nous substituons  $\psi'(s, s') = [\phi(s') \Rightarrow \psi(s, s')]$  à  $\psi(s, s')$  dans (-Y), nous obtenons (-Y-):

$$\begin{array}{lcl}
 & & [\exists I \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}) . \forall \underline{s}, \underline{s}', \underline{a} \in S, a \in A. \\
 (-Y-E) & \epsilon(\underline{s}) \Rightarrow I(\underline{s}, \underline{s}) & \\
 & \wedge & \\
 (-Y-i) & [\exists \underline{s}' \in S. \epsilon(\underline{s}) \wedge I(\underline{s}, \underline{s}') \wedge t_a(\underline{s}', \underline{s})] \Rightarrow I(\underline{s}, \underline{s}) & (-Y-) \\
 & \wedge & \\
 (-Y-s) & [\epsilon(\underline{s}) \wedge I(\underline{s}, \underline{s}') \wedge \phi(\underline{s}')] \Rightarrow \psi(\underline{s}, \underline{s}') & \\
 & \iff & \\
 & [\forall p \in Z\langle S, A, t, e \rangle, i \in |p|. (\epsilon(p_0) \wedge \phi(p_i)) \Rightarrow \psi(p_0, p_i)] & 
 \end{array}$$

#### 4.2.1.2.2 Transformation par déduction/prédiction

Pour l'affectation

4:  $x := x - y;$   
5:

la condition de vérification due à Floyd est "déductive":

$$P_5(x, y, x, y, q) \leftarrow [\exists x'. P_4(x, y, x', y, q) \wedge x = x' - y]$$

la plus forte post-condition déduite de la précondition doit entraîner la post-condition. la condition de vérification due à Hoare est "prédictive":

$$P_4(x, y, x, y, q) \Rightarrow P_5(x, y, x - y, y, q)$$

La précondition doit entraîner la plus faible précondition prédite à partir de la post-condition. Ces deux conditions de vérification sont équivalentes.

Cette remarque se généralise comme suit:

La condition de vérification (-J-.i)

$$[\forall \Delta, \Delta' \in S, a \in A. [\exists \Delta' \in S. \varepsilon(\Delta) \wedge I(\Delta, \Delta') \wedge t_a(\Delta', \Delta)] \Rightarrow I(\Delta, \Delta)]$$

est équivalente à

$$\begin{aligned} & [\forall \Delta, \Delta' \in S, a \in A. [\varepsilon(\Delta) \wedge I(\Delta, \Delta')] \Rightarrow [t_a(\Delta', \Delta) \Rightarrow I(\Delta, \Delta)]] \\ \Leftrightarrow & [\forall \Delta, \Delta' \in S, a \in A. [\varepsilon(\Delta) \wedge I(\Delta, \Delta')] \Rightarrow [\forall \Delta \in S. t_a(\Delta', \Delta) \Rightarrow I(\Delta, \Delta)]] \\ \Leftrightarrow & [\forall \Delta, \Delta \in S, a \in A. [\varepsilon(\Delta) \wedge I(\Delta, \Delta)] \Rightarrow \neg [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge \neg I(\Delta, \Delta')]] \end{aligned}$$

Alors à partir de (-J-), nous dérivons le principe d'induction équivalent

(-J̃-) :

$$\begin{array}{l} \boxed{[\exists I \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}) . \forall \Delta, \Delta, \bar{\Delta} \in S, a \in A.} \\ \begin{array}{l} (-\tilde{J}-. \varepsilon) \quad \varepsilon(\Delta) \Rightarrow I(\Delta, \Delta) \\ (-\tilde{J}-. i) \quad \wedge \quad [\varepsilon(\Delta) \wedge I(\Delta, \Delta)] \Rightarrow \neg [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge \neg I(\Delta, \Delta')] \\ (-\tilde{J}-. \sigma) \quad \wedge \quad [\varepsilon(\Delta) \wedge I(\Delta, \bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow \psi(\Delta, \bar{\Delta}) \end{array} \quad (-\tilde{J}-) \\ \Leftrightarrow \\ [\forall p \in \Sigma \leq S, A, t, \text{tt}, i \in |p|. [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_0, p_i)] \end{array}$$

#### 4.2.1.2.3 Transformation par inversion

Définissons l'inverse  $p^{-1}$  d'une trace finie

$$p = \Delta_0 \xrightarrow{a_0} \Delta_1 \dots \Delta_{m-2} \xrightarrow{a_{m-2}} \Delta_{m-1} = \langle m, \Delta, a \rangle$$

comme étant :

$$p^{-1} = \Delta_{m-1} \xrightarrow{a_{m-2}} \Delta_{m-2} \dots \Delta_1 \xrightarrow{a_0} \Delta_0 = \langle m, \Delta', a' \rangle$$

$$\text{où } \forall i \in m. \Delta'_i = \Delta_{m-i-1} \text{ et } \forall i \in (m-1). a'_i = a_{m-i-2}$$

et l'inverse d'un ensemble  $\Sigma$  de traces comme étant l'ensemble  $\Sigma^{-1}$  des inverses des traces de  $\Sigma$  :

$$\Sigma^{-1} = \{p^{-1} : p \in \Sigma\}$$

Démontrer pour tout  $p$  qu'on a la propriété d'invariance  
 $\forall i \in |p|. [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_0, p_i)$

est équivalent, d'après le théorème 4.1.3<sup>ve</sup>, à la preuve que pour tous les préfixes finis  $q \in \Sigma^{<\omega}$  des traces de  $\Sigma$ , nous avons :

$$[\varepsilon(q_0) \wedge \sigma(q_{|q|+1})] \Rightarrow \psi(q_0, q_{|q|+1})$$

en posant  $\langle S, A, \Sigma^{<\omega} \rangle = \text{Pref}^{<\omega}(\langle S, A, \Sigma \rangle)$ , ou bien encore en raisonnant sur les traces inverses :

$$\forall q \in (\Sigma^{<\omega})^{-1}. [\sigma(q_0) \wedge \varepsilon(q_{|q|+1})] \Rightarrow \psi^{-1}(q_0, q_{|q|+1})$$

Dans le cas particulier où tout état  $p_i$  d'une trace  $p$  de  $\Sigma$  satisfaisant  $\varepsilon$  est origine du suffixe de la trace de  $p$  commençant à  $p_i$  :

$$\forall p \in \Sigma, i \in |p|. [\varepsilon(p_i) \Rightarrow (p^{>i} \in \Sigma)]$$

la propriété ci-dessus est équivalente à la propriété d'invariance suivante qui porte sur les inverses de préfixes finis de  $\Sigma$  :

$$\forall p \in (\Sigma^{<\omega})^{-1}, i \in |p|. [\sigma(p_0) \wedge \varepsilon(p_i)] \Rightarrow \psi^{-1}(p_0, p_i)$$

Dans ce cas, toute preuve d'invariance portant sur  $S, A, \Sigma$ ,  $\varepsilon, \sigma, \psi$  peut se faire en raisonnant respectivement sur  $S, A, (\Sigma^{<\omega})^{-1}$ ,  $\sigma, \varepsilon, \psi^{-1}$ .

Dans le cas particulier où l'ensemble de traces est engendré par un système de transition  $\langle S, A, t, \varepsilon \rangle$ , nous obtenons :

$$[\forall \underline{A}, \bar{A} \in S. [\sigma(\underline{A}) \wedge (t^*(\underline{A}, \bar{A}))^{-1} \wedge \varepsilon(\bar{A})] \Rightarrow \psi^{-1}(\underline{A}, \bar{A})]$$

qui peut s'écrire puisque  $(t^*(\underline{a}, \bar{a}))^{-1} = t^{-1}*(\underline{a}, \bar{a})$ ,

$$[\forall \underline{a}, \bar{a} \in S. [\sigma(\underline{a}) \wedge t^{-1}*(\underline{a}, \bar{a}) \wedge \varepsilon(\bar{a})] \Rightarrow \psi^{-1}(\underline{a}, \bar{a})]$$

Ceci peut se démontrer en utilisant le principe d'induction (-I-) où  $\varepsilon, t, \sigma, \psi$  sont respectivement choisis comme  $\sigma, t^{-1}, \varepsilon, \psi^{-1}$  d'où les conditions de vérification suivantes :

$$[\exists I \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}', \bar{a} \in S, a \in A .$$

$$\begin{aligned} & \sigma(\underline{a}) \Rightarrow I(\underline{a}, \underline{a}) \\ \wedge & [\exists \underline{a}' \in S. \sigma(\underline{a}) \wedge I(\underline{a}, \underline{a}') \wedge t_a^{-1}(\underline{a}', \underline{a})] \Rightarrow I(\underline{a}, \underline{a}) \\ \wedge & [\sigma(\underline{a}) \wedge I(\underline{a}, \bar{a}) \wedge \varepsilon(\bar{a})] \Rightarrow \psi^{-1}(\underline{a}, \bar{a})] \end{aligned}$$

soit  $J$  l'inverse  $I^{-1}$  de  $I$ . Ces conditions de vérification sont équivalentes à :

$$[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}', \bar{a} \in S, a \in A .$$

$$\begin{aligned} & \sigma(\underline{a}) \Rightarrow J(\underline{a}, \underline{a}) \\ \wedge & [\exists \underline{a}' \in S. t_a^{-1}(\underline{a}', \underline{a}) \wedge J(\underline{a}', \underline{a}) \wedge \sigma(\underline{a})] \Rightarrow J(\underline{a}, \underline{a}) \\ \wedge & [\varepsilon(\bar{a}) \wedge J(\bar{a}, \underline{a}) \wedge \sigma(\underline{a})] \Rightarrow \psi^{-1}(\underline{a}, \bar{a})] \end{aligned}$$

Renommant les variables muettes  $\underline{a}, \bar{a}$  respectivement en  $\bar{a}, \underline{a}$ , nous obtenons

$$[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}', \bar{a} \in S, a \in A .$$

$$\begin{aligned} & \sigma(\bar{a}) \Rightarrow J(\bar{a}, \bar{a}) \\ \wedge & [\exists \underline{a}' \in S. t_a^{-1}(\underline{a}', \underline{a}) \wedge J(\underline{a}', \bar{a}) \wedge \sigma(\bar{a})] \Rightarrow J(\underline{a}, \bar{a}) \\ \wedge & [\varepsilon(\underline{a}) \wedge J(\underline{a}, \bar{a}) \wedge \sigma(\bar{a})] \Rightarrow \psi^{-1}(\bar{a}, \underline{a})] \end{aligned}$$

utilisant la définition des relations inverses, nous venons de démontrer que le principe d'induction  $(-I^{-1})$  est correct et sémantiquement complet :

$$[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}', \bar{a} \in S, a \in A .$$

|                         |                                                                                                                                                                                 |             |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| $(-I^{-1}.\sigma)$      | $\sigma(\bar{a}) \Rightarrow J(\bar{a}, \bar{a})$                                                                                                                               |             |
| $(-I^{-1}.i)$           | $\wedge [\exists \underline{a}' \in S. t_a^{-1}(\underline{a}', \underline{a}) \wedge J(\underline{a}', \bar{a}) \wedge \sigma(\bar{a})] \Rightarrow J(\underline{a}, \bar{a})$ | $(-I^{-1})$ |
| $(-I^{-1}.\varepsilon)$ | $\wedge [\varepsilon(\underline{a}) \wedge J(\underline{a}, \bar{a}) \wedge \sigma(\bar{a})] \Rightarrow \psi^{-1}(\bar{a}, \underline{a})]$                                    |             |
|                         | $\iff$                                                                                                                                                                          |             |
|                         | $[\forall p \in \Sigma \times S, A, t, \#, i \in  p . [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_0, p_i)]$                                                        |             |

Ce principe d'induction est à la base de la méthode de Morris-Wegbreit [77] dite "subgoal induction".

Plus généralement, la transformation notée  $-1$  consiste à remplacer la preuve :

$$P(S, A, E, \varepsilon, \delta, \psi) \Leftrightarrow \exists I. \text{Co}[[S, A, E, \varepsilon, \delta, \psi]](I)$$

par une preuve :

$$P'(S, A, E^{-1}, \delta, \varepsilon, \psi^{-1}) \Leftrightarrow \exists J. \text{Co}'[[S, A, E^{-1}, \delta, \varepsilon, \psi^{-1}]](J)$$

(où  $J = I^{-1}$ ) quand :

$$P(S, A, E, \varepsilon, \delta, \psi) \Leftrightarrow P'(S, A, E^{-1}, \delta, \varepsilon, \psi^{-1})$$

#### 4.2.1.2.4 Transformation contrapositive

Utilisant la propriété que  $\neg\neg J = J$ , nous pouvons réécrire les conditions de vérification  $(-J^{-1})$  comme suit :

$$[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{A}, \bar{A}, \bar{\delta} \in S, a \in A .$$

$$\begin{aligned} & [E(\underline{A}) \wedge \neg J(\underline{A}, \bar{A}) \wedge \sigma(\bar{A})] \Rightarrow \psi(\underline{A}, \bar{A}) \\ & \wedge [\exists \bar{A}' \in S. t_a(\underline{A}, \bar{A}') \wedge \neg J(\bar{A}', \bar{A}) \wedge \sigma(\bar{A})] \Rightarrow \neg J(\underline{A}, \bar{A}) \\ & \wedge [\sigma(\bar{A}) \Rightarrow \neg J(\bar{A}, \bar{A})] \end{aligned}$$

Posons  $\bar{J} = \neg J$  et utilisons le fait que  $P \Rightarrow Q$  si et seulement si  $\neg Q \Rightarrow \neg P$  dans la condition ci-dessus. Nous obtenons la condition équivalente suivante :

$$\begin{aligned} & [\exists \bar{J} \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{A}, \bar{A}', \bar{\delta} \in S, a \in A . \\ & [E(\underline{A}) \wedge \neg \psi(\underline{A}, \bar{A}) \wedge \sigma(\bar{A})] \Rightarrow \bar{J}(\underline{A}, \bar{A}) \\ & \wedge [\exists \bar{A} \in S. \bar{J}(\underline{A}, \bar{A}) \wedge t_a(\underline{A}, \bar{A}') \wedge \sigma(\bar{A})] \Rightarrow \bar{J}(\bar{A}', \bar{A}) \\ & \wedge [\sigma(\bar{A}) \Rightarrow \neg \bar{J}(\bar{A}, \bar{A})] \end{aligned}$$

à partir de laquelle nous concluons que le principe d'induction  $(\overline{-J^{-1}})$  est équivalent à  $(-J^{-1})$  et est donc correct et sémantiquement complet :

$$\begin{array}{lcl}
 & & [\exists \bar{J} \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \forall \underline{\Delta}, \bar{\Delta}, \bar{\delta} \in S, a \in A. \\
 (-J^{-1}. \varepsilon) & & [\varepsilon(\underline{\Delta}) \wedge \neg \psi(\underline{\Delta}, \bar{\Delta}) \wedge \sigma(\bar{\delta})] \Rightarrow \bar{J}(\underline{\Delta}, \bar{\Delta}) \\
 (-J^{-1}. i) & \wedge & [\exists \bar{\Delta}' \in S. \bar{J}(\bar{\Delta}', \bar{\Delta}) \wedge t_a(\bar{\Delta}', \bar{\Delta}) \wedge \sigma(\bar{\delta})] \Rightarrow \bar{J}(\bar{\Delta}, \bar{\Delta}) \\
 (-J^{-1}. \sigma) & \wedge & \sigma(\bar{\delta}) \Rightarrow \neg \bar{J}(\bar{\Delta}, \bar{\Delta}) \\
 & \iff & \\
 & & [\forall p \in \Sigma \times S, A, t, tt, i \in |p|. [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_0, p_i)]
 \end{array}
 \quad (-J^{-1})$$

Nous obtenons ainsi une nouvelle méthode de preuve par l'absurde.

#### Exemple 4.2.1.2.4-1

En utilisant cette méthode pour démontrer la correction partielle du programme 4.2.1.2-1, nous procédons comme suit :

Cette méthode de preuve étant contrapositive, les invariants locaux décrivent ce qui n'arrivera pas pendant l'exécution du programme :

$$P_1(x, y, q, \bar{x}, \bar{q}) = [(x = \bar{q} \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)]$$

$$P_2(x, y, q, \bar{x}, \bar{q}) = [(x = (\bar{q} - q) \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)]$$

$$P_3(x, y, q, \bar{x}, \bar{q}) = [(x = (\bar{q} - q) \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)]$$

$$P_4(x, y, q, \bar{x}, \bar{q}) = [(x = (\bar{q} - q + 1) \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)]$$

$$P_5(x, y, q, \bar{x}, \bar{q}) = [(x = (\bar{q} - q) \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)]$$

$$P_6(x, y, q, \bar{x}, \bar{q}) = [((x = (\bar{q} - q) \times y + \bar{x}) \Rightarrow (\bar{x} \geq y)) \wedge (x < y)]$$

Soient  $x, y, q$  et  $\bar{x}, \bar{y}, \bar{q}$  les valeurs initiales et finales des variables  $x, y, q$  du programme. Si le programme n'était pas partiellement correct, alors on aurait  $\neg [x = \bar{q} \times y + \bar{x} \wedge \bar{x} < y]$  et donc  $P_1(x, y, q, \bar{x}, \bar{q})$  serait vrai d'après la première condition de vérification :

$$P_1(x, y, q, \bar{x}, \bar{y}) \Leftarrow \neg [x = \bar{q} \times y + \bar{x} \wedge \bar{x} < y]$$

Puis par induction sur le nombre  $n$  de pas de calcul durant l'exécution du programme, l'hypothèse que  $P_1(x, y, q, \bar{x}, \bar{q})$  est vrai et les conditions

de vérification impliquent que les  $P_i(x, y, q, \bar{x}, \bar{q})$ ,  $i = 1, \dots, 6$  sont vrais :

$$P_2(x, y, q, \bar{x}, \bar{q}) \Leftarrow [\exists q'. P_1(x, y, q', \bar{x}, \bar{q}) \wedge q = 0]$$

$$P_3(x, y, q, \bar{x}, \bar{q}) \Leftarrow [(P_2(x, y, q, \bar{x}, \bar{q}) \vee P_5(x, y, q, \bar{x}, \bar{q})) \wedge (x \leq y)]$$

$$P_4(x, y, q, \bar{x}, \bar{q}) \Leftarrow [\exists q'. P_3(x, y, q', \bar{x}, \bar{q}) \wedge q = q' + 1]$$

$$P_5(x, y, q, \bar{x}, \bar{q}) \Leftarrow [\exists x'. P_4(x', y, q, \bar{x}, \bar{q}) \wedge x = x' - y]$$

$$P_6(x, y, q, \bar{x}, \bar{q}) \Leftarrow [(P_3(x, y, q, \bar{x}, \bar{q}) \vee P_5(x, y, q, \bar{x}, \bar{q})) \wedge (x > y)]$$

si nous supposons que l'exécution du programme se termine, alors la dernière condition de vérification :

$$\neg P_6(x, y, q, \bar{x}, \bar{q}) \Leftarrow [x = \bar{x} \wedge q = \bar{q}]$$

implique que  $P_6(x, y, q, \bar{x}, \bar{q})$  n'est pas vrai, donc contradiction. Nous avons donc montré par l'absurde que le programme est partiellement correct.

□

#### 4.2.1.2.5 Transformation relation/assertion

Dans le cas d'une assertion invariante

$$\forall p \in \Sigma \langle S, A, t, tt \rangle, i \in |P|. [\varepsilon(p_0) \wedge \delta(p_i)] \Rightarrow \psi(p_i)$$

l'invariant  $I$  utilisé dans le principe d'induction  $(-I)$  peut également être vraie. Nous obtenons le principe d'induction :

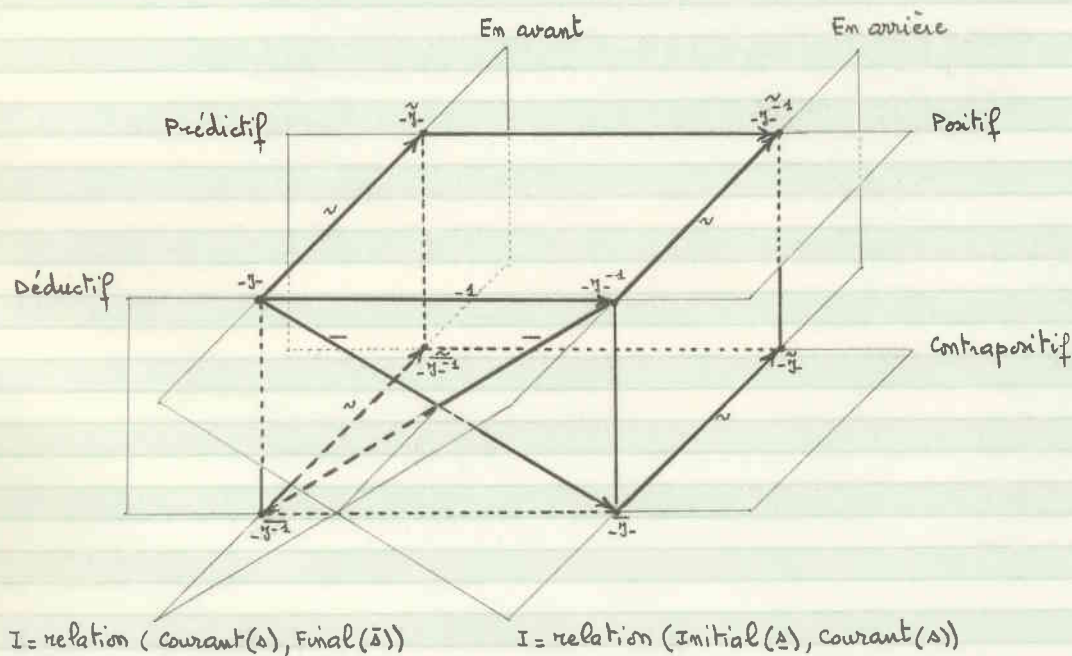
|                                                                                                                                                   |                                                                                                     |       |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------|
| $[\exists i \in (S \rightarrow \{tt, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\Delta} \in S, a \in A .$                                         |                                                                                                     |       |
| (-i-ε)                                                                                                                                            | $\varepsilon(\Delta) \Rightarrow i(\Delta)$                                                         |       |
| (-i-i)                                                                                                                                            | $\wedge$<br>$[\exists \Delta' \in S. i(\Delta') \wedge t_a(\Delta, \Delta')] \Rightarrow i(\Delta)$ | (-i-) |
| (-i-σ)                                                                                                                                            | $\wedge$<br>$[i(\bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow \psi(\bar{\Delta})$          |       |
| $\Leftrightarrow$<br>$[\forall p \in \Sigma \langle S, A, t, tt \rangle, i \in  P . [\varepsilon(p_0) \wedge \delta(p_i)] \Rightarrow \psi(p_i)]$ |                                                                                                     |       |

qui est à la base de la méthode de Floyd-Naur.

### 4.2.1.3 Principes d'induction dérivés par transformations

En partant du principe d'induction de base  $(-I)$  qui est correct et sémantiquement complet, nous obtenons des principes d'induction dérivés en utilisant les trois transformations  $\sim$ ,  $-1$  et  $-$  qui conservent la correction et la complétude sémantique, ce qui évite d'avoir à refaire les démonstrations pour chaque principe d'induction dérivé.

Nous pouvons représenter ces dérivations comme suit :

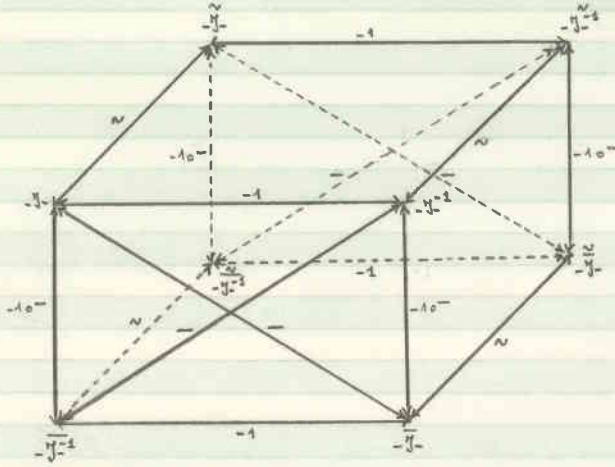


La liste des principes d'induction dérivés est la suivante :

$$[\forall p \in \Sigma \langle S, A, t, \# \rangle, i \in |p|. [E(p_i) \wedge S(p_i)] \Rightarrow \psi(p_i, p_i)]$$

|                  |                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | $\Leftrightarrow$                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                  | $[\exists I \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                               | $[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                                                        |
| $(\tilde{J}-)$   | $\begin{aligned} & E(\underline{a}) \Rightarrow I(\underline{a}, \underline{a}) \\ \wedge & [E(\underline{a}) \wedge I(\underline{a}, \underline{a})] \Rightarrow \neg [\exists a' \in S. t_a(\underline{a}, a') \wedge \neg I(\underline{a}, a')] \\ \wedge & [E(\underline{a}) \wedge I(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \psi(\underline{a}, \bar{a}) \end{aligned}$          | $\begin{aligned} & S(\bar{a}) \Rightarrow J(\bar{a}, \bar{a}) \\ \wedge & [J(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \neg [\exists a' \in S. \neg J(\underline{a}, \bar{a}) \wedge t_a(a', \underline{a})] \\ \wedge & [E(\underline{a}) \wedge J(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \psi(\underline{a}, \bar{a}) \end{aligned}$                                                            |
|                  | $\Leftrightarrow$                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                  | $[\exists I \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                               | $[\exists J \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                                                        |
| $(\tilde{J}-)$   | $\begin{aligned} & E(\underline{a}) \Rightarrow I(\underline{a}, \underline{a}) \\ \wedge & [\exists a' \in S. E(\underline{a}) \wedge I(\underline{a}, a') \wedge t_a(a', \underline{a})] \Rightarrow I(\underline{a}, \underline{a}) \\ \wedge & [E(\underline{a}) \wedge I(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \psi(\underline{a}, \bar{a}) \end{aligned}$                      | $\begin{aligned} & S(\bar{a}) \Rightarrow J(\bar{a}, \bar{a}) \\ \wedge & [\exists a' \in S. t_a(\underline{a}, a') \wedge J(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow J(\underline{a}, \bar{a}) \\ \wedge & [E(\underline{a}) \wedge J(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \psi(\underline{a}, \bar{a}) \end{aligned}$                                                                        |
|                  | $\Leftrightarrow$                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                  | $[\exists \bar{I} \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                         | $[\exists \bar{I} \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                                                  |
| $(\tilde{J}-^+)$ | $\begin{aligned} & [E(\underline{a}) \wedge \neg \psi(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \bar{J}(\underline{a}, \bar{a}) \\ \wedge & [\bar{J}(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \neg [\exists a' \in S. t_a(\underline{a}, a') \wedge \neg \bar{J}(\underline{a}, \bar{a})] \\ \wedge & S(\bar{a}) \Rightarrow \neg \bar{J}(\bar{a}, \bar{a}) \end{aligned}$ | $\begin{aligned} & [E(\underline{a}) \wedge \neg \psi(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \bar{I}(\underline{a}, \bar{a}) \\ \wedge & [E(\underline{a}) \wedge \bar{I}(\underline{a}, \underline{a})] \Rightarrow \neg [\exists a' \in S. \neg \bar{I}(\underline{a}, a') \wedge t_a(a', \underline{a})] \\ \wedge & E(\underline{a}) \Rightarrow \neg \bar{I}(\underline{a}, \underline{a}) \end{aligned}$ |
|                  | $\Leftrightarrow$                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                  | $[\exists \bar{J} \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                         | $[\exists \bar{I} \in (S \times S \rightarrow \{\#, \#\#\}) . \forall \underline{a}, \underline{a}, \bar{a} \in S, a \in A .$                                                                                                                                                                                                                                                                                                  |
| $(\bar{J}-^+)$   | $\begin{aligned} & [E(\underline{a}) \wedge \neg \psi(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \bar{J}(\underline{a}, \bar{a}) \\ \wedge & [\exists a' \in S. \bar{J}(\underline{a}, \bar{a}) \wedge t_a(a', \underline{a}) \wedge S(\bar{a})] \Rightarrow \bar{J}(\underline{a}, \bar{a}) \\ \wedge & S(\bar{a}) \Rightarrow \neg \bar{J}(\bar{a}, \bar{a}) \end{aligned}$             | $\begin{aligned} & [E(\underline{a}) \wedge \neg \psi(\underline{a}, \bar{a}) \wedge S(\bar{a})] \Rightarrow \bar{I}(\underline{a}, \bar{a}) \\ \wedge & [\exists a' \in S. E(\underline{a}) \wedge t_a(\underline{a}, a') \wedge \bar{I}(\underline{a}, a')] \Rightarrow \bar{I}(\underline{a}, \underline{a}) \\ \wedge & E(\underline{a}) \Rightarrow \neg \bar{I}(\underline{a}, \underline{a}) \end{aligned}$             |

Des applications supplémentaires des transformations  $\sim$ ,  $-1$  et  $-$  aux principes d'induction ci-dessus ne donnent pas de nouveaux principes d'induction. Plus généralement, le diagramme suivant commute :

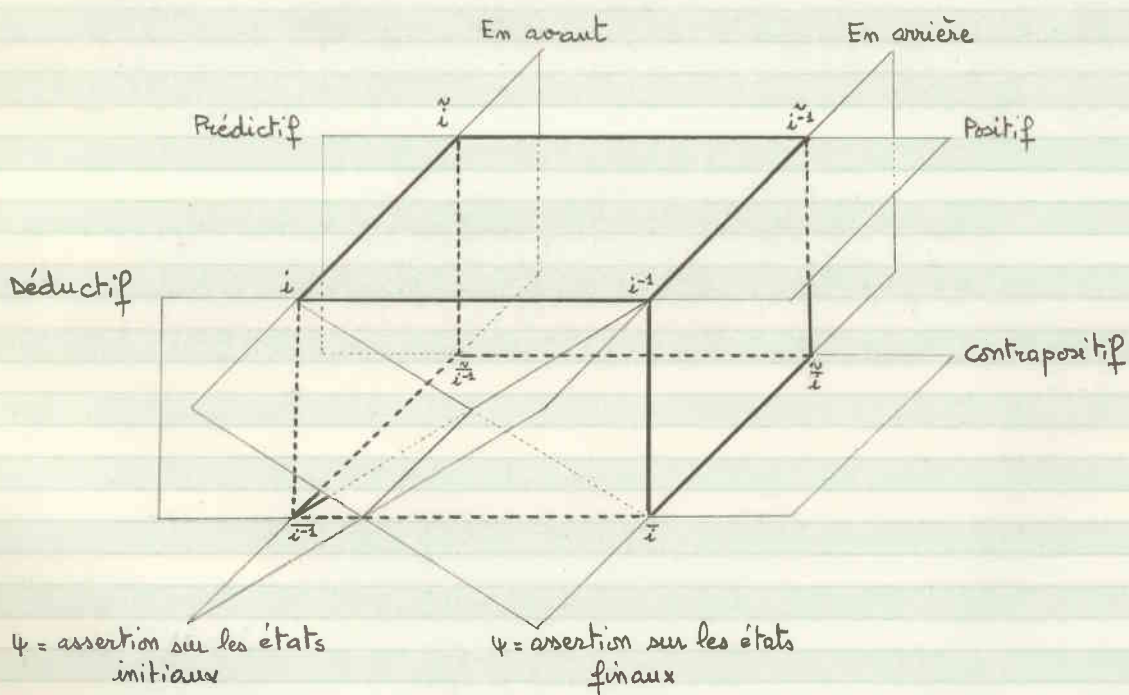


La transformation relation/assertion conduit aux principes d'induction suivants:

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | $\forall p \in Z \langle S, A, t, \{t\} \rangle, i \in  p . [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_i)$ $\iff$ $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} (-i-) \quad & \varepsilon(\Delta) \Rightarrow i(\Delta) \\ & \wedge [\exists \Delta' \in S. i(\Delta') \wedge t_a(\Delta', \Delta)] \Rightarrow i(\Delta) \\ & \wedge [i(\bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow \psi(\bar{\Delta}) \end{aligned}$ | $\forall p \in Z \langle S, A, t, \{t\} \rangle, i \in  p . [\varepsilon(p_0) \wedge \sigma(p_i)] \Rightarrow \psi(p_0)$ $\iff$ $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} & \sigma(\bar{\Delta}) \Rightarrow i(\bar{\Delta}) \\ & \wedge [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge i(\Delta')] \Rightarrow i(\Delta) \\ & \wedge [\varepsilon(\Delta) \wedge i(\Delta)] \Rightarrow \psi(\Delta) \end{aligned}$ $(-i^{-1})$ |
|  | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} (-\tilde{i}-) \quad & \varepsilon(\Delta) \Rightarrow i(\Delta) \\ & \wedge i(\Delta) \Rightarrow \neg [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge \neg i(\Delta')] \\ & \wedge [i(\bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow \psi(\bar{\Delta}) \end{aligned}$                                                                                                               | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} & \sigma(\bar{\Delta}) \Rightarrow i(\bar{\Delta}) \\ & \wedge i(\Delta) \Rightarrow \neg [\exists \Delta' \in S. \neg i(\Delta') \wedge t_a(\Delta', \Delta)] \\ & \wedge [\varepsilon(\Delta) \wedge i(\Delta)] \Rightarrow \psi(\Delta) \end{aligned}$ $(-\tilde{i}^{-1})$                                                                                                               |
|  | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} (-\bar{i}-) \quad & [\neg \psi(\bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow i(\bar{\Delta}) \\ & \wedge [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge i(\Delta')] \Rightarrow i(\Delta) \\ & \wedge \varepsilon(\Delta) \Rightarrow \neg i(\Delta) \end{aligned}$                                                                                                                 | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} & [\varepsilon(\Delta) \wedge \neg \psi(\Delta)] \Rightarrow i(\Delta) \\ & \wedge [\exists \Delta' \in S. i(\Delta') \wedge t_a(\Delta', \Delta)] \Rightarrow i(\Delta) \\ & \wedge \sigma(\bar{\Delta}) \Rightarrow \neg i(\bar{\Delta}) \end{aligned}$ $(-\bar{i}^{-1})$                                                                                                                 |
|  | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} (-\tilde{\bar{i}}-) \quad & [\neg \psi(\bar{\Delta}) \wedge \sigma(\bar{\Delta})] \Rightarrow i(\bar{\Delta}) \\ & \wedge i(\Delta) \Rightarrow \neg [\exists \Delta' \in S. \neg i(\Delta') \wedge t_a(\Delta', \Delta)] \\ & \wedge \varepsilon(\Delta) \Rightarrow \neg i(\Delta) \end{aligned}$                                                                                               | $[\exists i \in (S \rightarrow \{t, ff\}) . \forall \Delta, \bar{\Delta}, \bar{\sigma} \in S, a \in A.$ $\begin{aligned} & [\varepsilon(\Delta) \wedge \neg \psi(\Delta)] \Rightarrow i(\Delta) \\ & \wedge i(\Delta) \Rightarrow \neg [\exists \Delta' \in S. t_a(\Delta, \Delta') \wedge \neg i(\Delta')] \\ & \wedge \sigma(\bar{\Delta}) \Rightarrow \neg i(\bar{\Delta}) \end{aligned}$ $(-\tilde{\bar{i}}^{-1})$                                                                                               |

Notons cependant que tous les principes d'induction relationnels sont équivalents tandis que pour les principes d'induction assertionnels, qui servent à démontrer différentes propositions, seuls ceux d'une même colonne sont équivalents.

Nous pouvons classer ces principes d'induction assertionnels comme suit :



En appliquant la transformation par distinction/confusion des états initiaux, des états finaux et des états initiaux et finaux, nous obtenons 64 autres principes d'induction.

Nous verrons dans la suite que chaque principe d'induction peut donner lieu à un grand nombre de méthodes de preuves.

#### 4.2.1.4 Equivalence forte des principes d'induction dérivés

Tous les principes d'induction dans une même catégorie (c'est-à-dire permettant de démontrer une propriété d'invariance d'un certain type) sont corrects et sémantiquement complets. Par conséquent, s'il existe une preuve par une méthode  $M$  alors il existe également une preuve par toute autre méthode  $M'$  de la même catégorie.

En ce qui concerne les méthodes de Floyd [67] et Morris-Wegbreit [77], Dijkstra [82] a montré que toute preuve de correction partielle par "subgoal induction" peut se réécrire en une preuve par la méthode de Floyd (et conclut que la méthode de Morris-Wegbreit est inutile). Morris et Wegbreit avaient démontré la réciproque. Ce dernier résultat est généralisé dans Cousot-R [81] aux principes d'induction  $(-Y-)$  et  $(-Y^{-1}-)$  comme suit :

##### Théorème 4.2.1.4~1

Les conditions . $\epsilon$ , .i et . $\delta$  de  $(-Y^{-1}-)$  et . $\epsilon$ , .i et . $\delta$  de  $(-Y-)$  sont équivalentes en définissant :

$$I(\underline{A}, \bar{A}) = [\forall \bar{A} \in S. (J(\underline{A}, \bar{A}) \wedge \delta(\bar{A})) \Rightarrow \psi(\underline{A}, \bar{A})]$$

$$J(\underline{A}, \bar{A}) = [\forall \underline{A} \in S. (\epsilon(\underline{A}) \wedge I(\underline{A}, \bar{A})) \Rightarrow \psi(\underline{A}, \bar{A})]$$

et la remarque de Dijkstra se généralise de la même manière :

##### Théorème 4.2.1.4~2

Les conditions . $\epsilon$ , .i et . $\delta$  de  $(-Y^{-1}-)$  et . $\epsilon$ , .i et . $\delta$  de  $(-Y-)$  sont équivalentes en définissant :

$$I(\underline{A}, \bar{A}) = [\epsilon(\underline{A}) \wedge \forall \bar{A} \in S. (J(\underline{A}, \bar{A}) \wedge \delta(\bar{A})) \Rightarrow J(\underline{A}, \bar{A})]$$

$$J(\underline{A}, \bar{A}) = [\delta(\bar{A}) \wedge \forall \underline{A} \in S. (\epsilon(\underline{A}) \wedge I(\underline{A}, \bar{A})) \Rightarrow I(\underline{A}, \bar{A})]$$

Démonstration

- si  $J$  satisfait les conditions . $\bar{s}$ , . $i$  et . $\bar{e}$  de  $(-Y^{-1})$  alors nous avons :

$$(-Y-.e) \quad E(\underline{A}) \Rightarrow [E(\underline{A}) \wedge \forall \bar{A} \in S. ((J(\underline{A}, \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}, \bar{A}))] \Rightarrow I(\underline{A}, \underline{A})$$

$$\begin{aligned} (-Y-.i) \quad [I(\underline{A}, \underline{A}') \wedge t_q(\underline{A}', \underline{A})] &\Rightarrow [E(\underline{A}) \wedge \forall \bar{A} \in S. ((J(\underline{A}', \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}, \bar{A}) \wedge t(\underline{A}', \underline{A}))] \Rightarrow \\ &[E(\underline{A}) \wedge \forall \bar{A} \in S. ((J(\underline{A}', \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}, \bar{A})) \wedge \forall \bar{A}' \in S. ((J(\underline{A}, \bar{A}') \wedge \bar{s}(\bar{A}')) \Rightarrow J(\underline{A}', \bar{A}'))] \text{ (car} \\ &t_q(\underline{A}', \underline{A}) \Rightarrow [(J(\underline{A}', \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}', \bar{A})] \text{ d'après } (-Y^{-1}.i)) \Rightarrow \\ &[E(\underline{A}) \wedge \forall \bar{A} \in S. ((J(\underline{A}, \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}, \bar{A}))] \Rightarrow I(\underline{A}, \underline{A}) \end{aligned}$$

$$\begin{aligned} (-Y-.s) \quad [I(\underline{A}, \bar{A}) \wedge \bar{s}(\bar{A})] &\Rightarrow [E(\underline{A}) \wedge \forall \bar{A} \in S. ((J(\bar{A}, \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow J(\underline{A}, \bar{A}) \wedge J(\bar{A}, \bar{A}) \wedge \bar{s}(\bar{A}))] \\ &\text{(d'après la définition de } I \text{ et } (-Y^{-1}.s)) \Rightarrow [E(\underline{A}) \wedge J(\underline{A}, \bar{A}) \wedge \bar{s}(\bar{A}) \Rightarrow \\ &\psi(\underline{A}, \bar{A}) \text{ (d'après } (-Y^{-1}.e)). \end{aligned}$$

- Pour la réciproque, nous utilisons la transformation -1.

□

- Ces résultats se généralisent aisément à tous les principes d'induction dérivés de  $(-Y)$  par les transformations  $\sim$ ,  $-1$  et  $-$  :

Ceci signifie que si une preuve a été faite en utilisant une méthode  $M$  (c'est-à-dire que nous avons trouvé un invariant  $I$  satisfaisant les conditions de vérification  $CV$ ) et  $M'$  est une autre méthode qui nécessite la découverte d'un invariant  $I'$  satisfaisant  $CV'$ , alors nous pouvons définir formellement  $I'$  en fonction de  $I$  de sorte que  $CV(I)$  implique  $CV'(I')$  :

- Si  $M' = \tilde{M}$  alors  $I' = I$
- Si  $M' = \bar{M}$  alors  $I' = \neg I$
- Si  $M' = M^{-1}$  et  $M$  est une méthode positive où  $I$  relie un état initial à un état courant alors  $I'(\underline{A}, \bar{A}) = [\forall \underline{A} \in S. (E(\underline{A}) \wedge I(\underline{A}, \underline{A})) \Rightarrow \psi(\underline{A}, \bar{A})]$
- Si  $M' = (M^{-1})^{-1}$  et  $M$  est une méthode contrapositive où  $I$  relie un état initial à un état courant alors  $I'(\underline{A}, \bar{A}) = [\exists \underline{A} \in S. E(\underline{A}) \wedge I(\underline{A}, \underline{A}) \wedge \neg \psi(\underline{A}, \bar{A})]$
- Si  $M' = M^{-1}$  et  $M$  est une méthode positive où  $I$  relie un état courant à un état final alors  $I'(\underline{A}, \underline{A}) = [\forall \bar{A} \in S. (I(\underline{A}, \bar{A}) \wedge \bar{s}(\bar{A})) \Rightarrow \psi(\underline{A}, \bar{A})]$

- Si  $M' = M^{-1}$  et  $M$  est une méthode contrapositive où  $I$  relie un état courant à un état final alors  $I'(\underline{A}, \bar{A}) = [\exists \bar{B} \in S. I(\underline{A}, \bar{B}) \wedge \bar{B} \wedge \neg \psi(\underline{A}, \bar{B})]$

— De même quand  $\psi$  est une assertion, alors une preuve par un principe d'induction (M) peut également se faire par le principe d'induction (M') correspondant en choisissant

$$I(\underline{A}, \bar{A}) = [\varepsilon(\underline{A}) \wedge i(\bar{A})] \quad (\text{ou bien } I(\underline{A}, \bar{A}) = [i(\underline{A}) \wedge \varepsilon(\bar{A})])$$

La technique des variables auxiliaires (qui permet de recréer une preuve d'invariance d'une relation par une méthode relationnelle en une preuve assertionnelle pour un programme transformé où les valeurs initiales des variables sont mémorisées dans des variables auxiliaires) se généralise comme suit :

si  $[\forall p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |P|. \varepsilon(p_i) \Rightarrow \psi(p_0, p_i)]$  a été démontré par la méthode (M) utilisant l'invariant relationnel  $I$ , alors nous pouvons faire la même démonstration par la méthode (m) en posant :

- si  $I$  relie un état initial à un état courant,  
 $S' = S \times S, A' = A, \varepsilon'(\langle \underline{A}, \underline{A}' \rangle) = [\varepsilon(\underline{A}) \wedge \underline{A} = \underline{A}'], t'_a(\langle \underline{A}, \underline{A}' \rangle, \langle \underline{A}', \underline{A}'' \rangle) = [\underline{A} = \underline{A}' \wedge t_a(\underline{A}, \underline{A}')], \varepsilon'(\langle \underline{A}, \underline{A}' \rangle) = \varepsilon(\underline{A})$  avec l'invariant uneaire  $i(\langle \underline{A}, \underline{A}' \rangle) = I(\underline{A}, \underline{A}')$ .
- si  $I$  relie un état courant à un état final,  
 $S' = S \times S, A' = A, \varepsilon'(\langle \underline{A}, \bar{A} \rangle) = [\underline{A} = \bar{A} \wedge \varepsilon(\bar{A})], t'_a(\langle \underline{A}, \bar{A} \rangle, \langle \underline{A}', \bar{A}' \rangle) = [t_a(\underline{A}, \underline{A}') \wedge \bar{A} = \bar{A}'], \varepsilon'(\langle \underline{A}, \bar{A} \rangle) = \varepsilon(\underline{A})$  avec l'invariant uneaire  $i(\langle \underline{A}, \bar{A} \rangle) = I(\underline{A}, \bar{A})$ .

#### 4.2.2 PRINCIPES D'INDUCTION POUR UNE SEMANTIQUE NON CLOSE FERMEE PAR FUSIONS

Dans le cas d'une sémantique  $\langle S, A, \Sigma \rangle$  fermée par fusions (c'est-à-dire  $\underline{E}_{fus}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle$ ), l'invariance de  $\psi$  sous condition  $\phi$  pour  $\langle S, A, \Sigma \rangle$  est équivalente, d'après les théorèmes 4.1.3 v 3.7 et 4.1.3 v 4 à l'invariance de  $\psi$  sous condition  $\phi$  pour  $\underline{R}_{trans}(\langle S, A, \Sigma \rangle)$ . Ceci montre que dans ce cas il est toujours correct et sémantiquement complet de faire les preuves d'invariance en utilisant l'un quelconque des principes d'induction du paragraphe 4.2.1 pour le système de transition engendré par cette sémantique  $\langle S, A, \Sigma \rangle$ . C'est le cas en particulier (et d'après 4.1.3 v 3.8 et 4.1.3 v 3.9) pour le langage que nous avons considéré en 2.8.

#### 4.2.3 PRINCIPES D'INDUCTION POUR UNE SEMANTIQUE (NON CLOSE ET) NON FERMEE PAR FUSIONS

##### 4.2.3.1 Principes d'induction pour une sémantique non fermée par fusions définie par une condition sur les préfixes des traces engendrées par un système de transition

Dans le cas d'une sémantique  $\langle S, A, \Sigma \rangle$  non fermée par fusions, définie par une condition sur les préfixes des traces engendrées par un système de transition  $\langle S, A, t, E \rangle$  :

$$\langle S, A, \Sigma \rangle = \langle S, A, \{p \in \Sigma' : \langle S, A, \Sigma' \rangle = \underline{Pref}(\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle) \wedge Cp(p)\} \rangle$$

$\psi$  est invariante sous condition  $\phi$  pour  $\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$  si et seulement si  $\psi$  est invariante sous condition  $\phi$  pour  $\underline{Pref}(\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle)$  (cf. 4.1.3 v 3.1) et seulement si  $\psi$  est invariante sous condition  $\phi$  pour  $\langle S, A, \Sigma \rangle$  (cf. 4.1.3 v 1).

Il est donc toujours correct de faire une preuve d'invariance relative à  $\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$  (pour laquelle nous pouvons toujours utiliser l'un quelconque des principes d'induction du paragraphe 4.2.1) pour conclure relativement à  $\langle S, A, \Sigma \rangle$ . Cette approche n'est pas toujours complète (comme en témoigne 4.1.3v1).

Pour être complet nous proposons d'utiliser le principe d'induction suivant (ou ses transformées comme en 4.2.1.2) qui utilise des variables auxiliaires (dans la preuve mais pas dans le programme) permettant de cumuler des histoires. Ce principe d'induction est directement inspiré d'un principe d'induction similaire proposé dans Cousot-Cousot [82] pour des preuves de fatalité. Il peut se motiver comme suit :

Pour démontrer que :

$\psi$  est invariante pour  $\langle S, A, \Sigma \rangle$

il faut et il suffit en général de démontrer l'invariance d'une propriété plus forte :

$\exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). [J \text{ est invariante pour } \langle S, A, \Sigma \rangle \wedge J \Rightarrow \psi]$

D'après le théorème 4.1.3v2, il suffit de démontrer l'invariance pour les préfixes finis :

$\exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). [J \text{ est invariante pour } \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle \wedge J \Rightarrow \psi]$

En écrivant  $\forall p \in \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle. Q(p)$  pour abréger  $\forall p. [(\exists \Sigma'. [\langle S, A, \Sigma' \rangle = \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle \wedge p \in \Sigma']) \Rightarrow Q(p)]$ , ceci équivaut d'après 3.2.2 à :

$\exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). [\forall p \in \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle, i \in |p|. J(p_0, p_i) \wedge J \Rightarrow \psi]$

En procédant par induction sur  $i$ , nous obtenons :

$\exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \forall p \in \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle.$

$J(p_0, p_0)$   
 $\wedge$   
 $\forall i \in |p| \setminus \{0\}. J(p_0, p_{i-1}) \Rightarrow J(p_0, p_i)$   
 $\wedge$   
 $J \Rightarrow \psi$

Il suffit de faire la preuve pour  $i = |p|$  car si  $p \in \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle$  et  $i \in |p|$  alors  $p^{<i} \in \text{Pref}^{\omega}_{\text{fin}} \langle S, A, \Sigma \rangle$  :

$$\exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \forall p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle).$$

$$\begin{aligned} & J(p_0, p_0) \\ \wedge & [ |p| > 1 \wedge J(p_0, p_{|p|-1}) ] \Rightarrow J(p_0, p_{|p|}) \\ \wedge & J \Rightarrow \psi \end{aligned}$$

La condition  $(p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle) \wedge |p| > 1)$  équivaut à  $(\exists p' \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle), a \in A, \Delta \in S. p = p' \xrightarrow{a} \langle \Delta \rangle \wedge p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle))$ , ce qui donne :

$$\begin{aligned} & \exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \\ & \forall p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle). J(p_0, p_0) \\ \wedge & \forall p' \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle), a \in A, \Delta \in S. \\ & [ J(p'_0, p'_{|p'|}) \wedge p' \xrightarrow{a} \langle \Delta \rangle \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle) ] \Rightarrow J(p'_0, \Delta) \\ \wedge & J \Rightarrow \psi \end{aligned}$$

En introduisant le système de transition  $\langle S, A, t, \varepsilon \rangle$  engendré par  $\langle S, A, \Sigma \rangle$ , ceci peut s'écrire :

$$\begin{aligned} & \exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \\ & \forall \Delta \in S. \varepsilon(\Delta) \Rightarrow J(\Delta, \Delta) \\ \wedge & \forall p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle), a \in A, \Delta \in S. \\ & [ J(p_0, p_{|p|}) \wedge t_a(p_{|p|}, \Delta) \wedge p \xrightarrow{a} \langle \Delta \rangle \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle) ] \Rightarrow J(p_0, \Delta) \\ \wedge & J \Rightarrow \psi \end{aligned}$$

ou encore

$$\begin{aligned} & \exists J \in (S \times S \rightarrow \{\text{tt}, \text{ff}\}). \\ & \forall \Delta, \Delta', \bar{\Delta}, \bar{\Delta} \in S, p', \bar{p} \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle), a \in A. \\ & \varepsilon(\Delta) \Rightarrow [ \exists p \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle). [ p_0 = \Delta \wedge J(\Delta, \Delta) ] ] \\ \wedge & [ [ p'_0 = \Delta \wedge J(\Delta, \Delta') ] \wedge t_a(\Delta', \Delta) \wedge [ \Delta' = \bar{p}'_{|p'|} \wedge \bar{p}' \xrightarrow{a} \langle \Delta \rangle \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle) ] ] \\ & \Rightarrow [ p'_0 = \Delta \wedge J(\Delta, \Delta) ] \\ \wedge & [ \bar{p}_0 = \bar{\Delta} \wedge J(\bar{\Delta}, \bar{\Delta}) ] \Rightarrow \psi(\Delta, \bar{\Delta}) \end{aligned}$$

En posant

$$H = \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle)$$

$$F(\underline{a}, p) = [p_0 = \underline{a}]$$

$$R(\underline{a}', p', a, \underline{a}) = [\underline{a}' = p'_{|H|} \wedge p' \xrightarrow{a} \langle \underline{a} \rangle \in \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle)]$$

$$N(\underline{a}', p', a, \underline{a}, p) = [\underline{a}' = p'_{|H|} \wedge p = p' \xrightarrow{a} \langle \underline{a} \rangle]$$

$$I(\underline{a}, \bar{a}, p) = [p_0 = \underline{a} \wedge J(\underline{a}, \bar{a})]$$

nous obtenons

$$\exists I \in (S \times S \times H \rightarrow \{\text{tt}, \text{ff}\}) . \forall \underline{a}, \bar{a}, \underline{a}', \bar{a}' \in S, p, \bar{p} \in H, a \in A.$$

$$E(\underline{a}) \Rightarrow [\exists p \in H. F(\underline{a}, p) \wedge I(\underline{a}, \underline{a}, p)]$$

$$\wedge [I(\underline{a}, \underline{a}', p') \wedge t_a(\underline{a}', \underline{a}) \wedge R(\underline{a}', p', a, \underline{a})] \Rightarrow [\exists p \in H. N(\underline{a}', p', a, \underline{a}, p) \wedge I(\underline{a}, \underline{a}, p)]$$

$$\wedge I(\underline{a}, \bar{a}, \bar{p}) \Rightarrow \Psi(\underline{a}, \bar{a})$$

En pratique, nous évitons de raisonner sur les (préfixes des) traces de  $\Sigma$  en utilisant des "histoires" que nous interprétons comme des fonctions des préfixes des traces cumulant l'essentiel de l'information contenue dans ces préfixes. En comprenant  $H$  comme un ensemble d'histoires,  $F(\underline{a}, h)$  comme l'assertion que l'histoire  $h$  commence en l'état  $\underline{a}$ ,  $R(\underline{a}', h', a, \underline{a})$  comme l'assertion qu'au terme de l'histoire  $h'$  l'état  $\underline{a}'$  a un successeur  $\underline{a}$  par l'action  $a$  et  $N(\underline{a}', h', a, \underline{a}, h)$  comme l'assertion que l'information, qu'au terme de l'histoire  $h'$  l'état  $\underline{a}'$  a un successeur  $\underline{a}$  par l'action  $a$ , se cumule dans l'histoire  $h$ , nous obtenons le principe d'induction suivant :

## Théorème 4.2.3.1 v1

$$[ [\exists H, F \in (S \times H \rightarrow \{\text{tt}, \text{ff}\}), R \in (S \times H \times A \times S \rightarrow \{\text{tt}, \text{ff}\}), \\ N \in (S \times H \times A \times S \times H \rightarrow \{\text{tt}, \text{ff}\}) .$$

$$\text{Hut} \langle S, A, \Sigma \rangle (H, F, N, R)$$

$$\wedge (\exists I \in (S \times S \times H \rightarrow \{\text{tt}, \text{ff}\}). \forall \Delta, \Delta', \bar{\Delta}, \bar{\Delta}' \in S, h, \bar{h} \in H, a \in A.$$

$$\varepsilon(\Delta) \Rightarrow [\exists h \in H. F(\Delta, h) \wedge I(\Delta, \Delta, h)]$$

(-↑)]

$$\wedge [I(\Delta, \Delta', h') \wedge t_a(\Delta', \Delta) \wedge R(\Delta', h', a, \Delta)]$$

$$\Rightarrow [\exists h \in H. N(\Delta', h', a, \Delta, h) \wedge I(\Delta, \Delta, h)]$$

$$\wedge [I(\Delta, \bar{\Delta}, \bar{h}) \Rightarrow \psi(\Delta, \bar{\Delta})]$$

 $\Leftrightarrow$ 

$$[\forall p \in \Sigma, i \in |p|. \psi(p_0, p_i)]$$

ou

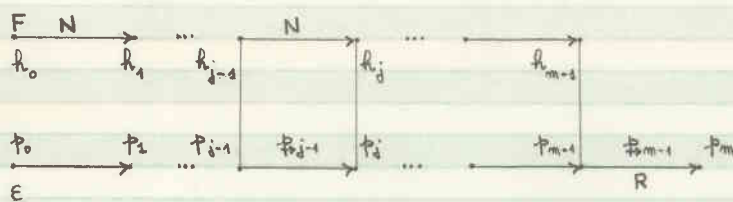
$$\text{Hut} \langle S, A, \Sigma \rangle (H, F, N, R) =$$

$$[\forall p \in \Sigma, m \in \omega. (1 \leq m < |p|) \Rightarrow$$

$$(\forall h \in (m \rightarrow H). [F(p_0, h_0) \wedge \forall j \in (m \vee 0). N(p_{j-1}, h_{j-1}, p_{j-1}, p_j, h_j)]$$

$$\Rightarrow R(p_{m-1}, h_{m-1}, p_{m-1}, p_m)])]$$

une formule qui se comprend mieux à l'aide du schéma suivant :



Démonstration

( $\Rightarrow$ ) Soit  $p \in \Sigma$ . Nous démontrons par induction sur  $m \in (|p| \vee 0)$  qu'il existe  $h \in (m \rightarrow H)$  tel que  $\forall j \in m. I(p_0, p_j, h_j) \wedge F(p_0, h_0) \wedge \forall j \in (m \vee 0). N(p_{j-1}, h_{j-1}, p_j, h_j)$ .  
 En effet pour  $m=1$ , nous avons  $\varepsilon(p_0) \Rightarrow \exists h_0 \in H. F(p_0, h_0) \wedge I(p_0, p_0, h_0)$ . Si par hypothèse d'induction le lemme est vrai pour  $m-1 \geq 1$  et  $m \in |p|$  alors nous avons  $I(p_0, p_{m-2}, h_{m-2})$  et d'après la condition Hist,  $R(p_{m-2}, h_{m-2}, p_{m-1}, h_{m-1})$  ce qui avec  $\vdash_{p_{m-2}} (p_{m-2}, p_{m-1})$  implique que  $\exists h_{m-1} \in H. N(p_{m-2}, h_{m-2}, p_{m-1}, h_{m-1}) \wedge I(p_0, p_{m-1}, h_{m-1})$ . Nous en déduisons que  $\forall j \in |p|. I(p_0, p_j, h_j)$  et donc  $\psi(p_0, p_j)$ .

( $\Leftarrow$ ) Si  $\forall p \in \Sigma, i \in |p|. \psi(p_0, p_i)$  alors nous pouvons choisir :

$$H = 2^{\Sigma \times \omega}, \quad F(\Delta, \langle T, m \rangle) = [T = \{p \in \Sigma : p_0 = \Delta\} \neq \emptyset \wedge m = 0], \quad N(\Delta', \langle T', m' \rangle, a, \Delta, \langle T, m \rangle) =$$

$$[m = m' + 1 \wedge T = \{p \in T' : m \in |p| \wedge p_m = \Delta' \wedge p_{m-1} = a \wedge p_m = \Delta\} \neq \emptyset], \quad R(\Delta', \langle T', m' \rangle, a, \Delta) =$$

$$[\exists p \in T'. (m' + 1) \in |p| \wedge p_{m'} = \Delta' \wedge p_{m-1} = a \wedge p_{m'+1} = \Delta] \text{ et } I(\Delta, \Delta, \langle T, m \rangle) =$$

$$[T = \{p \in \Sigma : m \in |p| \wedge p_0 = \Delta \wedge p_m = \Delta\} \neq \emptyset].$$

□

Observons que  $(\neg \uparrow \downarrow)$  est une généralisation de  $(\neg \downarrow)$  que nous retrouvons en choisissant  $F = tt$ ,  $R = tt$ ,  $N = tt$  et  $H = 1$ .

#### 4.2.3.2 Principes d'induction pour une sémantique non fermée par fusions définie par concordance avec une sémantique close

Il est toujours possible de définir une sémantique non close  $\langle S, A, \Sigma \rangle$  par concordance avec une sémantique close (engendrée par un système de transition  $\langle S^\#, A^\#, E^\#, \varepsilon^\# \rangle$ ) à une fonction  $f_\Delta^\# \in (S^\# \rightarrow S)$  entre états près (cf. 2.7.2.2 ~ 1) :

$$\langle S, A, \Sigma \rangle = \approx \langle f_\Delta^\# \rangle (\langle S^\#, A^\#, \Sigma \langle S^\#, A^\#, E^\#, \varepsilon^\# \rangle \rangle)$$

Par conséquent, pour démontrer que  $\psi \in (S \times S \rightarrow \{t, ff\})$  est invariante pour  $\langle S, A, \Sigma \rangle$  il est toujours correct et possible (d'après 4.1.3~3 et 4.1.3~4) d'utiliser l'un quelconque des principes d'induction du paragraphe 4.2.1 pour  $\langle S^{\#}, A^{\#}, E^{\#}, \varepsilon^{\#} \rangle$  et  $\psi^{\#}$  définie par  $\psi^{\#}(A_0, A_1) = \psi(f_{A_0}^{\#}(A_0), f_{A_1}^{\#}(A_1))$ .

Par exemple, en utilisant le principe d'induction  $(-I^{\#})$ , nous obtenons :

$$\begin{aligned}
 & [ \exists S^{\#}, A^{\#}, E^{\#} \in (A^{\#} \rightarrow (S^{\#} \times S^{\#} \rightarrow \{t, ff\})), \varepsilon^{\#} \in (S^{\#} \rightarrow \{t, ff\}), e^{\#} \in (S^{\#} \rightarrow S). \\
 & \quad \exists I \in (S^{\#} \times S^{\#} \rightarrow \{t, ff\}). \forall \underline{A}, \bar{A}, \bar{\Delta} \in S^{\#}, a \in A^{\#}. \\
 & \quad \begin{aligned}
 & \langle S, A, \Sigma \rangle = \nu \langle f_{A^{\#}}^{\#} \rangle (\langle S^{\#}, A^{\#}, \Sigma \langle S^{\#}, A^{\#}, E^{\#}, \varepsilon^{\#} \rangle \rangle) \\
 & \wedge \varepsilon^{\#}(\underline{A}) \Rightarrow I(\underline{A}, \underline{A}) \\
 & \wedge [\exists \Delta' \in S^{\#}. I(\underline{A}, \Delta') \wedge E_a^{\#}(\Delta', \bar{A})] \Rightarrow I(\underline{A}, \bar{A}) \\
 & \wedge I(\underline{A}, \bar{\Delta}) \Rightarrow \psi(f_{A^{\#}}^{\#}(\underline{A}), f_{A^{\#}}^{\#}(\bar{\Delta})) ] \\
 & \iff \\
 & [\forall p \in \Sigma, i \in |p|. \psi(p_0, p_i)]
 \end{aligned}
 \end{aligned}
 \tag{-I^{\#}}$$

#### 4.2.3.3 Equivalence forte des deux principes d'induction $(\uparrow I)$ et $(-I^{\#})$

##### Théorème 4.2.3.3~1

Toute preuve d'invariance de  $\psi$  pour une sémantique quelconque  $\langle S, A, \Sigma \rangle$  utilisant le principe d'induction  $(-I^{\#})$  peut se réécrire en une preuve utilisant  $(\uparrow I)$ , et réciproquement.

Démonstration

- Ayant trouvé  $s^\#, A^\#, t^\#, \varepsilon^\#, f_\Delta^\#$  et  $I^\#$  satisfaisant les conditions données en  $(-Y^\#)$ , nous pouvons toujours recréer cette preuve d'invariance en une preuve utilisant le principe d'induction  $(-\uparrow Y)$  en posant :

$$H = S^\# \times S^\#$$

$$F(\underline{\Delta}, \langle \underline{\Delta}^\#, \Delta^\# \rangle) = [\varepsilon^\#(\underline{\Delta}^\#) \wedge \Delta^\# = \underline{\Delta}^\# \wedge \underline{\Delta} = f_\Delta^\#(\Delta^\#)]$$

$$R(\Delta', \langle \underline{\Delta}^\#, \Delta^\# \rangle, a, \Delta) = [\Delta' = f_\Delta^\#(\Delta^\#) \wedge \exists \Delta^\# \in S^\#. (t_a^\#(\Delta^\#, \Delta^\#) \wedge \Delta = f_\Delta^\#(\Delta^\#))]$$

$$N(\Delta', \langle \underline{\Delta}^\#, \Delta^\# \rangle, a, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle) = [\Delta' = f_\Delta^\#(\Delta^\#) \wedge t_a^\#(\Delta^\#, \Delta^\#) \wedge \Delta = f_\Delta^\#(\Delta^\#)]$$

$$I(\underline{\Delta}, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle) = [I^\#(\underline{\Delta}^\#, \Delta^\#) \wedge \underline{\Delta} = f_\Delta^\#(\underline{\Delta}^\#) \wedge \Delta = f_\Delta^\#(\Delta^\#)]$$

En effet, nous avons bien :

.  $\text{Hist} \langle S, A, \Sigma \rangle (H, F, N, R)$  car si nous avons  $p \in \Sigma, m \in \omega, \forall i \in m. \langle \underline{\Delta}_i^\#, \Delta_i^\# \rangle \in H$  tels que  $1 < m < |p|$ ,  $F(p_0, \langle \underline{\Delta}_0^\#, \Delta_0^\# \rangle)$  et  $\forall j \in (m \vee 0). N(p_{j-1}, \langle \underline{\Delta}_{j-1}^\#, \Delta_{j-1}^\# \rangle, p_{j-1}, p_j, \langle \underline{\Delta}_j^\#, \Delta_j^\# \rangle)$  alors la séquence  $\Delta_0^\# \xrightarrow{p_0} \dots \xrightarrow{p_{m-1}} \Delta_{m-1}^\#$  est préfixe d'une trace  $p^\# \in \Sigma \langle S^\#, A^\#, t^\#, \varepsilon^\# \rangle$  telle que  $p = f_\Delta^\#(p^\#)$ . Par conséquent comme  $m < |p| = |p^\#|$ , nous avons  $p_{m-1} = f_\Delta^\#(p_{m-1}^\#) = f_\Delta^\#(\Delta_{m-1}^\#)$  et  $t_{p_{m-1}}^\#(p_{m-1}^\#, p_m^\#)$  donc  $t_{p_{m-1}}^\#(\Delta_{m-1}^\#, p_m^\#)$  et  $p_m = f_\Delta^\#(p_m^\#)$  soit  $R(p_{m-1}, \langle \underline{\Delta}_{m-1}^\#, \Delta_{m-1}^\# \rangle, p_{m-1}, p_m)$ .

. Si  $\varepsilon(\underline{\Delta})$  est vrai, alors  $\exists \underline{\Delta}^\# \in S^\#. (\varepsilon^\#(\underline{\Delta}^\#) \wedge \underline{\Delta} = f_\Delta^\#(\underline{\Delta}^\#))$  donc  $F(\underline{\Delta}, \langle \underline{\Delta}^\#, \Delta^\# \rangle)$  et  $I^\#(\underline{\Delta}^\#, \Delta^\#)$  donc  $F(\underline{\Delta}, \langle \underline{\Delta}^\#, \Delta^\# \rangle) \wedge I(\underline{\Delta}, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle)$ .

. Si  $I(\underline{\Delta}, \Delta', \langle \underline{\Delta}^\#, \Delta^\# \rangle) \wedge t_a(\Delta', \Delta) \wedge R(\Delta', \langle \underline{\Delta}^\#, \Delta^\# \rangle, a, \Delta)$  est vrai, alors  $\underline{\Delta} = f_\Delta^\#(\underline{\Delta}^\#) \wedge I^\#(\underline{\Delta}^\#, \Delta^\#) \wedge t_a(\Delta^\#, \Delta^\#) \wedge \Delta = f_\Delta^\#(\Delta^\#)$  impliquent  $N(\Delta', \langle \underline{\Delta}^\#, \Delta^\# \rangle, a, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle)$  et  $I^\#(\underline{\Delta}^\#, \Delta^\#)$  donc  $I(\underline{\Delta}, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle)$ .

. Si  $I(\underline{\Delta}, \Delta, \langle \underline{\Delta}^\#, \Delta^\# \rangle)$  est vrai alors nous avons  $I^\#(\underline{\Delta}^\#, \Delta^\#)$  qui implique  $\psi(f_\Delta^\#(\underline{\Delta}^\#), f_\Delta^\#(\Delta^\#)) = \psi(\underline{\Delta}, \Delta)$ .

- Réciproquement, ayant trouvé  $E, H, F, R, N$  et  $I$  satisfaisant les conditions de  $(-\uparrow Y)$ , nous pouvons toujours recréer cette preuve d'invariance en une preuve utilisant le principe d'induction  $(-Y^\#)$  comme suit :

Etant donnée  $p \in \Sigma$ , nous construisons  $p^\# \in \Sigma \times \langle S \times H, A \rangle$  comme suit:  
 Comme  $\varepsilon(p_0) \Rightarrow \exists h_0 \in H. F(p_0, h_0) \wedge I(p_0, p_0, h_0)$ , nous choisissons  $p_0^\# = \langle p_0, h_0 \rangle$ .  
 Nous avons aussi  $R(p_0, h_0, p_0, p_1)$  et  $I(p_0, p_0, h_0) \wedge t_{p_0}(p_0, p_1)$  qui  
 impliquent  $\exists h_1 \in H. N(p_0, h_0, p_0, p_1, h_1) \wedge I(p_0, p_1, h_1)$ . Nous posons alors  
 $p_1^\# = \langle p_1, h_1 \rangle$ . Ayant construit  $p_j^\#$  pour  $j = 0, \dots, m-2$  et  
 $p_j^\# = \langle p_j, h_j \rangle$  pour  $j = 0, \dots, m-1$  avec  $1 < m < |p|$  tel que  $F(p_0, h_0) \wedge$   
 $\forall j \in (m \cup 0). N(p_{j-1}, h_{j-1}, p_{j-1}, p_j, h_j) \wedge \forall j \in m. I(p_0, p_j, h_j)$ , nous avons  $t_{p_{m-1}}(p_{m-1}, p_m)$   
 et  $R(p_{m-1}, h_{m-1}, p_{m-1}, p_m)$  et donc  $\exists h_m \in H. N(p_{m-1}, h_{m-1}, p_{m-1}, p_m, h_m) \wedge$   
 $I(p_0, p_m, h_m)$ . Nous posons alors  $p_{m-1}^\# = p_{m-1}^\#$  et  $p_m^\# = \langle p_m, h_m \rangle$ .

Posons maintenant :

$$\Sigma^\# = \{p^\# : p \in \Sigma\}$$

$$S^\# = S \times \Sigma^\# \times \omega$$

$$A^\# = A$$

$$\varepsilon^\#(\langle \Delta, T, m \rangle) = [T = \{p^\# \in \Sigma^\# : p_0^\#(0) = \Delta\} \neq \emptyset \wedge m = 0]$$

$$t_a^\#(\langle \Delta', T', m' \rangle, \langle \Delta, T, m \rangle) = [m = m' + 1 \wedge T = \{p^\# \in T' : m \in |p^\#| \wedge p_{m'}^\#(0) = \Delta' \wedge p_m^\# = a \wedge p_m^\#(0) = \Delta\} \neq \emptyset]$$

$$I^\#(\langle \Delta, I, m \rangle, \langle \Delta, T, m \rangle) = [T = \{p^\# \in \Sigma^\# : m \in |p^\#| \wedge p_0^\#(0) = \Delta \wedge p_m^\#(0) = \Delta\} \neq \emptyset]$$

$$f_\Delta^\#(\langle \Delta, T, m \rangle) = \Delta$$

alors, nous avons bien :

$$\bullet \quad \langle S, A, \Sigma \rangle = \approx \langle f_\Delta^\# \rangle (\langle S^\#, A^\#, \Sigma \times \langle S^\#, A^\#, t^\#, \varepsilon^\# \rangle) \text{ par construction.}$$

$$\bullet \quad \varepsilon^\#(\langle \Delta, I, m \rangle) = [T = \{p^\# \in \Sigma^\# : p_0^\#(0) = \Delta\} \neq \emptyset \wedge m = 0] \Rightarrow I^\#(\langle \Delta, I, m \rangle, \langle \Delta, I, m \rangle).$$

$$\bullet \quad \exists \langle \Delta', T', m' \rangle \in S^\#. (I^\#(\langle \Delta, I, m \rangle, \langle \Delta', T', m' \rangle) \wedge t_a^\#(\langle \Delta', T', m' \rangle, \langle \Delta, T, m \rangle)) \Leftrightarrow$$

$$\exists \langle \Delta', T', m' \rangle \in S^\#. [T' = \{p^\# \in \Sigma^\# : m' \in |p^\#| \wedge p_0^\#(0) = \Delta \wedge p_{m'}^\#(0) = \Delta'\} \neq \emptyset \wedge m = m' + 1 \wedge$$

$$T = \{p^\# \in T' : m \in |p^\#| \wedge p_{m'}^\#(0) = \Delta' \wedge p_m^\# = a \wedge p_m^\#(0) = \Delta\} \neq \emptyset] \Rightarrow [T = \{p^\# \in \Sigma : m \in |p^\#| \wedge p_0^\#(0) = \Delta \wedge p_m^\#(0) = \Delta\} \neq \emptyset] = I^\#(\langle \Delta, I, m \rangle, \langle \Delta, T, m \rangle).$$

$$\bullet \quad I^\#(\langle \Delta, I, m \rangle, \langle \bar{\Delta}, \bar{T}, \bar{m} \rangle) = [T = \{p^\# \in \Sigma^\# : \bar{m} \in |p^\#| \wedge p_0^\#(0) = \Delta \wedge p_{\bar{m}}^\#(0) = \bar{\Delta}\} \neq \emptyset] \Rightarrow$$

$$\{p \in \Sigma : \bar{m} \in |p| \wedge p_0 = \Delta \wedge p_{\bar{m}} = \bar{\Delta}\} \neq \emptyset \Rightarrow \psi(\Delta, \bar{\Delta}) \Rightarrow \psi(f_\Delta^\#(\langle \Delta, I, m \rangle), f_{\bar{\Delta}}^\#(\langle \bar{\Delta}, \bar{T}, \bar{m} \rangle)).$$

□

### 4.3 CONSTRUCTION D'UNE METHODE DE PREUVE D'INVARIANCE A PARTIR D'UNE SEMANTIQUE OPERATIONNELLE ET D'UN PRINCIPE D'INDUCTION PAR DECOMPOSITION DE L'INVARIANT GLOBAL EN INVARIANTS LOCAUX

#### 4.3.1 FORMALISATION DE LA CONSTRUCTION

Nous montrons comment construire formellement une méthode de preuve pour un langage de programmation étant donné une sémantique opérationnelle, un principe d'induction, un langage d'assertions et sa sémantique.

##### 4.3.1.1 Définition de la sémantique opérationnelle

Pour construire formellement une méthode de preuve de propriétés d'invariance pour un langage de programmation  $\mathcal{P}_r$  il faut commencer par en définir la sémantique opérationnelle,

$$\mathcal{P}_r \in \mathcal{P}_r \longrightarrow \langle S[\mathcal{P}_r], A[\mathcal{P}_r], \Sigma[\mathcal{P}_r] \rangle$$

ce qui donne  $\varepsilon[\mathcal{P}_r]$  et  $t[\mathcal{P}_r]$  (cf. 2.3), ou bien

$$\mathcal{P}_r \in \mathcal{P}_r \longrightarrow \langle S[\mathcal{P}_r], A[\mathcal{P}_r], t[\mathcal{P}_r], \varepsilon[\mathcal{P}_r] \rangle$$

ce qui donne  $\Sigma[\mathcal{P}_r]$  (cf. 2.4).

#### 4.3.1.2 Définition de la propriété invariante à démontrer

Il faut ensuite définir la propriété d'invariance d'intérêt en définissant un ensemble  $\mathcal{P}$  de propriétés et une application

$$\langle P_r \in \mathcal{P}_r, P \in \mathcal{P} \rangle \longrightarrow \psi[P_r, P] \in (S[P_r] \times S[P_r] \rightarrow \{\text{tt}, \text{ff}\})$$

de sorte que la spécification " $P_r$  a la propriété  $P$ " signifie :

$$\forall p \in S[P_r], i \in |p|. \psi[P_r, P](p_0, p_i)$$

#### 4.3.1.3 Choix d'un principe d'induction

Il faut ensuite choisir un principe d'induction qui, de manière générale a la forme :

$$[\exists I \in A_S[S]. \text{Cv}[S, A, \Sigma, t, \varepsilon, \psi](I)]$$

En remplaçant  $S, A, \Sigma, t, \varepsilon$  et  $\psi$  par leurs définitions dans ce principe d'induction, nous obtenons les conditions de vérification d'une propriété  $P$  pour un programme  $P_r$  quelconque. Il s'agit de déterminer l'application :

$$\langle P_r, P \rangle \longrightarrow \langle \tilde{A}_S[P_r, P], \tilde{\text{Cv}}[P_r, P] \rangle$$

de sorte qu'une preuve ait la forme :

$$[\exists \tilde{I} \in \tilde{A}_S[P_r, P]. \tilde{\text{Cv}}[P_r, P](\tilde{I})]$$

et consiste donc à inventer un invariant  $\tilde{I}$  (qui se présente généralement sous forme d'une conjonction d'invariants locaux associés à certains points du programme) puis à démontrer qu'il satisfait certaines conditions de vérification  $\tilde{\text{Cv}}[P_r, P]$ .

On peut choisir  $\tilde{A}_S[P_r, P] = A_S[S[P_r]]$  et  $\tilde{\text{Cv}}[P_r, P] = \text{Cv}[S[P_r], A[P_r], \Sigma[P_r], t[P_r], \varepsilon[P_r], \psi[P_r]]$  mais ce choix ne permet pas de faire toutes les simplifications désirables.

C'est pourquoi nous avons proposé une autre démarche (Cousot-Cousot [80c], Cousot-R [81], Cousot-Cousot [82a], Cousot-Cousot [84]) qui se justifie comme suit :

La méthode de preuve doit être correcte :

$$[\exists \tilde{I} \in \tilde{A}_S[P_r, P]. C\tilde{v}[P_r, P](\tilde{I})] \Rightarrow [\forall p \in \Sigma[P_r], i \in |p|. \psi[P_r, P](p_i, p_i)]$$

et comme le principe d'induction choisi est correct et (sémantiquement) complet, ceci équivaut à :

$$[\exists \tilde{I} \in \tilde{A}_S[P_r, P]. C\tilde{v}[P_r, P](\tilde{I})] \Rightarrow [\exists I \in A_S[S]. C_v[S, A, \Sigma, t, \varepsilon, \psi](I)]$$

ou encore :

$$\forall \tilde{I} \in \tilde{A}_S[P_r, P].$$

$$C\tilde{v}[P_r, P](\tilde{I}) \Rightarrow \exists I \in A_S[S]. C_v[S, A, \Sigma, t, \varepsilon, \psi](I)$$

d'où nous déduisons qu'il doit exister une fonction  $\gamma \in (A_S[P_r, P] \rightarrow A_S[S[P_r]])$  telle que :

$$C\tilde{v}[P_r, P] \Rightarrow C_v[S, A, \Sigma, t, \varepsilon, \psi] \circ \gamma$$

De façon similaire, la méthode de preuve doit être (sémantiquement) complète, d'où nous déduisons qu'il doit exister une fonction  $\alpha \in (A_S[S[P_r]] \rightarrow \tilde{A}_S[P_r, P])$  telle que :

$$C_v[S, A, \Sigma, t, \varepsilon, \psi] \Rightarrow C\tilde{v}[P_r, P] \circ \alpha$$

La fonction  $\gamma[P_r, P]$  donne la signification  $\gamma[P_r, P](\tilde{I})$  des invariants "locaux"  $\tilde{I} \in \tilde{A}_S[P_r, P]$  en terme des invariants "globaux" de  $A_S[S[P_r]]$ , tandis que la fonction  $\alpha[P_r, P]$  donne la représentation  $\alpha[P_r, P](I)$  de l'invariant "global"  $I \in A_S[S[P_r]]$  par des invariants "locaux" de  $\tilde{A}_S[P_r, P]$ . Ces éléments conduisent à poursuivre la construction de la méthode de preuve comme suit :

#### 4.3.1.4 Choix d'un langage pour exprimer les invariants locaux

En pratique l'invariant utilisé pour faire la preuve n'est pas un élément de  $As[S][Pr]$  (ce qui donnerait un invariant global pour tout le programme) mais il s'exprime généralement de manière équivalente mais plus aisée comme élément d'un certain ensemble  $As[Pr, P]$  (que nous appellerons conventionnellement "langage"), le plus souvent sous forme d'invariants locaux associés à divers points du programme.

##### Exemple 4.3.1.4-1

Considérons l'exemple 4.2.1.1-1, nous avons pour le programme  $P_2$  :

```

1:
   $Q := 0;$ 
2:
  while  $x \geq y$  do
3:
     $Q := Q + 1;$ 
4:
     $X := X - Y;$ 
5:
  od;
6:

```

$C = \{1, \dots, 6\}$  états de contrôle

$V = \{X, Y, Q\}$  variables

$\mathcal{M} = (V \rightarrow \mathbb{Z})$  états mémoire

$S = C \times \mathcal{M}$  états

Un invariant  $I \in As[S] = (S \times S \rightarrow \{\text{tt}, \text{ff}\})$  est exprimé comme un vecteur  $\tilde{I} = \langle P_1, \dots, P_6 \rangle$  où  $P_i \in (\mathbb{Z}^5 \rightarrow \{\text{tt}, \text{ff}\})$ ,  $i = 1, \dots, 6$ . Par conséquent  $As[Pr, \varphi] = \prod_{c \in C} (\mathbb{Z}^5 \rightarrow \{\text{tt}, \text{ff}\})$

□

#### 4.3.1.5 Définition de la sémantique du langage exprimant les invariants locaux

La sémantique du langage  $A\delta[P_c, P]$  exprimant les invariants locaux se définit en terme de  $A\delta[S[P_c]]$  au moyen de deux fonctions :

$$\alpha[P_c, P] \in (A\delta[S[P_c]] \rightarrow A\delta[P_c, P])$$

$$\delta[P_c, P] \in (A\delta[P_c, P] \rightarrow A\delta[S[P_c]])$$

##### Exemple 4.3.1.5-1

La signification de  $\tilde{I} = \langle P_1, \dots, P_6 \rangle$  (cf. exemple 4.2.1.1-1) est  $I = \delta(\tilde{I})$  tel que  $I(\underline{s}, \underline{s}) = [\exists i \in C, \underline{x}, \underline{y}, \underline{q}, x, y, q \in \mathbb{Z}. \underline{s} = \langle 1, \langle \underline{x}, \underline{y}, \underline{q} \rangle \rangle \wedge \underline{s} = \langle i, \langle x, y, q \rangle \rangle \wedge P_i(\underline{x}, \underline{y}, x, y, q)]$ . Ceci formalise le fait que  $P_i(\underline{x}, \underline{y}, x, y, q)$  est vrai entre les valeurs initiales  $\underline{x}, \underline{y}$  et les valeurs courantes  $x, y, q$  des variables  $x, y, q$  du programme quand le contrôle est au point  $i$ .

Réciproquement, un invariant  $I \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$  peut être représenté par  $\alpha(I) = \langle P_1, \dots, P_6 \rangle$  tel que pour tout  $i = 1, \dots, 6$ ,  $P_i(\underline{x}, \underline{y}, x, y, q) = [\exists q \in \mathbb{Z}. I(\langle 1, \langle \underline{x}, \underline{y}, q \rangle \rangle, \langle i, \langle x, y, q \rangle \rangle)]$ .

□

#### 4.3.1.6 Propriétés du langage exprimant les invariants locaux et sa sémantique

##### 4.3.1.6.1 Treillis complets des invariants locaux

Dans les principes d'induction que nous avons considérés au paragraphe 4.2,  $A_S[S]$  était de la forme  $(s \rightarrow \{tt, ff\})$ ,  $(s \wedge s \rightarrow \{tt, ff\})$ , etc., etc., donc toujours un treillis complet booléen  $\langle A_S[S], \Rightarrow, \vee, \wedge, tt, ff, \neg \rangle$ .

Quand on choisit le langage  $A_S[P_r, P]$  pour exprimer les invariants locaux, il faut pouvoir exprimer qu'un invariant est plus fort ou plus faible qu'un autre de façon à pouvoir traduire dans  $C_r$  l'implication qui figure dans  $C_r$ . Il est donc nécessaire de disposer d'un ordre partiel  $\varepsilon$  sur  $A_S$  correspondant à  $\Rightarrow$  sur  $A_S$ . Ceci nous conduira à choisir  $\langle A_S[P_r, P], \varepsilon \rangle$  comme un ensemble partiellement ordonné.

Il arrive souvent que  $\langle A_S[P_r, P], \varepsilon \rangle$  soit un treillis complet. En effet, la propriété que la conjonction d'invariants est invariante doit également être conservée pour  $A_S[P_r, P]$  car elle permet de combiner des preuves indépendantes en une seule sans aucune vérification supplémentaire. Ceci signifie que  $A_S[P_r, P]$  doit être muni d'une opération borne inférieure  $\prod_{i \in \Delta} \tilde{I}_i$  de  $\{\tilde{I}_i : i \in \Delta\}$  pour  $\varepsilon$ . Enfin l'invariant  $tt$  de  $A_S[S[P_r]]$  exprime qu'on ne sait rien sur le programme  $P_r$ . Nous pouvons toujours ajouter l'équivalent à  $A_S[P_r, P]$  sous la forme d'un supremum  $\top$  tel que  $\forall \tilde{I} \in A_S[P_r, P]. \tilde{I} \varepsilon \top$ . Avec ces hypothèses  $\langle A_S[P_r, P], \varepsilon, \prod, \top \rangle$  est un inf-demi-treillis complet avec supremum et par conséquent c'est un treillis complet  $\langle A_S[P_r, P], \varepsilon, \prod, \sqcup, \top, \perp \rangle$ .

Quand nous utiliserons les principes d'induction contrapositifs qui utilisent la négation, nous serons amenés à choisir  $\langle A_S[P_r, P], \varepsilon, \sqcup, \prod, \top, \neg \rangle$  comme étant un treillis complet booléen.

Exemple 4.3.1.6.1-1

Dans l'exemple 4.3.1.4-1, nous avons :

$$S = C \times (V \rightarrow \mathbb{Z}) \quad \text{où } C = \{1, \dots, 6\} \quad \text{et } V = \{x, y, \varphi\}$$

$A_S[S] = (S \times S \rightarrow \{\text{tt}, \text{ff}\})$  qui est un treillis complet booléen pour  $\Rightarrow, \vee, \wedge, \text{tt}, \text{ff}, \neg$ .

$A_S^{\sim}[P, \psi] = \prod_{c \in C} (\mathbb{Z}^S \rightarrow \{\text{tt}, \text{ff}\})$  qui est un treillis complet booléen pour

$\exists, \cup, \cap, \tau, \perp, \neg$  définis par :

$$\langle P_1, \dots, P_6 \rangle \in \langle Q_1, \dots, Q_6 \rangle \quad \text{si et seulement si} \quad \bigwedge_{i=1}^6 P_i \Rightarrow Q_i$$

$$\bigcup_{i \in \Delta} \langle P_1^i, \dots, P_6^i \rangle = \langle \bigvee_{i \in \Delta} P_1^i, \dots, \bigvee_{i \in \Delta} P_6^i \rangle \quad \text{et de même pour } \cap \text{ et } \wedge$$

$$\perp = \langle \text{ff}, \dots, \text{ff} \rangle \quad \text{et de même pour } \tau \text{ et } \text{tt}$$

$$\neg \langle P_1, \dots, P_6 \rangle = \langle \neg P_1, \dots, \neg P_6 \rangle$$

□

## 4.3.1.6.2 Correspondance entre invariants locaux et globaux

La paire  $(\alpha, \gamma)$  de fonctions  $\alpha \in (A_S \rightarrow A_S^{\sim})$  et  $\gamma \in (A_S^{\sim} \rightarrow A_S)$  entre invariants globaux  $A_S$  et invariants locaux  $A_S^{\sim}$  a souvent des propriétés intéressantes qui peuvent être exploitées pour construire la méthode de preuve. Ces propriétés sont les suivantes :

## 4.3.1.6.2.1 Correspondance monotone

Le fait que l'ordre  $\in$  sur  $A_S^{\sim}$  corresponde à l'implication  $\Rightarrow$  sur  $A_S$  s'exprime par la monotonie de  $\alpha$  et de  $\gamma$  :

$$(a) \quad \forall I_1, I_2 \in A_S. [(I_1 \Rightarrow I_2) \Rightarrow (\alpha(I_1) \in \alpha(I_2))]$$

$$(b) \quad \forall \tilde{I}_1, \tilde{I}_2 \in A_S^{\sim}. [(\tilde{I}_1 \in \tilde{I}_2) \Rightarrow (\gamma(\tilde{I}_1) \Rightarrow \gamma(\tilde{I}_2))]$$

## 4.3.1.6.2.2 Demi-correspondance de Galois

Supposant  $\alpha$  et  $\gamma$  monotones (et donc que  $\in$  correspond à  $\Rightarrow$ ), il se peut qu'on ait  $\neg(I \Rightarrow \gamma(\tilde{I}))$  et  $\alpha(I) \in \tilde{I}$ . Supposons par exemple que  $I$  soit le plus fort invariant pour une sémantique  $\langle S, A, \Sigma \rangle$  (c'est-à-dire que  $I(\Delta, \Delta) = [\exists p \in \Sigma, i \in |p|, p_0 = \Delta \wedge p_i = \Delta]$ ). Alors (le sens de)  $\tilde{I} \in \tilde{A\Delta}$  est invariant si et seulement si  $I \Rightarrow \gamma(\tilde{I})$ . Nous voudrions pouvoir transposer ce raisonnement dans  $\tilde{A\Delta}$  c'est-à-dire que  $\alpha(I) \in \tilde{I}$ . Ceci nous amène à supposer que  $\alpha$  et  $\gamma$  sont monotones et que

$$\forall I \in A\Delta, \tilde{I} \in \tilde{A\Delta}. (\alpha(I) \in \tilde{I}) \Rightarrow (I \Rightarrow \gamma(\tilde{I}))$$

ou encore, de manière équivalente

- (a)  $\alpha \in (A\Delta \rightarrow \tilde{A\Delta})$  est monotone
- (b)  $\gamma \in (\tilde{A\Delta} \rightarrow A\Delta)$  est monotone
- (c)  $\gamma \circ \alpha$  est extensive

## 4.3.1.6.2.3 Quasi-correspondance de Galois

Quand  $(\alpha, \gamma)$  est une demi-correspondance de Galois, il se peut que  $\gamma \circ \alpha(I) \not\Rightarrow J$  même s'il existe  $\tilde{I}$  tel que  $I \Rightarrow \gamma(\tilde{I}) \Rightarrow J$ . Autrement dit la représentation  $\alpha(I)$  de  $I$  dans  $\tilde{A\Delta}$  donne moins d'informations que si on avait choisi  $\alpha(I) = \tilde{I}$ . Pour que  $\alpha(I)$  soit la meilleure représentation possible de  $I \in A\Delta$  dans  $\tilde{A\Delta}$ , il faut supposer que

$$\gamma \circ \alpha(I) \Rightarrow \bigwedge \{ \gamma(\tilde{I}) : I \Rightarrow \gamma(\tilde{I}) \}$$

pour éviter que  $\gamma \circ \alpha(I) \not\Rightarrow J$  lorsque  $I \Rightarrow \gamma(\tilde{I}) = J$ . Comme  $\gamma \circ \alpha$  est extensive cette condition revient à supposer que  $\gamma \circ \alpha(I) = \bigwedge \{ \gamma(\tilde{I}) : I \Rightarrow \gamma(\tilde{I}) \}$ , soit

- (a)  $\alpha \in (A\Delta \rightarrow \tilde{A\Delta})$  est monotone
- (b)  $\gamma \in (\tilde{A\Delta} \rightarrow A\Delta)$  est monotone
- (c)  $\gamma \circ \alpha$  est extensive
- (d)  $\forall I \in A\Delta. [\gamma \circ \alpha(I) = \bigwedge \{ \gamma(\tilde{I}) : I \Rightarrow \gamma(\tilde{I}) \}]$

Observons que les propriétés (c) et (d) sont indépendantes et que  $\gamma \circ \alpha$  est l'unique opérateur de fermeture supérieure  $e \in (A_s \rightarrow A_s)$  tel que  $e[A_s] = \gamma[A_s^*]$ . Autrement dit, à la représentation près, seuls les éléments de  $A_s$  qui appartiennent à  $e[A_s]$  sont disponibles quand on raisonne dans  $A_s^*$  au lieu de  $A_s$ . Tout invariant  $I$  devra être approché par un invariant plus faible, le meilleur étant  $e(I)$ . En général  $e(I) \neq I$  et cette perte d'information peut évidemment conduire à des problèmes d'incomplétude mais ce n'est pas toujours le cas.

#### 4.3.1.6.2.4 Correspondance de Galois

Bien que  $(\alpha, \gamma)$  soit une quasi-correspondance de Galois,  $\alpha$  peut ne pas être un morphisme complet pour les disjonctions. Comme le plus fort invariant pour les principes d'induction (positifs) s'exprime comme une disjonction (en général infinie) ceci peut avoir pour conséquence que la représentation du plus fort invariant par  $\alpha$  dans  $A_s^*$  conduise à une perte d'information qui est source d'incomplétude. La propriété que  $\alpha$  est un morphisme complet pour la disjonction et donc de manière équivalente que la paire  $(\alpha, \gamma)$  est une correspondance de Galois peut donc être utile :

- (a)  $\alpha \in (A_s \rightarrow A_s^*)$  est monotone
- (b)  $\gamma \in (A_s^* \rightarrow A_s)$  est monotone
- (c)  $\gamma \circ \alpha$  est extensive
- (e)  $\alpha \circ \gamma$  est reductive

(on trouvera des définitions équivalentes en annexe II).

## 4.3.1.6.2.5 Correspondance de Galois surjective

Les éléments de  $\tilde{A}_S \vee \alpha[A_S]$  ne servent à représenter aucune information de  $A_S$  et peuvent être éliminés en choisissant  $\tilde{A}_S' = \alpha[A_S]$ . Dans ces conditions, nous avons les propriétés :

- (a)  $\alpha \in (A_S \rightarrow \tilde{A}_S)$  est monotone
- (b)  $\gamma \in (\tilde{A}_S \rightarrow A_S)$  est monotone
- (c)  $\gamma \circ \alpha$  est extensive
- (e)  $\alpha \circ \gamma$  est réductive
- (f)  $\alpha$  est surjective

que nous avons fréquemment utilisées (cf. par exemple à Cousot-Cousot [76], Cousot-Cousot [77a]).

## 4.3.1.6.2.6 Correspondance de Galois injective

Si  $I_1, I_2 \in A_S$  sont différents et ont la même représentation  $\alpha(I_1) = \alpha(I_2)$  dans  $\tilde{A}_S$ , il y a une perte d'information en passant de  $A_S$  à  $\tilde{A}_S$  par  $\alpha$  puisque des invariants différents ne peuvent plus être distingués. Ceci peut conduire à des problèmes d'incomplétude sémantique puisque la propriété  $I_1 \neq I_2$  ne peut pas s'exprimer dans  $\tilde{A}_S$ . Ceci nous amène aux conditions suivantes :

- (a)  $\alpha \in (A_S \rightarrow \tilde{A}_S)$  est monotone
- (b)  $\gamma \in (\tilde{A}_S \rightarrow A_S)$  est monotone
- (c)  $\gamma \circ \alpha$  est extensive
- (e)  $\alpha \circ \gamma$  est réductive
- (g)  $\alpha$  est injective

## 4.3.1.6.2.7 Isomorphisme complet

Enfin, dans le cas où les raisonnements dans  $A_\Delta$  ou  $\tilde{A}_\Delta$  sont équivalents,  $\alpha$  est un isomorphisme complet et  $\gamma$  est son inverse de sorte que les hypothèses suivantes sont satisfaites :

$$(k) \quad \alpha \left( \bigvee_{i \in \Delta} I_i \right) = \bigcup_{i \in \Delta} \alpha(I_i)$$

$$(l) \quad \alpha \left( \bigwedge_{i \in \Delta} I_i \right) = \bigcap_{i \in \Delta} \alpha(I_i)$$

$$(j) \quad \alpha \text{ est bijective}$$

$$(k) \quad \gamma = \alpha^{-1}$$

Exemple 4.3.1.6.2.7-1

Dans l'exemple 4.3.1.4-1 poursuivi en 4.3.1.5-1 et 4.3.1.6.1-1, nous avons :

$$\alpha_1(I)_i(x, y, x, y, q) = [\exists q \in \mathbb{Z}. I(\langle 1, \langle x, y, q \rangle \rangle, \langle i, \langle x, y, q \rangle \rangle)]$$

$$\gamma_1(\tilde{I})(\underline{a}, \underline{a}) = [\exists i \in C, x, y, q, x, y, q \in \mathbb{Z}. \underline{a} = \langle 1, \langle x, y, q \rangle \rangle \wedge \underline{a} = \langle i, \langle x, y, q \rangle \rangle \wedge \tilde{I}_i(x, y, x, y, q)]$$

de sorte que les conditions (a) et (b) sont satisfaites mais pas la condition (c). Nous pouvons également choisir :

$$\alpha_2(I)_i(x, y, x, y, q) = [\exists j \in C, q \in \mathbb{Z}. I(\langle j, \langle x, y, q \rangle \rangle, \langle i, \langle x, y, q \rangle \rangle)]$$

$$\gamma_2(\tilde{I})(\underline{a}, \underline{a}) = [\exists j, i \in C, x, y, q, x, y, q \in \mathbb{Z}. \underline{a} = \langle j, \langle x, y, q \rangle \rangle \wedge \underline{a} = \langle i, \langle x, y, q \rangle \rangle \wedge \tilde{I}_i(x, y, x, y, q)]$$

qui satisfont les conditions (a), (b), (c), (d), (e), (f), (h) mais pas (g), (i), (j) et (k). (Avec ce deuxième choix et contrairement au premier, il n'est pas supposé que  $I(\underline{a}, \underline{a}) \Rightarrow \varepsilon(\underline{a})$ ).

□

### 4.3.1.7 Dérivation de conditions de vérification correctes

Etant donné un principe d'induction correct et sémantiquement complet

$$[\exists I \in A_S. C_V(I)]$$

et une correspondance  $\gamma \in (A_{\tilde{S}} \rightarrow A_S)$  entre  $A_{\tilde{S}}$  et  $A_S$ , toute preuve

$$[\exists \tilde{I} \in A_{\tilde{S}}. \tilde{C}_V(\tilde{I})]$$

telle que

$$\tilde{C}_V \Rightarrow C_V \circ \gamma$$

est correcte. On peut donc choisir  $\tilde{C}_V[P_r, P]$  comme étant  $C_V[S[P_r], A[P_r], \Sigma[P_r], t[P_r], e[P_r], \psi[P_r, P]] \circ \gamma[P_r, P]$ . Par diverses manipulations algébriques on cherchera à exprimer cette condition sous forme d'une conjonction de conditions plus simples correspondant chacune à une commande élémentaire du programme  $P_r$ . Des simplifications sont possibles puisque c'est une implication et non pas une égalité qui est requise. La méthode de preuve obtenue de cette manière est correcte par construction. Puisque le résultat soit valable non pas pour un programme  $P_r$  particulier mais pour le langage  $P_r$  considéré il faut procéder par induction sur la syntaxe du langage.

#### Exemple 4.3.1.7-1

Dans le cas des principes d'induction pour l'invariance, la condition de vérification

$$[\exists I \in A_S. C_V(I)]$$

peut s'écrire sous forme d'une conjonction :

$$[\exists I \in A_S. C_{V_e}(I) \wedge C_{V_t}(I) \wedge C_{V_s}(I)]$$

Par exemple pour le principe d'induction (-3), nous avons :

$$Cv_E(I) = [\forall \underline{A} \in S. E(\underline{A}) \Rightarrow I(\underline{A}, \underline{A})]$$

$$Cv_i(I) = [\forall \underline{A}, \underline{A}' \in S. [\exists \underline{A}' \in S, a \in A. E(\underline{A}) \wedge I(\underline{A}, \underline{A}') \wedge t_a(\underline{A}', \underline{A})] \Rightarrow I(\underline{A}, \underline{A})]$$

$$Cv_\delta(I) = [\forall \underline{A}, \bar{\underline{A}} \in S. [E(\underline{A}) \wedge I(\underline{A}, \bar{\underline{A}})] \Rightarrow \psi(\underline{A}, \bar{\underline{A}})]$$

Dans ce cas, on choisira le plus souvent

$$\tilde{Cv}(\tilde{I}) = \tilde{Cv}_E(\tilde{I}) \wedge \tilde{Cv}_i(\tilde{I}) \wedge \tilde{Cv}_\delta(\tilde{I})$$

satisfaisant :

$$\tilde{Cv}_E(\tilde{I}) \Rightarrow Cv_E(\gamma(\tilde{I}))$$

$$\tilde{Cv}_i(\tilde{I}) \Rightarrow Cv_i(\gamma(\tilde{I}))$$

$$\tilde{Cv}_\delta(\tilde{I}) \Rightarrow Cv_\delta(\gamma(\tilde{I}))$$

De plus dans le cas des principes d'induction pour l'invariance le terme  $Cv_i(I)$  est de la forme  $F(I) \Rightarrow I$  (ou dualement  $I \Rightarrow F(I)$ ).

Par exemple, dans le cas du principe d'induction (-3), nous avons

$$Cv_i(I) = [F(I) \Rightarrow I]$$

où

$$F(I)(\underline{A}, \underline{A}') = [\exists \underline{A}' \in S, a \in A. E(\underline{A}) \wedge I(\underline{A}, \underline{A}') \wedge t_a(\underline{A}', \underline{A})]$$

ou bien, nous pouvons intégrer le terme  $Cv_E(I)$  dans  $Cv_i(I)$  ce qui donne

$$F(I)(\underline{A}, \underline{A}') = [\exists \underline{A}' \in S, a \in A. E(\underline{A}) \wedge ([\underline{A} = \underline{A}'] \vee [I(\underline{A}, \underline{A}') \wedge t_a(\underline{A}', \underline{A})])]$$

Dans les deux cas la condition  $\tilde{Cv}_i(\tilde{I}) \Rightarrow Cv_i(\gamma(\tilde{I}))$  s'écrit :

$$\tilde{Cv}_i(\tilde{I}) \Rightarrow [F(\gamma(\tilde{I})) \Rightarrow \gamma(\tilde{I})]$$

quand  $(\alpha, \gamma)$  est une demi-correspondance de Galois (cf. 4.3.1.6.2.2), on est amené à choisir

$$\tilde{Cv}_i(\tilde{I}) \Rightarrow [\alpha \circ F \circ \gamma(\tilde{I}) \subseteq \tilde{I}]$$

car  $\alpha \circ F \circ \gamma(\tilde{I}) \subseteq \tilde{I}$  implique que  $F \circ \gamma(\tilde{I}) \Rightarrow \gamma(\tilde{I})$ . Posons  $\tilde{F} = \alpha \circ F \circ \gamma$ . A la suite de Cousot-Cousot [80c], observons que si  $(\alpha, \gamma)$  est une correspondance de Galois injective (cf. 4.3.1.6.2.7) alors  $\alpha \circ F = \tilde{F} \circ \alpha$ , (en effet,  $\alpha$  étant injective nous avons  $\gamma \circ \alpha = 1_{\alpha}$  et donc  $\alpha \circ F = \alpha \circ F \circ \gamma \circ \alpha = \tilde{F} \circ \alpha$ ).

Supposons maintenant que ( $\alpha$  n'étant pas injective), il existe un opérateur  $\tilde{F}$  sur  $\tilde{A}_0$  tel que  $\alpha \circ F = \tilde{F} \circ \alpha$ . Nous avons alors

[ $F \in \tilde{F}$ , l'inégalité peut être stricte et il y a égalité si (mais pas seulement si)  $\alpha$  est surjective]

(En effet  $\tilde{F} = \alpha \circ F \circ \gamma = \tilde{F} \circ \alpha \circ \gamma \in \tilde{F}$  car  $\alpha \circ \gamma$  est réductive. L'inégalité peut être stricte comme le montre l'exemple suivant :  $A_0 = \tilde{A}_0 = \{a, b\}$ ,  $a \neq b$ ,  $\alpha(a) = \alpha(b) = a$ ,  $\gamma(a) = \gamma(b) = b$ ,  $F(x) = x$ ,  $\tilde{F}(x) = x$ ,  $F(a) = F(b) = a$ . Si  $\alpha$  est surjective alors  $\alpha \circ \gamma$  est l'identité et donc  $\tilde{F} = F$  mais la réciproque n'est pas vraie comme le montre l'exemple suivant :  $A_0 = \tilde{A}_0 = \{a, b\}$ ,  $a \neq b$ ,  $F(x) = \tilde{F}(x) = \alpha(x) = a$ ,  $\gamma(x) = b$ .)

Dans ces conditions, on peut choisir

$$Co_{\gamma}(\tilde{I}) = [\tilde{F}(\tilde{I}) \in \tilde{I}]$$

où  $\tilde{F}$  est un opérateur sur  $\tilde{A}_0$  tel que  $F \in \tilde{F} \in \tilde{F}$  (puisque  $\tilde{F}(\tilde{I}) \in \tilde{I}$  implique  $\tilde{F}(\tilde{I}) \in \tilde{I}$  donc  $\alpha \circ F \circ \gamma(\tilde{I}) \in \tilde{I}$  soit  $F \circ \gamma(\tilde{I}) \Rightarrow \gamma(\tilde{I})$  et donc  $Co_{\gamma}(\gamma(\tilde{I}))$ ). Si  $\alpha$  n'est pas surjective le treillis complet des  $\tilde{F} \in (A_0 \rightarrow A_0)$  tels que  $F \in \tilde{F} \in \tilde{F}$  n'est pas réduit à un seul élément de sorte qu'il est possible d'envisager diverses conditions de vérifications.

Parmi ces conditions de vérification,  $\tilde{F}$  peut nécessiter de trouver un invariant  $\tilde{I}$  plus fort que pour  $F$  puisque  $\tilde{F}(\tilde{I}) \in \tilde{I}$  implique  $\tilde{F}(\tilde{I}) \in \tilde{I}$  mais la réciproque n'est pas vraie. Cet argument en faveur du choix de  $\tilde{F}$  doit être modulé par le fait qu'en pratique  $\tilde{F}$  conduit à des conditions de vérification plus simples. En pratique, un équilibre doit être trouvé entre des invariants plus faibles et des conditions de vérification plus compliquées ou des invariants plus forts et des conditions de vérification plus simples.

□

#### 4.3.1.8 Vérification de la complétude sémantique

La vérification de la complétude sémantique consiste à montrer que :

$$[\exists I \in A_S. Cr(I)] \Rightarrow [\exists \tilde{I} \in A_{\tilde{S}}. C\tilde{v}(\tilde{I})]$$

Si nous choisissons  $\tilde{I}$  comme étant la représentation de  $I$  par  $\alpha \in (A_S \rightarrow A_{\tilde{S}})$ , nous obtenons la condition suffisante de complétude :

$$Cr \Rightarrow C\tilde{v} \circ \alpha$$

(celle-ci étant d'ailleurs nécessaire pour un  $\alpha$  convenablement choisi).

##### Exemple 4.3.1.8-1

Dans le cas particulier que nous rencontrerons assez souvent où  $\gamma \circ \alpha = \underline{1}$  (cf. 4.3.1.6.2.6, 4.3.1.6.2.7) et  $C\tilde{v} = Cr \circ \gamma$  auquel cas la condition suffisante de correction  $C\tilde{v} \Rightarrow Cr \circ \gamma$  est trivialement satisfaite, nous avons  $C\tilde{v} \circ \alpha = Cr \circ \gamma \circ \alpha = Cr$  et la condition suffisante de complétude  $Cr \Rightarrow C\tilde{v} \circ \alpha$  est également trivialement satisfaite.

□

##### Exemple 4.3.1.8-2

Pour poursuivre l'exemple 4.3.1.7-1, dans le cas où

$$Cr(I) = [F(I) \Rightarrow I \wedge I \Rightarrow \psi]$$

$$\wedge C\tilde{v}(\tilde{I}) = [\bar{F}(\tilde{I}) \in \tilde{I} \wedge \tilde{I} \in \alpha(\psi)]$$

où  $\alpha$  est monotone,  $\bar{F} \in \tilde{F}$  et  $\alpha \circ F = \tilde{F} \circ \alpha$ , il faut vérifier que :

$$[F(I) \Rightarrow I \wedge I \Rightarrow \psi] \Rightarrow [\bar{F}(\alpha(I)) \in \alpha(I) \wedge \alpha(I) \in \alpha(\psi)]$$

Nous avons bien  $(I \Rightarrow \psi)$  qui implique  $\alpha(I) \in \alpha(\psi)$  par monotonie de sorte qu'il suffit de vérifier que

$$[F(I) \Rightarrow I] \Rightarrow [\bar{F}(\alpha(I)) \in \alpha(I)]$$

ce qui est évident car  $[F(I) \Rightarrow I] \Rightarrow [\alpha \circ F(I) \in \alpha(I)]$  (par monotonie)  $\Rightarrow [\tilde{F}(\alpha(I)) \in \alpha(I)]$

(car  $\alpha \circ F = \tilde{F} \circ \alpha$ )  $\Rightarrow [\bar{F}(\alpha(I)) \in \alpha(I)]$  (car  $\bar{F} \in \tilde{F}$ ).

□

### 4.3.2 EXEMPLES DE CONSTRUCTIONS

#### 4.3.2.1 Construction d'une méthode de preuve de non-terminaison, d'absence d'erreurs à l'exécution et d'invariance globale de programmes séquentiels par l'absurde

##### 4.3.2.1.1 La non-terminaison est une propriété d'invariance

Soit  $P_s \in \mathcal{P}_s$  un programme séquentiel et  $\phi \in ((\mathcal{V} \rightarrow \mathcal{D}) \rightarrow \{\text{tt}, \text{ff}\})$  une caractérisation des valeurs possibles des variables à l'entrée du programme. Le programme  $P_s$  ne se termine pas normalement si aucun état de sortie ne peut être atteint durant l'exécution :

$$\forall p \in \Sigma \langle S[P_s], A[P_s], t[P_s], \varepsilon \rangle, i \in |p|. [\sigma(p_i) \Rightarrow \psi(p_i)]$$

où

$$\varepsilon \langle L, M \rangle = [(P_s \equiv L : p) \wedge \phi(M)]$$

$$\sigma \langle L, M \rangle = \text{tt}$$

$$\psi \langle L, M \rangle = [\neg (P_s \equiv p L :)]$$

##### 4.3.2.1.2 Choix d'un principe d'induction

Puisque  $\psi$  est une assertion sur les états finaux, d'après le paragraphe 4.2.1.3, nous pouvons utiliser les principes d'induction  $(-i)$ ,  $(-\tilde{i})$ ,  $(-\bar{i})$ ,  $(-\ddot{i})$ . Nous choisissons  $(-\tilde{i})$  comme exemple car ce principe d'induction n'est pas du tout conventionnel.  $(-\tilde{i})$  est de la forme :

$$[\exists T \in \mathcal{A}_s[P_s]. \text{Cv}[P_s](\varepsilon, \sigma)(\psi)(T)]$$

où

$$A_{\Delta} \llbracket P_s \rrbracket = (S \llbracket P_s \rrbracket \rightarrow \{\#, \#f\})$$

$$Cv_{\sigma} \llbracket P_s \rrbracket(\varepsilon, \sigma)(\psi)(I) = Cv_{\sigma} \llbracket P_s \rrbracket(\psi, \sigma)(I) \wedge Cv_{\mu} \llbracket P_s \rrbracket(I) \wedge Cv_{\varepsilon} \llbracket P_s \rrbracket(\varepsilon)(I)$$

$$Cv_{\sigma} \llbracket P_s \rrbracket(\psi, \sigma)(I) = [\forall \bar{a} \in S \llbracket P_s \rrbracket. [\neg \psi(\bar{a}) \wedge \sigma(\bar{a})] \Rightarrow I(\bar{a})]$$

$$Cv_{\mu} \llbracket P_s \rrbracket(I) = [\forall \Delta \in S \llbracket P_s \rrbracket, a \in A \llbracket P_s \rrbracket. I(\Delta) \Rightarrow \neg [\exists \Delta' \in S \llbracket P_s \rrbracket. \neg I(\Delta') \wedge \vdash \llbracket P_s \rrbracket_a(A', \Delta)]]$$

$$Cv_{\varepsilon} \llbracket P_s \rrbracket(\varepsilon)(I) = [\forall \underline{\Delta} \in S \llbracket P_s \rrbracket. \varepsilon(\underline{\Delta}) \Rightarrow \neg I(\underline{\Delta})]$$

#### 4.3.2.1.3 Choix d'un langage pour exprimer les invariants locaux

Nous décidons de représenter un invariant global  $I \in A_{\Delta} \llbracket P_s \rrbracket$  par un vecteur  $\tilde{I}$  d'invariants locaux sur les états mémoires et associés à chaque point du programme. Alors

$$\tilde{A}_{\Delta} \llbracket P_s \rrbracket = \prod_{L \in C \llbracket P_s \rrbracket} (M \llbracket P_s \rrbracket \rightarrow \{\#, \#f\})$$

où

$$C \llbracket P_s \rrbracket = \{L \in \mathcal{L}. P_s \equiv \beta L : \delta\}$$

$$M \llbracket P_s \rrbracket = (\{x \in \mathcal{V} : P_s \equiv \alpha x \beta\} \rightarrow \mathcal{D})$$

La signification de ce vecteur d'invariants locaux est que l'invariant  $\tilde{I}(L)(M)$  associé au point  $L$  est vrai pour l'état mémoire  $M$  de tout état  $\langle L, M \rangle$  du programme ayant  $L$  comme état de contrôle. Plus formellement

$$\gamma \llbracket P_s \rrbracket \in (\tilde{A}_{\Delta} \llbracket P_s \rrbracket \rightarrow A_{\Delta} \llbracket P_s \rrbracket)$$

$$\gamma \llbracket P_s \rrbracket(\tilde{I})(\langle L, M \rangle) = \tilde{I}(L)(M)$$

Réciproquement, un invariant global  $I \in A_{\Delta} \llbracket P_s \rrbracket$  est représenté par  $\alpha \llbracket P_s \rrbracket(I) \in \tilde{A}_{\Delta} \llbracket P_s \rrbracket$ , c'est-à-dire un vecteur  $\tilde{I}$  d'invariants locaux sur les états mémoire  $M$  tels que :

$$\forall L \in C \llbracket P_s \rrbracket. \tilde{I}(L)(M) = I(\langle L, M \rangle)$$

## 4.3.2.1.4 Dérivation de conditions de vérification correctes

Nous avons à construire  $\tilde{Cv} \llbracket Ps \rrbracket$  tel que :

$$\tilde{Cv} \llbracket Ps \rrbracket(\epsilon, \varsigma)(\psi)(\tilde{I}) \Rightarrow Cv \llbracket Ps \rrbracket(\epsilon, \varsigma)(\chi \llbracket Ps \rrbracket(\tilde{I}))$$

Puisque  $Cv \llbracket Ps \rrbracket$  est une conjonction de trois conditions de vérification, nous choisissons  $\tilde{Cv} \llbracket Ps \rrbracket$  de la même forme, c'est-à-dire :

$$\tilde{Cv} \llbracket Ps \rrbracket(\epsilon, \varsigma)(\psi)(\tilde{I}) = \tilde{Cv}_{\varsigma} \llbracket Ps \rrbracket(\psi, \varsigma)(\tilde{I}) \wedge \tilde{Cv}_i \llbracket Ps \rrbracket(\tilde{I}) \wedge \tilde{Cv}_{\epsilon} \llbracket Ps \rrbracket(\epsilon)(\tilde{I})$$

Pour garantir la condition de correction ci-dessus, nous choisissons simplement :

$$\tilde{Cv}_{\varsigma} \llbracket Ps \rrbracket(\psi, \varsigma)(\tilde{I}) = Cv_{\varsigma} \llbracket Ps \rrbracket(\psi, \varsigma)(\chi \llbracket Ps \rrbracket(\tilde{I}))$$

$$\tilde{Cv}_i \llbracket Ps \rrbracket(\tilde{I}) = Cv_i \llbracket Ps \rrbracket(\chi \llbracket Ps \rrbracket(\tilde{I}))$$

$$\tilde{Cv}_{\epsilon} \llbracket Ps \rrbracket(\epsilon)(\tilde{I}) = Cv_{\epsilon} \llbracket Ps \rrbracket(\epsilon)(\chi \llbracket Ps \rrbracket(\tilde{I}))$$

## 4.3.2.1.4.1 Base

$$\tilde{Cv}_{\varsigma} \llbracket Ps \rrbracket(\psi, \varsigma)(\tilde{I})$$

$$= Cv_{\varsigma} \llbracket Ps \rrbracket(\psi, \varsigma)(\chi \llbracket Ps \rrbracket(\tilde{I}))$$

$$= [\forall \bar{a} \in S \llbracket Ps \rrbracket. [\neg \psi(\bar{a}) \wedge \varsigma(\bar{a})] \Rightarrow \chi \llbracket Ps \rrbracket(\tilde{I})(\bar{a})]$$

Puisque  $S \llbracket Ps \rrbracket = C \llbracket Ps \rrbracket \times M \llbracket Ps \rrbracket$ , nous avons :

$$= [\forall L \in C \llbracket Ps \rrbracket, M \in M \llbracket Ps \rrbracket. [\neg \psi(\langle L, M \rangle) \wedge \varsigma(\langle L, M \rangle)] \Rightarrow \chi \llbracket Ps \rrbracket(\tilde{I})(\langle L, M \rangle)]$$

Remplaçons  $\varsigma, \psi$  et  $\chi$  par leurs définitions :

$$= [\forall L \in C \llbracket Ps \rrbracket, M \in M \llbracket Ps \rrbracket. (Ps \equiv \beta L:) \Rightarrow \tilde{I}(L)(M)]$$

$Ps$  a un et un seul point de sortie d'où :

$$= [(Ps \equiv \beta L:) \wedge \forall M \in M \llbracket Ps \rrbracket. \tilde{I}(L)(M)]$$

## 4.3.2.1.4.2 Induction

$$\begin{aligned}
& \check{Cv}_i \llbracket Ps \rrbracket (\check{I}) \\
&= Cv_i \llbracket Ps \rrbracket (\delta \llbracket Ps \rrbracket (\check{I})) \\
&= [\forall \Delta \in S \llbracket Ps \rrbracket, a \in A \llbracket Ps \rrbracket. \delta \llbracket Ps \rrbracket (\check{I})(\Delta) \Rightarrow \neg [\exists \Delta' \in S \llbracket Ps \rrbracket. \neg \delta \llbracket Ps \rrbracket (\check{I})(\Delta') \wedge t \llbracket Ps \rrbracket_q(\Delta', \Delta)]] \\
&= [\forall \Delta \in S \llbracket Ps \rrbracket, a \in A \llbracket Ps \rrbracket. \delta \llbracket Ps \rrbracket (\check{I})(\Delta) \Rightarrow [\forall \Delta' \in S \llbracket Ps \rrbracket. t \llbracket Ps \rrbracket_q(\Delta', \Delta) \Rightarrow \delta \llbracket Ps \rrbracket (\check{I})(\Delta')]] \\
&= [\forall L \in C \llbracket Ps \rrbracket, M \in M \llbracket Ps \rrbracket, a \in A \llbracket Ps \rrbracket. \check{I}(L)(M) \Rightarrow [\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket. \\
&\quad t \llbracket Ps \rrbracket_q(\langle L', M' \rangle, \langle L, M \rangle) \Rightarrow \check{I}(L')(M')]] \\
&= \bigwedge_{L \in C \llbracket Ps \rrbracket} [\forall M \in M \llbracket Ps \rrbracket. \check{I}(L)(M) \Rightarrow [\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket. \\
&\quad (\text{cond} \llbracket Ps \rrbracket(L', L)(M') \wedge \text{succ} \llbracket Ps \rrbracket(L')(M', M)) \Rightarrow \check{I}(L')(M')]]
\end{aligned}$$

Nous avons une conjonction de conditions de vérification, une pour chaque point du programme, et de la forme :

$$\check{I}(L)(M) \Rightarrow [\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket. (\text{cond} \llbracket Ps \rrbracket(L', L)(M') \wedge \text{succ} \llbracket Ps \rrbracket(L')(M', M)) \Rightarrow \check{I}(L')(M')]$$

cette condition peut se décomposer aux différents cas correspondant à la définition de cond  $\llbracket Ps \rrbracket$  :

(a) Si  $(Ps \equiv L: p)$  alors  $L$  désigne le point d'entrée du programme, alors  $\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket. \neg \text{cond} \llbracket Ps \rrbracket(L', L)(M')$  et la condition de vérification est donc vraie trivialement.

(b) Si  $(Ps \equiv pL: \text{skip}; L: \delta)$  ou  $(Ps \equiv pL: \text{else } Ps' fi; L: \delta)$  ou  $(Ps \equiv pL: fi; L: \delta)$  alors cond  $\llbracket Ps \rrbracket(L', L)(M)$  est vrai et succ  $\llbracket Ps \rrbracket(L')(M', M)$  implique  $M = M'$  de sorte que la condition de vérification peut être simplifiée en :

$$\check{I}(L)(M) \Rightarrow \check{I}(L')(M)$$

(c) Si  $(Ps \equiv pL: v := E; L: \delta)$  alors nous devons vérifier que

$$\check{I}(L)(M) \Rightarrow [\forall M' \in M \llbracket Ps \rrbracket. [M' \in \text{dom}(E[E]) \wedge M = M'[v \leftarrow E[E](M)]] \Rightarrow \check{I}(L')(M')]$$

Noter que  $M'$  doit être de la forme  $M[v \leftarrow m]$  où  $m \in \mathcal{S}$  est la valeur de  $v$  avant l'affectation. Alors cette condition peut se simplifier en :

$$\check{I}(L)(M) \Rightarrow [\forall m \in \mathcal{S}. [M[v \leftarrow m] \in \text{dom}(E[E]) \wedge M(v) = E[E](M[v \leftarrow m])] \Rightarrow \check{I}(L')(M[v \leftarrow m])]$$

Si  $(Ps \equiv pL: v := ?; L: \delta)$ , nous obtenons de même :

$$\check{I}(L)(M) \Rightarrow [\forall m \in \mathcal{S}. \check{I}(L')(M[v \leftarrow m])]$$

(d) Si  $(P_s \equiv \beta L' : \text{if } B \text{ then } L : \delta)$  ou  $(P_s \equiv \beta L' : \text{while } B \text{ do } L : \delta)$  ou  $(P_s \equiv \beta \text{ while } B \text{ do } L : C_0; \dots; L_m : C_m; L' : \text{od}; \delta)$  nous obtenons

$$\tilde{I}(L)(M) \Rightarrow [(M \in \text{dom}(B[B])) \wedge B[B](M) = \text{tt}) \Rightarrow \tilde{I}(L')(M)]$$

(e) Si  $(P_s \equiv \beta L' : \text{if } B \text{ then } P_s' \text{ else } L : \delta)$  ou  $(P_s \equiv \beta L' : \text{while } B \text{ do } P_s' \text{ od}; L : \delta)$  ou  $(P_s \equiv \alpha \text{ while } B \text{ do } L_0 : C_0; \dots; L_m : C_m; L' : \text{od}; L : \delta)$

$$\tilde{I}(L)(M) \Rightarrow [(M \in \text{dom}(B[B])) \wedge B[B](M) = \text{ff}) \Rightarrow \tilde{I}(L')(M)]$$

#### 4.3.2.1.4.3 Contradiction

$$\begin{aligned} C_{\forall \epsilon}^{\tilde{I}} [P_s](\epsilon)(\tilde{I}) &= C_{\forall \epsilon} [P_s](\epsilon)(\chi[P_s](\tilde{I})) \\ &= [\forall \Delta \in S[P_s]. \epsilon(\Delta) \Rightarrow \neg \chi[P_s](\tilde{I})(\Delta)] \\ &= [\forall L \in C[P_s], M \in M[P_s]. [(P_s \equiv L : \beta) \wedge \phi(M)] \Rightarrow \neg \tilde{I}(L)(M)] \\ &= [(P_s \equiv L : \beta) \wedge (\forall M \in M[P_s]. \phi(M) \Rightarrow \neg \tilde{I}(L)(M))] \end{aligned}$$

#### 4.3.2.1.5 Résumé informel des conditions de vérification

En utilisant des notations mnémoniques, nous pouvons récapituler les conditions de vérification ci-dessus comme suit ( $P_i$  est l'invariant sur les variables du programme associé au point  $L_i$ ) :

## - Base

 $\beta L_1:$  $P_1$ 

## - Induction

## . Commande nulle

 $\beta L_1: \underline{\text{skip}}; L_2: \delta$  $P_2 \Rightarrow P_1$ 

## . Commande d'affectation

 $\beta L_1: V := E; L_2: \delta$  $P_2 \Rightarrow [\forall m \in \mathcal{D}. V = E[V \leftarrow m] \Rightarrow P_1[V \leftarrow m]]$  $\beta L_1: V := ?; L_2: \delta$  $P_2 \Rightarrow [\forall m \in \mathcal{D}. P_1[V \leftarrow m]]$ 

## . Commande conditionnelle

 $\beta L_1: \underline{\text{if } B \text{ then}}$  $L_2: \delta$  $L_3:$ else $L_4: \delta'$  $L_5:$ fi; $L_6: \beta'$  $P_2 \Rightarrow [B \Rightarrow P_1]$  $P_4 \Rightarrow [\neg B \Rightarrow P_1]$  $P_6 \Rightarrow [P_3 \wedge P_5]$ 

## . Commande itérative

 $\beta L_1: \underline{\text{while } B \text{ do}}$  $L_2: \delta$  $L_3:$ od; $L_4: \beta'$  $P_2 \Rightarrow [B \Rightarrow (P_1 \wedge P_3)]$  $P_4 \Rightarrow [\neg B \Rightarrow (P_1 \wedge P_3)]$ 

## - Contradiction

 $L_1: \beta$  $\phi \Rightarrow \neg P_1$

## 4.3.2.1.6 Exemple de preuve avec cette méthode

soit à prouver que le programme suivant

```

1:   Q := 0;
2:   while x > y do
3:       Q := Q + 1;
4:       x := x - y;
5:   od;
6:

```

ne se termine pas normalement quand les valeurs initiales  $x, y$  des variables  $x, y$  sont telle que  $x \geq 0$  et  $y = 0$ . Nous supposons que le domaine  $\mathcal{D}$  de valeurs de chaque variable est l'ensemble des entiers compris entre deux bornes min et max. Ces bornes sont supposées telles que  $\min < 0 < \max$ .

Les conditions de vérification (après des simplifications triviales) sont les suivantes (où  $x, y, q \in [\min, \max]$ ) :

$$P_0(x, y, q)$$

$$P_0(x, y, q) \Rightarrow [(x < y) \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_5(x, y, q) \Rightarrow P_4(x + y, y, q)$$

$$P_4(x, y, q) \Rightarrow P_3(x, y, q - 1)$$

$$P_3(x, y, q) \Rightarrow [(x \geq y) \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_2(x, y, q) \Rightarrow [\forall q' \in [\min, \max]. (q = 0) \Rightarrow P_1(x, y, q')]$$

$$[(0 \leq x \leq \max) \wedge (y = 0)] \Rightarrow \neg P_2(x, y, q)$$

Intuitivement, par l'absurde, si  $y = 0$  le programme ne peut se terminer que si  $x < 0$ , en contradiction avec l'hypothèse sur la valeur initiale de  $x$ . Ceci peut se démontrer formellement en utilisant les invariants suivants :

$$P_i(x, y, q) = [(y = 0) \Rightarrow (x < 0)] \quad \text{pour } i = 1, \dots, 5$$

$$P_0(x, y, q) = \text{tt}$$

## 4.3.2.1.7 Vérification de la complétude sémantique

Conformément au paragraphe 7.5, nous devons vérifier que  
 $\forall I \in A_S \llbracket P_S \rrbracket. C_v \llbracket P_S \rrbracket(\epsilon, \sigma)(\psi)(I) \Rightarrow C_v \llbracket P_S \rrbracket(\epsilon, \sigma)(\psi)(\alpha \llbracket P_S \rrbracket(I))$

En utilisant le fait que  $C_v \llbracket P_S \rrbracket(\epsilon, \sigma)(\psi)(\tilde{I}) = C_v \llbracket P_S \rrbracket(\epsilon, \sigma)(\psi)(\delta \llbracket P_S \rrbracket(\tilde{I}))$ , il est  
 suffisant de vérifier que  $\delta \llbracket P_S \rrbracket(\alpha \llbracket P_S \rrbracket(I)) = I$ , qui est trivial car  $\delta \llbracket P_S \rrbracket$   
 est une bijection entre  $A_S \llbracket P_S \rrbracket$  et  $A_S \llbracket P_S \rrbracket$  (et  $\alpha \llbracket P_S \rrbracket$  est son inverse).

4.3.2.1.8 Preuve d'absence d'erreurs à l'exécution, par l'absurde  
pour des programmes séquentiels

Soit  $P_S \in \mathcal{P}_S$  un programme séquentiel et  $\phi \in (\mathcal{V} \rightarrow \mathcal{B}) \rightarrow \{\text{tt}, \text{ff}\}$  une  
 caractérisation des valeurs initiales possibles des variables du programme.  
 L'exécution du programme  $P_S$  ne conduit pas à une erreur d'exécution  
 si et seulement si tout état atteint durant l'exécution et qui n'est pas un  
 état de sortie a un état successeur, c'est-à-dire

$$\forall p \in \Sigma \times S \llbracket P_S \rrbracket, A \llbracket P_S \rrbracket, t \llbracket P_S \rrbracket, \epsilon, i \in |p|. [\sigma(p_i) \Rightarrow \psi(p_i)]$$

où

$$\epsilon(\langle L, M \rangle) = [(P_S \equiv L; \beta) \wedge \phi(M)]$$

$$\sigma(\langle L, M \rangle) = [\neg (P_S \equiv \alpha L;)]$$

$$\psi(\langle L, M \rangle) = [\exists L' \in C \llbracket P_S \rrbracket, M' \in M \llbracket P_S \rrbracket, a \in A \llbracket P_S \rrbracket. t \llbracket P_S \rrbracket_a(\langle L, M \rangle, \langle L', M' \rangle)]$$

$$\text{avec } M \llbracket P_S \rrbracket = (\{x \in \mathcal{V} : P_S \equiv x; \beta\} \rightarrow \mathcal{B}), \quad C \llbracket P_S \rrbracket = \{L \in \mathcal{L} : P_S \equiv L; \beta\}$$

Noter que la seule différence avec le paragraphe 4.3.2.1.1 est  $\sigma$  et  $\psi$ .  
 Alors, en choisissant le principe d'induction et le langage, pour exprimer  
 les invariants locaux, considérés aux paragraphes 4.3.2.1.2 et 4.3.2.1.3, nous  
 obtenons les mêmes conditions de vérification qu'en 4.3.2.1.4 excepté  
 pour la base :

$$\check{C}v_g \llbracket Ps \rrbracket (\psi, \delta)(\check{I})$$

$$= Cv_g \llbracket Ps \rrbracket (\psi, \delta)(\delta \llbracket Ps \rrbracket (\check{I}))$$

$$= [\forall \bar{\delta} \in S \llbracket Ps \rrbracket. [\neg \psi(\bar{\delta}) \wedge \delta(\bar{\delta})] \Rightarrow \delta \llbracket Ps \rrbracket (\check{I})(\bar{\delta})]$$

$$= [\forall L \in C \llbracket Ps \rrbracket, M \in M \llbracket Ps \rrbracket. [\neg (Ps \equiv \beta L) \wedge (\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket, a \in A \llbracket Ps \rrbracket.$$

$$\neg \vdash \llbracket Ps \rrbracket_a (\langle L, M \rangle, \langle L', M' \rangle)] \Rightarrow \check{I}(L)(M)]$$

Alors pour tous les points  $L$  du programme  $Ps$  excepté l'état de sortie, nous devons prouver :

$$[\forall L' \in C \llbracket Ps \rrbracket, M' \in M \llbracket Ps \rrbracket, a \in A \llbracket Ps \rrbracket. \neg \vdash \llbracket Ps \rrbracket_a (\langle L, M \rangle, \langle L', M' \rangle)] \Rightarrow \check{I}(L)(M)$$

Cette condition se décompose aux différents cas correspondant à  $\vdash \llbracket Ps \rrbracket_a$  :

(a) Si  $(Ps \equiv \beta L: \underline{\text{skip}}; L': \delta)$  ou  $(Ps \equiv \beta L: \underline{\text{else } Ps' \text{ fi}}; L': \delta)$  ou  $(Ps \equiv \beta L: \underline{\text{fi}}; L': \delta)$  ou  $(Ps \equiv \beta L: V := ?; L': \delta)$  alors le membre de gauche est faux et la condition de vérification est identiquement vraie.

(b) Si  $(Ps \equiv \beta L: V := E; L': \delta)$  alors nous obtenons :

$$[M \notin \underline{\text{dom}}(\llbracket E \rrbracket)] \Rightarrow \check{I}(L)(M)$$

(c) Si  $(Ps \equiv \beta L: \underline{\text{if } B \text{ then } \delta})$  ou  $(Ps \equiv \beta L: \underline{\text{while } B \text{ do } \delta})$  ou  $(Ps \equiv \beta \underline{\text{while } B \text{ do } } L_0: C_0; \dots; L_m: C_m; L: \text{od}; \delta)$  alors nous obtenons

$$[M \notin \underline{\text{dom}}(\llbracket B \rrbracket)] \Rightarrow \check{I}(L)(M)$$

Nous pouvons récapituler ces conditions de vérifications comme suit :

## - Commande nulle

$$\beta L_1: \underline{\text{skip}}; L_2: \delta$$

$$P_2 \Rightarrow P_1$$

## - Commande d'affectation

$$\beta L_1: V := E; L_2: \delta$$

$$\neg \text{dom}(E) \Rightarrow P_1$$

$$P_2 \Rightarrow [\forall m \in \mathcal{D}. V = E[V \leftarrow m] \Rightarrow P_1[V \leftarrow m]]$$

$$\beta L_1: V := ?; L_2: \delta$$

$$P_2 \Rightarrow [\forall m \in \mathcal{D}. P_1[V \leftarrow m]]$$

## - Commande conditionnelle

$$\beta L_1: \underline{\text{if}} B \underline{\text{then}}$$

$$\neg \text{dom}(B) \Rightarrow P_1$$

$$L_2: \delta$$

$$P_2 \Rightarrow [B \Rightarrow P_1]$$

$$L_3:$$

$$\underline{\text{else}}$$

$$L_4: \delta'$$

$$P_4 \Rightarrow [\neg B \Rightarrow P_1]$$

$$L_5:$$

$$\underline{\text{fi}}$$

$$L_6: \beta'$$

$$P_6 \Rightarrow [P_3 \wedge P_5]$$

## - Commande itérative

$$\beta L_1: \underline{\text{while}} B \underline{\text{do}}$$

$$\neg \text{dom}(B) \Rightarrow P_1$$

$$L_2: \delta$$

$$P_2 \Rightarrow [B \Rightarrow (P_1 \wedge P_3)]$$

$$L_3:$$

$$\underline{\text{od}};$$

$$L_4: \beta'$$

$$P_4 \Rightarrow [\neg B \Rightarrow (P_1 \wedge P_3)]$$

## - Point d'entrée

$$L_1: \beta$$

$$\phi \Rightarrow \neg P_1$$

Pour notre programme pris comme exemple

```

1:
  q := 0;
2:
  while x > y do
3:
    q := q + 1;
4:
    x := x - y;
5:
  od;
6:

```

nous pouvons montrer qu'il n'y a pas d'erreurs à l'exécution quand les valeurs initiales  $x, y$  de  $x, y$  sont telles que  $x \geq 0 \wedge y > 0$  et  $\mathcal{D} = [\min, \max]$  avec  $\min < 0 < \max$ .

Les conditions de vérification (après des simplifications triviales) sont:

$$[(q+1) > \max] \Rightarrow P_3(x, y, q)$$

$$[(x-y) < \min] \vee [(x-y) > \max] \Rightarrow P_4(x, y, q)$$

$$P_2(x, y, q) \Rightarrow [(x < y) \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_5(x, y, q) \Rightarrow P_4(x+y, y, q)$$

$$P_4(x, y, q) \Rightarrow P_3(x, y, q-1)$$

$$P_3(x, y, q) \Rightarrow [x \geq y] \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_2(x, y, q) \Rightarrow [\forall q' \in [\min, \max]. (q=0) \Rightarrow P_1(x, y, q')]$$

$$[x \geq 0 \wedge y > 0] \Rightarrow \neg P_1(x, y, q)$$

Intuitivement, si initialement  $x \geq 0$  et  $y > 0$  la seule erreur d'exécution possible est un débordement à la commande 3:  $q := q + 1$ ; . Puisque  $q$  reste positif ceci peut arriver seulement si la valeur initiale  $x$  de  $x$  (c'est-à-dire  $q \times y + x$  en terme des valeurs courantes des variables) est supérieure à  $\max$ , en contradiction avec le fait que  $x \in \mathcal{D} = [\min, \max]$ .

Ce raisonnement est formalisé en montrant que les invariants locaux suivants satisfont les conditions de vérification (où  $x, y, q \in [\min, \max]$ ):

$$P_1(x, y, q) = [\neg(x \geq 0 \wedge y > 0)]$$

$$P_2(x, y, q) = P_5(x, y, q) = [(x \geq 0 \wedge y > 0 \wedge q \geq 0) \Rightarrow (q \times y + x > \max)]$$

$$P_3(x, y, q) = [(x > y > 0 \wedge q \geq 0) \Rightarrow (q \times y + x > \max)]$$

$$P_4(x, y, q) = [(x \geq y > 0 \wedge q \geq 1) \Rightarrow ((q-1) \times y + x > \max)]$$

$$P_6(x, y, q) = \text{ff}$$

#### 4.3.2.1.9 Preuve d'invariance globale, par l'absurde pour des programmes séquentiels

Soit  $P_s \in \mathcal{P}_s$  un programme séquentiel. Un invariant global d'un programme  $P_s$  est une assertion  $\delta \in ((\mathcal{V} \rightarrow \mathcal{B}) \rightarrow \{\text{tt}, \text{ff}\})$  sur les valeurs des variables qui est vraie tout le temps durant l'exécution du programme. c'est-à-dire

$$[\forall p \in \Sigma \langle S[P_s], A[P_s], E[P_s], \varepsilon \rangle, i \in |p|. [\delta(p_i) \Rightarrow \psi(p_i)]]$$

où

$$\varepsilon(\langle L, M \rangle) = [(P_s \equiv L : \beta) \wedge \phi(M)]$$

$$\delta(\langle L, M \rangle) = \text{tt}$$

$$\psi(\langle L, M \rangle) = \delta(M)$$

En choisissant le principe d'induction et le langage, pour exprimer les invariants locaux, considérés aux paragraphes 4.3.2.1 et 4.3.2.2, nous obtenons les conditions de vérifications du paragraphe 4.3.2.1.4 excepté pour la base :

$$\begin{aligned} & C\tilde{v}_\delta[P_s](\psi, \varepsilon)(\tilde{I}) \\ &= C\tilde{v}_\delta[P_s](\psi, \varepsilon)(\delta[P_s](\tilde{I})) \\ &= [\forall \tilde{\delta} \in S[P_s]. [\neg \psi(\tilde{\delta}) \wedge \varepsilon(\tilde{\delta})] \Rightarrow \delta[P_s](\tilde{I})(\tilde{\delta})] \\ &= [\forall L \in C[P_s], M \in M[P_s]. \neg \delta(M) \Rightarrow \tilde{I}(L)(M)] \end{aligned}$$

qui s'écrit informellement ( $L_i$  désignant une étiquette quelconque du programme):

- Base

$$\beta L_i : \beta'$$

$$\neg \delta \Rightarrow P_i$$

Pour notre programme pris comme exemple,

```

1:   Q := 0;
2:   while x > y do
3:       Q := Q + 1;
4:       X := X - y;
5:   od;
6:

```

les conditions de vérification sont les suivantes (où  $x, y, q \in [\min, \max]$ ) :

$$\neg \delta(x, y, q) \Rightarrow P_i(x, y, q) \quad i=1, \dots, 6$$

$$P_6(x, y, q) \Rightarrow [(x < y) \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_5(x, y, q) \Rightarrow P_4(x+y, y, q)$$

$$P_4(x, y, q) \Rightarrow P_5(x, y, q-1)$$

$$P_3(x, y, q) \Rightarrow [(x \geq y) \Rightarrow (P_2(x, y, q) \wedge P_5(x, y, q))]$$

$$P_2(x, y, q) \Rightarrow [\forall q' \in [\min, \max]. (q=0) \Rightarrow P_1(x, y, q')]$$

$$\phi(x, y, q) \Rightarrow \neg P_1(x, y, q)$$

Pour prouver que  $\phi(x, y, q) = (x \geq 0 \wedge y \geq 0 \wedge q \geq 0)$  est un invariant global de ce programme, nous pouvons choisir les invariants locaux suivants :

$$P_1(x, y, q) = P_2(x, y, q) = P_5(x, y, q) = [x < 0 \vee y \leq 0 \vee q < 0]$$

$$P_3(x, y, q) = P_4(x, y, q) = [(x \geq y) \Rightarrow (x < 0 \vee y \leq 0 \vee q < 0)]$$

$$P_6(x, y, q) = [(x < y) \Rightarrow (x < 0 \vee y \leq 0 \vee q < 0)]$$

#### 4.3.2.2 Extension de la méthode de Morris-Wegbreit dite "Subgoal induction" aux programmes parallèles et généralisation à d'autres propriétés d'invariance

La méthode de Morris-Wegbreit [77] a été conçue pour démontrer la correction partielle de programmes séquentiels. Etant données des spécifications d'entrée  $\phi \in ((V \rightarrow D) \rightarrow \{tt, ff\})$  et de sortie  $\psi \in ((V \rightarrow D)^2 \rightarrow \{tt, ff\})$  d'un programme  $P_\pi$ , il s'agit de démontrer la propriété d'invariance :

$$\forall p \in \Sigma \langle S[P_\pi], A[P_\pi], t[P_\pi], \varepsilon \rangle, i \in |p|. [\sigma(p_i) \Rightarrow \Psi(p_0, p_i)]$$

où

$$\varepsilon(\langle L, M \rangle) = [(P_\pi \equiv L : \beta) \wedge \phi(M)]$$

$$\sigma(\langle L, M \rangle) = [P_\pi \equiv \beta L : ]$$

$$\Psi(\langle \underline{L}, \underline{M} \rangle, \langle \bar{L}, \bar{M} \rangle) = \psi(\underline{M}, \bar{M})$$

Nous allons montrer dans un premier temps que l'essence de la méthode de Morris-Wegbreit [77] consiste à appliquer le principe d'induction  $(\cdot)^{-1}$ , c'est-à-dire à une correspondance  $(\alpha, \delta)$  entre invariants près à démontrer que

$$[\exists I \in A_\Delta[P_\pi]. C\sigma[P_\pi](\varepsilon, \sigma)(\Psi)(I)]$$

où

$$A_\Delta[P_\pi] = (S[P_\pi]^2 \rightarrow \{tt, ff\})$$

$$C\sigma[P_\pi](\varepsilon, \sigma)(\Psi)(I) = [C\sigma_\sigma[P_\pi](\sigma)(I) \wedge C\sigma_i[P_\pi](\sigma)(I) \wedge C\sigma_\varepsilon[P_\pi](\varepsilon, \sigma)(\Psi)(I)]$$

avec

$$C\sigma_\sigma[P_\pi](\sigma)(I) = [\forall \bar{\alpha} \in S[P_\pi]. \sigma(\bar{\alpha}) \Rightarrow I(\bar{\alpha}, \bar{\alpha})]$$

$$C\sigma_i[P_\pi](\sigma)(I) = [\forall \alpha, \alpha', \bar{\alpha} \in S[P_\pi], \alpha \in A[P_\pi]. (t_\alpha[P_\pi](\alpha, \alpha') \wedge I(\alpha', \bar{\alpha}) \wedge \sigma(\bar{\alpha})) \Rightarrow I(\alpha, \bar{\alpha})]$$

$$C\sigma_\varepsilon[P_\pi](\varepsilon, \sigma)(\Psi)(I) = [\forall \underline{\alpha}, \bar{\alpha} \in S[P_\pi]. (\varepsilon(\underline{\alpha}) \wedge I(\underline{\alpha}, \bar{\alpha}) \wedge \sigma(\bar{\alpha})) \Rightarrow \Psi(\underline{\alpha}, \bar{\alpha})]$$

Ensuite nous généraliserons la méthode aux programmes parallèles et à d'autres propriétés d'invariance (Cousot-R[81]).

## 4.3.2.2.1 Programmes séquentiels

Nous considérons des programmes séquentiels comme ils ont été définis au paragraphe 2.8.1.

## 4.3.2.2.1.1 Choix d'un langage pour exprimer les invariants locaux et sa sémantique

Nous choisissons

$$A_{\Delta}^{\vee} [Ps] = \prod_{L \in C[Ps]} (M[Ps] \xrightarrow{L} \{\text{tt}, \text{ff}\})$$

où

$$C[Ps] = \{L \in \mathcal{L} : Ps \equiv \beta L; \delta\}$$

$$M[Ps] = (\{x \in \mathcal{V} : Ps \equiv \alpha x \beta\} \rightarrow \mathcal{D})$$

pour pouvoir associer une relation sur les états mémoire à chaque point du programme. La signification de ce vecteur d'invariants locaux est défini par la fonction sémantique

$$\gamma[Ps] \in (A_{\Delta}^{\vee} [Ps] \rightarrow A_{\Delta} [Ps])$$

$$\gamma[Ps](\tilde{I})(\langle L, M \rangle, \langle \bar{L}, \bar{M} \rangle) = \tilde{I}(L)(M, \bar{M})$$

Intuitivement, quand le contrôle est en  $L$ , la relation  $\tilde{I}(L)$  est vraie entre l'état mémoire courant  $M$  et l'état mémoire (final)  $\bar{M}$  (qui correspond au point  $\bar{L}$  de sortie de  $Ps$ ).

## 4.3.2.2.1.2 Dérivation de conditions de vérification correctes

Nous avons à construire  $Cv[Ps]$  tel que

$$Cv[Ps](\epsilon, \delta)(\Psi)(\tilde{I}) \Rightarrow Cv[Ps](\epsilon, \delta)(\Psi)(\gamma[Ps](\tilde{I}))$$

$Cv[Ps]$  étant une conjonction de trois conditions, nous choisissons  $Cv[Ps]$  également comme conjonction de trois conditions, chacune satisfaisant le critère de correction.

## 4.3.2.2.1.2.1 Finalisation

$$\begin{aligned}
& \text{Cv}_{\bar{\sigma}} \llbracket P_s \rrbracket (\bar{\sigma}) (\bar{\chi} \llbracket P_s \rrbracket (\bar{I})) \\
&= [\forall \bar{a} \in S \llbracket P_s \rrbracket. \bar{\sigma}(\bar{a}) \Rightarrow \bar{\chi} \llbracket P_s \rrbracket (\bar{I})(\bar{a}, \bar{a})] \\
&= [\forall \bar{L} \in C \llbracket P_s \rrbracket, \bar{M} \in M \llbracket P_s \rrbracket. \bar{\sigma}(\langle \bar{L}, \bar{M} \rangle) \Rightarrow \bar{\chi} \llbracket P_s \rrbracket (\bar{I})(\langle \bar{L}, \bar{M} \rangle, \langle \bar{L}, \bar{M} \rangle)] \\
&= [\forall \bar{L} \in C \llbracket P_s \rrbracket, \bar{M} \in M \llbracket P_s \rrbracket. (P_s \equiv \beta \bar{L} :) \Rightarrow \bar{I}(\bar{L})(\bar{M}, \bar{M})] \\
&= [\forall \bar{M} \in M \llbracket P_s \rrbracket. \bar{I}(\bar{L})(\bar{M}, \bar{M})] \quad \text{où} \quad P_s \equiv \alpha \bar{L} : \\
&= \text{Cv}_{\bar{\sigma}}^{\sim} \llbracket P_s \rrbracket (\bar{\sigma}) (\bar{I})
\end{aligned}$$

Intuitivement, cette condition de vérification établit que l'invariant  $\bar{I}(\bar{L})$  associé au point de sortie doit être vrai pour toute exécution qui ne termine.

## 4.3.2.2.1.2.2 Induction

$$\begin{aligned}
& \text{Cv}_{\downarrow} \llbracket P_s \rrbracket (\bar{\sigma}) (\bar{\chi} \llbracket P_s \rrbracket (\bar{I})) \\
&= [\forall \lambda, \lambda', \bar{a} \in S \llbracket P_s \rrbracket, a \in A \llbracket P_s \rrbracket. (\bar{t} \llbracket P_s \rrbracket_a(\lambda, \lambda') \wedge \bar{\chi} \llbracket P_s \rrbracket (\bar{I})(\lambda', \bar{a}) \wedge \bar{\sigma}(\bar{a})) \Rightarrow \bar{\chi} \llbracket P_s \rrbracket (\bar{I})(\lambda, \bar{a})] \\
&= [\forall L \in C \llbracket P_s \rrbracket, M, \bar{M} \in M \llbracket P_s \rrbracket. \\
&\quad [\exists L' \in C \llbracket P_s \rrbracket, M' \in M \llbracket P_s \rrbracket. \text{cond} \llbracket P_s \rrbracket (L, L')(M) \wedge \text{succ} \llbracket P_s \rrbracket (L)(M, M') \wedge \bar{I}(L')(M', \bar{M})] \Rightarrow \bar{I}(L)(M, \bar{M})] \\
&= \text{Cv}_{\downarrow}^{\sim} \llbracket P_s \rrbracket (\bar{\sigma}) (\bar{I})
\end{aligned}$$

Intuitivement cette condition de vérification établit que si un pas atomique du programme peut conduire du point  $L$  où l'état mémoire est  $M$  à son successeur immédiat  $L'$  avec l'état mémoire  $M'$  tel que  $\text{succ} \llbracket P_s \rrbracket (L)(M, M')$  alors l'invariant local  $\bar{I}(L')(M', \bar{M})$  après ce pas doit impliquer l'invariant local  $\bar{I}(L)(M, \bar{M})$  avant ce pas.

Suivant la nature de la commande étiquetée par  $L$  et en utilisant les définitions de  $\text{cond} \llbracket P_s \rrbracket$  et  $\text{succ} \llbracket P_s \rrbracket$ , nous pouvons la décomposer en sous-cas. Par exemple, si  $L$  désigne une boucle while, nous obtenons :

$$[(P_s \equiv \beta L: \underline{\text{while}} B \underline{\text{do}} L': \delta) \wedge M \in \underline{\text{dom}}(B[B]) \wedge B[B](M) = \text{tt} \wedge \tilde{I}(L')(M, M)] \\ \Rightarrow \tilde{I}(L)(M, \bar{M})$$

et

$$[(P_s \equiv \beta L: \underline{\text{while}} B \underline{\text{do}} P_s' \underline{\text{od}}; L': \delta) \wedge M \in \underline{\text{dom}}(B[B]) \wedge B[B](M) = \text{ff} \wedge \tilde{I}(L')(M, \bar{M})] \\ \Rightarrow \tilde{I}(L)(M, \bar{M})$$

#### 4.3.2.2.1.2.3 Initialisation

$$\begin{aligned} Cw_\varepsilon [P_s](\varepsilon, \sigma)(\Psi)(\delta[P_s](\tilde{I})) \\ &= [\forall \underline{A}, \bar{A} \in S[P_s]. (\varepsilon(\underline{A}) \wedge \delta[P_s](\tilde{I})(\underline{A}, \bar{A}) \wedge \sigma(\bar{A})) \Rightarrow \Psi(\underline{A}, \bar{A})] \\ &= [\forall \underline{L}, \bar{L} \in C[P_s], \underline{M}, \bar{M} \in M[P_s]. \\ &\quad (P_s \equiv \underline{L}: \beta \wedge \tilde{I}(\underline{L})(\underline{M}, \bar{M}) \wedge P_s \equiv \beta \bar{L}:.) \Rightarrow (\phi(\underline{M}) \Rightarrow \psi(\underline{M}, \bar{M}))] \\ &= [(\phi(\underline{M}) \wedge \tilde{I}(\underline{L})(\underline{M}, \bar{M})) \Rightarrow \psi(\underline{M}, \bar{M})] \quad \text{où } P_s \equiv \underline{L}: \alpha \\ &= \tilde{Cw}_\varepsilon [P_s](\varepsilon, \sigma)(\Psi)(\tilde{I}) \end{aligned}$$

Intuitivement, la spécification d'entrée et l'invariant local associé au point d'entrée doivent impliquer la spécification de sortie.

#### 4.3.2.2.1.3 Vérification de la complétude sémantique

Définissons

$$\alpha[P_s] \in (A_\delta[P_s] \rightarrow A_\lambda[P_s])$$

$$\alpha[P_s](I)(L)(M, \bar{M}) = [\forall \bar{L} \in C[P_s]. (P_s \equiv \beta \bar{L}:.) \Rightarrow I(\langle L, M \rangle, \langle \bar{L}, \bar{M} \rangle)]$$

qui spécifie comment une hypothèse d'induction  $I \in A_\delta[P_s]$  peut être codée par un vecteur d'invariants locaux  $\tilde{I}(L)$  associés à chaque point  $L$  du programme  $P_s$ .

Ayant choisi  $\tilde{Cw}[P_s](\varepsilon, \sigma)(\Psi) = Cw[P_s](\varepsilon, \sigma)(\Psi) \circ \delta[P_s]$ , la vérification de la complétude sémantique est :

$\forall I \in A_0[[P_5]]. C_v[[P_5]](\epsilon, \sigma)(\Psi)(\alpha[[P_5]](I)) = C_v[[P_5]](\epsilon, \sigma)(\Psi)(\gamma_0 \alpha[[P_5]](I)) \leftarrow C_v[[P_5]](\epsilon, \sigma)(\Psi)(I)$   
 car  $C_v[[P_5]](\epsilon, \sigma)(\Psi)$  est monotone et  $\gamma_0 \alpha$  est extensive.

Ce résultat réfute l'argument de Misra [78] que "subgoal induction is not guaranteed to prove a correct program correct".

#### 4.3.2.2.1.4 Résumé des conditions de vérification pour la preuve de correction partielle de programmes séquentiels par induction en arrière

Nous utiliserons des notations mnémoriques informelles.  $\underline{x}$  et  $\bar{x}$  désignent les vecteurs des valeurs respectivement initiales et finales des variables du programme et  $P_i$  est l'invariant associé au point  $L_i$ .

## - Finalisation

$$P_{L_1}: \quad \forall \bar{x}. P_1(\bar{x}, \bar{x})$$

## - Induction

## . Commande nulle

$$\beta L_1: \underline{\text{skip}}; L_2: \delta \quad P_0 \Rightarrow P_1$$

## . Commande d'affectation

$$\beta L_1: V := E; L_2: \delta \quad P_0[V \leftarrow E] \Rightarrow P_1$$

$$\beta L_1: V := ?; L_2: \delta \quad \forall m \in \mathcal{D}. P_0[V \leftarrow m] \Rightarrow P_1$$

## . Commande conditionnelle

$$\beta L_1: \underline{\text{if } B \text{ then}}$$

$$L_2: \delta$$

$$L_3:$$

else

$$L_4: \delta'$$

$$L_5:$$

$$\underline{\text{fi}};$$

$$L_6: \beta'$$

$$[(P_2 \wedge B) \vee (P_4 \wedge \neg B)] \Rightarrow P_1$$

$$P_0 \Rightarrow (P_3 \wedge P_5)$$

## . Commande itérative

$$\beta L_1: \underline{\text{while } B \text{ do}}$$

$$L_2: \delta$$

$$L_3:$$

od;

$$L_4: \beta'$$

$$[(P_2 \wedge B) \vee (P_4 \wedge \neg B)] \Rightarrow (P_1 \wedge P_3)$$

## - Initialisation

$$L_1: \beta \quad \forall x, \bar{x}. [(\phi(x) \wedge P_1(x, \bar{x})) \Rightarrow \psi(x, \bar{x})]$$

#### 4.3.2.2.1.5 Preuves d'autres propriétés d'invariance de programmes séquentiels par induction en arrière

Morris et Wegbreit affirment que "a drawback of subgoal induction is that it cannot be used to prove invariants about non-terminating programs". Le problème est de montrer que lorsque l'exécution d'un programme  $P_S$  commence dans un état mémoire initial  $\underline{M}$  satisfaisant une spécification d'entrée  $\phi$  et atteint un point  $L$  quelconque du programme avec l'état mémoire  $M$ , alors l'invariant  $\psi(L)(M)$  doit être vrai. Formellement

$$\forall p \in \Sigma \langle S[P_S], A[P_S], t[P_S], \varepsilon \rangle, i \in |p|. [\varepsilon(p_i) \Rightarrow \Psi(p_0, p_i)]$$

où

$$\varepsilon(\langle L, M \rangle) = [(P_S \equiv L : \beta) \wedge \phi(M)]$$

$$\varepsilon(\langle L, M \rangle) = \text{tt}$$

$$\Psi(\langle \underline{L}, \underline{M} \rangle, \langle \bar{L}, \bar{M} \rangle) = \psi(\bar{L})(\bar{M})$$

Les définitions ci-dessus de  $\varepsilon$  et  $\Psi$  diffèrent du cas de la correction partielle et donc de "subgoal induction", de sorte que la remarque de Morris-Wegbreit est justifiée pour "subgoal induction" mais non pour toutes les méthodes de preuve par induction en arrière.

Pour être complet, construisons une méthode de preuve d'invariants par induction en arrière.  $\Psi$  est maintenant une assertion sur les états finaux (final signifiant quelconque dans ce cas). D'après 4.2.1.3, nous pourrions utiliser le principe d'induction (-i-) contrapositionnel en arrière. La démarche reste la même et les conditions de vérification sont similaires à celles du paragraphe 4.3.2.2.1.2 pour le pas d'induction 4.3.2.2.1.2.2 tandis que 4.3.2.2.1.2.1 et 4.3.2.2.1.2.3 deviennent :

- Finalisation

$$\vdash L_i : \delta$$

$$\neg \psi_i \Rightarrow P_i$$

- Initialisation

$$L_1 : \beta$$

$$\phi \Rightarrow \neg P_1$$

### Exemple

Illustrons cette méthode par le (contre)exemple simple de Morris-Wegbreit[77], qui consiste à montrer que  $x > 0$  dans le programme suivant :

```

1:      x := 1
2:      while true do
3:          x := x + 1;
4:      od;
5:

```

Nous choisissons  $\phi(x) = \text{tt}$ ,  $\psi_1(x) = \text{tt}$ ,  $\psi_i(x) = [x > 0]$ ,  $i = 2, \dots, 5$ . Les conditions de vérification sont :

$$\neg \psi_i \Rightarrow P_i \quad i = 1, \dots, 5$$

$$P_2 [x \leftarrow 1] \Rightarrow P_1$$

$$[(P_3 \wedge \text{true}) \vee (P_5 \wedge \neg \text{true})] \Rightarrow (P_2 \wedge P_4)$$

$$P_4 [x \leftarrow x + 1] \Rightarrow P_3$$

$$\phi \Rightarrow \neg P_1$$

Ces conditions de vérification sont trivialement satisfaites par

$$P_i = \neg \psi_i, \quad i = 1, \dots, 5.$$

□

#### 4.3.2.2.2 Programmes parallèles asynchrones

##### 4.3.2.2.2.1 Choix d'un langage pour exprimer les invariants locaux et sa sémantique

Pour exprimer une relation entre états courants et finaux, nous associons une relation à chaque point du programme qui ne soit pas dans une section critique. Dans le prélude et le postlude, c'est une relation entre états mémoire courant et final. A chaque point d'un processus c'est une relation entre les valeurs courantes des états de contrôle des autres processus, l'état mémoire courant et l'état mémoire final. Ainsi pour un programme

$$Ppa \equiv Ps \parallel Ppa_0 \parallel \dots \parallel Ppa_i \parallel \dots \parallel Ppa_{m-1} \parallel Ps' \quad (m > 1)$$

nous choisissons

$$\begin{aligned} \tilde{A}_s[Ppa] = \{ \tilde{I} : & [\exists \bar{L} \in C[Ps]. \tilde{I}(\bar{L}) \in (M[Ppa]^* \rightarrow \{\text{tt}, \text{ff}\})] \\ & \vee [\exists i \in m, L \in C[Ppa_i]. \\ & \quad \tilde{I}(i)(L) \in ((\prod_{j \in (m \setminus i)} C[Ppa_j] \times M[Ppa]^*) \rightarrow \{\text{tt}, \text{ff}\})] \\ & \vee [\exists \bar{L} \in C[Ps']. \tilde{I}(\bar{L}) \in (M[Ppa]^* \rightarrow \{\text{tt}, \text{ff}\})] \} \end{aligned}$$

la signification d'un tel vecteur  $\tilde{I}$  est définie formellement par la fonction sémantique  $\gamma[Ppa]$ . Nous avons  $\gamma[Ppa](\tilde{I}) = I$  où, par cas :

- $I(\langle L, M \rangle, \langle \bar{L}, \bar{M} \rangle) = \tilde{I}(L)(M, \bar{M})$  quand  $L \in (C[Ps] \cup C[Ps'])$  et  $Ppa \equiv \beta \bar{L}$ ;
- $I(\langle L_0, \dots, L_{m-1}, M \rangle, \langle \bar{L}, \bar{M} \rangle) = \bigwedge_{i \in m} \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M})$   
quand  $L_i \in C[Ppa_i], i \in m$  et  $Ppa \equiv \beta \bar{L}$ ;

$(\gamma[Ppa](\tilde{I}))(\Delta, \bar{\Delta})$  n'a pas besoin d'être définie quand  $\bar{\Delta}$  n'est pas un état final.

## 4.3.2.2.2 Construction de conditions de vérification correctes

Les conditions de finalisation et d'initialisation sont similaires à 4.3.2.2.1.2.1 et 4.3.2.2.1.2.3 respectivement et ne présentent pas de difficultés. Le point principal consiste à trouver  $\tilde{Cv}_i \llbracket Ppa \rrbracket$  tel que :

$$\tilde{Cv}_i \llbracket Ppa \rrbracket(s)(\tilde{I}) \Rightarrow Cv_i \llbracket Ppa \rrbracket(s)(\delta \llbracket Ppa \rrbracket(\tilde{I}))$$

où

$$\begin{aligned} Cv_i \llbracket Ppa \rrbracket(s)(\delta \llbracket Ppa \rrbracket(\tilde{I})) &= [\forall \Delta, \Delta', \bar{\Delta} \in S \llbracket Ppa \rrbracket, a \in A \llbracket Ppa \rrbracket. (t \llbracket Ppa \rrbracket_a(\Delta, \Delta') \wedge \delta \llbracket Ppa \rrbracket(\tilde{I})(\Delta', \bar{\Delta}) \wedge \delta(\bar{\Delta}) \Rightarrow \delta \llbracket Ppa \rrbracket(\tilde{I})(\Delta, \bar{\Delta})] \\ &= [\forall \Delta, \Delta' \in S \llbracket Ppa \rrbracket, \bar{M} \in M \llbracket Ppa \rrbracket, a \in A \llbracket Ppa \rrbracket. \\ &\quad (t \llbracket Ppa \rrbracket_a(\Delta, \Delta') \wedge \delta \llbracket Ppa \rrbracket(\tilde{I})(\Delta', \langle \bar{L}, \bar{M} \rangle) \wedge Ppa \equiv \beta \bar{L} :) \Rightarrow \delta \llbracket Ppa \rrbracket(\tilde{I})(\Delta, \langle \bar{L}, \bar{M} \rangle)] \end{aligned}$$

Nous décomposons cette condition de vérification en sous-cas selon la forme de  $s$  définie par  $S \llbracket Ppa \rrbracket$  :

Cas 1 :  $\Delta = \langle L, M \rangle$  où  $L \in C \llbracket Ps \rrbracket$  et  $M \in M \llbracket Ppa \rrbracket$

Cas 1.1 :  $\neg (Ps \equiv \beta L :)$

Ce cas a été traité en 4.3.2.2.1 pour les programmes séquentiels.

Cas 1.2 :  $(Ps \equiv \beta L :)$

D'après la définition 2.8.2.2.4 de  $t \llbracket Ppa \rrbracket_a$ ,  $\Delta'$  est nécessairement de la forme  $\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle$  où  $Ppa \equiv \beta L : \llbracket L_0 : \beta_0 \parallel \dots \parallel L_{m-1} : \beta_{m-1} \rrbracket \beta'$  et  $a = \beta$  de sorte que par définition de  $\delta \llbracket Ppa \rrbracket$ , la condition de vérification est dans ce cas

$$\bigwedge_{i \in n} \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M}) \Rightarrow \tilde{I}(L)(M, \bar{M})$$

Intuitivement, la conjonction des invariants d'entrée de chaque processus doit impliquer l'invariant de sortie du prélude.

Cas 2 :  $\Delta = \langle L, M \rangle$  où  $L \in \llbracket Ps \rrbracket$  et  $M \in M \llbracket Ppa \rrbracket$

Ce cas a été traité en 4.3.2.2.1 pour les programmes séquentiels.

Cas 3 :  $\Lambda = \langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle$

où  $Pp \equiv P_s \llbracket Pra_0 \parallel \dots \parallel Pra_{m-1} \rrbracket; Ps'$ ,  $L_i \in C \llbracket Pra_i \rrbracket$  et  $M \in M \llbracket Pp \rrbracket$

Suivant la forme possible de  $s'$ , nous distinguons deux cas :

Cas 3.1 :  $s' = \langle L', M' \rangle$  où  $L' \in (C \llbracket Ps' \rrbracket \vee C \llbracket Ps' \rrbracket)$  et  $M' \in (\mathcal{V} \rightarrow \mathcal{D})$

D'après la définition 3.8.2.2.4 de  $t \llbracket Pp \rrbracket_a$  nous avons nécessairement  $M = M'$  et  $Pp \equiv \beta \llbracket \beta_0 L_0; \dots; \beta_{m-1} L_{m-1}; \rrbracket; L'; \beta'$  et  $a = \beta'$  de sorte que par définition de  $\tilde{s} \llbracket Pp \rrbracket(\tilde{I})$ , la condition de vérification est dans ce cas

$$\tilde{I}(L')(M, \bar{M}) \Rightarrow \bigwedge_{i \in m} \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M})$$

Intuitivement, l'invariant d'entrée du postlude doit impliquer la conjonction des invariants de sortie de chaque processus.

Cas 3.2 :  $s' = \langle \langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle$  où  $L'_i \in C \llbracket Pra_i \rrbracket, i \in m$  et  $M' \in M \llbracket Pp \rrbracket$

Par définition de  $t \llbracket Pp \rrbracket_a$ , nous décomposons ce cas en deux sous-cas suivant que la transition de  $s$  à  $s'$  correspond ou non à l'exécution d'une section critique.

Cas 3.2.1 : Transition ne correspondant pas à une section critique

Par définition de  $t \llbracket Pp \rrbracket_a$  et  $\tilde{s} \llbracket Pp \rrbracket$ , la condition de vérification équivaut à :

$$\begin{aligned} & [ [\exists i \in m. Pp \equiv \beta \llbracket Pra_0 \parallel \dots \parallel Pra_i \parallel \dots \parallel Pra_{m-1} \rrbracket \beta' \wedge (\forall j \in (m \setminus i). L'_j = L_j) \\ & \quad \wedge t \llbracket Pra_i \rrbracket_a(\langle L_i, M \rangle, \langle L'_i, M' \rangle) \wedge \bigwedge_{j \in m} \tilde{I}(L'_j)(L'_0, \dots, L'_{j-1}, L'_{j+1}, \dots, L'_{m-1}, M', \bar{M}) ] \\ & \Rightarrow \bigwedge_{k \in m} \tilde{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M, \bar{M}) ] \end{aligned}$$

Cette condition se décompose en une conjonction de conditions de vérification correspondant aux processus  $Pra_i, i \in m$  :

$$\begin{aligned} & [ [ \text{cond} \llbracket Pra_i \rrbracket(L_i, L'_i)(M) \wedge \text{succ} \llbracket Pra_i \rrbracket(L_i)(M, M') \\ & \quad \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M}) \\ & \quad \wedge \bigwedge_{j \in (m \setminus i)} \tilde{I}(L'_j)(L'_0, \dots, L'_{j-1}, L'_{j+1}, \dots, L'_{m-1}, M', \bar{M}) ] \\ & \Rightarrow \bigwedge_{k \in m} \tilde{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M, \bar{M}) ] \end{aligned}$$

De nouveau, cette condition se décompose en sous-cas suivant  $k$  :

Cas 3.2.1.1 : Preuve séquentielle ( $k=i$ )

$$\begin{aligned} & [[\text{cond } \llbracket \text{Pra}_i \rrbracket(L_i, L'_i)(M) \wedge \text{succ } \llbracket \text{Pra}_i \rrbracket(L_i)(M, M') \\ & \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M}) \wedge \text{context}(i, i)] \\ & \Rightarrow \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M})] \end{aligned}$$

où

$$\text{context}(i, k) = \bigwedge_{j \in \{m-1, i, k\}} \tilde{I}(L_j)(L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{j-1}, L_{j+1}, \dots, L_{m-1}, M', \bar{M})$$

Cette condition de vérification correspond au cas des preuves séquentielles (cf. 4.3.2.2.1) excepté pour le terme context( $i, i$ ). Elle signifie que si l'invariant  $\tilde{I}(L'_i)$  est vrai après la transition du point  $L_i$  au point  $L'_i$ , alors l'invariant  $\tilde{I}(L_i)$  doit être vrai avant cette transition. De plus, le terme context( $i, i$ ) établit que nous pouvons utiliser dans la preuve toute l'information disponible sur les autres processus  $\text{Pra}_j, j \neq i$  avant la transition. Pour avoir des preuves séquentielles similaires dans les cas de programmes séquentiels ou parallèles, nous négligeons le terme context( $i, i$ ) et choisissons la condition de vérification plus forte :

$$\begin{aligned} & [[\text{cond } \llbracket \text{Pra}_i \rrbracket(L_i, L'_i)(M) \wedge \text{succ } \llbracket \text{Pra}_i \rrbracket(L_i)(M, M') \wedge \\ & \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M})] \\ & \Rightarrow \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M}) \end{aligned}$$

Notons que cette simplification est correcte puisque la condition de vérification ci-dessus implique l'originale. C'est également sémantiquement complet car, intuitivement, l'information donnée par context( $i, i$ ) peut si nécessaire être incorporée en  $\tilde{I}(L'_i)$  en chaque point  $L'_i$  de chaque processus  $\text{Pra}_i$ .

Cas 3.2.1.2 : Absence d'interférences ( $k \neq i$ )

Pour  $k \in (m \setminus i)$ , nous devons montrer :

$$\begin{aligned} & [[\text{cond } \llbracket \text{Pr}_{a_i} \rrbracket (L_i, L'_i)(M) \wedge \text{succ } \llbracket \text{Pr}_{a_i} \rrbracket (L_i)(M, M') \\ & \quad \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M}) \\ & \quad \wedge \tilde{I}(L_k)(L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M', \bar{M}) \wedge \text{context}(i, k)] \\ & \Rightarrow \tilde{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M, \bar{M})] \end{aligned}$$

Intuitivement, les invariants  $\tilde{I}(L_k)$  dans un processus  $\text{Pr}_{a_k}$  ne doivent pas être invalidés par l'exécution de commandes dans d'autres processus  $\text{Pr}_{a_i}$ ,  $i \neq k$ . Comme ci-dessus, il est correct (et complet) de négliger le terme context( $i, k$ ).

Nous devrions maintenant détailler ces conditions de vérification suivant la nature de la commande désignée par  $L_i$ . Ceci est simple et nous donnerons seulement les résultats en 4.3.2.2.2.4.

Cas 3.2.2 Transition correspondant à une section critique

Lorsque  $P \rightarrow q \equiv P_s \llbracket \text{Pr}_{a_0} \rrbracket \dots \llbracket \text{Pr}_{a_i} \rrbracket \dots \llbracket \text{Pr}_{a_{m-1}} \rrbracket ; P_{s'}$ ,  $\text{Pr}_{a_i} \equiv \beta_{L_i} : \{ P_s'' \}$ ;  $L'_i : P'$   
 $P_s'' \equiv \underline{L}_1 : \delta \bar{L}_2$ , nous devons, d'après la condition de vérification, montrer que :

$$\text{cs-proof} ( \vdash \llbracket P_s'' \rrbracket^* ( \langle \underline{L}_1, M \rangle, \langle \bar{L}_2, M' \rangle ) )$$

où

$$\text{cs-proof} (P) =$$

$$\begin{aligned} & [[ P \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M}) \wedge \\ & \quad \bigwedge_{j \in (m \setminus i)} \tilde{I}(L'_j)(L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{j-1}, L_{j+1}, \dots, L_{m-1}, M', \bar{M}) ] \\ & \Rightarrow \bigwedge_{k \in m} \tilde{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M, \bar{M}) ] \end{aligned}$$

Comme c'est le cas dans toutes les preuves d'invariance, il n'est pas toujours nécessaire de caractériser exactement la relation  $\vdash \llbracket P_s'' \rrbracket^* ( \langle \underline{L}_1, M \rangle, \langle \bar{L}_2, M' \rangle )$  entre les états d'entrée et de sortie de la section critique. Une approximation  $\Psi$  pourra être utilisée car la formule ci-dessus est égale à :

$$[\exists \psi \in (S[P_s''])^2 \rightarrow \{t, ff\}).$$

$$\begin{aligned} & t[P_s'']^*(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle) \Rightarrow \psi(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle) \\ & \wedge \text{cs-proof}(\psi(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle)) \end{aligned}$$

Puisque nous avons à montrer que  $\psi$  est invariant, nous pouvons appliquer le principe d'induction ( $-V'$ ). Nous obtenons :

$$[\exists \psi \in (S[P_s''])^2 \rightarrow \{t, ff\}).$$

$$[\exists J \in (S[P_s''])^2 \rightarrow \{t, ff\}).$$

$$(\forall M' \in (V \rightarrow \mathcal{D}). J(\langle \bar{L}_2, M' \rangle, \langle \bar{L}_2, M' \rangle))$$

$$\wedge (\forall L_1, L_2 \in C[P_s''], M_1, M_2, M' \in M[Ppa]).$$

$$(t[P_s''](\langle L_1, M_1 \rangle, \langle L_2, M_2 \rangle) \wedge J(\langle L_2, M_2 \rangle, \langle \bar{L}_2, M' \rangle)) \Rightarrow J(\langle L_1, M_1 \rangle, \langle \bar{L}_2, M' \rangle)$$

$$\wedge (\forall M, M' \in (V \rightarrow \mathcal{D}). J(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle) \Rightarrow \psi(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle))$$

$$\wedge [\text{cs-proof}(\psi(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle))]$$

Après simplification pour éliminer  $\psi$ , nous obtenons

$$[\exists J \in (S[P_s''])^2 \rightarrow \{t, ff\}).$$

$$\forall M \in (V \rightarrow \mathcal{D}). J(\langle \bar{L}_2, M \rangle, \langle \bar{L}_2, M \rangle)$$

$$\wedge \forall L_1, L_2 \in C[P_s''], M_1, M_2, M' \in M[Ppa].$$

$$(t[P_s''](\langle L_1, M_1 \rangle, \langle L_2, M_2 \rangle) \wedge J(\langle L_2, M_2 \rangle, \langle \bar{L}_2, M' \rangle)) \Rightarrow J(\langle L_1, M_1 \rangle, \langle \bar{L}_2, M' \rangle)$$

$$\wedge \text{cs-proof}(J(\langle L_1, M \rangle, \langle \bar{L}_2, M' \rangle))]$$

Si nous appliquons 4.3.2.2.1.2.1 et 4.3.2.2.1.2.2, c'est équivalent à

$$[\exists \tilde{J} \in \prod_{L \in C[P_s'']} (M[Ppa])^2 \rightarrow \{t, ff\}).$$

$$\forall M' \in (V \rightarrow \mathcal{D}). \tilde{J}(\bar{L}_2)(M', M')$$

$$\wedge \forall L, L' \in C[P_s''], M, M', M'' \in M[Ppa].$$

$$(\text{cond}[P_s''](L, L')(M) \wedge \text{succ}[P_s''](L)(M, M'') \wedge \tilde{J}(L')(M'', M')) \Rightarrow \tilde{J}(L)(M, M')$$

$$\wedge \text{cs-proof}(\tilde{J}(L_1)(M, M'))]$$

Ainsi la condition de vérification pour les sections critiques a été divisée en deux sous-problèmes. Premièrement, le corps  $P_s''$  de la section critique doit être traité indépendamment de son contexte de sorte à inventer en chaque point  $L$  une relation  $\tilde{J}(L)(M, M')$  entre

l'état mémoire courant  $M$  et l'état mémoire final  $M'$ , et à montrer que  $\tilde{I}$  est invariant en utilisant la méthode de preuve par induction en arrière récapitulée en 4.3.2.2.1.4. Puis faire la preuve cs-proof ( $\tilde{I}(L_1)(M, M')$ ) où la section critique est considérée comme atomique et sa sémantique définie par  $\tilde{I}(L_1)(M, M')$ . Comme pour les autres commandes, cette preuve peut se décomposer en :

- une preuve séquentielle (en omettant context( $i, i$ ))
 
$$[\tilde{I}(L_1)(M, M') \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M})]$$

$$\Rightarrow \tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M})$$
- une preuve d'absence d'interférences (en omettant context( $i, k$ ))
 
$$[\tilde{I}(L_1)(M, M') \wedge \tilde{I}(L'_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M', \bar{M}) \wedge$$

$$\tilde{I}(L_k)(L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M', \bar{M})]$$

$$\Rightarrow \tilde{I}(L_k)(L_0, \dots, L_{i-1}, L_{k+1}, \dots, L_{m-1}, M, \bar{M})$$

#### 4.3.2.2.3 Vérification de la complétude sémantique

Définissons  $\alpha[\text{Ppa}] \in (A_s[\text{Ppa}] \rightarrow A_s[\text{Ppa}])$  qui spécifie comment une hypothèse d'induction  $I \in A_s[\text{Ppa}]$  peut être codée par un vecteur d'invariants  $\tilde{I}(L)$  associés en chaque point  $L$  du programme  $\text{Ppa}$

$$\text{Ppa} \equiv \text{Ps} [\text{Pra}_0 \parallel \dots \parallel \text{Pra}_i \parallel \dots \parallel \text{Pra}_{m-1}]; \text{Ps}'$$

par cas :

$$- \alpha[\text{Ppa}](I)(L)(M, \bar{M}) = [\forall \bar{L} \in C[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{L} :) \Rightarrow I(\langle L, M \rangle, \langle \bar{L}, \bar{M} \rangle)]$$

quand  $L \in (C[\text{Ps}] \cup C[\text{Ps}'])$

$$- \forall i \in m, L \in C[\text{Pra}_i],$$

$$\alpha[\text{Ppa}](I)(i)(L)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M}) =$$

$$[\forall \bar{L} \in C[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{L} :) \Rightarrow I(\langle \langle L_0, \dots, L_{i-1}, L, L_{i+1}, \dots, L_{m-1} \rangle, M \rangle, \langle \bar{L}, \bar{M} \rangle)]$$

La preuve de complétude (sémantique) montre qu'il est complet d'omettre les termes context(i,i) dans la preuve séquentielle et context(i,k) dans la preuve d'absence d'interférences. Elle montre aussi que la preuve d'absence d'interférences peut être simplifiée en omettant le terme  $\tilde{I}(L_R)$  à gauche de l'implication. (Cependant, en pratique ce terme est utile puisque  $\tilde{I}(L'_i)$  pourra s'écrire plus simplement).

Nous devons montrer que :

$$\forall I \in \text{As}[\text{Ppa}]. \text{Cv}[\text{Ppa}](\varepsilon, \delta)(\psi)(I) \rightarrow \text{Cv}^{\sim}[\text{Ppa}](\varepsilon, \delta)(\psi)(\alpha[\text{Ppa}](I))$$

La preuve suit les cas considérés en 4.3.2.2.1.2. Nous traitons uniquement le cas 3.2.1 (puisque le cas 3.2.2 est similaire tandis que les cas restants se traitent comme en 4.3.2.2.1.2).

Le terme correspondant à  $\text{Cv}^{\sim}[\text{Ppa}](\varepsilon, \delta)(\psi)(\alpha[\text{Ppa}](I))$  dans le cas 3.2.1 est la conjonction d'une preuve séquentielle et d'une preuve d'absence d'interférences :

$$\begin{aligned} & [ [(\text{cond}[\text{Pra}_i](L_i, L'_i)(M) \wedge \text{succ}[\text{Pra}_i](L_i)(M, M') \wedge (\forall \bar{I} \in \text{C}[\text{Ppa}]. \\ & \quad (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{m-1} \rangle, M' \rangle, \langle \bar{L}, \bar{M} \rangle))) \\ & \Rightarrow (\forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \bar{L}, \bar{M} \rangle)) ] \\ & \wedge \\ & \bigwedge_{k \in (m+1)} [ (\text{cond}[\text{Pra}_i](L_i, L'_i)(M) \wedge \text{succ}[\text{Pra}_i](L_i)(M, M') \\ & \quad \wedge (\forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{m-1} \rangle, M' \rangle, \langle \bar{L}, \bar{M} \rangle))) \\ & \Rightarrow (\forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \bar{L}, \bar{M} \rangle)) ] ] \end{aligned}$$

qui est impliqué par :

$$\begin{aligned} & [ [(\text{cond}[\text{Pra}_i](L_i, L'_i)(M) \wedge \text{succ}[\text{Pra}_i](L_i)(M, M') \\ & \quad \wedge (\forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{m-1} \rangle, M' \rangle, \langle \bar{L}, \bar{M} \rangle))) \\ & \Rightarrow (\forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow I(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \bar{L}, \bar{M} \rangle)) ] \end{aligned}$$

qui est lui-même impliqué par

$$\begin{aligned} & [ \forall \bar{I} \in \text{C}[\text{Ppa}]. (\text{Ppa} \equiv \beta \bar{I} : ) \Rightarrow (\text{cond}[\text{Pra}_i](L_i, L'_i)(M) \wedge \text{succ}[\text{Pra}_i](L_i)(M, M') \wedge \\ & \quad I(\langle \langle L_0, \dots, L_{i-1}, L'_i, L_{i+1}, \dots, L_{m-1} \rangle, M' \rangle, \langle \bar{L}, \bar{M} \rangle)) ] \Rightarrow I(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \bar{L}, \bar{M} \rangle) ] \end{aligned}$$

qui est le terme de  $\text{Cv}[\text{Ppa}](\varepsilon, \delta)(\psi)(I)$  correspondant à la transition considérée au cas 3.2.1.

#### 4.3.2.2.4 Résumé des conditions de vérification pour la preuve de correction partielle de programmes parallèles asynchrones par induction en arrière

$$Ppa \equiv \alpha_0 l_0 : \beta_0 \parallel Ppa_0 \parallel \dots \parallel Ppa_{i-1} \parallel \alpha l_{ij} : \beta \parallel Ppa_{i+1} \parallel \dots \parallel Ppa_{m-1} \parallel ; \alpha_1 l_1 : \beta_1 ; \bar{l} :$$

Les invariants  $P_0(x, \bar{x})$  associé au point  $l_0$  du prélude et  $P_1(x, \bar{x})$  associé au point  $l_1$  du postlude relient la valeur courante  $x$  à la valeur finale  $\bar{x}$  des variables (quand l'exécution est en  $\bar{l}$ ). L'invariant de sortie  $P_{\bar{e}}(\bar{x}, \bar{x})$  porte sur la valeur finale  $\bar{x}$  des variables.

L'invariant  $P_{ij}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, x, \bar{x})$  associé au point  $j$  du processus  $i$  relie les états de contrôle  $c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}$  des autres processus, les valeurs courantes  $x$  et les valeurs finales  $\bar{x}$  des variables.

#### - Preuve séquentielle

Les conditions de vérification pour le prélude, le postlude et chaque processus  $Ppa_i$ ,  $i \in m$  sont les mêmes que pour les programmes séquentiels (cf. 4.3.2.2.1.4) plus

#### • Finalisation du parallélisme

$$\alpha \parallel \beta_0 l_0 : \parallel \dots \parallel \beta_{m-1} l_{m-1} : \parallel ; l_f : \beta$$

$$P_p(x, \bar{x}) \Rightarrow \bigwedge_{i \in m} P_i(l_0, \dots, l_{i-1}, l_{i+1}, \dots, l_{m-1}, x, \bar{x})$$

#### • Section critique

$$\begin{array}{l} \alpha l_{i1} : \delta \\ \quad l_{i2} : \beta \\ \quad \quad l_{i3} : \\ \quad \quad \quad \delta_i \\ \quad \quad \quad l_{i4} : \delta \end{array}$$

A chaque point  $l_{ij}$  du corps  $l_{i2} : \beta l_{i3}$ , un invariant  $P_{ij}(x, x')$  relie les valeurs courantes  $x$  en  $l_{ij}$  aux valeurs finales  $x'$  en  $l_{i3}$  des variables. Les conditions de vérification sont celles des programmes séquentiels (excepté pour l'initialisation) :

$$[P_{i2}(x, x') \wedge P_{i4}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, x', \bar{x})] \\ \Rightarrow P_{i4}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, x, \bar{x})$$

• Initialisation du parallélisme

$$\alpha l_d : \parallel l_0 : \alpha_0 \parallel \dots \parallel l_{m-1} : \alpha_{m-1} \parallel \beta$$

$$[\bigwedge_{i \in m} P_i(l_0, \dots, l_{i-1}, l_{i+1}, \dots, l_{m-1}, x, \bar{x})] \Rightarrow P_d(x, \bar{x})$$

- Preuve d'absence d'interférences

Pour tout point  $l_{kj}$  de tout processus  $\text{Proc}_k$ ,  $k \in (m \cup i)$

• Commande nulle

$$\alpha l_{i1} : \text{skip}; l_{i2} : \beta$$

$$(P_{i2}[c_k \leftarrow l_{kj}] \wedge P_{kj}[c_i \leftarrow l_{i2}]) \Rightarrow P_{kj}[c_i \leftarrow l_{i1}]$$

• Commande d'affectation

$$\alpha l_{i1} : v := E; l_{i2} : \beta$$

$$(P_{i2}[c_k \leftarrow l_{kj}, v \leftarrow E] \wedge P_{kj}[c_i \leftarrow l_{i2}, v \leftarrow E]) \Rightarrow P_{kj}[c_i \leftarrow l_{i1}]$$

$$\alpha l_{i1} : v := ?; l_{i2} : \beta$$

$$\forall m \in \mathbb{D}. (P_{i2}[c_k \leftarrow l_{kj}, v \leftarrow m] \wedge P_{kj}[c_i \leftarrow l_{i2}, v \leftarrow m]) \Rightarrow P_{kj}[c_i \leftarrow l_{i1}]$$

• Commande conditionnelle

$\alpha l_{i1}: \text{if } B \text{ then}$

$l_{i2}:$   
 $l_{i3}:$   $\beta$

else

$l_{i4}:$   $\gamma$   
 $l_{i5}:$

fi;

$l_{i6}:$   $\delta$

$$[(P_{i2}[c_R \leftarrow l_{Rj}] \wedge B \wedge P_{Rj}[c_i \leftarrow l_{i2}]) \vee (P_{i4}[c_R \leftarrow l_{Rj}] \wedge \neg B \wedge P_{Rj}[c_i \leftarrow l_{i4}])] \Rightarrow P_{Rj}[c_i \leftarrow l_{i1}]$$

$$(P_{i6}[c_R \leftarrow l_{Rj}] \wedge P_{Rj}[c_i \leftarrow l_{i6}]) \Rightarrow (P_{Rj}[c_i \leftarrow l_{i3}] \wedge P_{Rj}[c_i \leftarrow l_{i5}])$$

• Commande itérative

$\alpha l_{i1}: \text{while } B \text{ do}$

$l_{i2}:$   
 $l_{i3}:$   $\beta$

od;

$l_{i4}:$   $\gamma$

$$[(P_{i2}[c_R \leftarrow l_{Rj}] \wedge B \wedge P_{Rj}[c_i \leftarrow l_{i2}]) \vee (P_{i4}[c_R \leftarrow l_{Rj}] \wedge \neg B \wedge P_{Rj}[c_i \leftarrow l_{i4}])] \Rightarrow [P_{Rj}[c_i \leftarrow l_{i1}] \wedge P_{Rj}[c_i \leftarrow l_{i3}]]$$

• Section critique

$\alpha l_{i1}: \{$   
 $l_{i2}:$   
 $l_{i3}:$   $\beta$   
 $l_{i4}:$   $\gamma$   
 $\}$

$$[P_{i2}(x, x') \wedge P_{i4}(c_0, \dots, c_{R-1}, l_{Rj}, c_{R+1}, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, x', \bar{x}) \\ \wedge P_{Rj}(c_0, \dots, c_{R-1}, c_{R+1}, \dots, c_{i-1}, l_{i4}, c_{i+1}, \dots, c_{m-1}, x', \bar{x})]$$

$$\Rightarrow P_{Rj}(c_0, \dots, c_{R-1}, c_{R+1}, \dots, c_{i-1}, l_{i1}, c_{i+1}, \dots, c_{m-1}, x, \bar{x})$$

## 4.3.2.2.5 Exemples

Exemple 4.3.2.2.5-1

Un exemple très simple pris dans Owicki-Gries [76]. Nous devons montrer que le programme suivant :

```

0:  ||
    11:  ✕
        12:
            x := x+1;
        13:  ✕
    14:  ||
        21:  ✕
            22:
                x := x+1;
            23:  ✕
        24:  ||;
3:

```

est partiellement correct pour :

$$\phi(x) = (x = 0)$$

$$\psi(x, \bar{x}) = (\bar{x} = 2)$$

Les conditions de vérification sont les suivantes :

- Finalisation :

$$P_3(\bar{x}, \bar{x})$$

$$P_3(x, \bar{x}) \Rightarrow [P_{24}(14, x, \bar{x}) \wedge P_{14}(24, x, \bar{x})]$$

- Preuve séquentielle pour le processus 2 :

$$P_{23}(x', x')$$

$$P_{23}(x+1, x') \Rightarrow P_{22}(x, x')$$

$$[P_{22}(x, x') \wedge P_{24}(c_1, x', \bar{x})] \Rightarrow P_{21}(c_1, x, \bar{x})$$

- Absence d'interférences du processus 2 avec la preuve du processus 1 :

$$[P_{22}(x, x') \wedge P_{24}(11, x', \bar{x}) \wedge P_{11}(24, x', \bar{x})] \Rightarrow P_{11}(21, x, \bar{x})$$

$$[P_{22}(x, x') \wedge P_{24}(14, x', \bar{x}) \wedge P_{14}(24, x', \bar{x})] \Rightarrow P_{14}(21, x, \bar{x})$$

- De la même manière, nous avons une preuve séquentielle pour le processus 1 et une preuve d'absence d'interférences du processus 1 avec la preuve du processus 2.

- Initialisation :

$$[P_{11}(z_1, x, \bar{x}) \wedge P_{21}(11, x, \bar{x})] \Rightarrow P_0(x, \bar{x})$$

$$[\phi(x) \wedge P_0(x, \bar{x})] \Rightarrow \psi(x, \bar{x})$$

Esquisse de la preuve :

```

0: {x = x̄ - 2}
  II
    11: {(c₂ = 21 ∧ x = x̄ - 2) ∨ (c₂ = 24 ∧ x = x̄ - 1)}
        ↙
          12: {x = x' - 1}
            x := x + 1;
          13: {x = x'}
        ↘
    14: {(c₂ = 21 ∧ x = x̄ - 1) ∨ (c₂ = 24 ∧ x = x̄)}
  II
    21: {(c₁ = 11 ∧ x = x̄ - 2) ∨ (c₁ = 14 ∧ x = x̄ - 1)}
        ↙
          22: {x = x' - 1}
            x := x + 1;
          23: {x = x'}
        ↘
    24: {(c₁ = 11 ∧ x = x̄ - 1) ∨ (c₁ = 14 ∧ x = x̄)}
  II;
3: {x = x̄}
  
```

□

Exemple 4.3.2.2.5-2

Le programme parallèle asynchrone suivant calcule  $f = m!$  quand

$m > 1$ .

```

0:  n1:=1; n2:=n;
1:  [
      11: f1:=1;
      12: while (n1+2)<n2 do
      13:   n1:=n1+1;
      14:   f1:=f1*n1;
      15: od;
      ||
      21: f2:=n2;
      22: while (n1+2)<n2 do
      23:   n2:=n2-1;
      24:   f2:=f2*n2;
      25: od;
      26: od;
3:  ];
4:  if (n1+1)=n2 then f:=f1*f2; else f:=f1*f2*(n1+1); fi;

```

Les conditions de vérification sont les suivantes :

. Finalisation :

$$\begin{aligned}
 & \forall \bar{m}, \bar{m}_1, \bar{m}_2, \bar{f}_1, \bar{f}_2, \bar{f} . P_4 (\langle \bar{m}, \bar{m}_1, \bar{m}_2, \bar{f}_1, \bar{f}_2, \bar{f} \rangle, \langle \bar{m}, \bar{m}_1, \bar{m}_2, \bar{f}_1, \bar{f}_2, \bar{f} \rangle) \\
 & [(P_4 [f \leftarrow f_1 f_2] \wedge (m_1+1)=m_2) \vee (P_4 [f \leftarrow f_1 f_2 (m_1+1)] \wedge (m_1+1) \neq m_2)] \Rightarrow P_3 \\
 & P_3 \Rightarrow (P_{36} [c_1 \leftarrow 16] \wedge P_{16} [c_2 \leftarrow 26])
 \end{aligned}$$

. Preuve séquentielle du processus 2 :

$$\begin{aligned}
 & [(P_{23} \wedge (m_1+2) < m_2) \vee (P_{26} \wedge (m_1+2) \geq m_2)] \Rightarrow [P_{22} \wedge P_{25}] \\
 & P_{25} [f_2 \leftarrow f_2 \times m_2] \Rightarrow P_{24} \\
 & P_{24} [m_2 \leftarrow m_2 - 1] \Rightarrow P_{23} \\
 & P_{22} [f_2 \leftarrow m_2] \Rightarrow P_{21}
 \end{aligned}$$

. Preuve d'absence d'interférences du processus 2 avec la preuve du processus 1 :

Pour  $j=1, \dots, 6$

$$\begin{aligned}
 & [(P_{23} [c_1 \leftarrow 1j] \wedge (m_1+2) < m_2 \wedge P_{1j} [c_2 \leftarrow 23]) \vee (P_{26} [c_1 \leftarrow 1j] \wedge (m_1+2) \geq m_2 \wedge P_{1j} [c_2 \leftarrow 26])] \\
 & \Rightarrow (P_{1j} [c_2 \leftarrow 23] \wedge P_{1j} [c_2 \leftarrow 25]) \\
 & (P_{25} [c_1 \leftarrow 1j, f_2 \leftarrow f_2 \times m_2] \wedge P_{1j} [c_2 \leftarrow 25, f_2 \leftarrow f_2 \times m_2]) \Rightarrow P_{1j} [c_2 \leftarrow 24] \\
 & (P_{24} [c_1 \leftarrow 1j, m_2 \leftarrow m_2 - 1] \wedge P_{1j} [c_2 \leftarrow 24, m_2 \leftarrow m_2 - 1]) \Rightarrow P_{1j} [c_2 \leftarrow 23] \\
 & (P_{22} [c_1 \leftarrow 1j, f_2 \leftarrow m_2] \wedge P_{1j} [c_2 \leftarrow 23, f_2 \leftarrow m_2]) \Rightarrow P_{1j} [c_2 \leftarrow 21]
 \end{aligned}$$

. La preuve séquentielle du processus 1 et la preuve d'absence d'interférences du processus 1 avec la preuve du processus 2 sont similaires.

Initialisation :

$$(P_{11}[c_2 \leftarrow 21] \wedge P_{21}[c_1 \leftarrow 11]) \Rightarrow P_1$$

$$P_1[m_1 \leftarrow 1, m_2 \leftarrow m] \Rightarrow P_0$$

$$(m > 1 \wedge P_0) \Rightarrow (\bar{f} = \bar{m}! \wedge \bar{m} = m)$$

L'esquisse de la preuve est :

Nous posons  $\pi(a, b) = (a \leq b \rightarrow a \times (a-1) \times \dots \times b \mid 1)$  et

$$I = (m = \bar{m} \wedge [(\bar{m}_1 + 1 = m_2 \wedge \bar{f} = \bar{f}_1 \times \bar{f}_2) \vee (\bar{m}_1 + 1 \neq m_2 \wedge \bar{f} = \bar{f}_1 \times \bar{f}_2 \times (\bar{m}_1 + 1))])$$

$$0: \{(n > 1) \Rightarrow (\bar{f} = n! \wedge \bar{n} = n)\}$$

$$n_1 := 1; n_2 := n;$$

$$1: \{(n_1 < n_2) \Rightarrow (1 \leq \bar{n}_2 - \bar{n}_1 \leq 2 \wedge \bar{f}_1 = \pi(n_1 + 1, \bar{n}_1) \wedge \bar{f}_2 = \pi(\bar{n}_2, n_2) \wedge I)\}$$

||

$$11: \{[(c_2 \in \{21, 22\} \wedge n_1 < n_2) \vee (c_2 = 23 \wedge n_1 + 2 < n_2) \vee (c_2 \in \{24, 25\} \wedge n_1 + 1 < n_2) \vee (c_2 = 26 \wedge n_1 < n_2 \leq n_1 + 2)] \Rightarrow [\bar{f}_1 = \pi(n_1 + 1, \bar{n}_1) \wedge \sup(n_1, \bar{n}_2 - 2) \leq \bar{n}_1 < \bar{n}_2 \wedge I]\}$$

$$f_1 := 1;$$

$$12: \{[(c_2 \in \{21, 22\} \wedge n_1 < n_2) \vee (c_2 = 23 \wedge n_1 + 2 < n_2) \vee (c_2 \in \{24, 25\} \wedge n_1 + 1 < n_2) \vee (c_2 = 26 \wedge n_1 < n_2 \leq n_1 + 2)] \Rightarrow [\bar{f}_1 = f_1 \times \pi(n_1 + 1, \bar{n}_1) \wedge \sup(n_1, \bar{n}_2 - 2) \leq \bar{n}_1 < \bar{n}_2 \wedge I]\}$$

while  $(n_1 + 2) < n_2$  do

$$13: \{[(c_2 \in \{21, 22, 23\} \wedge n_1 + 2 < n_2) \vee (c_2 \in \{24, 25, 26\} \wedge n_1 + 1 < n_2)] \Rightarrow [\bar{f}_1 = f_1 \times \pi(n_1 + 1, \bar{n}_1) \wedge \sup(n_1 + 1, \bar{n}_2 - 2) \leq \bar{n}_1 < \bar{n}_2 \wedge I]\}$$

$$n_1 := n_1 + 1;$$

$$14: \{[(c_2 \in \{21, 22, 23\} \wedge n_1 + 1 < n_2) \vee (c_2 \in \{24, 25, 26\} \wedge n_1 < n_2)] \Rightarrow [\bar{f}_1 = f_1 \times \pi(n_1, \bar{n}_1) \wedge \sup(n_1, \bar{n}_2 - 2) \leq \bar{n}_1 < \bar{n}_2 \wedge I]\}$$

$$f_1 := f_1 * n_1;$$

$$15: \{[(c_2 \in \{21, 22, 23\} \wedge n_1 + 1 < n_2) \vee (c_2 \in \{24, 25, 26\} \wedge n_1 < n_2)] \Rightarrow [\bar{f}_1 = f_1 \times \pi(n_1 + 1, \bar{n}_1) \wedge \sup(n_1, \bar{n}_2 - 2) \leq \bar{n}_1 < \bar{n}_2 \wedge I]\}$$

od;

$$16: \{[1 \leq n_2 - n_1 \leq 2] \Rightarrow [\bar{n}_2 - 2 \leq n_1 = \bar{n}_1 < \bar{n}_2 \wedge f_1 = \bar{f}_1 \wedge I]\}$$

||

$$21: \{[(c_1 \in \{11, 12\} \wedge n_1 < n_2) \vee (c_1 = 13 \wedge n_1 + 2 < n_2) \vee (c_1 \in \{14, 15\} \wedge n_1 + 1 < n_2) \vee (c_1 = 16 \wedge n_1 < n_2 \leq n_1 + 2)] \Rightarrow [\bar{f}_2 = \pi(\bar{n}_2, n_2) \wedge \bar{n}_1 < \bar{n}_2 \leq \inf(\bar{n}_1 + 2, n_2) \wedge I]\}$$

$$f_2 := n_2;$$

$$22: \{[(c_1 \in \{11, 12\} \wedge n_1 < n_2) \vee (c_1 = 13 \wedge n_1 + 2 < n_2) \vee (c_1 \in \{14, 15\} \wedge n_1 + 1 < n_2) \vee (c_1 = 16 \wedge n_1 < n_2 \leq n_1 + 2)] \Rightarrow [\bar{f}_2 = \pi(\bar{n}_2, n_2 - 1) \times f_2 \wedge \bar{n}_1 < \bar{n}_2 \leq \inf(\bar{n}_1 + 2, n_2) \wedge I]\}$$

while  $(n_1 + 2) < n_2$  do

$$23: \{[(c_1 \in \{11, 12, 13\} \wedge n_1 + 2 < n_2) \vee (c_1 \in \{14, 15, 16\} \wedge n_1 + 1 < n_2)] \Rightarrow [\bar{f}_2 = \pi(\bar{n}_2, n_2 - 1) \times f_2 \wedge \bar{n}_1 < \bar{n}_2 \leq \inf(\bar{n}_1 + 2, n_2 - 1) \wedge I]\}$$

$$n_2 := n_2 - 1;$$

$$24: \{[(c_1 \in \{11, 12, 13\} \wedge n_1 + 1 < n_2) \vee (c_1 \in \{14, 15, 16\} \wedge n_1 < n_2)] \Rightarrow [\bar{f}_2 = \pi(\bar{n}_2, n_2) \times f_2 \wedge \bar{n}_1 < \bar{n}_2 \leq \inf(\bar{n}_1 + 2, n_2) \wedge I]\}$$

$$f_2 := f_2 * n_2;$$

$$25: \{[(c_1 \in \{11, 12, 13\} \wedge n_1 + 1 < n_2) \vee (c_1 \in \{14, 15, 16\} \wedge n_1 < n_2)] \Rightarrow [\bar{f}_2 = \pi(\bar{n}_2, n_2 - 1) \times f_2 \wedge \bar{n}_1 < \bar{n}_2 \leq \inf(\bar{n}_1 + 2, n_2) \wedge I]\}$$

od;

$$26: \{[1 \leq n_2 - n_1 \leq 2] \Rightarrow [\bar{f}_2 = f_2 \wedge \bar{n}_1 < \bar{n}_2 = n_2 \leq \bar{n}_1 + 2 \wedge I]\}$$

||;

$$3: \{n_1 = \bar{n}_1 \wedge n_2 = \bar{n}_2 \wedge f_1 = \bar{f}_1 \wedge f_2 = \bar{f}_2 \wedge I\}$$

$$\text{if } (n_1 + 1) = n_2 \text{ then } f := f_1 \times f_2; \text{ else } f := f_1 \times f_2 \times (n_1 + 1); \text{ fi};$$

$$4: \{n = \bar{n} \wedge n_1 = \bar{n}_1 \wedge n_2 = \bar{n}_2 \wedge f_1 = \bar{f}_1 \wedge f_2 = \bar{f}_2 \wedge f = \bar{f}\}$$



Remarque 4.3.2.2.5-3 (Un principe d'induction avant-arrière symétrique)

Les invariants associés en chaque point de chaque processus sont de la forme  $A \Rightarrow R$  où  $A$  dépend des valeurs courantes des états de contrôle des autres processus et des valeurs courantes des variables et  $R$  est une relation entre les valeurs courantes et finales des variables.  $A$  décrit ce qui a été accompli jusqu'ici et  $R$  décrit ce qui reste à faire. Ceci aurait été encore plus clair si nous avions utilisé des invariants redondants qui décrivent plus précisément le comportement du programme, par exemple

$$\begin{aligned}
 4: & \{ [m > 1 \wedge f = m!] \Rightarrow [m = \bar{m} \wedge m_1 = \bar{m}_1 \wedge m_2 = \bar{m}_2 \wedge f_1 = \bar{f}_1 \wedge f_2 = \bar{f}_2 \wedge f = \bar{f}] \} \\
 23: & \{ [ [(c_1 = 11 \wedge m_1 = 1 \wedge m_1 + 2 < m_2) \vee (c_1 \in \{12, 13\} \wedge m_1 \geq 1 \wedge f_1 = m_1! \wedge m_1 + 2 < m_2) \vee \\
 & (c_1 = 14 \wedge m_1 > 1 \wedge f_1 = (m_1 - 1)! \wedge m_1 + 1 < m_2) \vee (c_1 \in \{15, 16\} \wedge m_1 > 1 \wedge f_1 = m_1! \wedge m_1 + 1 < m_2) ] \\
 & \wedge [m_2 \leq m \wedge f_2 = \pi(m_2, m)] ] \Rightarrow [ \bar{f}_2 = \pi(\bar{m}_2, m_2 - 1) \times f_2 \wedge \bar{m}_1 < \bar{m}_2 \leq \inf(\bar{m}_1 + 2, m_2) \wedge \bar{f}_1 = \bar{f}_2 ] \}
 \end{aligned}$$

Cette possibilité offerte par l'induction en arrière devrait être contrastée avec l'induction en avant (Lamport [77], Owicki-Gries [76a] qui permet de spécifier ce qui a été fait (i.e.  $A$ ) mais non ce qui reste à faire (i.e.  $R$ ). Alors pour comprendre le programme, il faut inventer ce qui reste à faire à partir de ce qui a été fait et de la spécification de sortie.

Cette constatation pour le programme "factorielle" est en fait générale. La preuve est que  $J = (A \Rightarrow R)$  où  $A(\Delta) = [\exists \underline{\Delta} \in S[\llbracket P_r \rrbracket \cdot \varepsilon(\underline{\Delta}) \wedge t[\llbracket P_r \rrbracket^*(\underline{\Delta}, \Delta)]$  et  $R(\Delta, \bar{\Delta}) = [t[\llbracket P_r \rrbracket^*(\Delta, \bar{\Delta}) \wedge \delta(\bar{\Delta})]$ , est toujours un invariant pour l'induction positive en arrière ( $-V^{-1}$ ). Noter que nous aurions pu démontrer la complétude sémantique pour l'induction assertionnelle en avant (-i) en utilisant  $A$ . Alors en utilisant le principe d'induction en arrière ( $-V^{-1}$ ) on peut spécifier le maximum d'information  $A$  sur l'état courant du programme, qui pourrait se faire en utilisant le principe d'induction en avant (-i-), plus une certaine information  $R$  sur ce qui reste à faire. Ceci ne permet pas toutefois de spécifier de relation entre

l'état courant et l'état initial du programme (par exemple dans le programme 4.3.2.2.2.5-2, on peut exprimer que  $m = \bar{m}$  à la ligne 0: et donc que les valeurs initiales  $\underline{m}$  et finales  $\bar{m}$  de  $m$  sont égales, mais on ne peut pas exprimer que  $\underline{m} = m = \bar{m}$  en tout point du programme). Pour ce faire nous proposons d'utiliser un invariant  $K(\underline{s}, s, \bar{s})$  qui est la conjonction de l'invariant  $I(\underline{s}, s)$  utilisé dans le principe d'induction (-Y-) et de l'invariant  $J(s, \bar{s})$  utilisé dans le principe d'induction (-Y-'), ce qui conduit au principe d'induction suivant :

$$[\exists K \in (S^3 \rightarrow \{\text{tt}, \text{ff}\}) \cdot \forall \underline{s}, s, s', \bar{s} \in S, a \in A.$$

$$[E(\underline{s}) \wedge S(\underline{s})] \Rightarrow [K(\underline{s}, \underline{s}, \bar{s}) \wedge K(\underline{s}, \bar{s}, \bar{s})]$$

$$\wedge [E(\underline{s}) \wedge K(\underline{s}, s', \bar{s}) \wedge t_a(s', s) \wedge S(\bar{s})] \Leftrightarrow$$

$$[E(\underline{s}) \wedge t_a(s', s) \wedge K(\underline{s}, s, \bar{s}) \wedge S(\bar{s})]$$

$$\wedge [E(\underline{s}) \wedge (K(\underline{s}, \underline{s}, \bar{s}) \vee K(\underline{s}, \bar{s}, \bar{s})) \wedge S(\bar{s})] \Rightarrow \psi(\underline{s}, \bar{s})]$$

$$\Leftrightarrow$$

$$[\forall p \in \Sigma \langle S, A, t, \epsilon \rangle, i \in |p| \cdot (S(p_i) \Rightarrow \psi(p_0, p_i))]$$

(-Y'Y-')

Pour la preuve de correction, nous avons  $[\forall p \in \Sigma \langle S, A, t, \epsilon \rangle, i \in |p|, j \in i \cdot (S(p_i) \Rightarrow K(p_0, p_j, p_i))]$ . Pour la preuve de complétude sémantique, il suffit de choisir  $K(\underline{s}, s, \bar{s}) = [\exists p \in \Sigma \langle S, A, t, \epsilon \rangle, i \in |p|, j \in i \cdot p_0 = \underline{s} \wedge p_j = s \wedge p_i = \bar{s} \wedge S(\bar{s})]$ .

□

#### 4.3.2.2.3 Construction d'une méthode d'absence d'interblocages dans les programmes parallèles asynchrones par induction en arrière

La méthode de preuve développée au paragraphe 4.3.2.2.2 pour les programmes parallèles asynchrones se généralise sans difficultés pour les programmes parallèles synchrones définis en 2.8.5. Par exemple pour la sémantique libérale des sémaphores considérée en 2.8.5.3, nous obtenons les conditions de vérification suivantes :

##### - Preuve séquentielle

$$\bullet \alpha l_{i_1} : \#(se); l_{i_2} : \beta$$

$$(se > 0 \wedge P_{i_2}[se \leftarrow se-1]) \Rightarrow P_{i_1}$$

$$\bullet \alpha l_{i_1} : \#(se); l_{i_2} : \beta$$

$$P_{i_2}[se \leftarrow se+1] \Rightarrow P_{i_1}$$

##### - Preuve d'absence d'interférences

(pour chaque point  $l_{kj}$  de chaque processus  $Pr_{kj}$ ,  $k \in (m \cup i)$ ) :

$$\bullet \alpha l_{i_1} : \#(se); l_{i_2} : \beta$$

$$(se > 0 \wedge P_{i_2}[c_k \leftarrow l_{kj}, se \leftarrow se-1] \wedge P_{kj}[c_i \leftarrow l_{i_2}, se \leftarrow se-1]) \Rightarrow P_{kj}[c_i \leftarrow l_{i_2}]$$

$$\bullet \alpha l_{i_1} : \#(se); l_{i_2} : \beta$$

$$(P_{i_2}[c_k \leftarrow l_{kj}, se \leftarrow se+1] \wedge P_{kj}[c_i \leftarrow l_{i_2}, se \leftarrow se+1]) \Rightarrow P_{kj}[c_i \leftarrow l_{i_2}]$$

L'exécution d'un programme synchrone est globalement bloquée si les processus n'ont pas tous terminé et si tous les processus dont l'exécution n'est pas terminée sont en attente pour prendre un sémaphore. Plus formellement, si

$$Pps \equiv Ps \parallel Pr_0 \parallel \dots \parallel Pr_{m-1} \parallel Ps'$$

alors Pps est (globalement) bloqué dans l'état  $\Delta$  si et seulement si  $\beta[Pps](\Delta)$  est vrai, avec :

$$\beta[Pps](\Delta) = [\exists L_0 \in C[Proc_0], \dots, L_{m-1} \in C[Proc_{m-1}], M \in M[Pps]. \Delta = \langle L_0, \dots, L_{m-1}, M \rangle \wedge \\ [[\forall i \in m. (Proc_i = \alpha L_i : \neg (Se); \delta \wedge M(Se) \leq 0) \vee (Proc_i = \alpha L_i :)] \\ \wedge [\exists j \in m. \neg (Proc_j = \alpha L_j :)]]]$$

Un programme Pps est exempt d'interblocage si aucune exécution de Pps ne conduit à un état où Pps est bloqué, c'est-à-dire :

$$\forall p \in \Sigma \langle S[Pps], A[Pps], t[Pps], \varepsilon \rangle, i \in |p|. \neg \beta[Pps](p_i)$$

c'est une propriété d'invariance où  $\varepsilon(\bar{\Delta}) = tt$  et  $\psi(\underline{\Delta}, \bar{\Delta}) = \neg \beta[Pps](\bar{\Delta})$  ne dépend pas des états initiaux. D'après 4.2.1.3, l'absence d'interblocage peut être prouvée par induction en arrière, en utilisant le principe d'induction contrapositif  $(-i)$ .

Le choix d'un langage pour exprimer les invariants locaux et sa sémantique est similaire à celui de la correction partielle (cf. 4.3.2.2.1) excepté qu'au point  $L_i$  du processus  $i$ , nous avons un invariant local de la forme :

$$\tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M)$$

au lieu de

$$\tilde{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M, \bar{M})$$

puisque'il n'est plus nécessaire de relier les états courants et finaux.

La dérivation des conditions de vérification correspondant au pas d'induction de  $(-i)$  est la même que dans les paragraphes 4.3.2.2.2 et 4.3.2.2.3 excepté que les états mémoires finaux sont omis. Les cas qui restent sont :

## . Initialisation

$$[\forall \underline{A} \in S[Pps]. \varepsilon(\underline{A}) \Rightarrow \neg \gamma[Pps](\check{I})(\underline{A})]$$

$$= [\forall \underline{L} \in C[Pps], M \in (V \rightarrow X). (Pps \equiv \underline{L} : \alpha \wedge \phi(M)) \Rightarrow \neg \check{I}(\underline{L})(M)]$$

## . Finalisation

$$[\forall \bar{A} \in S[Pps]. \beta[Pps](\bar{A}) \Rightarrow \gamma[Pps](\check{I})(\bar{A})]$$

quand  $\bar{A}$  n'est pas de la forme  $\langle L_0, \dots, L_{m-1}, M \rangle$ ,  $\beta[Pps](\bar{A})$  est faux de sorte que la condition est trivialement vérifiée, sinon elle est équivalente à :

$$[\forall L_0 \in C[Pro_0], \dots, L_{m-1} \in C[Pro_{m-1}], M \in M[Pps].$$

$$[Pps \equiv Ps[Pro_0 \parallel \dots \parallel Pro_{m-1}], Ps' \wedge (\exists j \in m. \neg (Pro_j \equiv \alpha L_j :)) \wedge$$

$$(\forall i \in m. (Pro_i \equiv \alpha L_i :)) \vee (Pro_i \equiv \alpha L_i : p(se); \delta \wedge M(se) \leq 0)]$$

$$\Rightarrow \bigwedge_{k \in m} \check{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M)]$$

Informellement, pour tout tuple  $L_0, \dots, L_{m-1}$  d'étiquettes telles que

- $L_i$  désigne une commande  $L_i : p(se);$   
(auquel cas bloqué $[Pps](i, L_i)(M) = [M(se) \leq 0]$ )

ou bien

- $L_i$  désigne le point de sortie du processus  $Pro_i$   
(auquel cas bloqué $[Pps](i, L_i)(M) = \text{tt}$ )

et telles qu'elles ne désignent pas toutes des points de sortie, nous devons montrer que :

$$\bigwedge_{i \in m} \text{bloqué}[Pps](i, L_i)(M) \Rightarrow \bigwedge_{i \in m} \check{I}(L_i)(L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M)$$

Exemple 4.3.2.2.3-1

Considérons le programme très simple suivant :

```

0:  ||
    || 11: while true do 12:  $\phi(m)$ ; 13:  $\psi(m)$ ; 14: od; 15:
    || 21: while true do 22:  $\phi(m)$ ; 23:  $\psi(m)$ ; 24: od; 25:
3:  ||;

```

Les conditions de vérification sont :

- Initialisation

$$\phi(m) \Rightarrow \neg P_0(m)$$

- Induction

$$[P_{11}(21, m) \wedge P_{21}(11, m)] \Rightarrow P_0(m)$$

• Preuve séquentielle du processus 1 :

$$P_{12}(c_2, m) \Rightarrow [P_{11}(c_2, m) \wedge P_{14}(c_2, m)]$$

$$[m > 0 \wedge P_{13}(c_2, m-1)] \Rightarrow P_{12}(c_2, m)$$

$$P_{14}(c_2, m+1) \Rightarrow P_{13}(c_2, m)$$

• Absence d'interférences du processus 1 avec la preuve du processus 2 :

Pour  $j = 1, \dots, 5$

$$[P_{12}(2j, m) \wedge P_{2j}(12, m)] \Rightarrow [P_{2j}(11, m) \wedge P_{2j}(14, m)]$$

$$[m > 0 \wedge P_{13}(2j, m-1) \wedge P_{2j}(13, m-1)] \Rightarrow P_{2j}(12, m)$$

$$[P_{14}(2j, m+1) \wedge P_{2j}(14, m+1)] \Rightarrow P_{2j}(13, m)$$

• La preuve séquentielle du processus 2 et la preuve d'absence d'interférences du processus 2 avec la preuve du processus 1 est similaire.

$$P_3(m) \Rightarrow [P_{15}(25, m) \wedge P_{25}(15, m)]$$

- Finalisation

• Interblocage possible en 12 et 22 :

$$m \leq 0 \Rightarrow [P_{12}(22, m) \wedge P_{22}(12, m)]$$

. Interblocage possible en 12 et 25 :

$$m \leq 0 \Rightarrow [P_{12}(25, m) \wedge P_{25}(12, m)]$$

. Interblocage possible en 15 et 22 :

$$m \leq 0 \Rightarrow [P_{15}(22, m) \wedge P_{22}(15, m)]$$

L'esquisse de la preuve est :

0:  $\{m < 1\}$

[[

11:  $\{[m \leq 0 \wedge c_2 \in \{21, 22, 24\}] \vee [m < 0 \wedge c_2 = 23] \vee [c_2 = 25]\}$

while true do

12:  $\{[m \leq 0 \wedge c_2 \in \{21, 22, 24\}] \vee [m < 0 \wedge c_2 = 23] \vee [c_2 = 25]\}$

$\bar{p}(m);$

13:  $\{[m < 0 \wedge c_2 \in \{21, 22, 24\}] \vee [m < -1 \wedge c_2 = 23] \vee [c_2 = 25]\}$

$\bar{v}(m);$

14:  $\{[m \leq 0 \wedge c_2 \in \{21, 22, 24\}] \vee [m < 0 \wedge c_2 = 23] \vee [c_2 = 25]\}$

od;

15:  $\{\bar{tt}\}$

||

21:  $\{[m \leq 0 \wedge c_1 \in \{11, 12, 14\}] \vee [m < 0 \wedge c_1 = 13] \vee [c_1 = 15]\}$

while true do

22:  $\{[m \leq 0 \wedge c_1 \in \{11, 12, 14\}] \vee [m < 0 \wedge c_1 = 13] \vee [c_1 = 15]\}$

$\bar{p}(m);$

23:  $\{[m < 0 \wedge c_1 \in \{11, 12, 14\}] \vee [m < -1 \wedge c_1 = 13] \vee [c_1 = 15]\}$

$\bar{v}(m);$

24:  $\{[m \leq 0 \wedge c_1 \in \{11, 12, 14\}] \vee [m < 0 \wedge c_1 = 13] \vee [c_1 = 15]\}$

od;

25:  $\{\bar{tt}\}$

]],

3:  $\{\bar{tt}\}$

Informellement, en chaque point du programme nous donnons une condition sur  $m$  qui est nécessaire (mais peut-être pas suffisante) pour que le programme soit bloqué plus tard. Puisque cette condition n'est pas satisfaite par les états d'entrée quand  $m \geq 1$ , le programme ne peut pas être bloqué.

□

#### 4.3.2.2.4 Construction d'une méthode de preuve d'exclusion mutuelle dans les programmes parallèles asynchrones par induction en arrière

Deux sections d'un programme sont en exclusion mutuelle si elles ne contiennent pas de commandes qui peuvent s'exécuter en même temps. Supposons que  $cs_i$  (respectivement  $cs_j$ ) est un prédicat qui caractérise l'ensemble des étiquettes appartenant à la section critique du processus  $Pro_i$  (respectivement  $Pro_j$ ) du programme :

$$Pps \equiv Ps \llbracket Pro_0 \parallel \dots \parallel Pro_{m-1} \rrbracket; Ps'$$

Les sections critiques sont en exclusion mutuelle si et seulement si :

$$\forall p \in \Sigma \langle S \llbracket Pps \rrbracket, A \llbracket Pps \rrbracket, L \llbracket Pps \rrbracket, \varepsilon \rangle, k \in |p|. \text{me} \llbracket Pps \rrbracket(i, j)(cs_i, cs_j)(p_k)$$

où

$$\text{me} \llbracket Pps \rrbracket(i, j)(cs_i, cs_j)(\bar{A}) = [\forall L_0 \in C \llbracket Pro_0 \rrbracket, \dots, L_{m-1} \in C \llbracket Pro_{m-1} \rrbracket, M \in M \llbracket Pps \rrbracket.$$

$$\bar{A} = \langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle \Rightarrow \neg (cs_i(L_i) \wedge cs_j(L_j))]$$

L'exclusion mutuelle est une propriété d'invariance qui peut être démontrée en utilisant le principe d'induction contrapositif en arrière ( $\neg \bar{i}$ ). Les conditions de vérification sont alors similaires à celles pour l'absence d'interblocages, excepté pour la finalisation qui est :

- Finalisation

$$\begin{aligned} & [\neg \text{me} \llbracket Pps \rrbracket(i, j)(cs_i, cs_j)(\bar{A}) \Rightarrow \gamma \llbracket Pps \rrbracket(\bar{i})(\bar{A})] \\ & = [ (cs_i(L_i) \wedge cs_j(L_j)) \Rightarrow \bigwedge_{k \in m} \tilde{I}(L_k)(L_0, \dots, L_{k-1}, L_{k+1}, \dots, L_{m-1}, M) ] \end{aligned}$$

Informellement, pour toutes les étiquettes  $L_i$  de  $Pro_i$  telles que  $cs_i(L_i)$  et  $L_j$  de  $Pro_j$  telles que  $cs_j(L_j)$ ,

$$\tilde{I}(L_i)(c_0, \dots, c_{j-1}, L_j, c_{j+1}, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, M)$$

et

$$\tilde{I}(L_j)(c_0, \dots, c_{j-1}, c_{j+1}, \dots, c_{i-1}, L_i, c_{i+1}, \dots, c_{m-1}, M)$$

doivent être vrais. De plus, pour toutes les étiquettes  $L_k$  du processus  $k$ ,  $k \in (m \cup \{i, j\})$  nous devons avoir :

$$[cs_i(c_i) \wedge cs_j(c_j)] \Rightarrow \tilde{I}(L_k)(c_0, \dots, c_{k-1}, c_{k+1}, \dots, c_{m-1}, M)$$

### Exemple 4.3.2.2.4-1

Les points 13 et 23 du programme 4.3.2.2.3-1 sont en exclusion mutuelle quand  $m \leq 1$ . Les conditions de vérification sont celles de 4.3.2.2.3-1 excepté pour la finalisation qui est :

$$P_{13}(23, m)$$

$$P_{23}(13, m)$$

L'esquisse de la preuve est :

```

0: {m>1}
  [
    11: {[m>0 ∧ c2=23] ∨ [m>1 ∧ c2 ∈ {21, 22, 24}]}
        while true do
    12:   {[m>0 ∧ c2=23] ∨ [m>1 ∧ c2 ∈ {21, 22, 24}]}
        P(m);
    13:   {[c2=23] ∨ [m>0 ∧ c2 ∈ {21, 22, 24}]}
        V(m);
    14:   {[m>0 ∧ c2=23] ∨ [m>1 ∧ c2 ∈ {21, 22, 24}]}
        od;
    15: {PP}
  ]
  ||
    21: {[m>0 ∧ c1=13] ∨ [m>1 ∧ c1 ∈ {11, 12, 14}]}
        while true do
    22:   {[m>0 ∧ c1=13] ∨ [m>1 ∧ c1 ∈ {11, 12, 14}]}
        P(m);
    23:   {[c1=13] ∨ [m>0 ∧ c1 ∈ {11, 12, 14}]}
        V(m);
    24:   {[m>0 ∧ c1=13] ∨ [m>1 ∧ c1 ∈ {11, 12, 14}]}
        od;
    25: {PP}
  ];
3: {PP}

```



#### 4.3.2.2.5 Construction d'une méthode de preuve de non-termination de programmes parallèles par induction en arrière

Un programme  $P_{ps}$  ne se termine pas si et seulement si  $\forall p \in \Sigma \langle S[P_{ps}], A[P_{ps}], t[P_{ps}], \epsilon \rangle, i \in |p|. \psi(p_i)$

où

$$\psi(\bar{a}) = \neg [\exists \bar{L} \in C[P_{ps}], \bar{M} \in M[P_{ps}]. \bar{a} = \langle \bar{L}, \bar{M} \rangle \wedge P_{ps} \equiv \alpha \bar{L} : ]$$

C'est une propriété que nous pouvons montrer par induction en arrière en utilisant  $(\neg i)$ . Les conditions de vérification sont celles du paragraphe 4.3.2.2.4 excepté pour la finalisation qui est :

$$\forall \bar{M} \in M[P_{ps}]. \check{I}(\bar{L})(\bar{M}) \quad \text{où } P_{ps} \equiv \alpha \bar{L} :$$

#### Exemple 4.3.2.2.5-1

Les conditions de vérification sont celles de l'exemple 4.3.2.2.4-1 excepté pour la finalisation qui est :

$$P_3(m)$$

Esquisse de la preuve :

0: {ff}

||

11: {ff} while true do 12: {ff}  $p(m)$ ; 13: {ff}  $v(m)$ ; 14: {ff} od; 15: {tt}

||

21: {ff} while true do 22: {ff}  $p(m)$ ; 23: {ff}  $v(m)$ ; 24: {ff} od; 25: {tt}

||,

3: {tt}

□

#### 4.3.2.2.6 Conclusion sur la preuve de propriétés d'invariance de programmes par induction en arrière

La méthode de Morris-Wegbreit [77] dite "subgoal induction" n'a jamais remporté le succès qu'a eu la méthode de Floyd [67]. Diverses raisons ont été avancées dont certaines sont incorrectes (comme l'incomplétude sémantique Misra [78], cf. 4.3.2.2.1.3) ou superficielles (comme les limitations concernant les programmes qui ne se terminent pas Morris-Wegbreit [77], cf. 4.3.2.2.1.5). Dijkstra [82, p. 224-225] démontre un cas particulier du théorème 4.2.1.4<sup>22</sup> à propos d'un programme séquentiel consistant en une boucle "while" et conclut (à la page xiii) que "we can ignore subgoal induction because it is nothing but the Invariance Theorem in a complicated disguise". Cette conclusion porte sur l'équivalence des méthodes de preuve et reste donc valable pour la méthode de preuve de correction partielle que nous avons introduite (en généralisant la méthode de Morris-Wegbreit dite "subgoal induction") quand on la compare par exemple aux méthodes de preuve d'invariance en avant à la Lamport [7], Owicki-Gries [76a] (qui généralisent la méthode de Floyd [67]). Cependant comme nous l'avons remarqué en 4.3.2.2.2.5-3, l'invariant  $A$  associé en tout point d'un programme pour une méthode de preuve en avant peut être utilisé pour une méthode de preuve en arrière dans la forme  $A \Rightarrow R$ . Quand la preuve est utilisée comme commentaires, ceci est utile pour le lecteur du programme puisque  $A$  décrit ce qui a été fait jusqu'à ce point et  $R$  ce qui reste à faire.  $R$  doit être inventé par le lecteur du programme quand les méthodes de preuve en avant sont utilisées.

Ces arguments nous semblent en fait peu convaincants. La véritable raison de l'insuccès de l'induction en arrière nous semble être que pour les méthodes de preuve en avant, les mêmes invariants peuvent être utilisés pour la correction partielle, l'absence d'interblocages,

l'exclusion mutuelle, l'absence d'erreurs à l'exécution, la non-termination, etc. Ceci parce que dans l'induction en avant, le même principe d'induction  $(-I)$  peut être utilisé pour la preuve de toutes ces propriétés d'invariance. Ce qui n'est pas le cas pour les méthodes de preuve en arrière pour lesquelles nous devons utiliser deux principes d'induction  $(-I^{-1})$  et  $(-i)$  qui conduisent à des conditions de vérifications différentes.

Un compromis heureux pourrait consister à utiliser une combinaison des principes d'induction en avant et en arrière comme nous l'avons proposé en 4.3.2.2.5-3. Pour les programmes parallèles, par exemple, il faut souvent démontrer une propriété de la forme  $\forall p \in \Sigma\langle S, A, t, \varepsilon \rangle, i \in |p|. (\psi(p_i) \Rightarrow \psi(p_0, p_i))$  (comme la correction partielle) et une propriété de la forme  $\forall p \in \Sigma\langle S, A, t, \varepsilon \rangle, i \in |p|. \Gamma(p_i)$  (où  $\Gamma$  est une conjonction de conditions correspondant par exemple à l'absence d'interblocages globaux permanents, l'exclusion mutuelle de certaines sections critiques, etc.). On pourra alors choisir le principe d'induction suivant :

$$[\exists K \in (S^3 \rightarrow \{\text{tt}, \text{ff}\}) . \forall \underline{A}, \underline{A}', \underline{\bar{A}} \in S, a \in A$$

$$\begin{aligned} & [E(\underline{A}) \wedge \psi(\underline{\bar{A}})] \Rightarrow [K(\underline{A}, \underline{A}, \underline{\bar{A}}) \wedge K(\underline{A}, \underline{\bar{A}}, \underline{\bar{A}})] \\ & \wedge [E(\underline{A}) \wedge K(\underline{A}, \underline{A}, \underline{\bar{A}}) \wedge t_a(\underline{A}, \underline{A}') \wedge \psi(\underline{\bar{A}})] \Leftrightarrow \\ & \quad [E(\underline{A}) \wedge t_a(\underline{A}, \underline{A}') \wedge K(\underline{A}, \underline{A}', \underline{\bar{A}}) \wedge \psi(\underline{\bar{A}})] \\ & \wedge [E(\underline{A}) \wedge (K(\underline{A}, \underline{A}, \underline{\bar{A}}) \vee K(\underline{A}, \underline{\bar{A}}, \underline{\bar{A}})) \wedge \psi(\underline{\bar{A}})] \Rightarrow \psi(\underline{A}, \underline{\bar{A}}) \\ & \wedge [E(\underline{A}) \wedge K(\underline{A}, \underline{A}, \underline{\bar{A}})] \Rightarrow \Gamma(\underline{A}) ] \end{aligned}$$

$$\Leftrightarrow$$

$$[\forall p \in \Sigma\langle S, A, t, \varepsilon \rangle, i \in |p|. [\forall j \in i. \neg \psi(p_j) \wedge \psi(p_i)] \Rightarrow [\forall j \in i. \Gamma(p_j) \wedge \psi(p_0, p_i)]]$$

(La preuve de corréction consiste essentiellement à démontrer que  $\forall p \in \Sigma\langle S, A, t, \varepsilon \rangle, i \in |p|. [\forall j \in i. \neg \psi(p_j) \wedge \psi(p_i)] \Rightarrow [\forall j \in i. K(p_0, p_j, p_i)]$  et

la preuve de complétude se fait en choisissant  $\kappa(\Delta, \Delta, \bar{\Delta}) =$   
 $[\exists p \in \Sigma \langle S, A, t, \varepsilon \rangle, i \in |p|, j \leq i. p_0 = \Delta \wedge p_j = \Delta \wedge p_i = \bar{\Delta} \wedge \forall j \in i. \neg \sigma(p_j) \wedge \sigma(p_i)]$ .

Ce principe d'induction est implicitement utilisé dans les preuves informelles que donnent Ricart-Agrawala [81].

#### 4.3.2.3 Construction d'une méthode de preuve pour les programmes parallèles communicants

En appliquant la méthode de construction définie au paragraphe 4.3.1, nous avons proposé dans Cousot-Cousot [80a] une méthode de preuve pour un sous-ensemble de CSP dont l'originalité consistait à proposer une décomposition des preuves pour les programmes CSP en des preuves pour chaque processus (ne faisant référence qu'à l'état du processus), pour chaque canal (faisant référence à l'état de tous les processus au moment des communications (la référence aux seuls états des processus qui communiquent sur le canal étant incomplète)) et en des preuves d'absence d'interférence (entre les assertions associées aux canaux et l'exécution des processus). L'idée essentielle était de considérer qu'entre l'initialisation et l'attente du premier rendez-vous, ou entre deux attentes de rendez-vous ou entre l'attente du dernier rendez-vous et la terminaison, l'exécution d'un processus est indivisible. Cette idée n'est plus valable pour les programmes communicants considérés au paragraphe 2.8.3 pour la raison que les processus peuvent, entre deux communications par les canaux, interférer au moyen des variables globales partagées.

Soit

$$P_{pc} \equiv P_s \llbracket P_{rc_0} \parallel \dots \parallel P_{rc_{n-1}} \rrbracket; P_{s'}$$

Pour construire une méthode de preuve basée sur le principe d'induction (I), nous pouvons utiliser la décomposition définie par:

$$As \llbracket P_{pc} \rrbracket = (s \llbracket P_{pc} \rrbracket^2 \rightarrow \{tt, ff\})$$

$$\tilde{As} \llbracket P_{pc} \rrbracket = (\tilde{Pr\acute{e}l} \llbracket P_{pc} \rrbracket \cup \tilde{Pr\acute{o}c} \llbracket P_{pc} \rrbracket \cup \tilde{P\acute{o}stl} \llbracket P_{pc} \rrbracket)$$

Une relation reliant les états mémoire courants et initiaux est associée à chaque point du prélude et du postlude :

$$\text{Prél} \llbracket Ppc \rrbracket = \{ \tilde{I} : \exists \underline{L} \in C \llbracket Ps \rrbracket. \tilde{I}(\underline{L}) \in (M \llbracket Ppc \rrbracket^i \rightarrow \{tt, ff\}) \}$$

$$\text{Postl} \llbracket Ppc \rrbracket = \{ \tilde{I} : \exists \bar{L} \in C \llbracket Ps' \rrbracket. \tilde{I}(\bar{L}) \in (M \llbracket Ppc \rrbracket^i \rightarrow \{tt, ff\}) \}$$

Une relation reliant l'état mémoire et contrôle courant à l'état mémoire initial est associée à chaque point de chaque processus :

$$\text{Préc} \llbracket Ppc \rrbracket = \{ \tilde{I} : \exists i \in m, L \in C \llbracket Proc_i \rrbracket. \tilde{I}(i)(L) \in (M \llbracket Ppc \rrbracket \times (\prod_{j \in (m \setminus i)} C \llbracket Proc_j \rrbracket) \times M \llbracket Ppc \rrbracket \rightarrow \{tt, ff\}) \}$$

La signification d'un tel vecteur  $\tilde{I}$  est définie formellement par la fonction sémantique  $\gamma \llbracket Ppc \rrbracket(\tilde{I}) = I$ , où par cas :

$$I(\langle \underline{L}, \underline{M} \rangle, \langle L, M \rangle) = \tilde{I}(\underline{L})(\underline{M}, M) \quad \text{quand} \quad Ppc \equiv \underline{L} : \beta \wedge L \in (C \llbracket Ps \rrbracket \cup C \llbracket Ps' \rrbracket)$$

$$I(\langle \underline{L}, \underline{M} \rangle, \langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle) =$$

$$\bigwedge_{i \in m} \tilde{I}(L_i)(\underline{M}, L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, M) \quad \text{quand} \quad Ppc \equiv \underline{L} : \beta$$

Nous ne donnerons pas le détail de la construction des conditions de vérification correspondant à cette décomposition, ni la vérification de la complétude sémantique. En résumé, les conditions de vérification sont les suivantes (si on exclut les commandes de communication, ce sont celles proposées par Lamport [76a] et sont similaires aux conditions de Owicki-Gries [77] (sauf pour l'usage de variables auxiliaires qui est remplacé par celui des compteurs ordinaux). Avec les commandes de communication elles sont similaires à celles proposées par Levin [78] (les variables auxiliaires étant remplacés par des compteurs ordinaux)) :

Les invariants  $P_i(\underline{x}, x)$  associés aux points  $L_i$  des prélude et postlude relient l'état initial  $\underline{x}$  à l'état courant  $x$  des variables quand l'exécution est en  $L_i$ .

Les invariants  $P_{ij}(\underline{x}, c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, x)$  associés aux points  $L_{ij}$  du processus  $Proc_i$  relient l'état initial  $\underline{x}$  des variables à l'état

courant  $\alpha$  des variables et l'état  $c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}$  de contrôle des autres processus  $Proc_0, \dots, Proc_{i-1}, Proc_{i+1}, \dots, Proc_{m-1}$  quand l'exécution est au point  $L_{ij}$  du processus  $Proc_i$ .

### - Preuve séquentielle

(pour le prélude  $P_s$ , le postlude  $P_s'$  et chaque processus  $Proc_0, \dots, Proc_{m-1}$ )

#### . Initialisation (prélude)

$L_1: \alpha$

$\forall \bar{x}. P_1(\bar{x}, \bar{x})$

#### . Commande nulle

$\alpha L_1: \underline{skip}; L_2: \beta$

$P_1 \Rightarrow P_2$

#### . Commande d'affectation

$\alpha L_1: V := E; L_2: \beta$

$P_1 \Rightarrow P_2 [V \leftarrow E]$

$\alpha L_1: V := ?; L_2: \beta$

$\forall v \in \mathcal{D}. P_1 \Rightarrow P_2 [V \leftarrow v]$

#### . Commande conditionnelle

$\alpha L_1: \underline{if} B \underline{then}$

$L_2: \beta$

$[P_1 \wedge B] \Rightarrow P_2$

$L_3: \beta$

$[P_1 \wedge \neg B] \Rightarrow P_4$

else

$L_4: \gamma$

$[P_3 \vee P_5] \Rightarrow P_6$

$L_5: \gamma$

fi;

$L_6: \delta$

#### . Itération

$\alpha L_1: \underline{while} B \underline{do}$

$L_2: \beta$

$(P_1 \vee P_3) \Rightarrow [(B \Rightarrow P_2) \wedge (\neg B \Rightarrow P_4)]$

$L_3: \beta$

od;

$L_4: \delta$

. Section critique (dans le processus  $\text{Proc}_i$ )

$$\alpha L_{i_1} : \begin{array}{l} L_{i_2} : \\ L_{i_3} : \end{array} \beta \\ L_{i_4} : \gamma$$

A chaque point  $L_{ij}$  du corps  $L_{i_2} : \beta L_{i_3} :$  de la section critique, une relation invariante  $P_{ij}(x', x)$  relie les valeurs courantes  $x$  en  $L_{ij}$  aux valeurs initiales  $x'$  en  $L_{i_2}$  des variables dans la section critique. Les conditions de vérification correspondantes sont celles des programmes séquentiels avec  $\forall x'. P_{i_2}(x', x)$ . Il faut y ajouter

$$[P_{i_1}[x \leftarrow x'] \wedge P_{i_3}(x', x)] \Rightarrow P_{i_4}$$

. Sortie d'une commande alternative (dans le processus  $\text{Proc}_i$ )

$$\alpha \underline{ae} \beta_0; L_{i_0} : \underline{or} \dots \underline{or} \beta_{e-1}; L_{i_{e-1}} : \underline{es}; L_{i_e} : \beta$$

$$\forall j \in e. P_{i_j} \Rightarrow P_{i_e}$$

- Initialisation / Finalisation du parallélisme

$$\alpha L_m : [L_0 : \beta_0 \parallel \dots \parallel L_{m-1} : \beta_{m-1}] \beta$$

$$P_m(x, x) \Rightarrow [\bigwedge_{i \in m} P_i(x, L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, x)]$$

$$\alpha [\beta_0 L_0 : \parallel \dots \parallel \beta_{m-1} L_{m-1} :]; L_m : \beta$$

$$[\bigwedge_{i \in m} P_i(x, L_0, \dots, L_{i-1}, L_{i+1}, \dots, L_{m-1}, x)] \Rightarrow P_m(x, x)$$

### - Preuve de correction des communications

- $ch!E$  équivaut à se true;  $ch!E$  then skip; es se traite comme ci-dessous
- $ch?v$  équivaut à se true;  $ch?v$  then skip; es se traite comme ci-dessous
- Pour toutes paires de commandes alternatives dans des processus différents  $i$  et  $j$ ,

$$\alpha L_{i_1}: \underline{se} \dots \underline{or} B_1; ch!E \text{ then } L_{i_2}: \beta \text{ or } \dots \underline{es} \gamma$$

$$\alpha' L_{j_1}: \underline{se} \dots \underline{or} B_2; ch?v \text{ then } L_{j_2}: \beta' \text{ or } \dots \underline{es} \gamma'$$

$$(P_{i_1}[c_i \leftarrow L_{j_1}] \wedge B_1 \wedge P_{j_1}[c_i \leftarrow L_{i_1}] \wedge B_2)$$

$$\Rightarrow (P_{i_2}[c_j \leftarrow L_{j_2}, v \leftarrow E] \wedge P_{j_2}[c_i \leftarrow L_{i_2}, v \leftarrow E])$$

### - Preuve d'absence d'interférences

Pour tout point  $L_{k_e}$  de tout processus  $Proc_k$ ,  $k \in (m \cup i)$ ,

- Commande nulle

$$\alpha L_{i_1}: \underline{skip}; L_{i_2}: \beta$$

$$(P_{i_1}[c_k \leftarrow L_{k_e}] \wedge P_{k_e}[c_i \leftarrow L_{i_1}]) \Rightarrow P_{k_e}[c_i \leftarrow L_{i_2}]$$

- Commande d'affectation

$$\alpha L_{i_1}: v := E; L_{i_2}: \beta$$

$$(P_{i_1}[c_k \leftarrow L_{k_e}] \wedge P_{k_e}[c_i \leftarrow L_{i_1}]) \Rightarrow P_{k_e}[c_i \leftarrow L_{i_2}, v \leftarrow E]$$

$$\alpha L_{i_1}: v := ?; L_{i_2}: \beta$$

$$(P_{i_1}[c_k \leftarrow L_{k_e}] \wedge P_{k_e}[c_i \leftarrow L_{i_1}]) \Rightarrow (\forall v \in D. P_{k_e}[c_i \leftarrow L_{i_2}, v \leftarrow v])$$

## . Commande conditionnelle

 $\alpha L_{i_1}: \text{if } B \text{ then}$ 
 $L_{i_2}: \beta$ 
 $L_{i_3}:$ 
 $\text{else}$ 
 $L_{i_4}: \gamma$ 
 $L_{i_5}:$ 
 $\text{fi};$ 
 $L_{i_6}: \delta$ 

$$(P_{i_1}[c_R \leftarrow L_{R\ell}] \wedge P_{R\ell}[c_i \leftarrow L_{i_2}]) \Rightarrow [(B \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_2}]) \wedge (\neg B \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_4}])]$$

$$[(P_{i_1}[c_R \leftarrow L_{R\ell}] \wedge P_{R\ell}[c_i \leftarrow L_{i_3}]) \vee (P_{i_1}[c_R \leftarrow L_{R\ell}] \wedge P_{R\ell}[c_i \leftarrow L_{i_5}])] \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_6}]$$

## . Iteration

 $\alpha L_{i_1}: \text{while } B \text{ do}$ 
 $L_{i_2}: \beta$ 
 $L_{i_3}:$ 
 $\text{od};$ 
 $L_{i_4}: \gamma$ 

$$[(P_{i_1}[c_R \leftarrow L_{R\ell}] \wedge P_{R\ell}[c_i \leftarrow L_{i_2}]) \vee (P_{i_1}[c_R \leftarrow L_{R\ell}] \wedge P_{R\ell}[c_i \leftarrow L_{i_3}])]$$

$$\Rightarrow [(B \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_2}]) \wedge (\neg B \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_4}])]$$

## . Section critique

 $\alpha L_{i_1}: \{$ 
 $L_{i_2}: \beta$ 
 $L_{i_3}:$ 
 $\};$ 
 $L_{i_4}: \gamma$ 

$$[P_{i_1}[c_R \leftarrow L_{R\ell}, x \leftarrow x'] \wedge P_{R\ell}[c_i \leftarrow L_{i_2}, x \leftarrow x'] \wedge P_{i_3}(x', x)] \Rightarrow P_{R\ell}[c_i \leftarrow L_{i_4}]$$

. Communication ( $i \neq k, j \neq k$ )

$\alpha L_{i_1}; \underline{ae} \dots \underline{or} B_1; \text{ch!}E \text{ then } L_{i_2}; \beta \underline{or} \dots \underline{es} \delta$

$\alpha' L_{j_1}; \underline{ae} \dots \underline{or} B_2; \text{ch?}v \text{ then } L_{j_2}; \beta' \underline{or} \dots \underline{es} \delta'$

$[P_{i_1}[c_j \leftarrow L_{j_1}, c_k \leftarrow L_{k_1}] \wedge B_1 \wedge P_{j_2}[c_i \leftarrow L_{i_2}, c_k \leftarrow L_{k_2}] \wedge B_2 \wedge P_{k_1}[c_i \leftarrow L_{i_1}, c_j \leftarrow L_{j_1}]]$

$\Rightarrow P_{k_1}[c_i \leftarrow L_{i_2}, c_j \leftarrow L_{j_2}, v \leftarrow E]$

#### 4.3.2.4 Comparaison des méthodes de preuve pour les programmes parallèles connues dans la littérature

La comparaison des méthodes de preuve de propriétés d'invariance de programmes parallèles (comme Ashcroft [77], Hoare [75], Howard [76], Keller [76], Lamport [77], Mazurkiewicz [77], Newton [75], Owicki-Gries [76a, 76b], etc.) est souvent difficile à cause de la diversité des formalismes syntaxiques qui sont utilisés pour représenter les algorithmes. Nous proposons de les comparer en montrant que toutes ces méthodes dérivent du même principe d'induction  $(-I-)$  (ou ses variantes  $(-I')$ ,  $(-i)$ ,  $(i)$  ou leurs transformés par  $\nu$ ) et ne diffèrent que par la façon de décomposer l'invariant global utilisé dans  $(-I-)$  en invariants locaux associés à des points du programme.

##### 4.3.2.4.1 Utilisation d'un seul invariant global

Dans les méthodes de preuve d'invariance de Ashcroft [75] et Keller [76], un seul invariant est utilisé dans la preuve. Ashcroft justifie sa méthode de preuve par un théorème qui peut s'exprimer en utilisant notre formalisme comme suit :

$$[\forall \Delta, \Delta' \in S.$$

$$\varepsilon(\Delta) \Rightarrow \psi(\Delta)$$

$$\wedge [\exists \underline{\Delta} \in S. \varepsilon(\underline{\Delta}) \wedge t^*(\underline{\Delta}, \Delta) \wedge \psi(\Delta) \wedge (\exists q \in A. t_q(\Delta, \Delta'))] \Rightarrow \psi(\Delta')]$$

$$\Leftrightarrow$$

$$[\forall \Delta \in S. (\exists \underline{\Delta} \in S. \varepsilon(\underline{\Delta}) \wedge t^*(\underline{\Delta}, \Delta)) \Rightarrow \psi(\Delta)]$$

Puis Ashcroft remarque ensuite qu'on ne connaît pas exactement l'ensemble des états  $\Delta$  satisfaisant  $[\exists \underline{\Delta} \in S. \varepsilon(\underline{\Delta}) \wedge t^*(\underline{\Delta}, \Delta)]$  de sorte que "we could have left this term out of the verification condition entirely".

Il ajoute "however, if the impossibility of reaching certain states is crucial for certain properties of a program to hold" then we can "explicitly incorporate the impossibility into the assertions we wish to prove valid and check the above conditions for all states".

Autrement dit, on peut être amené à remplacer  $\psi$  par  $I$  tel que  $\forall s \in S. I(s) \Rightarrow \psi(s)$ . Il s'agit donc bien du principe d'induction (-i) dont la correction a été démontrée ultérieurement par Keller [76].

Dans les deux cas, on choisit l'ensemble des actions comme étant l'ensemble des affectations et tests de chaque processus du programme, de sorte que la méthode consiste à utiliser un seul invariant global et autant de conditions de vérification qu'il y a d'actions. Dans ce cas la méthode de preuve consiste à appliquer directement le principe d'induction. Cependant l'utilisation d'un seul invariant pour décrire le comportement d'un grand programme peut être inadéquate.

#### Exemple 4.3.2.4.1-1

Si on veut démontrer la correction partielle du programme  $\llbracket 11: x := x+1; 12: \parallel 21: x := x+2; 22: \rrbracket$  relativement à une spécification  $\phi, \psi$ , il faudra trouver un invariant  $I$  tel que :

$$\phi(x) \Rightarrow I(11, 21, x)$$

$$[I(11, c_2, x') \wedge x = x' + 1] \Rightarrow I(12, c_2, x)$$

$$[I(c_1, 21, x') \wedge x = x' + 2] \Rightarrow I(c_2, 22, x)$$

$$I(12, 22, x) \Rightarrow \psi(x)$$

□

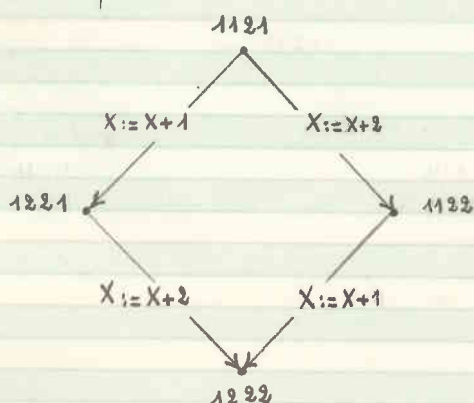
#### 4.3.2.4.2 Utilisation d'invariants sur les variables associés à chaque état de contrôle

Une des premières tentatives de décomposition de l'invariant global a été faite par Ashcroft-Manna [70]. Leur technique consiste à transformer un programme parallèle en un programme non déterministe équivalent puis à appliquer la méthode de Floyd-Hoare-Naur qui utilise un invariant local, portant sur les variables du programme, associé à chaque point du programme transformé.

Exemple 4.3.2.4.2-1

$[11: X := X+1; 12: \parallel 21: X := X+2; 22:]$

est transformé en :



auquel on applique la méthode de Floyd [67], ce qui donne :

$$\phi(x) \Rightarrow I_{1121}(x)$$

$$[I_{1121}(x') \wedge x = x' + 1] \Rightarrow I_{1221}(x)$$

$$[I_{1121}(x') \wedge x = x' + 2] \Rightarrow I_{1122}(x)$$

$$[I_{1221}(x') \wedge x = x' + 2] \Rightarrow I_{1222}(x)$$

$$[I_{1122}(x') \wedge x = x' + 1] \Rightarrow I_{1222}(x)$$

$$I_{1222}(x) \Rightarrow \psi(x)$$

□

Une autre façon d'expliquer la même idée qui évite d'avoir à transformer le programme consiste à dire que la méthode de Floyd [67] et Ashcroft-Manna [70] s'applique à des programmes dont l'ensemble des états est  $S = C \times M$  où  $C$  est l'ensemble des états de contrôle et  $M$  l'ensemble des états mémoires et consiste à appliquer le principe d'induction (-i) avec la décomposition :

$$A_\Delta = (C \times M \rightarrow \{\text{tt}, \text{ff}\})$$

$$A_\Delta^\sim = (C \rightarrow (M \rightarrow \{\text{tt}, \text{ff}\}))$$

$$\alpha \in (A_\Delta \rightarrow A_\Delta^\sim)$$

$$\alpha(I)_c(m) = I(\langle c, m \rangle)$$

$$\gamma \in (A_\Delta^\sim \rightarrow A_\Delta)$$

$$\gamma(\tilde{I})(\langle c, m \rangle) = \tilde{I}_c(m)$$

ce qui conduit évidemment à des méthodes correctes et sémantiquement complètes (cf. 4.3.1.7-1 et 4.3.1.8-2, les hypothèses de 4.3.1.6.2.7 étant satisfaites).

Si cette décomposition est appliquée à un programme parallèle  $\llbracket Pr_0 \rrbracket \dots \llbracket Pr_{m-1} \rrbracket$  où chaque processus  $Pr_i$  a  $m_i$  points de contrôle, il y a  $m_0 \times m_1 \times \dots \times m_{m-1}$  invariants locaux à considérer, ce qui conduit évidemment à des preuves très longues. Ceci peut être évité en utilisant des décompositions moins fines.

#### 4.3.2.4.3 Utilisation d'invariants sur les variables associés à chaque point de contrôle du programme

Considérons un programme dont l'ensemble des états est  $S = C_0 \times \dots \times C_{m-1} \times M$  où  $C_i, i \in m$  est l'ensemble des points de contrôle du  $i$ ème processus et  $M$  l'état mémoire.

On peut remplacer l'invariant global utilisé dans le principe d'induction (i) par des invariants locaux portant sur l'état mémoire et associés à chaque point de contrôle. Ceci revient à choisir (on suppose  $i \neq j \Rightarrow (C_i \cap C_j = \emptyset)$ ) :

$$A\Delta = (C_0 \times \dots \times C_{m-1} \times M \rightarrow \{\text{tt}, \text{ff}\})$$

$$\tilde{A}\Delta = \tilde{A}\Delta_0 \times \dots \times \tilde{A}\Delta_{m-1} \quad \text{où} \quad \tilde{A}\Delta_i = (C_i \rightarrow (M \rightarrow \{\text{tt}, \text{ff}\}))$$

$$\alpha \in (A\Delta \rightarrow \tilde{A}\Delta)$$

$$\alpha(I)_{i,c}(m) = (\exists c_0 \in C_0, \dots, c_{i-1} \in C_{i-1}, c_{i+1} \in C_{i+1}, \dots, c_{m-1} \in C_{m-1}.$$

$$I(\langle \langle c_0, \dots, c_{i-1}, c, c_{i+1}, \dots, c_{m-1} \rangle, m \rangle)$$

$$\gamma \in (\tilde{A}\Delta \rightarrow A\Delta)$$

$$\gamma(\tilde{I})(\langle \langle c_0, \dots, c_{m-1} \rangle, m \rangle) = [\forall i \in m. \tilde{I}_{i,c_i}(m)]$$

#### Exemple 4.3.2.4.3-1 Méthode de Owicki-Gries [76a]

Pour démontrer la correction partielle de :

$$\llbracket 11: X := X+1; \quad 12: \parallel \quad 21: X := X+2; \quad 22: \rrbracket$$

par la méthode de Owicki-Gries [76a], il faut vérifier les conditions suivantes :

$$\phi \Rightarrow [I_{11}(x) \wedge I_{21}(x)]$$

$$[I_{11}(x') \wedge x = x'+1] \Rightarrow I_{12}(x)$$

$$[I_{21}(x') \wedge x = x'+2] \Rightarrow I_{22}(x)$$

$$[I_{11}(x') \wedge I_{21}(x') \wedge x = x' + 2] \Rightarrow I_{11}(x)$$

$$[I_{12}(x') \wedge I_{21}(x') \wedge x = x' + 2] \Rightarrow I_{12}(x)$$

$$[I_{21}(x') \wedge I_{11}(x') \wedge x = x' + 1] \Rightarrow I_{21}(x)$$

$$[I_{22}(x') \wedge I_{11}(x') \wedge x = x' + 1] \Rightarrow I_{22}(x)$$

$$[I_{12}(x) \wedge I_{22}(x)] \Rightarrow \psi(x)$$

(ces conditions s'obtiennent à partir du principe d'induction (-i) en utilisant 4.3.1.7-1, ce point sera illustré au paragraphe suivant).

□

Keller [76] et Owicki-Gries [76a] ont montré que la méthode de preuve correspondante n'est pas complète en utilisant un argument intuitif basé sur un exemple. Ce raisonnement peut être fait plus rigoureusement en observant que partant du principe d'induction (-i) qui s'écrit :

$$[\exists I \in A\Delta. (f(I) \Rightarrow I) \wedge (I \Rightarrow \psi)]$$

où

$$f(I)(\Delta) = [\varepsilon(\Delta) \vee (\exists \Delta' \in S. I(\Delta') \wedge t(\Delta', \Delta))]$$

la méthode de preuve consiste à démontrer que

$$[\exists \tilde{I} \in \tilde{A}\Delta. \alpha \circ f \circ \chi(\tilde{I}) \Rightarrow \tilde{I} \wedge \tilde{I} \Rightarrow \alpha(\psi)]$$

où

$$\tilde{I} \Rightarrow \tilde{J} \text{ si et seulement si } \forall i \in m, c \in C_i, m \in M. \tilde{I}_{i,c}(m) \Rightarrow \tilde{J}_{i,c}(m)$$

Posons  $\tilde{f} = \alpha \circ f \circ \chi$ , c'est un opérateur monotone (pour  $\Rightarrow$ ) sur le treillis complet  $\tilde{A}\Delta$ . D'après le théorème de Tarski (Tarski [55], Cousot-Cousot [79]) le plus fort invariant  $\tilde{I}$  satisfaisant  $\tilde{f}(\tilde{I}) \Rightarrow \tilde{I}$  est le plus petit point fixe  $\text{lfp}(\tilde{f})$  de  $\tilde{f}$  (tel que  $\tilde{f}(\text{lfp}(\tilde{f})) = \text{lfp}(\tilde{f})$  et si  $\tilde{f}(\tilde{I}) \Rightarrow \tilde{I}$  alors  $\text{lfp}(\tilde{f}) \Rightarrow \tilde{I}$ ). Si on peut trouver un programme et un  $\psi$  tels que  $\text{lfp}(\tilde{f}) \not\Rightarrow \alpha(\psi)$ , alors la méthode est incomplète. Il suffit de choisir le programme

0:  $x := 0$ ; 1:  $\llbracket 11: x := x + 1; 12: \parallel 21: x := x + 1; 22: \rrbracket$

pour lequel  $\text{ffp}(\tilde{f})$  est :

$$I_0(x) = \text{true}$$

$$I_1(x) = [x=0]$$

$$I_{11}(x) = [x \geq 0] \quad I_{21}(x) = [x \geq 0]$$

$$I_{12}(x) = [x > 0] \quad I_{22}(x) = [x > 0]$$

qui ne permet pas de démontrer l'invariance de  $\psi(x) = [0 \leq x \leq 2]$ .

Remarquons que si  $m=1$ , la situation est celle de 4.3.2.4.2 et la méthode est sémantiquement complète.

Quand  $m > 1$ , la méthode est incomplète car il n'est pas possible d'exprimer dans  $\tilde{A}^s$  que certains états sont inaccessibles (dans le contre-exemple, on ne peut pas exprimer que lorsque l'exécution est en 11 avec  $x=1$  dans le processus 1, alors c'est qu'elle est en 22 dans le processus 2). Pour ce faire, il faut pouvoir exprimer des relations entre les états de contrôle des divers processus. On peut utiliser à cet effet des compteurs ordinaux (comme dans Lamport[77]) ou des variables auxiliaires (comme dans Owicki-Gries[76a,76b], Levin[79]).

#### 4.3.2.4.4 Utilisation d'invariants sur l'état de contrôle et les variables associés à chaque point de contrôle du programme

Nous pouvons extraire de Newton[75] l'idée exprimée plus clairement dans Lamport[77] qui consiste à associer à chaque point de contrôle du programme une assertion portant sur l'état mémoire et l'état de contrôle du programme.

Nous avons donc :

$$S = (C_0 \times \dots \times C_{m-1}) \times M$$

$$A_S = (S \rightarrow \{\text{tt}, \text{ff}\})$$

$$A_S^\vee = A_{S_0}^\vee \times \dots \times A_{S_{m-1}}^\vee \quad \text{où} \quad A_{S_i}^\vee = (C_i \rightarrow (C_0 \times \dots \times C_{i-1} \times C_{i+1} \times \dots \times C_{m-1} \times M \rightarrow \{\text{tt}, \text{ff}\}))$$

$$\alpha \in (A_S \rightarrow A_S^\vee)$$

$$\alpha(I)_{i,c} (c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m) = I(\langle \langle c_0, \dots, c_{i-1}, c, c_{i+1}, \dots, c_{m-1} \rangle, m \rangle)$$

$$\gamma \in (A_S^\vee \rightarrow A_S)$$

$$\gamma(\tilde{I})(\langle \langle c_0, \dots, c_{m-1} \rangle, m \rangle) = [\forall i \in m. \tilde{I}_{i,c_i} (c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m)]$$

Comme  $(\alpha, \gamma)$  est une correspondance de Galois injective, nous pouvons construire une méthode de preuve correcte et sémantiquement complète comme en 4.3.1.4-1 et 4.3.1.8-2.

#### Exemple 4.3.2.4.4-1

Considérons l'application du principe d'induction (-i) aux programmes parallèles asynchrones de la forme :

$$Ppq \equiv \llbracket Pra_0 \parallel \dots \parallel Pra_{m-1} \rrbracket$$

(cf. 2.8.2, les prélude et postlude étant vides et les affectations aléatoires et sections critiques étant exclues pour alléger la présentation) en utilisant la décomposition de l'invariant global en invariants locaux définie ci-dessus.

Le principe d'induction (-i) s'écrit :

$$[\exists I \in A_S. \varepsilon \Rightarrow I \wedge F(I) \Rightarrow I \wedge I \Rightarrow \psi]$$

où

$$\varepsilon(\langle \langle c_0, \dots, c_{m-1} \rangle, m \rangle) = [\forall i \in m. \exists \underline{c}_i \in \mathcal{C}. c_i = \underline{c}_i \wedge Pra_i \equiv \underline{c}_i : \beta]$$

$$F(I)(A) = [\exists \lambda' \in S, i \in m. I(\lambda') \wedge t \llbracket Ppq \rrbracket_i (\lambda', \lambda)]$$

$$t \llbracket Ppq \rrbracket_i (\langle \langle c'_0, \dots, c'_{m-1} \rangle, m' \rangle, \langle \langle c_0, \dots, c_{m-1} \rangle, m \rangle) =$$

$$[\forall j \in (m \vee i). c'_j = c_j \wedge t \llbracket Pra_i \rrbracket (\langle c'_i, m' \rangle, \langle c_i, m \rangle)]$$

Comme en 4.3.1.7-1, posons  $\tilde{F} = \alpha \circ F \circ \gamma$ . Nous avons

$$\begin{aligned}
 & \tilde{F}(\tilde{I})_{i,c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m) \\
 &= [\exists \lambda' \in S, k \in M. \gamma(\tilde{I})(\lambda') \wedge t[\text{Pra}]_k(\lambda', \langle c_0, \dots, c_{m-1} \rangle, m)] \\
 &= [\exists c'_0 \in C_0, \dots, c'_{m-1} \in C_{m-1}, m' \in M, k \in M. \forall j \in M. \tilde{I}_{j,c'_j}(c'_0, \dots, c'_{j-1}, c'_{j+1}, \dots, c'_{m-1}, m) \\
 &\quad \wedge \forall j \in (m \vee k). c'_j = c_j \wedge t[\text{Pra}_k](\langle c'_k, m' \rangle, \langle c_k, m \rangle)] \\
 &= [\exists c'_i \in C_i, m' \in M. \tilde{I}_{i,c'_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m') \\
 &\quad \wedge \text{contexte}\{i\}(\tilde{I})(c_0, \dots, c_{i-1}, c'_i, c_{i+1}, \dots, c_{m-1}, m') \wedge t[\text{Pra}_i](\langle c'_i, m' \rangle, \langle c_i, m \rangle)] \\
 &\vee [\exists k \in (m \vee i), c'_k \in C_k, m' \in M. \tilde{I}_{i,c'_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{k-1}, c'_k, c_{k+1}, \dots, c_{m-1}, m') \\
 &\quad \wedge \tilde{I}_{k,c'_k}(c_0, \dots, c_{k-1}, c_{k+1}, \dots, c_{m-1}, m') \wedge \text{contexte}\{i, k\}(\tilde{I})(c_0, \dots, c_{k-1}, c'_k, c_{k+1}, \dots, c_{m-1}, m') \\
 &\quad \wedge t[\text{Pra}_k](\langle c'_k, m' \rangle, \langle c_k, m \rangle)]
 \end{aligned}$$

ou

$$\text{contexte}\{i\}(\tilde{I})(c_0, \dots, c_{m-1}, m) = \forall j \in (m \vee i). \tilde{I}_{j,c_j}(c_0, \dots, c_{j-1}, c_{j+1}, \dots, c_{m-1}, m)$$

Le premier terme correspond à la preuve séquentielle (du processus  $i$ ), le deuxième terme correspond à la preuve d'absence d'interférences (entre  $i$  et tout  $k \neq i$ ), le terme de contexte apportant l'information disponible dans les autres processus n'apparaissant pas dans l'import [77].

Remarquons que le terme correspondant à l'absence d'interférences disparaît quand  $m=1$  et que dans ce cas les conditions de vérification correspondent exactement dans tous les cas à celle de la méthode de Floyd [67].

Nous illustrons la condition de vérification :

$$\forall i \in M, c_i \in C_i.$$

$$\tilde{F}(\tilde{I})_{i,c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m) \Rightarrow \tilde{I}_{i,c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m)$$

sur l'exemple suivant :

I

$$11: \{ \tilde{I}_{11}(c_2, x) \}$$

$$X := X+1;$$

$$12: \{ \tilde{I}_{12}(c_2, x) \}$$

II

$$21: \{ \tilde{I}_{21}(c_1, x) \}$$

$$X := X+2;$$

$$22: \{ \tilde{I}_{22}(c_1, x) \}$$

I

ce qui donne :

- Preuve séquentielle :

$$[ \tilde{I}_{11}(c_2, x') \wedge (c_2 = 21 \Rightarrow \tilde{I}_{21}(11, x')) \wedge (c_2 = 22 \Rightarrow \tilde{I}_{22}(11, x')) \wedge x = x'+1 ] \Rightarrow \tilde{I}_{12}(c_2, x)$$

$$[ \tilde{I}_{21}(c_1, x') \wedge (c_1 = 11 \Rightarrow \tilde{I}_{11}(21, x')) \wedge (c_1 = 12 \Rightarrow \tilde{I}_{12}(21, x')) \wedge x = x'+2 ] \Rightarrow \tilde{I}_{22}(c_1, x)$$

- Absence d'interférences :

$$[ \tilde{I}_{11}(21, x') \wedge \tilde{I}_{21}(11, x') \wedge x = x'+2 ] \Rightarrow \tilde{I}_{11}(22, x)$$

$$[ \tilde{I}_{12}(21, x') \wedge \tilde{I}_{21}(12, x') \wedge x = x'+2 ] \Rightarrow \tilde{I}_{12}(22, x)$$

$$[ \tilde{I}_{21}(11, x') \wedge \tilde{I}_{11}(21, x') \wedge x = x'+1 ] \Rightarrow \tilde{I}_{21}(12, x)$$

$$[ \tilde{I}_{22}(11, x') \wedge \tilde{I}_{11}(22, x') \wedge x = x'+1 ] \Rightarrow \tilde{I}_{22}(12, x)$$

Nous définissons maintenant  $\tilde{F} \in (A^{\tilde{\Delta}} \rightarrow A^{\tilde{\Delta}})$  par :

$$\tilde{F}(\tilde{I})_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m)$$

$$= [ \exists c'_i \in C_i, m' \in M. \tilde{I}_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m') \wedge \\ \vdash \llbracket \text{Pr}_{c_i} \rrbracket (\langle c'_i, m' \rangle, \langle c_i, m \rangle) ]$$

$$\vee [ \exists k \in (m \vee i), c'_k \in C_k, m' \in M. \tilde{I}_{k, c'_k}(c_0, \dots, c_{k-1}, c_{k+1}, \dots, c_{m-1}, m') \wedge \\ \vdash \llbracket \text{Pr}_k \rrbracket (\langle c'_k, m' \rangle, \langle c_k, m \rangle) ]$$

Nous observons alors que  $\alpha \circ F = \tilde{F} \circ \alpha$ . Les conditions de vérification correspondant à  $\tilde{F}(\tilde{I}) \Rightarrow \tilde{I}$  sont similaires à celles introduites par Newton [75] (bien que la similitude soit difficile à saisir car Newton utilise une définition des programmes parallèles un peu singulière).

sur notre exemple, nous obtenons :

- Preuve séquentielle :

$$[\tilde{I}_{11}(c_2, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{12}(c_2, x)$$

$$[\tilde{I}_{21}(c_1, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{22}(c_1, x)$$

- Absence d'interférences :

$$[\tilde{I}_{21}(11, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{11}(22, x)$$

$$[\tilde{I}_{21}(12, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{12}(22, x)$$

$$[\tilde{I}_{11}(21, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{21}(12, x)$$

$$[\tilde{I}_{11}(22, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{22}(12, x)$$

Comme  $\tilde{F} \neq \tilde{F}$ , nous obtenons un treillis complet de conditions de vérification  $\bar{F}(\tilde{I}) \Rightarrow \tilde{I}$  où  $\tilde{F} \Rightarrow \bar{F} \Rightarrow \tilde{F}$  correspondant chacune à une méthode de preuve particulière. Par exemple, la méthode de Lamport [77] (aussi bien que celle de Owicki-Gries [76a] si on avait utilisé la définition de  $\alpha$  et  $\gamma$  donnée en 4.3.2.4.3) correspond à :

$$\begin{aligned} \bar{F}(\tilde{I})_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{n-1}, m) \\ = [\exists c'_i \in C_i, m' \in M. \tilde{I}_{i, c'_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{n-1}, m') \wedge t[\text{Prq}_i](\langle c'_i, m' \rangle, \langle c_i, m \rangle)] \\ \vee [\exists R \in (M \times i), c'_R \in C_R, m' \in M. \tilde{I}_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{R-1}, c'_R, c_{R+1}, \dots, c_{n-1}, m') \wedge \\ \tilde{I}_{R, c'_R}(c_0, \dots, c_{R-1}, c_{R+1}, \dots, c_{n-1}, m') \wedge t[\text{Prq}_R](\langle c'_R, m' \rangle, \langle c_R, m \rangle)] \end{aligned}$$

D'après 4.3.1.7-1 la méthode de Lamport [77] est donc correcte et d'après 4.3.1.8-2, elle est sémantiquement complète.

Pour notre exemple, nous obtenons :

- Preuve séquentielle :

$$[\tilde{I}_{11}(c_2, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{12}(c_2, x)$$

$$[\tilde{I}_{21}(c_1, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{22}(c_1, x)$$

- Absence d'interférences

$$[\tilde{I}_{11}(21, x') \wedge \tilde{I}_{21}(11, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{11}(22, x)$$

$$[\tilde{I}_{12}(21, x') \wedge \tilde{I}_{21}(12, x') \wedge x = x' + 2] \Rightarrow \tilde{I}_{12}(22, x)$$

$$[\tilde{I}_{21}(11, x') \wedge \tilde{I}_{11}(21, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{21}(12, x)$$

$$[\tilde{I}_{12}(11, x') \wedge \tilde{I}_{11}(22, x') \wedge x = x' + 1] \Rightarrow \tilde{I}_{12}(12, x)$$

Observons que bien que toutes les méthodes dérivées d'un  $F$  tel que  $\tilde{F} \Rightarrow F \Rightarrow \tilde{F}$  soient sémantiquement complètes, il se peut que pour certains programmes, on puisse montrer que des assertions sont invariantes en utilisant  $\tilde{F}$  et qu'on ne puisse pas le faire en utilisant  $\tilde{F}$ . C'est le cas dans notre exemple pour :

$$\tilde{I}_{11}(c_2, x) = [\text{pair}(x)]$$

$$\tilde{I}_{21}(c_1, x) = [x = 0 \vee x = 1]$$

$$\tilde{I}_{12}(c_2, x) = [x = 1 \vee x = 3]$$

$$\tilde{I}_{22}(c_1, x) = [x = 2 \vee x = 3]$$

Toutefois, on peut toujours renforcer l'invariant et remplacer

$$\tilde{I}_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{n-1}, m)$$

par

$$\tilde{I}_{i, c_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{n-1}, m) \wedge \text{contexte } \{i\}(\tilde{I})(c_0, \dots, c_{n-1}, m)$$

donc toutes les conditions de vérification  $F(\tilde{I}) \Rightarrow \tilde{I}$  sont fortement équivalentes.

□

#### Exemple 4.3.2.4.4-2

Nous démontrons la correction partielle du programme 2.8.2.3, mais au lieu d'utiliser le principe d'induction (i), nous utilisons (-1-), de manière à pouvoir relier la valeur courante  $n$  de la variable  $N$  à sa valeur initiale  $\underline{n}$ .

Puisque les deux processus sont symétriques, nous n'avons besoin de raisonner que sur le processus 1. Nous allons montrer que la relation :

$$\text{Inv}(\underline{n}, c_2, m, p_1, p_2) = [(c_2 = 21 \wedge p_1 = 2^{n-m}) \vee (c_2 \neq 21 \wedge p_1 \times p_2 = 2^{n-m})]$$

est invariante dans le processus 1 après initialisation de la variable  $p_1$ . Puisque la correction partielle découle de l'invariant quand  $c_2 = 25$  (le processus 2 a terminé) et  $0 \leq m \leq 1$ , nous allons aussi montrer que la valeur de  $N$  après l'exécution de la commande parallèle est soit 0 ou 1. Puisque la valeur initiale  $\underline{n}$  de  $N$  est supposée positive, la seule difficulté est quand  $\underline{n} > 1$ . Dans ce cas  $N$  est décrémente jusqu'à atteindre la valeur 2. D'une part, chacun des deux processus peut tester que  $N > 1$  avant qu'il ne soit décrémente par l'autre, le décrémente et terminer. Dans ce cas  $N$  vaudra 0 après l'exécution de la commande parallèle. D'autre part, un processus peut tester si  $N > 1$  et le décrémente avant que l'autre ne teste si  $N > 1$ . Alors les deux processus terminent et  $N = 1$  après exécution de la commande parallèle. Pour une preuve d'invariance, ces arguments opérationnels peuvent s'exprimer "indépendamment du temps", d'où les invariants locaux suivants :

$$\tilde{I}_{11}(\underline{n}, c_2, m, p_1, p_2) = \tilde{I}_{12}(\underline{n}, c_2, m, p_1, p_2)$$

$$\tilde{I}_{12}(\underline{n}, c_2, m, p_1, p_2) = [\text{Inv}(\underline{n}, c_2, m, p_1, p_2) \wedge [(c_2 \in \{21, 22\} \wedge m = \underline{n} \wedge m > 0) \vee (c_2 = 23 \wedge m > 1) \vee (c_2 = 24 \wedge m > 1) \vee (c_2 = 25 \wedge 0 \leq m \leq 1)]]$$

$$\tilde{I}_{13}(\underline{n}, c_2, m, p_1, p_2) = [\text{Inv}(\underline{n}, c_2, m, p_1, p_2) \wedge [(c_2 \in \{21, 22, 23\} \wedge m > 1) \vee (c_2 = 24 \wedge m > 1) \vee (c_2 = 25 \wedge m = 1)]]$$

$$\tilde{I}_{14}(\underline{n}, c_2, m, p_1, p_2) = [\text{Inv}(\underline{n}, c_2, m, p_1, p_2) \wedge [(c_2 \in \{21, 22, 23\} \wedge m > 0) \vee (c_2 = 24 \wedge m > 0) \vee (c_2 = 25 \wedge 0 \leq m \leq 1)]]$$

$$\tilde{I}_{15}(\underline{n}, c_2, m, p_1, p_2) = [\text{Inv}(\underline{n}, c_2, m, p_1, p_2) \wedge [(c_2 = 23 \wedge m = 1) \vee (c_2 \neq 23 \wedge 0 \leq m \leq 1)]]$$

$$\tilde{I}_1(\underline{n}, m, p_1, p_2) = [p_1 \times p_2 = 2^{n-m} \wedge 0 \leq m \leq 1]$$

$$\tilde{I}_2(\underline{n}, p) = [p = 2^n]$$

C'est simple de montrer que ces invariants locaux satisfont les conditions de vérification suivantes :

- Initialisation :

$$[m \geq 0] \Rightarrow [\tilde{I}_{11}(m, 21, m, p_1, p_2) \wedge \tilde{I}_{21}(m, 11, m, p_2, p_1)]$$

- Preuve séquentielle :

$$\tilde{I}_{11}(m, c_2, m, p'_1, p_2) \Rightarrow \tilde{I}_{12}(m, c_2, m, 1, p_2)$$

$$[\tilde{I}_{12}(m, c_2, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}_{13}(m, c_2, m, p_1, p_2)$$

$$[\tilde{I}_{12}(m, c_2, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}_{15}(m, c_2, m, p_1, p_2)$$

$$\tilde{I}_{13}(m, c_2, m', p'_1, p_2) \Rightarrow \tilde{I}_{14}(m, c_2, m'-1, 2p'_1, p_2)$$

$$[\tilde{I}_{14}(m, c_2, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}_{13}(m, c_2, m, p_1, p_2)$$

$$[\tilde{I}_{14}(m, c_2, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}_{15}(m, c_2, m, p_1, p_2)$$

- Preuve d'absence d'interférences :

Pour  $k=1, \dots, 5$

$$[\tilde{I}_{1k}(m, 21, m, p_1, p'_2) \wedge \tilde{I}_{21}(m, 1k, m, p_1, p'_2)] \Rightarrow \tilde{I}_{1k}(m, 22, m, p_1, 1)$$

$$[\tilde{I}_{1k}(m, 22, m, p_1, p_2) \wedge \tilde{I}_{22}(m, 1k, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}_{1k}(m, 23, m, p_1, p_2)$$

$$[\tilde{I}_{1k}(m, 22, m, p_1, p_2) \wedge \tilde{I}_{22}(m, 1k, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}_{1k}(m, 25, m, p_1, p_2)$$

$$[\tilde{I}_{1k}(m, 23, m', p_1, p'_2) \wedge \tilde{I}_{23}(m, 1k, m', p_1, p'_2)] \Rightarrow \tilde{I}_{1k}(m, 24, m'-1, p_1, 2 \times p'_2)$$

$$[\tilde{I}_{1k}(m, 24, m, p_1, p_2) \wedge \tilde{I}_{24}(m, 1k, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}_{1k}(m, 23, m, p_1, p_2)$$

$$[\tilde{I}_{1k}(m, 24, m, p_1, p_2) \wedge \tilde{I}_{24}(m, 1k, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}_{1k}(m, 25, m, p_1, p_2)$$

- Finalisation :

$$[\tilde{I}_{15}(m, 25, m, p_1, p_2) \wedge \tilde{I}_{25}(m, 15, m, p_2, p_1)] \Rightarrow \tilde{I}_1(m, m, p_1, p_2)$$

$$[\tilde{I}_1(m, m, p_1, p_2) \wedge m=0] \Rightarrow \tilde{I}_2(m, p_1 \times p_2)$$

$$[\tilde{I}_1(m, m, p_1, p_2) \wedge m \neq 0] \Rightarrow \tilde{I}_2(m, 2 \times p_1 \times p_2)$$

$$\tilde{I}_2(m, p) \Rightarrow [p = 2^m]$$

Noter que pour tout le programme, nous avons obtenu 47 conditions de vérification, ce qui est évidemment énorme mais peut être réduit considérablement en choisissant une décomposition moins fine (cf. 4.3.2.4.6-3).

□

#### 4.3.2.4.5 Utilisation d'invariants sur les variables et des variables auxiliaires associés à chaque point de contrôle du programme

Après s'être rendus compte que l'utilisation d'invariants sur les variables associés à chaque point de contrôle du programme (comme en 4.3.2.4.3 qui leur semblait la généralisation la plus naturelle de la méthode de Floyd [67] au cas des programmes parallèles) était incomplète, Owicki-Gries [76a] ont introduit la technique des variables auxiliaires. Nous montrons que ces variables auxiliaires ne sont qu'un moyen de raisonner implicitement sur les états de contrôle du programme.

Pour présenter la technique, nous considérons un programme asynchrone  $Pp \equiv \llbracket Pp_0 \rrbracket \dots \llbracket Pp_{m-1} \rrbracket$  dont, pour simplifier, le prélude et le postlude sont supposés vides. L'ensemble des états du programme est donc  $S = (C_0 \times \dots \times C_{m-1}) \times M$  où  $C_i$  est l'ensemble des points de contrôle du processus  $Pp_i$  et  $M$  l'ensemble des états mémoire. Notons  $L_i$  le point d'entrée du processus  $Pp_i$  (de sorte que  $Pp_i \equiv L_i : \alpha$ ) et  $C = \bigcup_{i=0}^{m-1} C_i$  l'ensemble des points de contrôle du programme.

En chaque point  $L \in C$  du programme  $Pp$  est associé un commentaire qui est une relation  $\psi_L \in (M^2 \rightarrow \{\text{tt}, \text{ff}\})$  ne portant que sur les états mémoire et qui s'interprète comme signifiant que :

$$\psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$$

$$\psi(\langle \langle L_0, \dots, L_{m-1} \rangle, m \rangle, \langle \langle L'_0, \dots, L'_{m-1} \rangle, m' \rangle) = \left[ \bigwedge_{i=0}^{m-1} (L_i = L'_i \wedge \psi_{L_i}(m, m')) \right]$$

est invariante. Remarquons que  $\psi$  permet de décrire les états de contrôle du programme mais sous une forme extrêmement limitée (puisque par exemple, il est possible d'exprimer qu'un point  $L$  du programme est inaccessible en choisissant  $\psi_L(m, m) = \text{false}$  mais il n'est pas possible d'exprimer de relations entre les états de contrôle des

différents processus du programme, comme par exemple que les points  $L_i$  et  $L_j$  sont mutuellement exclusifs).

Pour démontrer l'invariance de  $\Psi$  pour  $P_p$ , la technique de Owicki-Gries [76a] consiste à considérer un programme transformé  $P_{p'}$  de la forme  $\llbracket P_{p'_0} \parallel \dots \parallel P_{p'_{m-1}} \rrbracket$  où chaque processus  $P_{p'_i}$  a essentiellement le même comportement que  $P_{p_i}$  mais contient des commandes d'affectation à des variables auxiliaires n'apparaissant pas dans  $P_p$  et des invariants  $\psi'_L$  (portant sur les variables de  $P_p$  mais également sur les variables auxiliaires) qui impliquent les  $\psi_L$ .

Plus précisément, l'ensemble  $VA$  des variables auxiliaires du programme  $P_{p'}$  est le plus grand ensemble de variables qui n'apparaissent qu'en membres gauches d'affectations ou en membres droits d'affectations à des variables auxiliaires.

$P_p$  s'obtient à partir de  $P_{p'}$  en supprimant les affectations aux variables auxiliaires c'est-à-dire en répétant un nombre fini de fois la transformation  $\alpha_L: V:=E; \beta \rightarrow \alpha \beta$  où  $V \in VA$  (puis en effectuant s'il y a lieu certaines simplifications évidentes comme par exemple,  $1: \& 2: V:=E; 3: \& 4:$  est équivalent à  $1: V:=E; 4:$  ou  $1: \underline{\text{skip}}; 2: \underline{\text{skip}}; 3:$  est équivalent à  $1: \underline{\text{skip}}; 3:$ ).

L'ensemble des états mémoire de  $P_{p'}$  peut donc être compris (à un isomorphisme près) comme étant  $M \times M'$  où  $M' = (VA \rightarrow \mathbb{D})$ . Comme précédemment, notons  $C'_i$  l'ensemble des points de contrôle du processus  $P_{p'_i}$ , i.e.m.,  $L'_i$  le point d'entrée du processus  $P_{p'_i}$  et  $C' = \bigcup_{i \in m} C'_i$ .

En chaque point  $L' \in C'$  du programme  $P_{p'}$  est associé un commentaire qui est une relation  $\psi'_L \in ((M \times M')^2 \rightarrow \{\text{tt}, \text{ff}\})$  ne portant que sur les variables de  $P_p$  et les variables auxiliaires et qui s'interprète comme précédemment par l'invariance de  $\Psi'$  telle que :

$$\Psi'(\langle L_0, \dots, L_{m-1} \rangle, m, \langle L'_0, \dots, L'_{m-1} \rangle, m') = \bigwedge_{i=0}^{m-1} (L_i = L'_i \wedge \psi'_{L'_i}(m, m'))$$

Les  $\psi'_L$  doivent être choisis de sorte que pour tous  $\underline{m}, \underline{m}', m, m' \in \mathcal{D}$ , on ait :

$$\psi'_L(\langle \underline{m}, \underline{m}' \rangle, \langle m, m' \rangle) \Rightarrow \psi_L(\underline{m}, m)$$

#### 4.3.2.4.5.1 Correction de la méthode

Observons que la sémantique du programme  $P_{pa}$  s'obtient à partir de la sémantique du programme auxiliaire  $P_{pa'}$  par élimination des états inobservables (appartenant à  $((C_0 \times \dots \times C_{m-1}) \vee (C_0 \times \dots \times C_{n-1})) \times M \times M'$ ) puis par correspondance à une fonction  $f_\Delta \in ((C_0 \times \dots \times C_{m-1}) \times M \times M' \rightarrow (C_0 \times \dots \times C_{m-1}) \times M)$  entre états près (définie par  $f_\Delta(\langle \langle C_0, \dots, C_{m-1} \rangle, m, m' \rangle) = \langle \langle C_0, \dots, C_{m-1} \rangle, m \rangle$ ). La preuve formelle, simple mais fastidieuse, repose sur le lemme 2.5.4 v1.2. De plus, on a  $\Psi'(\Delta, \Delta) \Rightarrow \Psi(f_\Delta(\Delta), f_\Delta(\Delta))$  quand  $\Delta \in ((C_0 \times \dots \times C_{m-1}) \times M \times M')$ .

Ceci permet de conclure que l'utilisation de variables auxiliaires est correcte dans la mesure où une propriété invariante pour le programme auxiliaire l'est également pour le programme original obtenu par élimination des variables auxiliaires. C'est la conséquence du théorème 4.1.1 v5 et du théorème 4.1.2 v1 (qui s'applique au cas où les états initiaux des programmes  $P_{pa}$  et  $P_{pa'}$  sont identiques et se généralise aisément).

Remarquons que, d'après le théorème 4.1.1 v5, la méthode de Owicki-Gries se généralise de manière évidente aux preuves d'invariance conditionnelle (dans la mesure où en pratique  $\phi$  ne porterait pas sur les variables auxiliaires).

De la même façon, on constate que la limitation de l'usage des variables auxiliaires aux affectations dans méthode de Owicki-Gries est inutile. Par exemple, les variables auxiliaires peuvent être utilisées dans les tests dans la mesure où ces modifications laissent le flot de contrôle du programme globalement inchangé.

## 4.3.2.4.5.2 Complétude sémantique de la méthode

Pour démontrer la correction partielle de l'exemple :

0:  $x := 0$ ; 1:  $\llbracket 11: x := x+1; 12: \parallel 21: x := x+1; 22: \rrbracket$ ; 2:

(utilisé pour démontrer que la méthode 4.3.2.4.3 est incomplète), on peut transformer le programme comme suit :

0:  $\{ 01: x := 0; 02: v1 := 11; 03: v2 := 21; 04: \}$ ;

1:  $\llbracket 11:$   
 $\{ 111: x := x+1; 112: v1 := 12; 113: \}$ ;  
 $12:$   
 $\parallel$   
 $21:$   
 $\{ 211: x := x+1; 212: v2 := 22; 213: \}$ ;  
 $22:$   
 $\rrbracket$ ;

2:

et utiliser les invariants :

$$\psi'_1(x, v1, v2) = [v1 = 11 \wedge v2 = 21 \wedge x = 0]$$

$$\psi'_{11}(x, v1, v2) = [v1 = 11 \wedge ((v2 = 21 \wedge x = 0) \vee (v2 = 22 \wedge x = 1))]$$

$$\psi'_{12}(x, v1, v2) = [v1 = 12 \wedge ((v2 = 21 \wedge x = 1) \vee (v2 = 22 \wedge x = 2))]$$

$$\psi'_{21}(x, v1, v2) = [v2 = 21 \wedge ((v1 = 11 \wedge x = 0) \vee (v1 = 12 \wedge x = 1))]$$

$$\psi'_{22}(x, v1, v2) = [v2 = 22 \wedge ((v1 = 11 \wedge x = 1) \vee (v1 = 12 \wedge x = 2))]$$

$$\psi'_2(x, v1, v2) = [x = 2]$$

qui sont ceux qu'on aurait utilisé avec la méthode (à la différence près que les variables  $v1$  et  $v2$  sont utilisées pour simuler les compteurs ordinaires des deux processus).

De manière plus générale, considérons un programme parallèle asynchrone  $P_{pa}$  de la forme  $\llbracket P_{pa0} \parallel \dots \parallel P_{pa_{n-1}} \rrbracket$  et des  $\psi_L, L \in C$  tels que  $\psi_L$ , défini comme précédemment soit invariant. Il faut montrer qu'on peut trouver  $\psi'$  dont on peut démontrer l'invariance pour un programme transformé  $P_{pa}'$  sans se référer aux compteurs ordinaires du programme  $P_{pa}'$ .

Construisons  $Ppa'$  en simulant le compteur ordinal de chaque processus  $Prq_i$  au moyen d'une variable auxiliaire  $V_i$  (n'apparaissant pas dans le programme  $Ppa$ ). Pour cela nous supposons qu'il existe un ensemble de valeurs de cette variable qui, au moyen d'un codage  $c$  bijectif, est équipotent à l'ensemble des étiquettes apparaissant dans le processus  $Prq_i$  et nous supposons également qu'aux étiquettes  $L$  de  $Ppa$  correspondent des étiquettes  $L', L'_1, L'_2, \dots$  n'apparaissant pas dans  $Ppa$  et ces étiquettes étant toutes deux à deux distinctes.  $Ppa'$  s'obtient à partir de  $Ppa$  par la transformation  $\tau$  suivante :

$$\tau(L : \llbracket Prq_0 \rrbracket \dots \llbracket Prq_{m-1} \rrbracket ; \bar{L}) =$$

$$L : V_0 := L_0 ; L'_1 : V_1 := L_1 ; \dots ; L'_{m-1} : V_{m-1} := L_{m-1} ; L'_m :$$

$$\llbracket \tau_0(Prq_0) \rrbracket \dots \llbracket \tau_{m-1}(Prq_{m-1}) \rrbracket ; \bar{L} :$$

$$\tau_i(L :) = L :$$

$$\tau_i(L1 : Ea ; L2 : \alpha) =$$

$$L1 : \{ L1'1 : Ea ; L1'2 : V_i := c(L2) ; L1'3 : \} ; \tau_i(L2 : \alpha)$$

$$\text{quand } Ea \text{ est } \underline{skip}, V_i := E, V_i := ? \text{ ou } \{ Ps \}$$

$$\tau_i(L1 : \text{if } B \text{ then } L2 : \alpha ; L3 : \underline{else} \text{ } L4 : \beta ; L5 : \underline{fi} ; L6 : \delta)$$

$$L1 : \{ L1'1 : \text{if } B \text{ then}$$

$$L1'2 : T_i := \underline{true} ; L1'3 : V_i := c(L2) ; L1'3 :$$

else

$$L1'5 : T_i := \underline{false} ; L1'6 : V_i := c(L3) ; L1'4 :$$

$$\underline{fi} ; L1'8 : \} ;$$

$$L1'' : \text{if } T_i \text{ then}$$

$$\tau_i(L2 : \alpha ; L3 : ) \quad V_i := c(L6) ; L3''' :$$

else

$$\tau_i(L4 : \beta ; L5 : ) \quad V_i := c(L6) ; L5''' :$$

$$\underline{fi} ;$$

$$\tau(L6 : \delta)$$

$\tau_i(L1: \text{while } B \text{ do } L2: \alpha; L3: \text{od}; L4: \beta) =$

$L1: \& L1'1: \text{if } B \text{ then}$

$L1'2: T_i := \text{true}; L1'3: V_i := c(L2); L1'4:$

else

$L1'5: T_i := \text{false}; L1'6: V_i := c(L4); L1'7:$

fi;  $L1'8: \&$ ;

$L1'': \text{while } T_i \text{ do}$

$\tau_i(L2: \alpha; L3:)$

$\& L3'1: \text{if } B \text{ then}$

$L3'2: T_i := \text{true}; L3'3: V_i := c(L2); L3'4:$

else

$L3'5: T_i := \text{false}; L3'6: V_i := c(L4); L3'7:$

fi;  $L3'8: \&$ ;

$L3'':$

od;

$\tau_i(L4: \beta)$

Définissons maintenant  $\psi'$  comme caractérisant exactement les descendants des états d'entrée de Ppa (au codage des états de contrôle de Ppa par des valeurs des variables auxiliaires de Ppa' pris).

Pour tous états  $\underline{A}, \underline{A}$  de Ppa, définissons :

$$\psi''(\underline{A}, \underline{A}) = [\varepsilon(\underline{A}) \wedge (\bigwedge_{i \in m} t[\text{Ppa}_i])^*(\underline{A}, \underline{A})]$$

avec

$$\varepsilon(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle) = (\bigwedge_{i \in m} L_i = \underline{L}_i)$$

de sorte que  $\psi''$  est invariante pour Ppa. Posons alors :

- Pour toutes les étiquettes  $L$  qui figurent dans Ppa et Ppa', remarquons que la transformation de Ppa en Ppa' est telle que lorsque le contrôle est au point  $L$  du  $i$ ème processus de Ppa', on a  $v_i = c(L)$ . on choisit donc :

$$\Psi'_L(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle) = \\ \Psi''(\langle \langle c^{-1}(\underline{M}'(V_0)), \dots, c^{-1}(\underline{M}'(V_{m-1})) \rangle, \underline{M} \rangle, \\ \langle \langle c^{-1}(M'(V_0)), \dots, c^{-1}(M'(V_{m-1})) \rangle, M \rangle)$$

de sorte que l'assertion en L dans Ppa' est le plus fort invariant de Ppa en L obtenu en transcrivant le contrôle en terme de variables auxiliaires.

- Pour toutes les étiquettes notées  $L_i$  qui figurent dans une section critique de Ppa' mais pas dans Ppa, on utilise des assertions intermédiaires liant les valeurs courantes des variables (y compris auxiliaires) aux valeurs (symboliques) de ces mêmes variables au début de la section critique, ce qui permet de traiter les sections critiques comme une action atomique (cf. 4.3.2.2.4).

- Les autres étiquettes figurant dans Ppa' mais pas dans Ppa ont été introduites à cause des tests (dans une commande alternative ou itérative). Pour les étiquettes notées  $L''$  qu'on trouve sous la forme

...  $L: \text{if } B \text{ then}$   
                    $L'2: T_i := \text{true}; L'3: V_i := c(L2); L'4:$   
                   else  
                    $L'5: T_i := \text{false}; L'6: V_i := c(L4); L'7:$   
                   fi;  $L'8: \text{fi};$   
 $L'': \dots$

dans Ppa', on choisit:

$$\Psi'_{L''}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle) = [M \in \text{dom}(B[B])] \wedge \\ [B[B](M) \wedge M'(T_i) \wedge M'(V_i) = c(L2) \wedge \Psi'_{L'2}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle)] \\ \vee \\ [\neg B[B](M) \wedge \neg M'(T_i) \wedge M'(V_i) = c(L4) \wedge \Psi'_{L'4}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle)]$$

- Pour les étiquettes notées  $L''$  qu'on trouve sous la forme  
...  $L_1$ :

$$V_i := c(L_2);$$

$L''$ : ...

dans  $Ppa$  et qui précèdent un else ou un fi, on choisira

$$\Psi'_{L''}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle) =$$

$$[\Psi'_{L_1}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle) \wedge M'(V_i) = c(L_2)]$$

on vérifie alors aisément (bien que les calculs soient très fastidieux) que les conditions de vérification correspondant à l'application de la méthode 4.3.2.4.3 pour  $\Psi'$  et  $Ppa'$  sont satisfaites (car  $\Psi''$  satisfait les conditions de vérification de la méthode 4.3.2.4.4 pour  $Ppa$ ) et que

$$\Psi'_{L''}(\langle \underline{M}, \underline{M}' \rangle, \langle M, M' \rangle) \Rightarrow \Psi_L(\underline{M}, M)$$

pour toutes les étiquettes figurant dans  $Ppa$  et  $Ppa'$ .

#### 4.3.2.4.6 Utilisation d'invariants sur l'état de contrôle et les variables associés à chaque processus du programme

Si on utilise les décompositions 4.3.2.4.3 ou 4.3.2.4.4 pour un programme  $\llbracket \text{Pra}_0 \rrbracket \dots \llbracket \text{Pra}_{m-1} \rrbracket$  où chaque processus  $\text{Pra}_i, i \in m$  a  $m_i$  points de contrôle, il y a  $\sum_{i \in m} (m_i - 1)$  conditions de vérifications correspondant à la preuve séquentielle et  $\sum_{i \in m} \sum_{j \in m, i \neq j} \sum_{k \in (m \cup i)} (m_k - 1) = \sum_{i \neq j} m_i (m_j - 1)$  conditions de vérification correspondant à la preuve d'absence d'interférences.

Pour éviter la prolifération des conditions de vérification correspondant à la preuve d'absence d'interférences, on peut choisir une décomposition moins fine, comme celle proposée par Lamport [80] qui consiste à associer un invariant global à chaque processus du programme.

#### Exemple 4.3.2.4.6-1

Pour démontrer la correction partielle du programme

$\llbracket 11: X := X + 1; 12: \parallel 21: X := X + 2; 22: \rrbracket$

il faut vérifier les conditions suivantes :

$$\phi(x) \Rightarrow [I_1(11, 21, x) \wedge I_2(11, 21, x)]$$

$$[I_1(11, c_2, x') \wedge x = x' + 1] \Rightarrow I_1(12, c_2, x)$$

$$[I_2(c_1, 21, x') \wedge x = x' + 2] \Rightarrow I_2(c_1, 22, x)$$

$$[I_1(c_1, 21, x') \wedge x = x' + 2] \Rightarrow I_1(c_1, 22, x)$$

$$[I_2(11, c_2, x') \wedge x = x' + 1] \Rightarrow I_2(12, c_2, x)$$

$$[I_1(12, 22, x) \wedge I_2(12, 22, x)] \Rightarrow \psi(x)$$

□

La décomposition choisie est la suivante (Cousot-Cousot [84]) :

$$S = (C_0 \times \dots \times C_{m-1}) \times M$$

$$A\Delta = (S \rightarrow \{\#, \#\#\})$$

$$A\check{\Delta} = (m \rightarrow (C_0 \times \dots \times C_{m-1} \times M \rightarrow \{\#, \#\#\}))$$

$$\alpha \in (A\Delta \rightarrow A\check{\Delta})$$

$$\alpha(I)_i (c_0, \dots, c_{m-1}, m) = I(\langle\langle c_0, \dots, c_{m-1} \rangle, m \rangle)$$

$$\gamma \in (A\check{\Delta} \rightarrow A\Delta)$$

$$\gamma(\check{I})(\langle\langle c_0, \dots, c_{m-1} \rangle, m \rangle) = \bigwedge_{i \in m} \check{I}_i(c_0, \dots, c_{m-1}, m)$$

### Exemple 4.3.2.4.6-2

La détermination des conditions de vérification pour démontrer la correction partielle de programmes parallèles asynchrones de la forme :

$$\llbracket \text{Pra}_0 \parallel \dots \parallel \text{Pra}_{m-1} \rrbracket$$

est tout à fait similaire à 4.3.2.4.3-1. Nous obtenons (Cousot-Cousot [84]):

- Initialisation

$$\phi(m) \Rightarrow \forall i \in m. \check{I}_i(\bar{L}_0, \dots, \bar{L}_{m-1}, m) \quad (\text{où } \text{Pra}_i \equiv \bar{L}_i : \alpha, i \in m)$$

- Preuve séquentielle

$$\begin{aligned} & [\check{I}_i(c_0, \dots, c_{i-1}, c'_i, c_{i+1}, \dots, c_{m-1}, m') \wedge \vdash \llbracket \text{Pra}_i \rrbracket(\langle c'_i, m' \rangle, \langle c_i, m \rangle)] \\ & \Rightarrow \check{I}_i(c_0, \dots, c_{i-1}, c_i, c_{i+1}, \dots, c_{m-1}, m) \end{aligned}$$

- Preuve d'absence d'interférences, ( $i \neq k$ ) :

$$\begin{aligned} & [\check{I}_i(c_0, \dots, c_{k-1}, c'_k, c_{k+1}, \dots, c_{m-1}, m') \wedge \vdash \llbracket \text{Pra}_k \rrbracket(\langle c'_k, m' \rangle, \langle c_k, m \rangle)] \\ & \Rightarrow \check{I}_i(c_0, \dots, c_{k-1}, c_k, c_{k+1}, \dots, c_{m-1}, m') \end{aligned}$$

- Finalisation

$$\forall i \in m. \check{I}_i(\bar{L}_0, \dots, \bar{L}_{m-1}, m) \Rightarrow \psi(m) \quad (\text{où } \text{Pra}_i \equiv \alpha \bar{L}_i : , i \in m)$$

□

Exemple 4.3.2.4.6-3

Nous illustrons la méthode de preuve 4.3.2.4.6-2 sur l'exemple 2.8.2.3. De manière évidente il ne peut s'agir que d'une reformulation de 4.3.2.4.4-2 en utilisant un invariant global par processus plutôt que des invariants locaux associés à chaque point de contrôle du programme. Comme les processus du programme sont symétriques, nous utilisons le même invariant  $\tilde{I}$  pour  $\tilde{I}_1$  et  $\tilde{I}_2$ , ce qui permet de n'avoir à raisonner que sur un seul processus.

Pour pouvoir désigner certains états de contrôle du programme, nous introduisons (cf. Cousot-Cousot [84]) :

$$\text{Not-started} = (c_1 = 11 \wedge c_2 = 21)$$

$$\text{Pre}_2\text{-started} = (c_1 = 11 \wedge c_2 \neq 21)$$

$$\text{Pre}_1\text{-started} = (c_1 \neq 11 \wedge c_2 = 21)$$

$$\text{Started} = (c_1 \neq 11 \wedge c_2 \neq 21)$$

L'idée centrale du programme 2.8.2.3 est de maintenir invariante la relation suivante :

$$\text{Inv}(\underline{m}, c_1, c_2, \underline{m}, p_1, p_2) = [( \text{Not-started} \wedge m = \underline{m} ) \vee ( \text{Pre}_2\text{-started} \wedge p_2 = 2^{m-m} ) \vee ( \text{Pre}_1\text{-started} \wedge p_1 = 2^{m-m} ) \vee ( \text{started} \wedge p_1 \times p_2 = 2^{m-m} )]$$

L'autre observation essentielle pour la preuve de correction partielle est que le programme peut se terminer seulement quand  $0 \leq m \leq 1$ . Pour montrer ceci, nous introduisons :

$$\text{Before-test} = [(c_1 \in \{11, 12\} \wedge c_2 \in \{21, 22\}) \vee (c_1 = 14 \wedge c_2 = 24)]$$

$$\text{After-test} = [(c_1 = 13 \wedge c_2 \in \{21, 22, 23\}) \vee (c_1 \in \{11, 12, 13\} \wedge c_2 = 23)]$$

$$\text{After-test-and-decrement} = [(c_1 = 14 \wedge c_2 \in \{21, 22, 23\}) \vee (c_1 \in \{11, 12, 13\} \wedge c_2 = 24)]$$

$$\text{one-decrement-left} = [(c_1 = 15 \wedge c_2 = 23) \vee (c_1 = 13 \wedge c_2 = 25)]$$

$$\text{No-decrement-left} = [(c_1 = 15 \wedge c_2 \neq 23) \vee (c_1 \neq 13 \wedge c_2 = 25)]$$

Pour chaque processus, nous pouvons choisir l'invariant global suivant:

$$\begin{aligned} \tilde{I}(m, c_1, c_2, m, p_1, p_2) = & [Inv(m, c_1, c_2, m, p_1, p_2) \wedge [(Before-test \wedge m \geq 0) \vee (After-test \wedge m > 1) \\ & \vee (After-test-and-decrement \wedge m \geq 1) \vee (One-decrement-left \wedge m = 1) \\ & \vee (No-decrement-left \wedge 0 \leq m \leq 1)]] \end{aligned}$$

qui satisfait les conditions de vérification suivantes,

- Initialisation

$$[m \geq 0] \Rightarrow \tilde{I}(m, 11, 21, m, p_1, p_2)$$

- Preuve séquentielle du processus 1

$$\tilde{I}(m, 11, c_2, m, p'_1, p_2) \Rightarrow \tilde{I}(m, 12, c_2, m, 1, p_2)$$

$$[\tilde{I}(m, 12, c_2, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}(m, 13, c_2, m, p_1, p_2)$$

$$[\tilde{I}(m, 12, c_2, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}(m, 15, c_2, m, p_1, p_2)$$

$$\tilde{I}(m, 13, c_2, m', p'_1, p_2) \Rightarrow \tilde{I}(m, 14, c_2, m'-1, 2 \times p'_1, p_2)$$

$$[\tilde{I}(m, 14, c_2, m, p_1, p_2) \wedge m > 1] \Rightarrow \tilde{I}(m, 13, c_2, m, p_1, p_2)$$

$$[\tilde{I}(m, 14, c_2, m, p_1, p_2) \wedge m \leq 1] \Rightarrow \tilde{I}(m, 15, c_2, m, p_1, p_2)$$

- La preuve d'absence d'interférences du processus 2 avec l'invariant global du processus 1 est exactement la preuve séquentielle du processus 2

- Finalisation

$$\tilde{I}(m, 15, 25, m, p_1, p_2) \Rightarrow \tilde{I}_1(m, m, p_1, p_2)$$

En comparaison avec 4.3.2.4.4-2, l'utilisation d'une décomposition moins fine conduit, pour cet exemple, à une factorisation naturelle de conditions de vérification similaires et donc seulement à 8 conditions de vérification (au lieu de 44 dans 4.3.2.4.4-2). Cet exemple montre que le choix de la bonne décomposition de l'invariant global en invariants locaux dépend du problème à résoudre.

□

## 4.3.2.4.7 Utilisation d'un invariant global et d'invariants locaux

Un certain nombre de méthodes de preuve de propriétés d'invariance des programmes utilisent un invariant global sur l'état mémoire (qui s'appelle le "resource invariant" chez Hoare [73], Owicki-Gries [76b], le "monitor invariant" chez Howard [76], le "global invariant" chez Apt-Francez-deRoover [80], etc.) en même temps que des invariants locaux associés à divers points de contrôle du programme et portant sur l'état mémoire et l'état de contrôle (ou bien sur l'état mémoire uniquement en utilisant des variables auxiliaires pour simuler l'état de contrôle).

Si  $S = (C_0 \times \dots \times C_{m-1}) \times M$  et  $A_S = (S \rightarrow \{\text{tt}, \text{ff}\})$ , cette décomposition se définit comme suit :

$$\tilde{A}_S = \tilde{A}_S^g \times \tilde{A}_S^l$$

$$\text{où } \tilde{A}_S^g = (M \rightarrow \{\text{tt}, \text{ff}\})$$

$$\tilde{A}_S^l = (\tilde{A}_{S_0}^l \times \dots \times \tilde{A}_{S_{m-1}}^l)$$

$$\tilde{A}_{S_i}^l = (C_i \rightarrow (C_0 \times \dots \times C_{i-1} \times C_{i+1} \times \dots \times C_{m-1} \times M \rightarrow \{\text{tt}, \text{ff}\})), \quad i \in m$$

$$\alpha \in (A_S \rightarrow \tilde{A}_S)$$

$$\alpha(I) = \langle \tilde{I}_g, \tilde{I}_l \rangle$$

$$\text{où } \tilde{I}_g(m) = (\exists c_0 \in C_0, \dots, c_{m-1} \in C_{m-1} . I(\langle c_0, \dots, c_{m-1} \rangle, m))$$

$$\tilde{I}_{l_{i,c_i}}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m) = I(\langle c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1} \rangle, m)$$

$$\gamma \in (\tilde{A}_S \rightarrow A_S)$$

$$\gamma(\langle \tilde{I}_g, \tilde{I}_l \rangle)(\langle c_0, \dots, c_{m-1} \rangle, m) = [\tilde{I}_g(m) \wedge \forall i \in m, c \in C_i . \tilde{I}_{l_{i,c}}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m)]$$

Cette décomposition est évidemment sémantiquement complète puisque l'invariant global  $\tilde{G}$  est redondant et les invariants locaux  $\tilde{I}$  peuvent être choisis comme en 4.3.2.4.4.

Pour concilier les avantages des décompositions 4.3.2.4.4 et 4.3.2.4.6 on peut envisager de faire porter l'invariant global  $\tilde{g}$  sur l'état mémoire mais également sur l'état de contrôle, ce qui donne :

$$\tilde{A\Delta g} = (C_0 \times \dots \times C_{m-1} \times M \rightarrow \{\text{tt}, \text{ff}\})$$

$$\tilde{g}(c_0, \dots, c_{m-1}, m) = I(\langle\langle c_0, \dots, c_{m-1} \rangle, m \rangle)$$

Dans le cas de programmes parallèles comportant des variables globales qui peuvent être modifiées par tous les processus et des variables locales qui ne sont visibles que dans le processus où elles sont déclarées, on peut imaginer de faire porter l'invariant global sur les variables globales et l'état de contrôle et de faire porter l'invariant local sur les variables locales visibles, sur les variables globales et sur l'état de contrôle. On a alors :

$$S = (C_0 \times \dots \times C_{m-1}) \times M \times (M'_0 \times \dots \times M'_{m-1})$$

$$A\Delta = (S \rightarrow \{\text{tt}, \text{ff}\})$$

$$A\tilde{\Delta} = \tilde{A\Delta g} \times \tilde{A\Delta l}$$

$$\text{où } \tilde{A\Delta g} = (C_0 \times \dots \times C_{m-1} \times M \rightarrow \{\text{tt}, \text{ff}\})$$

$$\tilde{A\Delta l} = (\tilde{A\Delta}_0 \times \dots \times \tilde{A\Delta}_{m-1})$$

$$\tilde{A\Delta}_i = (C_i \rightarrow (C_0 \times \dots \times C_{i-1} \times C_{i+1} \times \dots \times C_{m-1} \times M \times M'_i \rightarrow \{\text{tt}, \text{ff}\})), \quad i \in m$$

$$\alpha \in (A\Delta \rightarrow A\tilde{\Delta})$$

$$\alpha(I) = \langle \tilde{I}g, \tilde{I}l \rangle$$

$$\text{où } \tilde{I}g(c_0, \dots, c_{m-1}, m) = [\exists m'_0 \in M'_0, \dots, m'_{m-1} \in M'_{m-1}. I(\langle\langle c_0, \dots, c_{m-1} \rangle, m, \langle m'_0, \dots, m'_{m-1} \rangle \rangle)]$$

$$\tilde{I}l_{i, c'_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m, m') = [\exists m'_0 \in M'_0, \dots, m'_{i-1} \in M'_{i-1}, m'_{i+1} \in M'_{i+1}, \dots, m'_{m-1} \in M'_{m-1}. I(\langle\langle c_0, \dots, c_{m-1} \rangle, m, \langle m'_0, \dots, m'_{i-1}, m', m'_{i+1}, \dots, m'_{m-1} \rangle \rangle)]$$

$$\gamma \in (A\tilde{\Delta} \rightarrow A\Delta)$$

$$\gamma(\langle \tilde{I}g, \tilde{I}l \rangle)(\langle\langle c_0, \dots, c_{m-1} \rangle, m, \langle m'_0, \dots, m'_{m-1} \rangle \rangle) =$$

$$[\tilde{I}g(c_0, \dots, c_{m-1}, m) \wedge \forall i \in m. \tilde{I}l_{i, c'_i}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m, m'_i)]$$

on pourrait croire que l'utilisation de variables locales au lieu de variables globales permet de localiser les invariants en ce sens que l'invariant associé à un point de programme ne porte que sur les variables du programme qui sont visibles en ce point du programme. Cette approche n'est pas complète, comme le montre l'exemple suivant :

```

var M : integer ;
    F : boolean ;
    S1 : semaphore := 1 ;
    S2 : semaphore := 0 ;

```

```

M := 0 ; F := false ;

```

```

|| var T1 : integer ;

```

```

    T1 := 0 ;

```

```

    while T1 < 10 do

```

```

        p(S1) ;

```

```

        T1 := T1 + 1 ;

```

```

        v(S2) ;

```

```

    od ;

```

```

    p(S1) ;

```

```

    F := true ;

```

```

    v(S2) ;

```

```

    { M := M + T1 } ;

```

```

|| var T2 : integer ;

```

```

    T2 := 0 ;

```

```

    p(S2) ;

```

```

    while not F do

```

```

        T2 := T2 + 1 ;

```

```

        v(S1) ;

```

```

        p(S2) ;

```

```

    od ;

```

```

    { M := M + T2 } ;

```

```

|| ;

```

Pour démontrer que la valeur finale de  $M$  est nulle, il faut évidemment pouvoir établir une relation entre les valeurs finales de  $T_1$  et  $T_2$  et donc une relation entre les valeurs courantes de  $T_1$  et  $T_2$  ce qui est impossible avec la décomposition choisie. (Cet argument peut être formalisé à l'aide de points fixes comme en 4.3.2.4.3).

#### Exemple 4.3.2.4.7-1

La méthode de Levin [79] pour CSP (Hoare [78]) utilise une décomposition similaire mais qui ne porte pas sur l'état de contrôle, les invariants locaux portant sur les variables des processus de CSP et des variables auxiliaires et l'invariant global ne portant que sur des variables auxiliaires. La complétude sémantique de la méthode vient de l'utilisation des variables auxiliaires qui permettent de simuler les relations entre les états de contrôle des processus (comme en 4.3.2.4.5) mais également d'exprimer indirectement les relations entre variables locales des processus à l'aide de variables auxiliaires globales (la technique consistant à recopier les variables locales dans les variables auxiliaires globales après chaque modification des variables locales).

□

#### 4.3.2.4.8 Classification des méthodes de preuve d'invariance selon la finesse de la décomposition de l'invariant global en invariants locaux

Les méthodes de preuve de programmes peuvent être classées selon la classe des propriétés qu'elles permettent de démontrer (invariance conditionnelle, invariance, fatalité, ...).

Les diverses méthodes pour démontrer des propriétés de programmes dans une même classe peuvent être classées selon le principe d'induction dont elles découlent.

Dans le cas de propriétés d'invariance, les méthodes de preuve dérivant d'un même principe d'induction de la forme :

$$[\exists I \in A_S. Co(I)]$$

sont de la forme :

$$[\exists \tilde{I} \in A_S^V. C\tilde{v}(\tilde{I})]$$

et s'obtiennent au moyen d'une correspondance  $(\alpha, \gamma)$  entre  $A_S$  et  $A_S^V$  telle que  $C\tilde{v} \Rightarrow Co \circ \gamma$  (condition suffisante de correction, et éventuellement  $Co \Rightarrow C\tilde{v} \circ \alpha$  (condition suffisante de complétude)).

Nous dirons qu'une méthode  $M_1$  de preuve basée sur une décomposition  $(\tilde{A}_{S_1}, (\alpha_1, \gamma_1))$  de  $A_S$  peut se dériver d'une méthode  $M_2$  de preuve basée sur une décomposition  $(\tilde{A}_{S_2}, (\alpha_2, \gamma_2))$  de  $A_S$ , quand il existe une décomposition  $(\tilde{A}_S, (\alpha, \gamma))$  de  $\tilde{A}_{S_1}$  telle que  $\alpha_1 = \alpha \circ \alpha_2$  et  $\gamma_1 = \gamma_2 \circ \gamma$  (auquel cas nous écrirons  $M_1 \leftarrow M_2$ ).

#### Exemple 4.3.2.4.8-1

Si on a démontré une propriété d'invariance pour un programme  $\llbracket \text{Pro}_0 \rrbracket \dots \llbracket \text{Pro}_{n-1} \rrbracket$  en utilisant la méthode 4.3.2.4.6, c'est-à-dire en utilisant des invariants  $G_0, \dots, G_{n-1}$  sur l'état de contrôle et les variables associées

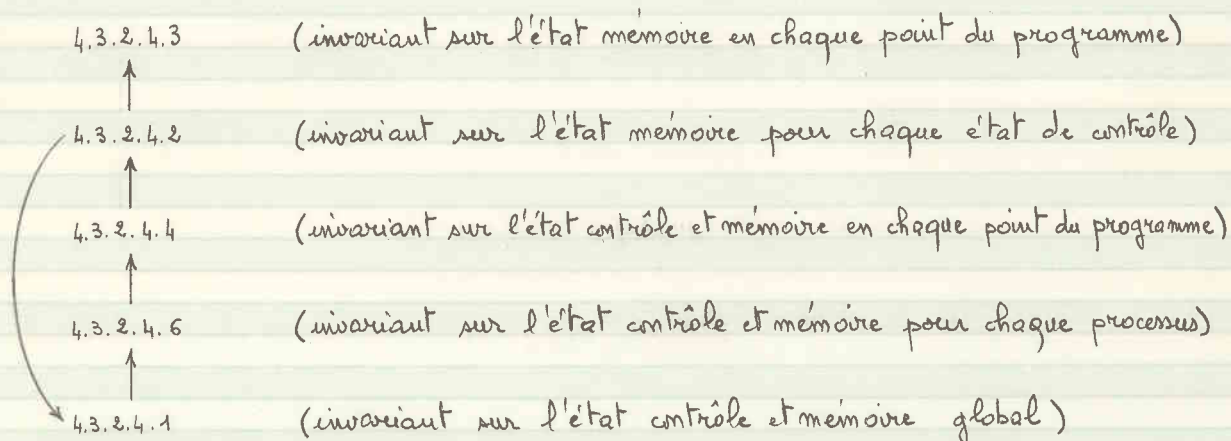
à chaque processus  $Proc_0, \dots, Proc_{m-1}$  du programme, on peut reformuler cette preuve puisqu'elle correspond à la méthode 4.3.2.4.4 en utilisant des invariants  $I_{i,c}$ ,  $i \in m$ ,  $c \in C_i$  définis par  $I = \alpha(G)$  tel que

$$I_{i,c}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m) = G_i(c_0, \dots, c_{i-1}, c, c_{i+1}, \dots, c_{m-1}, m)$$

Réciproquement, une preuve utilisant un invariant global par processus (cf. 4.3.2.4.6) peut se dériver d'une preuve utilisant des invariants locaux associés à chaque point du programme (cf. 4.3.2.4.4) en utilisant  $G = \delta(I)$  tel que

$$G_i(c_0, \dots, c_{m-1}, m) = \bigvee_{c \in C_i} I_{i,c}(c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, m)$$

Plus généralement, les méthodes de preuve considérées au paragraphe 4.3.2.4 peuvent se dériver les unes des autres comme suit:



□

La relation de dérivation est un préordre (transitive et réflexive). Nous dirons que deux méthodes de preuve (basées sur un même principe d'induction) sont "sémantiquement équivalentes" quand chacune se dérive de l'autre. La relation de dérivation induite sur l'ensemble des méthodes de preuve quotientée par la relation d'équivalence sémantique est un ordre partiel. Le supremum correspond au principe d'induction et l'infimum au choix  $\tilde{A}_s = 1$ ,  $\alpha(P) = 0$ ,  $\delta(0) = \text{tt}$ ,  $\tilde{C}_i(\tilde{I}) = \text{tt}$  (une méthode correcte mais qui permet de ne rien démontrer). On remarquera que cet ensemble ordonné contient le treillis complet des méthodes d'analyse sémantique des programmes considéré dans Cousot-Cousot [79a].

#### 4.3.2.5 Analyse sémantique des programmes

L'analyse sémantique d'un programme (Cousot P. [77, 78, 81], Cousot-Cousot [76, 77a, 77b, 77c, 77d, 79a, 80a, 84].) est une analyse statique (i.e. sans exécuter le programme) des propriétés dynamiques (i.e. à l'exécution) de ce programme.

##### 4.3.2.5.1 Analyse d'invariance "en avant"

Soit  $\langle s', A', \Sigma' \rangle$  la sémantique d'un programme. L'analyse sémantique d'invariance "en avant" consiste à caractériser la relation  $\underline{D}'(\phi, \langle s', A', \Sigma' \rangle)$  entre les états  $\underline{s}$  satisfaisant une condition  $\phi$  et leurs descendants possibles  $\bar{s}$  :

$$\underline{D}'(\phi, \langle s', A', \Sigma' \rangle) \in (S^2 \rightarrow \{\text{tt}, \text{ff}\})$$

$$\underline{D}'(\phi, \langle s', A', \Sigma' \rangle)(\underline{s}, \bar{s}) = [\exists p \in \Sigma', i, j \in |p|. (\phi(\underline{s}) \wedge p_i = \underline{s} \wedge i \leq j \wedge p_j = \bar{s})]$$

Il est toujours possible de se ramener au cas où la condition  $\phi$  ne porte que sur les états initiaux en considérant la sémantique  $\langle s, A, \Sigma \rangle$  telle que  $s = s'$ ,  $A = A'$  et

$$\Sigma = \{p \in \Sigma' : \phi(p_0) \wedge \exists q \in \Sigma'. p \rightarrow q\}$$

et en posant :

$$\underline{D}(\phi, \langle s, A, \Sigma \rangle) \in (S^2 \rightarrow \{\text{tt}, \text{ff}\})$$

$$\underline{D}(\phi, \langle s, A, \Sigma \rangle)(\underline{s}, \bar{s}) = [\exists p \in \Sigma, j \in |p|. (\phi(p_0) \wedge p_0 = \underline{s} \wedge p_j = \bar{s})]$$

car

$$\underline{D}'(\phi, \langle s', A', \Sigma' \rangle) = \underline{D}(\phi, \langle s, A, \Sigma \rangle)$$

La relation  $\underline{D}(\phi, \langle s, A, \Sigma \rangle)$  n'étant pas calculable pour tous les programmes, on ne peut envisager que de calculer des approximations supérieures  $\tilde{\underline{D}}(\phi, \langle s, A, \Sigma \rangle)$  telles que  $\underline{D}(\phi, \langle s, A, \Sigma \rangle) \Rightarrow \tilde{\underline{D}}(\phi, \langle s, A, \Sigma \rangle)$  (ou inférieures, ce qui se traite par dualité).

Soit  $\langle S, A, t, E \rangle$  le système de transition engendré par  $\langle S, A, \Sigma \rangle$ .  
 On a  $\underline{D}(\phi, \langle S, A, \Sigma \rangle) \Rightarrow \underline{D}(\phi, \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle)$  (en effet  $\underline{D}(\phi, \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle)$  est invariante pour  $\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$  et donc pour  $\langle S, A, \Sigma \rangle$  d'après 4.1.3v3).  
 Une première approximation supérieure consiste donc à raisonner sur le système de transition  $\langle S, A, t, E \rangle$  de sorte qu'en posant :

$$\underline{D}(\langle S, A, t, E \rangle) \in (S^2 \rightarrow \{\uparrow, \# \})$$

$$\underline{D}(\langle S, A, t, E \rangle)(\underline{A}, \bar{A}) = [\varepsilon(\underline{A}) \wedge t^*(\underline{A}, \bar{A})]$$

il s'agit de caractériser  $\underline{D}(\langle S, A, t, E, \phi \rangle)$ .

On remarque alors comme en 4.3.1.4-1 que  $\underline{D}(\langle S, A, t, E \rangle)$  est le plus petit point fixe  $\text{lfp}(F)$  pour  $\Rightarrow$  de l'opérateur  $F$  sur  $A_S = (S^2 \rightarrow \{\uparrow, \# \})$  défini par :

$$F(I)(\underline{A}, \bar{A}) = [(\varepsilon(\underline{A}) \wedge \bar{A} = \underline{A}) \vee (\exists \underline{A}' \in S, a \in A. I(\underline{A}, \underline{A}') \wedge t_a(\underline{A}', \bar{A}))]$$

de sorte qu'en utilisant une décomposition  $(\alpha, \gamma)$  des invariants globaux de  $\langle A_S, \Rightarrow \rangle$  en des invariants locaux de  $\langle \tilde{A}_S, E \rangle$  et un opérateur correspondant  $\tilde{F}$  monotone sur  $\tilde{A}_S$  tel que  $\alpha \circ F \circ \gamma \in \tilde{F}$ , on a  $\text{lfp}(F) \Rightarrow \gamma(\text{lfp}(\tilde{F}))$  quand  $(\alpha, \gamma)$  est une (demi-)correspondance de Galois et  $\langle \tilde{A}_S, E \rangle$  un treillis complet.

Si le treillis complet  $\langle \tilde{A}_S, E \rangle$  satisfait la condition de chaîne ascendante (toute chaîne strictement croissante pour  $\sqsubseteq$  est finie) alors  $\text{lfp}(\tilde{F})$  est calculable itérativement comme  $\bigcup_{n \geq 0} \tilde{F}^n(\alpha(\#))$ .

Si l'itération  $x^0 = \alpha(\#), \dots, x^{n+1} = \tilde{F}(x^n)$  peut ne pas converger en un nombre fini de pas, on utilisera une technique d'extrapolation de la plus petite solution  $\text{lfp}(\tilde{F})$  du système d'équations  $x = \tilde{F}(x)$  en calculant la limite  $\tilde{I} = \bigcup_{n \geq 0} \tilde{F}^{\nabla n}(\alpha(\#))$  où  $\tilde{F}^{\nabla}(x) = x \nabla \tilde{F}(x)$ , d'une itération croissante avec élargissement  $\nabla$  satisfaisant :

$$\forall x, y \in \tilde{A}_S. [(x \sqcup y) \sqsubseteq (x \nabla y)]$$

(ce qui assure que  $\text{lfp}(\tilde{F}) \in \tilde{I}$ ) et la condition qu'il n'existe pas de chaîne infinie strictement croissante de la forme  $x^0, \dots, x^{i+1} = x^i \nabla \tilde{F}(x^i), \dots$  (ce qui assure la convergence).

Si  $\tilde{I}$  n'est pas un point fixe de  $\tilde{F}$ , alors  $\text{lfp}(\tilde{F}) \in \tilde{F}(\tilde{I}) \subset \tilde{I}$  et l'approximation supérieure  $\tilde{I}$  de  $\text{lfp}(\tilde{F})$  peut être améliorée puisque  $\text{lfp}(\tilde{F}) \in \tilde{F}^n(\tilde{I})$  pour tout  $n \geq 0$ . De manière plus générale il est ensuite possible d'améliorer l'approximation  $\tilde{I}$  de  $\text{lfp}(\tilde{F})$  en calculant la limite  $\tilde{J} = \bigcap_{n \geq 0} \tilde{F}^{\Delta^n}(\tilde{I})$  où  $\tilde{F}^{\Delta}(x) = x \Delta \tilde{F}(x)$ , d'une itération décroissante avec rétrécissement  $\Delta$  tel que :

$$\forall x, y \in A_{\Delta}. [(x \sqcap y) \in (x \Delta y) \in y]$$

(de sorte que  $\text{lfp}(\tilde{F}) \in \tilde{J}$ ) et toute chaîne strictement décroissante de la forme  $x^0, \dots, x^{i+1} = x^i \Delta \tilde{F}(x^i), \dots$  est finie (de sorte que l'itération converge).

En pratique, la décomposition  $(\alpha, \gamma)$  est choisie sous la forme  $(\alpha_a \circ \alpha_p, \gamma_p \circ \gamma_a)$  où  $(\alpha_p, \gamma_p)$  est une décomposition utilisée pour une méthode de preuve (cf. par exemple à 4.3.2.4) et  $(\alpha_a, \gamma_a)$  introduit une approximation, qui peut être grossière, déterminée en fonction du problème posé. De ce fait l'équation  $x = \tilde{F}(x)$  se présente généralement sous la forme d'un système d'équations :

$$\begin{cases} x_i = \tilde{F}_i(x_0, \dots, x_{m-1}) \\ i \in m \end{cases}$$

Le calcul de  $\bigcup_{n \geq 0} \tilde{F}^n(\alpha(\text{ff}))$  peut alors se faire en utilisant toute stratégie chaotique voire asynchrone équivalente (Cousot-P [77]). En particulier, étant donné le graphe de dépendance du système d'équations ( $i$  dépend de  $j$  si le résultat de  $F_i(x_0, \dots, x_{m-1})$  dépend de  $x_j$ ), il peut être avantageux d'itérer en faisant localement converger les composantes  $x_i$  correspondant à une même composante fortement connexe du graphe de dépendance et ce d'une part en parcourant l'arbre des composantes fortement connexes en largeur et d'autre part en procédant récursivement de manière identique à l'intérieur de chaque

composante fortement connexe. Dans le cas d'itérations utilisant une extrapolation (élargissement ou rétrécissement), il suffit d'utiliser l'extrapolation pour les composantes  $x_i$  telles que  $i$  est un point de coupure du graphe de dépendance (les points de coupure étant choisis de sorte que tout cycle dans le graphe de dépendance passe par un point de coupure).

### Exemple 4.3.2.5.1-1

Pour étendre Cousot-Cousot [76, 77a] au cas des programmes parallèles de la forme :

$$\llbracket P_{x_0} \parallel \dots \parallel P_{x_{m-1}} \rrbracket$$

où

$$S = (C_0 \times \dots \times C_{m-1}) \times (\mathcal{V} \rightarrow \mathcal{D}) \quad \text{et}$$

$$\mathcal{D} = \{z \in \mathbb{Z} : l_i \leq z \leq h_i\}$$

on peut utiliser la décomposition :

$$A_\Delta = (S \rightarrow \{\perp, \text{ff}\})$$

$$A_\Delta^\vee = \prod_{i \in m} \prod_{c \in C_i} (\mathcal{V} \rightarrow \mathbb{Z} \times \mathbb{Z})$$

où

$$\alpha(I)_{i,c}[v] = \langle \inf \{v \in \mathbb{Z} : P(v)\}, \sup \{v \in \mathbb{Z} : P(v)\} \rangle \quad \text{si } \exists v. P(v) \\ = \perp \quad \text{si } \forall v. \neg P(v)$$

et

$$P(v) = [\exists c_0 \in C_0, \dots, c_{i-1} \in C_{i-1}, c_{i+1} \in C_{i+1}, \dots, c_{m-1} \in C_{m-1}, m \in (\mathcal{V} \rightarrow \mathcal{D}).$$

$$I(\langle \langle c_0, \dots, c_{i-1}, c, c_{i+1}, \dots, c_{m-1} \rangle, m[V \leftarrow v] \rangle)]$$

Cette décomposition ne permet pas d'exprimer de relations entre les compteurs ordinaires des différents processus. on obtiendra donc des résultats plus précis en utilisant la décomposition suivante :

$$A_\Delta^\vee = \prod_{i \in m} \prod_{c \in C_i} ((C_0 \times \dots \times C_{i-1} \times C_{i+1} \times \dots \times C_{m-1}) \rightarrow (\mathcal{V} \rightarrow \mathbb{Z} \times \mathbb{Z}))$$

où

$$\alpha(I)_{i,c} [c_0, \dots, c_{i-1}, c_{i+1}, \dots, c_{m-1}, v] = \langle \inf \{v \in \mathbb{Z} : Q(v)\}, \sup \{v \in \mathbb{Z} : Q(v)\} \rangle \quad \text{si } \exists v. Q(v)$$

$$= \perp \quad \text{si } \forall v. \neg Q(v)$$

et

$$Q(v) = [\exists m \in (\emptyset \rightarrow \mathbb{D}). I(\langle \langle c_0, \dots, c_{i-1}, c, c_{i+1}, \dots, c_{m-1} \rangle, m[v \leftarrow v] \rangle)]$$

Par exemple pour le programme 2.2.2.3 :

```

0: { m ≥ 0 }
  ||
  11: P1 := 1;
  12: while N > 1 do
  13:   * N := N - 1; P1 := 2 * P1 *;
  14: od;
  ||
  21: P2 := 1;
  22: while N > 1 do
  23:   * N := N - 1; P2 := 2 * P2 *;
  24: od;
  25: ||;
1: if N = 0 then P := P1 * P2 else P := 2 * P1 * P2 fi;
2:

```

L'invariant local approché  $D_{je}$  attaché au point  $e$  du processus  $j=1,2$  est choisi comme étant un intervalle de valeurs pour chaque variable  $v \in \mathcal{V}$  et chaque point de contrôle  $h \in C_j$  de l'autre processus  $\tilde{j}$  (où  $\tilde{1}=2$  et  $\tilde{2}=1$ ).  $D_{je}(\tilde{j}h)$  est alors un triplet  $\langle m, p_j, p_{\tilde{j}} \rangle$  des valeurs abstraites des variables  $N, P_j, P_{\tilde{j}}$  où chaque  $m, p_j, p_{\tilde{j}}$  est soit  $\perp$  (qui correspond à ff) ou un intervalle de valeurs numériques  $[a, b]$  tel que  $l_i \leq a \leq b \leq h_i$  où  $l_i$  et  $h_i$  sont respectivement les plus petit et plus grand entier representable en machine. Par exemple  $D_{14}(23) = \langle [1, h_{i-1}], [2, h_i], [1, h_i] \rangle$  signifie qu'au point 14 du processus 1, il est vrai que  $((1 \leq m \leq h_{i-1}) \wedge (2 \leq p_1 \leq h_i) \wedge (1 \leq p_2 \leq h_i))$  quand le contrôle est au point 23 du processus 2.

Nous utiliserons les notations suivantes :

- $\langle m, p_j, p_j^{\sim} \rangle [N] = m, \langle m, p_j, p_j^{\sim} \rangle [P_j] = p_j, \langle m, p_j, p_j^{\sim} \rangle [P_j^{\sim}] = p_j^{\sim}$
- $\langle m, p_1, p_2 \rangle [p_1/p_1'] = \langle m, p_1', p_2 \rangle$  (substitution)  
 $\langle m, p_1, p_2 \rangle [p_2/p_2'] = \langle m, p_1, p_2' \rangle$
- $\perp \wedge x = x \wedge \perp = \perp$  si  $x \in (\{ \perp \} \cup \{ [a, b] : a \leq b \})$  (conjonction approchée)  
 $[a, b] \wedge [c, d] = [\underline{\sup}(a, c), \underline{\inf}(b, d)]$   
 $\quad \quad \quad \text{ssi } \underline{\sup}(a, c) \leq \underline{\inf}(b, d)$   
 $\quad \quad \quad = \perp \quad \text{ssi } (b < c) \text{ ou } (d < a)$
- $\perp \vee x = x \vee \perp = x$  si  $x \in (\{ \perp \} \cup \{ [a, b] : a \leq b \})$  (disjonction approchée)  
 $[a, b] \vee [c, d] = [\underline{\inf}(a, c), \underline{\sup}(b, d)]$
- $\perp - 1 = \perp$   
 $[a, b] - 1 = [a-1, b-1] \wedge [li, hi]$  (décrémentatation approchée)
- $2 \times \perp = \perp$   
 $2 \times [a, b] = [2 \times a, 2 \times b] \wedge [li, hi]$  (décalage gauche approché)

Dans le système d'équations approchées de point fixe suivant nous supposons qu'initialement nous avons  $m \geq 0$ . Pour chaque équation nous distinguons un terme correspondant à la preuve séquentielle et un terme correspondant au contrôle d'absence d'interférences :

$$D_0 = \langle [0, hi], [li, hi], [li, hi] \rangle$$

$$D_{11}(2k) = D_0$$

$$D_{11}(2k) = \text{inter}_{11}(2k)$$

$$k=2, \dots, 5$$

$$D_{12}(2k) = D_{11}(2k) [p_1/[1,1]] \vee \text{inter}_{12}(2k)$$

$$k=1, \dots, 5$$

$$D_{13}(2k) = (D_{12}(2k) \wedge \langle [2, hi], [li, hi], [li, hi] \rangle) \vee (D_{14}(2k) \wedge \langle [2, hi], [li, hi], [li, hi] \rangle) \vee \text{inter}_{13}(2k)$$

$$k=1, \dots, 5$$

$$D_{14}(2k) = \langle D_{13}(2k) [N] - 1, 2 \times D_{13}(2k) [p_1], D_{13}(2k) [p_2] \rangle \vee \text{inter}_{14}(2k)$$

$$k=1, \dots, 5$$

$$D_{15}(2k) = (D_{12}(2k) \wedge \langle [li, 1], [li, hi], [li, hi] \rangle) \vee (D_{14}(2k) \wedge \langle [li, 1], [li, hi], [li, hi] \rangle) \vee \text{inter}_{15}(2k)$$

$$k=1, \dots, 5$$

où

$$\text{inter}_{1R}(21) = \langle 1, 1, 1 \rangle$$

$$\text{inter}_{1R}(22) = (D_{1R}(21) \wedge D_{21}(1R) [p2 / [1, 1]])$$

$$\text{inter}_{1R}(23) = (D_{1R}(22) \wedge D_{22}(1R) \wedge \langle [2, ki], [li, ki], [li, ki] \rangle)$$

$$\vee (D_{1R}(24) \wedge D_{24}(1R) \wedge \langle [2, ki], [li, ki], [li, ki] \rangle)$$

$$\text{inter}_{1R}(24) = \langle (D_{1R}(23)[N] \wedge D_{23}(1R)[N]) - 1, D_{1R}(23)[P1] \wedge D_{23}(1R)[P1], \\ 2 \times (D_{1R}(23)[P2] \wedge D_{23}(1R)[P2]) \rangle$$

$$\text{inter}_{1R}(25) = \vee (D_{1R}(22) \wedge D_{22}(1R) \wedge \langle [li, 1], [li, ki], [li, ki] \rangle)$$

$$(D_{1R}(24) \wedge D_{24}(1R) \wedge \langle [li, 1], [li, ki], [li, ki] \rangle)$$

... équations similaires pour le processus 2 ...

$$D_1 = D_{15}(25) \wedge D_{25}(15)$$

Ce système d'équations peut être résolu au moyen d'une stratégie itérative asynchrone (Cousot-P [77]). Initialement on pose  $D_{il}(il) = \langle 1, 1, 1 \rangle$  pour  $i=1, 2$ ,  $l \in C_i$ ,  $R \in C_i^*$ . Puis on itère appliquant n'importe quelle équation du système d'équations jusqu'à stabilisation.

La convergence peut être accélérée utilisant les techniques d'extrapolation décrites dans Cousot-Cousot [76, 77]. Ceci consiste à définir un opérateur d'élargissement  $\nabla$  tel que :

$$1 \nabla x = x$$

$$[a, b] \nabla [c, d] = [\text{if } c < a \text{ then } li \text{ else } a, \text{if } d > b \text{ then } hi \text{ else } b]$$

et remplacer les équations  $D_{j3}$ ,  $j=1, 2$  par :

$$D_{j3}(jR) = D_{j3}(jR) \nabla [(D_{j2}(jR) \wedge \langle [2, ki], [li, ki], [li, ki] \rangle) \vee \\ (D_{j4}(jR) \wedge \langle [2, ki], [li, ki], [li, ki] \rangle) \vee \\ \text{inter}_{j3}(jR)]$$

puis résoudre itérativement. Le résultat que nous avons obtenu (pour le processus 1) est :

| $k=1$        |                                                    |       | $k=2$                                          |                                                  |       | $k=3$                                            |       |       |
|--------------|----------------------------------------------------|-------|------------------------------------------------|--------------------------------------------------|-------|--------------------------------------------------|-------|-------|
| $n$          | $p^1$                                              | $p^2$ | $n$                                            | $p^1$                                            | $p^2$ | $n$                                              | $p^1$ | $p^2$ |
| $D_{11}(2k)$ | $\langle [0, k_i], [l_i, k_i], [l_i, k_i] \rangle$ |       | $\langle [0, k_i], [l_i, k_i], [1, 1] \rangle$ | $\langle [2, k_i], [l_i, k_i], [1, k_i] \rangle$ |       | $\langle [2, k_i], [l_i, k_i], [1, k_i] \rangle$ |       |       |
| $D_{12}(2k)$ | $\langle [0, k_i], [1, 1], [l_i, k_i] \rangle$     |       | $\langle [0, k_i], [1, 1], [1, 1] \rangle$     | $\langle [2, k_i], [1, 1], [1, k_i] \rangle$     |       | $\langle [2, k_i], [1, 1], [1, k_i] \rangle$     |       |       |
| $D_{13}(2k)$ | $\langle [2, k_i], [1, k_i], [l_i, k_i] \rangle$   |       | $\langle [2, k_i], [1, k_i], [1, 1] \rangle$   | $\langle [2, k_i], [1, k_i], [1, k_i] \rangle$   |       | $\langle [2, k_i], [1, k_i], [1, k_i] \rangle$   |       |       |
| $D_{14}(2k)$ | $\langle [1, k_i-1], [2, k_i], [l_i, k_i] \rangle$ |       | $\langle [1, k_i-1], [2, k_i], [1, 1] \rangle$ | $\langle [1, k_i-1], [2, k_i], [1, k_i] \rangle$ |       | $\langle [1, k_i-1], [2, k_i], [1, k_i] \rangle$ |       |       |
| $D_{15}(2k)$ | $\langle [0, 1], [1, k_i], [l_i, k_i] \rangle$     |       | $\langle [0, 1], [1, k_i], [1, 1] \rangle$     | $\langle [1, 1], [2, k_i], [1, k_i] \rangle$     |       | $\langle [1, 1], [2, k_i], [1, k_i] \rangle$     |       |       |

| $k=4$        |                                                    |       | $k=5$                                          |       |       |
|--------------|----------------------------------------------------|-------|------------------------------------------------|-------|-------|
| $n$          | $p^1$                                              | $p^2$ | $n$                                            | $p^1$ | $p^2$ |
| $D_{11}(2k)$ | $\langle [1, k_i-1], [l_i, k_i], [2, k_i] \rangle$ |       | $\langle [0, 1], [l_i, k_i], [1, k_i] \rangle$ |       |       |
| $D_{12}(2k)$ | $\langle [1, k_i-1], [1, 1], [2, k_i] \rangle$     |       | $\langle [0, 1], [1, 1], [1, k_i] \rangle$     |       |       |
| $D_{13}(2k)$ | $\langle [1, k_i-1], [1, k_i], [2, k_i] \rangle$   |       | $\langle [1, 1], [1, k_i], [2, k_i] \rangle$   |       |       |
| $D_{14}(2k)$ | $\langle [0, k_i-2], [2, k_i], [2, k_i] \rangle$   |       | $\langle [0, 1], [2, k_i], [1, k_i] \rangle$   |       |       |
| $D_{15}(2k)$ | $\langle [0, 1], [1, k_i], [2, k_i] \rangle$       |       | $\langle [0, 1], [1, k_i], [1, k_i] \rangle$   |       |       |

Observons que nous obtenons :

$$D_1 = \langle [0, 1], [1, k_i], [1, k_i] \rangle$$

ce qui montre que  $0 \leq N \leq 1$  à la sortie de la commande parallèle du programme, ce qui n'est pas complètement trivial à démontrer à la main.

□

#### Exemple 4.3.2.5.1-2

Pour étendre Cousot-Holbach [78] au cas des programmes parallèles, il suffit de considérer une décomposition de la forme  $(\alpha_a \circ \alpha_p, \gamma_p \circ \gamma_a)$  où  $(\alpha_p, \gamma_p)$  décompose  $(S^2 \rightarrow \{t, ff\})$  en  $\prod_{i \in m} \prod_{c \in c_i} (\mathbb{R}^m \rightarrow \{t, ff\})$  (ou  $\prod_{i \in m} \prod_{c \in c_i} (\mathbb{Z}^m \rightarrow \{t, ff\})$ ) tandis que la décomposition  $(\alpha_a, \gamma_a)$  consiste pour chaque composante  $i \in m$  à approcher les éléments  $p_i$  de  $(\mathbb{R}^m \rightarrow \{t, ff\})$  par le prédicat caractérisant l'enveloppe convexe de  $\{x : p_i(x)\}$ . Les opérateurs  $\sqcup$  et  $\sqcap$  correspondants ont été proposés par Holbach [78]. En particulier si  $p$  et  $q$  caractérisent deux polyèdres

convexes de  $\mathbb{R}^m$  tels que  $P \Rightarrow Q$  et  $P \neq Q$  alors l'élargissement  $P \vee Q$  de  $P$  par  $Q$  est obtenu en éliminant du système de contraintes linéaires  $P$  toutes les inéquations non satisfaites par  $Q$ . Lorsque la dimension du polyèdre caractérisé par  $P$  est strictement inférieure à  $m$ , on réécrit au préalable le système d'inéquations  $P$  sous une forme qui maximise le nombre de contraintes satisfaites par  $Q$ .

Par exemple, en choisissant la décomposition  $(\alpha_p, \delta_p)$  comme en 4.3.2.4.4 ce qui permet d'associer à chaque point du programme une relation linéaire entre l'état mémoire initial et les états de contrôle et mémoire courants, on obtient pour le programme 2.2.3.3.1 les résultats suivants (nous ignorons la variable  $\text{any}$  qui ne peut prendre qu'une seule valeur) :

0: {true}

11:  $\{c_2 \geq 21 \wedge c_3 \geq 31 \wedge c_3 \geq c_2 + 9 \wedge c_3 \geq 2c_2 - 14 \wedge 2c_3 \leq c_2 + 44 \wedge c_3 \leq c_2 + 11\}$

while true do

12:  $\{c_2 \geq 21 \wedge c_3 \geq 31 \wedge c_3 \geq c_2 + 9 \wedge c_3 \geq 2c_2 - 14 \wedge c_3 \leq 34 \wedge c_3 \leq c_2 + 11\}$

$P! \text{any};$

14:  $\{c_3 = 24 \wedge 21 \leq c_2 \leq 23\}$

$V! \text{any};$

13:  $\{c_2 \geq 21 \wedge c_3 \geq 32 \wedge c_3 \geq 2c_2 - 14 \wedge c_3 \leq 34 \wedge c_3 \leq c_2 + 12\}$

od;

15: {false}

||

21:  $\{c_1 \geq 11 \wedge c_3 \geq 31 \wedge c_2 \geq c_1 + 19 \wedge c_3 \geq 2c_1 + 6 \wedge 2c_3 \leq c_1 + 54 \wedge c_3 \leq c_2 + 21\}$ while true do22:  $\{c_1 \geq 11 \wedge c_3 \geq 31 \wedge c_2 \geq c_1 + 19 \wedge c_3 \geq 2c_1 + 6 \wedge c_3 \leq 34 \wedge c_3 \leq c_1 + 21\}$ 

P! any;

24:  $\{c_3 = 34 \wedge 11 \leq c_1 \leq 13\}$ 

V! any;

23:  $\{c_1 \geq 11 \wedge c_3 \geq 32 \wedge c_2 \geq 2c_1 + 6 \wedge c_3 \leq 34 \wedge c_3 \leq c_1 + 22\}$ od;25:  $\{\underline{\text{false}}\}$ 

||

31:  $\{11 \leq c_2 \leq 12 \wedge 21 \leq c_2 \leq 22\}$ while true do32:  $\{11 \leq c_1 \leq 13 \wedge 21 \leq c_2 \wedge 23 \wedge c_1 + c_2 \leq 35\}$ 

P? Any;

34:  $\{c_1 \leq 14 \wedge c_2 \leq 24 \wedge c_1 + c_2 \leq 37 \wedge 2c_2 + c_1 \geq 56 \wedge 2c_1 + c_2 \geq 46\}$ 

V? Any;

33:  $\{11 \leq c_1 \leq 13 \wedge 21 \leq c_2 \leq 23 \wedge c_1 + c_2 \geq 33\}$ od;35:  $\{\underline{\text{false}}\}$ 

||;

1:  $\{\underline{\text{false}}\}$ 

Ces invariants sont suffisants pour démontrer que les points 14 et 24 sont en exclusion mutuelle (puisque si le contrôle pouvait être simultanément en 14 et 24 on aurait  $c_1 = 14 \wedge \{c_3 = 24 \wedge 21 \leq c_2 \leq 23\}$  et  $c_2 = 24 \wedge \{c_3 = 34 \wedge 11 \leq c_1 \leq 13\}$  ce qui est faux!). Malheureusement le choix de la numérotation des points du programme a une influence sur la précision des résultats, comme le montre l'exemple suivant (le renumérotage interdisant l'expression de l'exclusion mutuelle à l'aide d'une conjonction d'inégalités linéaires):

0: {true}

11: {  $21 \leq c_2 \leq 24 \wedge 31 \leq c_3 \leq 34 \wedge 2c_2 - c_3 \geq 10 \wedge 2c_3 - c_2 \geq 40$  }

while true do

12: {  $21 \leq c_2 \leq 24 \wedge 31 \leq c_3 \leq 34 \wedge 2c_3 - c_2 \geq 40$  }

P! any;

13: {  $c_2 \leq 24 \wedge 32 \leq c_3 \leq 34 \wedge c_3 - c_2 \leq 12 \wedge c_3 + c_2 \geq 54$  }

V! any;

14: {  $21 \leq c_2 \leq 24 \wedge 32 \leq c_3 \leq 24$  }

od;

15: {false}

||

21: {  $11 \leq c_1 \leq 14 \wedge 31 \leq c_3 \leq 34 \wedge c_3 - 2c_1 \leq 10 \wedge 2c_3 - c_1 \geq 50$  }

while true do

22: {  $11 \leq c_1 \leq 14 \wedge 31 \leq c_3 \leq 34 \wedge 2c_3 - c_1 \geq 50$  }

P! any;

23: {  $c_1 \leq 14 \wedge 32 \leq c_3 \leq 34 \wedge c_3 - c_1 \leq 22 \wedge c_3 + c_1 \geq 44$  }

V! any;

24: {  $11 \leq c_1 \leq 14 \wedge 32 \leq c_3 \leq 24$  }

od;

25: {false}

||

31: {  $11 \leq c_1 \leq 12 \wedge 21 \leq c_2 \leq 22$  }

while true do

32: {  $11 \leq c_1 \leq 14 \wedge 21 \leq c_2 \leq 24$  }

P? Any;

33: {  $11 \leq c_1 \leq 14 \wedge 21 \leq c_2 \leq 24 \wedge c_1 + c_2 \leq 37 \wedge c_1 + c_2 \geq 33$  }

V? Any;

34: {  $11 \leq c_1 \leq 14 \wedge 21 \leq c_2 \leq 24 \wedge c_1 + c_2 \geq 33$  }

od;

35: {false}

||;

1: {false}

Ces résultats sont maintenant trop faibles pour démontrer l'exclusion mutuelle des points 13 et 23 du programme (puisque  $c_1 = 13 \wedge \{c_2 \leq 24 \wedge 32 \leq c_3 \leq 34 \wedge c_3 - c_2 \leq 12 \wedge c_3 + c_2 \geq 54\} \wedge c_2 = 23 \wedge \{c_1 \leq 14 \wedge 32 \leq c_3 \leq 34 \wedge c_3 - c_1 \leq 22 \wedge c_3 + c_1 \geq 44\}$  n'est pas identiquement faux).

Il est évidemment possible d'éviter cette perte d'information, par exemple en choisissant  $(\alpha_p, \delta_p)$  comme en 4.3.2.4.2, ce qui revient à associer une relation linéaire entre l'état mémoire initial et courant à toute valeur de l'état de contrôle. Dans ce cas l'analyse donne comme résultat l'attention true associée aux états de contrôle  $(11, 21, 31), (12, 21, 31), (11, 22, 31), (11, 21, 32), (12, 22, 31), (12, 21, 32), (11, 22, 32), (13, 21, 33), (12, 22, 32), (11, 23, 33), (14, 21, 34), (13, 22, 33), (12, 23, 33), (11, 24, 34), (12, 21, 34), (14, 22, 34), (14, 21, 32), (12, 24, 34), (11, 22, 34), (11, 24, 32), (12, 22, 34), (14, 22, 32), (12, 24, 32), (14, 23, 33), (13, 24, 33), (14, 24, 34), (14, 24, 32)$  et false associée aux autres états de contrôle. (on remarque que la réunion de ces points n'est pas convexe d'où la perte d'information avec la décomposition moins fine ci-dessus). Comme pour l'exemple 4.3.2.5.1-1, le coût de ce gain en précision est qu'il faut maintenant considérer un nombre d'invariants de l'ordre du produit et non plus de la somme des nombres de points de contrôle des processus du programme.

□

#### 4.3.2.5.2 Analyse d'invariance "en arrière"

On peut aussi s'intéresser à une approximation de la relation entre les états  $\bar{\alpha}$  satisfaisant une condition  $\psi$  et leurs ascendants possibles  $\underline{\alpha}$  :

$$\underline{A}(\psi, \langle S, A, \Sigma \rangle) \in (S^2 \rightarrow \{\text{tt}, \text{ff}\})$$

$$\underline{A}(\psi, \langle S, A, \Sigma \rangle)(\bar{\alpha}, \underline{\alpha}) = [\exists p \in \Sigma, i, j \in |p|. \psi(\bar{\alpha}) \wedge p_j = \bar{\alpha} \wedge i \leq j \wedge p_i = \underline{\alpha}]$$

ce qui se traite comme dans le cas précédent en appliquant la transformation  $-1$  (cf. 4.2.1.2.3) sur  $\text{Pref}^{\omega}(\langle S, A, \Sigma \rangle)$  et consiste donc à trouver une approximation supérieure de la plus petite solution d'un système d'équations  $X = \tilde{B}(X)$  tel que  $\alpha \circ B \circ \gamma \in \tilde{B}$  et

$$B(I)(\underline{A}, \bar{A}) = [(\exists \Delta' \in S, a \in A. t_a(\underline{A}, \Delta') \wedge I(\Delta', \bar{A})) \vee (\underline{A} = \bar{A} \wedge \Psi(\bar{A}))]$$

### Exemple 4.3.2.5.2-1

Soit à chercher une condition suffisante  $P(\underline{A})$  sur les états d'entrée d'un programme parallèle :

$$P_S [P_{\pi_0} \parallel \dots \parallel \alpha L : p \parallel \dots \parallel \alpha' L' : p' \parallel \dots \parallel P_{\pi_{m-1}}] ; P_{S'}$$

pour que les points  $L$  et  $L'$  dans deux processus différents  $P_{\pi_i}$  et  $P_{\pi_j}$  soient mutuellement exclusifs. Soit  $\langle S, A, \Sigma \rangle$  la sémantique du programme. Posons  $\mu \in (S \rightarrow \{\text{tt}, \text{ff}\})$  défini par  $\mu(\langle L, M \rangle) = \text{tt}$  et  $\mu(\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle) = \neg (L_i = L \wedge L_j = L')$ . Il s'agit de trouver  $P$  tel que :

$$\forall p \in \Sigma. (P(p_0) \Rightarrow \forall i \in p. \mu(p_i))$$

Soit  $\langle S, A, t, \varepsilon \rangle$  le système de transition engendré par  $\langle S, A, \Sigma \rangle$ . Il suffit de trouver  $P$  tel que :

$$\forall \underline{A} \in S. [\varepsilon(\underline{A}) \wedge P(\underline{A})] \Rightarrow [\forall \bar{A} \in S. t^*(\underline{A}, \bar{A}) \Rightarrow \mu(\bar{A})]$$

on peut donc choisir  $P(\underline{A})$  tel que :

$$[\varepsilon(\underline{A}) \wedge [\exists \bar{A} \in S. t^*(\underline{A}, \bar{A}) \wedge \neg \mu(\bar{A})]] \Rightarrow \neg P(\underline{A})$$

c'est-à-dire comme la négation d'une condition nécessaire sur les états d'entrée pour qu'ils aient comme descendants un état où le contrôle est simultanément en  $L$  et  $L'$ . Cette condition s'obtient comme approximation supérieure du plus petit point fixe de

$$b(I)(\underline{A}) = [(\exists \Delta' \in S, a \in A. t_a(\underline{A}, \Delta') \wedge I(\Delta')) \vee \neg \mu(\underline{A})]$$

□

## 4.3.2.5.3 Analyse d'invariance "avant-arrière"

L'analyse d'invariance "avant-arrière" consiste à trouver une approximation supérieure de l'ensemble des descendants des états initiaux (satisfaisant une condition initiale  $\phi$  portant par exemple sur les états d'entrée), qui satisfont une condition  $\delta$  (dérivée par exemple des déclarations) et sont ascendants des états finaux (satisfont une condition finale  $\psi$  portant par exemple sur les états de sortie):

$$\mathcal{DA}(\phi, \delta, \psi, \langle S, A, \Sigma \rangle) \in (S^2 \rightarrow \{\text{tt}, \text{ff}\})$$

$$\mathcal{DA}(\phi, \delta, \psi, \langle S, A, \Sigma \rangle)(\underline{A}, \bar{A}, \bar{A}) =$$

$$[\exists p \in \Sigma, i, j \in |p|. \phi(\underline{A}) \wedge p_0 = \underline{A} \wedge \delta(\underline{A}) \wedge p_i = \bar{A} \wedge \psi(\bar{A}) \wedge p_j = \bar{A} \wedge i \leq j]$$

Une première approximation supérieure consiste à raisonner sur le système de transition  $\langle S, A, t, \varepsilon \rangle$  engendré par la sémantique  $\langle S, A, \Sigma \rangle$ . On a:

$$\mathcal{DA}(\phi, \delta, \psi, \langle S, A, \Sigma \rangle)(\underline{A}, \bar{A}, \bar{A}) \Rightarrow (\bigsqcup_p (F)(\underline{A}, \bar{A}) \wedge \delta(\underline{A}) \wedge \bigsqcup_p (B)(\bar{A}, \bar{A}))$$

où comme précédemment :

$$F(I)(\underline{A}, \bar{A}) = [(\underline{A} = \bar{A} \wedge \phi(\underline{A})) \vee (\exists A' \in S, a \in A. I(\underline{A}, A') \wedge t_a(\underline{A}, \bar{A}))]$$

$$B(I)(\bar{A}, \bar{A}) = [(\exists A' \in S, a \in A. t_a(\underline{A}, A') \wedge I(A', \bar{A})) \vee (\bar{A} = \underline{A} \wedge \psi(\bar{A}))]$$

Soit  $\langle \tilde{A}, \tilde{\Sigma} \rangle$  une décomposition de  $\langle A, \Rightarrow \rangle$  par la correspondance de Galois  $(\alpha, \gamma)$  où  $A = (S^2 \rightarrow \{\text{tt}, \text{ff}\})$ . On pose  $\tilde{F} = \alpha \circ F \circ \gamma$ ,  $\tilde{B} = \alpha \circ B \circ \gamma$  et  $\tilde{\delta} = \alpha(\delta)$ . On définit l'opérateur  $\Pi$  sur  $\langle \tilde{A}, \tilde{\Sigma} \rangle$  tel que  $\forall P, Q \in \tilde{A}. [\alpha(\gamma(P) \wedge \gamma(Q)) \in (P \Pi Q)]$ , et  $\Delta$  est un opérateur de rétrécissement comme en 4.3.2.5.1. A la suite de Cousot-P[78], on peut calculer une approximation supérieure de  $\alpha(\mathcal{DA})$  où  $\tilde{\mathcal{DA}}(\underline{A}, \bar{A}, \bar{A}) = [\bigsqcup_p (F)(\underline{A}, \bar{A}) \wedge \delta(\underline{A}) \wedge \bigsqcup_p (B)(\bar{A}, \bar{A})]$

comme limite de la suite décroissante finie  $X^0, \dots, X^k, \dots$  telle que :

$$\tilde{\delta} = X^0$$

$$X^0 \Delta Z^0 = X^1 \quad \text{ou} \quad Z^0 \equiv \text{lfp}(f^0) \quad \text{et} \quad f^0(y) = X^0 \cap F(y)$$

$$X^1 \Delta Z^1 = X^2 \quad \text{ou} \quad Z^1 \equiv \text{lfp}(f^1) \quad \text{et} \quad f^1(y) = X^1 \cap B(y)$$

...

$$X^{2k} \Delta Z^{2k} = X^{2k+1} \quad \text{ou} \quad Z^{2k} \equiv \text{lfp}(f^{2k}) \quad \text{et} \quad f^{2k}(y) = X^{2k} \cap F(y)$$

$$X^{2k+1} \Delta Z^{2k+1} = X^{2k+2} \quad \text{ou} \quad Z^{2k+1} \equiv \text{lfp}(f^{2k+1}) \quad \text{et} \quad f^{2k+1}(y) = X^{2k+1} \cap B(y)$$

...

Pour calculer les  $Z^k$ , on utilise une itération chaotique croissante avec élargissement puis si la solution obtenue n'est pas un point fixe on l'améliore par une itération chaotique décroissante avec rétrécissement. Bien entendu si le treillis  $\langle A^2, \sqsubseteq \rangle$  satisfait la condition de chaîne ascendante (respectivement descendante) on peut choisir  $\nabla = \sqcup$  (respectivement  $\Delta = \sqcap$ ).

### Exemple 4.3.2.5.3-1

Si nous poursuivons l'analyse en avant du programme 2.8.2.3 donnée en exemple 4.3.2.5.1-1, par une analyse avant-arrière combinée partant de  $\delta(m, p_1, p_2, p) = (m \in [0, h_i] \wedge p_1, p_2, p \in [l_i, h_i])$  (donnée par les déclarations du programme), nous obtenons les résultats suivants (pour le processus 1, en notant  $\div$  la division entière et  $\lfloor x \rfloor = (x \div 2) \times 2$ ) :

|               | $k=1$                                                   |       |       | $k=2$                                               |       |       | $k=3$                                                        |       |       |
|---------------|---------------------------------------------------------|-------|-------|-----------------------------------------------------|-------|-------|--------------------------------------------------------------|-------|-------|
|               | $m$                                                     | $p_1$ | $p_2$ | $m$                                                 | $p_1$ | $p_2$ | $m$                                                          | $p_1$ | $p_2$ |
| $D_{i_1}(2k)$ | $\langle [0, h_i], [l_i, h_i], [l_i, h_i] \rangle$      |       |       | $\langle [0, h_i], [l_i, h_i], [1, 1] \rangle$      |       |       | $\langle [2, h_i], [l_i, h_i], [1, h_i \div 2] \rangle$      |       |       |
| $D_{i_2}(2k)$ | $\langle [0, h_i], [1, 1], [l_i, h_i] \rangle$          |       |       | $\langle [0, h_i], [1, 1], [1, 1] \rangle$          |       |       | $\langle [2, h_i], [1, 1], [1, h_i \div 2] \rangle$          |       |       |
| $D_{i_3}(2k)$ | $\langle [2, h_i], [1, h_i \div 2], [l_i, h_i] \rangle$ |       |       | $\langle [2, h_i], [1, h_i \div 2], [1, 1] \rangle$ |       |       | $\langle [2, h_i], [1, h_i \div 2], [1, h_i \div 2] \rangle$ |       |       |
| $D_{i_4}(2k)$ | $\langle [1, h_i - 1], [2, h_i], [l_i, h_i] \rangle$    |       |       | $\langle [1, h_i - 1], [2, h_i], [1, 1] \rangle$    |       |       | $\langle [1, h_i - 1], [2, h_i], [1, h_i \div 2] \rangle$    |       |       |
| $D_{i_5}(2k)$ | $\langle [0, 1], [1, h_i], [l_i, h_i] \rangle$          |       |       | $\langle [0, 1], [1, h_i], [1, 1] \rangle$          |       |       | $\langle [1, 1], [2, h_i], [1, h_i \div 2] \rangle$          |       |       |

|              | $k=4$                                                |       |       | $k=5$                                            |       |       |
|--------------|------------------------------------------------------|-------|-------|--------------------------------------------------|-------|-------|
|              | $m$                                                  | $p_1$ | $p_2$ | $m$                                              | $p_1$ | $p_2$ |
| $D_{11}(2k)$ | $\langle [1, k_i-1], [l_i, k_i], [2, [k_i]] \rangle$ |       |       | $\langle [0, 1], [l_i, k_i], [1, [k_i]] \rangle$ |       |       |
| $D_{12}(2k)$ | $\langle [1, k_i-1], [1, 1], [2, [k_i]] \rangle$     |       |       | $\langle [0, 1], [1, 1], [1, [k_i]] \rangle$     |       |       |
| $D_{13}(2k)$ | $\langle [1, k_i-1], [1, k_i+2], [2, [k_i]] \rangle$ |       |       | $\langle [1, 1], [1, k_i+2], [2, [k_i]] \rangle$ |       |       |
| $D_{14}(2k)$ | $\langle [0, k_i-2], [2, [k_i]], [2, [k_i]] \rangle$ |       |       | $\langle [0, 1], [2, [k_i]], [1, [k_i]] \rangle$ |       |       |
| $D_{15}(2k)$ | $\langle [0, 1], [1, [k_i]], [2, [k_i]] \rangle$     |       |       | $\langle [0, 1], [1, [k_i]], [1, [k_i]] \rangle$ |       |       |

On peut associer à chaque point  $1_j$  du programme l'invariant  $\chi(\bigcup_{k=1}^m D_{1j}(2k))$ , ce qui donne :

0:  $\{m \in [0, k_i]\}$

11:  $\{m \in [0, k_i] \wedge p_1 \in [l_i, k_i] \wedge p_2 \in [l_i, k_i]\}$

$P_1 := 1;$

12:  $\{m \in [0, k_i] \wedge p_1 \in [1, 1] \wedge p_2 \in [l_i, k_i]\}$

while  $N > 1$  do

13:  $\{m \in [1, k_i] \wedge p_1 \in [1, k_i+2] \wedge p_2 \in [l_i, k_i]\}$

$\{N := N-1; P_1 := 2 \times P_1\};$

14:  $\{m \in [0, k_i-1] \wedge p_1 \in [2, [k_i]] \wedge p_2 \in [l_i, k_i]\}$

od;

15:  $\{m \in [0, 1] \wedge p_1 \in [1, [k_i]] \wedge p_2 \in [l_i, k_i]\}$

||

21:  $\{m \in [0, k_i] \wedge p_1 \in [l_i, k_i] \wedge p_2 \in [l_i, k_i]\}$

$P_2 := 1;$

22:  $\{m \in [0, k_i] \wedge p_1 \in [1, 1] \wedge p_2 \in [l_i, k_i]\}$

while  $N > 1$  do

23:  $\{m \in [1, k_i] \wedge p_1 \in [l_i, k_i] \wedge p_2 \in [1, k_i+2]\}$

$\{N := N-1; P_2 := 2 \times P_2\};$

24:  $\{m \in [0, k_i-1] \wedge p_1 \in [l_i, k_i] \wedge p_2 \in [2, [k_i]]\}$

od;

25:  $\{m \in [0, 1] \wedge p_1 \in [l_i, k_i] \wedge p_2 \in [1, [k_i]]\}$

];

- 1:  $\{m \in [0,1] \wedge p_1 \in [1, l_i] \wedge p_2 \in [1, l_i]\}$   
 if  $N=0$  then  $P := P_1 \times P_2$  else  $P := 2 \times P_1 \times P_2$  fi;  
 2:  $\{m \in [0,1] \wedge p_1 \in [1, l_i] \wedge p_2 \in [1, l_i] \wedge p \in [1, l_i]\}$

Ces résultats permettent de placer des tests qui doivent être vérifiés pour éviter les erreurs à l'exécution (mais en moins grand nombre que ne le ferait un compilateur n'utilisant que les informations données par les déclarations) ainsi que des tests qu'il est nécessaire (mais en général pas suffisant) de vérifier pour que l'exécution du programme se termine sans erreurs à l'exécution et en ne passant que par des états satisfaisant la condition  $\delta$  (ces derniers ne sont pas placés par un compilateur classique, même pour les tests liés aux déclarations puisque les tests à l'exécution sont généralement placés au moment de l'affectation aux variables et pas au moment de leur utilisation). Pour les tests de la première sorte, on trouve :

- $p_1 \leq l_i + 2$  au point 13
- $p_2 \leq l_i + 2$  au point 23
- $p_1 \leq l_i + p_2$  au point 1 quand  $N=0$
- $p_1 \leq (l_i + 2) \div p_2$  au point 1 quand  $N \neq 0$

(Un compilateur classique placerait des tests inutiles comme  $n-1 \geq 0$  aux points 13 et 23 ou des tests plus complexes comme pour tester que  $l_i \leq p_1 \times p_2 \leq l_i$  ou  $l_i \leq 2 \times p_1 \times p_2 \leq l_i$  au point 1 puisque le signe de  $p_1$  et  $p_2$  n'est pas connu). Pour les tests de la seconde sorte, on trouve :

- $n \geq 0$  au point 0

(ce test figurerait dans un compilateur classique après la commande de lecture de la variable  $N$ ). L'intérêt d'une analyse en arrière pour introduire des tests nécessaires pour une terminaison normale est mieux illustré par l'exemple suivant (tiré de Cousot-P [78], p.(5)-53) :

```

const li = ...;    { plus petit entier, li < 0 }
      hi = ...;    { plus grand entier, hi > 1000 }
type integer = li..hi;
var N, K, I, J : integer
    T : array [0..1000] of integer;

1:  { m, k, i, j ∈ [li, hi] }
    read (N);

2:  { m ∈ [0, 1000] ∧ k, i, j ∈ [li, hi] }
    K := 0;

3:  { m ∈ [0, 1000] ∧ k ∈ [0, 0] ∧ i, j ∈ [li, hi] }
    while K ≤ N do

4:      { m, k ∈ [0, 1000] ∧ i, j ∈ [li, hi] }
        read (T[K]);

5:      { m, k ∈ [0, 1000] ∧ i, j ∈ [li, hi] }
        K := K + 1;

6:  od;

7:  { m ∈ [0, 1000] ∧ k ∈ [0, 1001] ∧ i, j ∈ [li, hi] }
    I := N;

8:  { i, j ∈ [0, 1000] ∧ k ∈ [0, 1001] ∧ j ∈ [li, hi] }
    while I <> 0 do

9:      { m ∈ [0, 1000] ∧ i ∈ [1, 1000] ∧ k, j ∈ [li, hi] }
        J := 0;

10:     { m ∈ [0, 1000] ∧ i ∈ [1, 1000] ∧ j ∈ [0, 0] ∧ k ∈ [li, hi] }
        while J <> I do

11:            { m ∈ [0, 1000] ∧ i ∈ [1, 1000] ∧ j ∈ [0, 999] ∧ k ∈ [li, hi] }
                if T[J] <= T[J+1] then

12:                    { m ∈ [0, 1000] ∧ i ∈ [1, 1000] ∧ j ∈ [0, 999] ∧ k ∈ [li, hi] }
                        K := T[J]; T[J] := T[J+1]; T[J+1] := K;

13:                    { m ∈ [0, 1000] ∧ i ∈ [1, 1000] ∧ j ∈ [0, 999] ∧ k ∈ [li, hi] }

```

fi;

14:  $\{m \in [0, 1000] \wedge i \in [1, 1000] \wedge i \in [0, 999] \wedge k \in [l_i, r_i]\}$

$J := J + 1;$

15:  $\{m \in [0, 1000] \wedge i, j \in [1, 1000] \wedge k \in [l_i, r_i]\}$

od;

16:  $\{m \in [0, 1000] \wedge i, j \in [1, 1000] \wedge k \in [l_i, r_i]\}$

$I := I - 1;$

17:  $\{m \in [0, 1000] \wedge i, j \in [1, 1000] \wedge k \in [l_i, r_i]\}$

od;

18:  $\{m \in [0, 1000] \wedge i \in [0, 0] \wedge j \in [1, 1000] \wedge k \in [l_i, r_i]\}$

Cette analyse conduit à introduire le test  $0 \leq m \leq 1000$  au point 2 (pour garantir que l'exécution se termine sans erreurs à l'exécution, ce test ne serait évidemment pas introduit par un compilateur classique). Dans ces conditions, il faut également introduire le test  $j \leq 999$  au point 11 (ce test étant en fait inutile).

□

## 4.4 REFERENCES

- APT K.R., FRANCEZ N., DE ROEVER W.P. [80], "A proof system for communicating sequential processes", TOPLAS 2, 3(1980), 359-385.
- ASHCROFT E.A. [75], "Proving assertions about parallel programs", J. of comp. and System Science, 10(1975), 110-135.
- ASHCROFT E.A., MANNA Z. [70], "Formalization of properties of parallel programs, Machine Intelligence, 6(1970), 17-41.
- COUSOT P. [77], "Asynchronous iterative methods for solving a fixed point system of monotone equations in a complete lattice", Rapport de Recherche n°88, IMAG, Université de Grenoble, (Mars 1978).
- COUSOT P. [78], "Méthodes itératives de construction et d'approximation de points fixes d'opérateurs monotones sur un treillis, analyse sémantique des programmes, Thèse d'Etat, Université de Grenoble, (Mars 1978).
- COUSOT P. [79], "Analysis of the behavior of dynamic discrete systems", Rapport de Recherche n°161, IMAG, Université de Grenoble, (Jan. 1979).
- COUSOT P. [81], "Semantic foundations of program analysis", dans "Program flow analysis, theory and applications", s.s. Muchnick & N.J. Jones (eds.), Prentice-Hall, (1981), 303-342.
- COUSOT R. [81], "Proving invariance properties of parallel programs by backward induction", Rapport de Recherche CRIN-81-P026, (1981).
- COUSOT P., COUSOT R. [76], "Static determination of dynamic properties of programs", Proc. 2nd Int. Symp. on Programming, Paris, Dunod, (Avril 1976), 106-130.

- COUSOT P., COUSOT R. [77a], "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints", Conf. Rec. of the 4th ACM Symp. on Principles of Programming Languages, Los Angeles, (Jan. 1977), 238-252.
- COUSOT P., COUSOT R. [77b], "Static determination of dynamic properties of generalized type unions", ACM Conf. on Language Design for Reliable Software, Raleigh, SIGPLAN Notices 12, 3 (1977), 77-94.
- COUSOT P., COUSOT R. [77c], "Static determination of dynamic properties of recursive procedures", Conf. on Formal Description of Programming Concepts, St. Andrews, Canada, North-Holland Pub. Co., (1977), 237-277.
- COUSOT P., COUSOT R. [77d], "Automatic synthesis of optimal invariant assertions: mathematical foundations", Proc. ACM Symp. on Artificial Intelligence & Programming Languages, Rochester, SIGPLAN Notices 12, 8 (1977), 1-12.
- COUSOT P., COUSOT R. [79a], "Systematic design of program analysis frameworks", Conf. Rec. of the 6th ACM Symp. on Principles of Programming Languages, San Antonio, Texas, (1979), 269-282.
- COUSOT P., COUSOT R. [79b], "Constructive versions of Tarski's fixed point theorems", Pacific Journal of Math., vol. 82, no. 1, (1979), 43-57.
- COUSOT P., COUSOT R. [80a], "Semantic analysis of communicating sequential processes", Automata, Languages and Programming, 7th Colloq., Lecture Notes in Computer Sci. 85, Springer-Verlag, (1980), 119-133.
- COUSOT P., COUSOT R. [80b], "Constructing program invariance proof methods", Proc. Int. Workshop on Program Construction, INRIA Ed., Tome 1, (1980).

- COUSOT P., COUSOT R. [80c], "Reasoning about program invariance proof methods", Rapport de Recherche CRIN-80-P050, (1980).
- COUSOT P., COUSOT R. [82a], "Induction principles for proving invariance properties of programs, dans "Tools and Notions for Program Construction", (D. Neel Ed.), Cambridge University Press, (1982), 75-119.
- COUSOT P., COUSOT R. [82b], "'A la Floyd' induction principles for proving inevitability properties of programs", Rapport de recherche LRIM-82-04, à paraître dans "Algebraic Methods in Programming", (M. Nivat & J. Reynolds, eds.), Cambridge University Press.
- COUSOT P., COUSOT R. [84], "Invariance proof methods and analysis techniques for parallel programs", dans "Automatic program construction techniques", (A. Diekmann et al., eds.), MAC MILLAN, (1984), 243-271.
- COUSOT P., HALBWACHS N. [78], "Automatic discovery of linear restraints among variables of a program", Conf. Rec. of the 5th ACM Symp. on Principles of Programming Languages, Tucson, Arizona, (1978), 84-97.
- DIJKSTRA E.W. [82], "Selected writings on computing: a personal perspective", Springer-Verlag, (1982).
- FLOYD R.W. [67], "Assigning meaning to programs", Proc. Symp. in Applied Math., AMS, Providence, RI, (1967), 19-32.
- HOARE C.A.R. [69], "An axiomatic basis for computer programming", CACM 12, 10 (1969), 576-580, 583.
- HOARE C.A.R. [72], "Toward a theory of parallel programming", dans "Operating Systems Techniques", (Hoare & Perott eds.), Academic Press, (1972).

- HOARE C.A.R. [75], "Parallel programming: an axiomatic approach", *computer Languages*, 1 (1975), 151-160.
- HOARE C.A.R. [78], "Communicating sequential processes", *CACM* 21, 8 (1978), 666-677.
- HOWARD J.H. [76], "Proving monitors", *CACM* 19, 5 (1976), 273-279.
- KELLER R.M. [76], "Formal verification of parallel programs", *CACM* 19, 7 (1976), 371-384.
- LAMPORT L. [77], "Proving the correctness of multiprocess programs", *IEEE Trans. on Soft. Eng.*, SE-2, 2 (1977), 125-143.
- LAMPORT L. [80], "The 'Hoare Logic' of concurrent programs", *Acta Informatica* 14, (1980), 21-37.
- LEVIN G.M. [79], "A proof technique for communicating sequential processes (with an example)", TR79-401, *Comp. Sci. Dept, Cornell U.*, N.Y., (1979).
- MANNA Z. [70], "Mathematical theory of partial correctness", *JCSS* 5, 3 (1970), 238-253.
- MAZURKIEWICZ A [77], "Concurrent program schemes and their interpretation", *Dept. Comp. Sci, Aarhus U., Denmark, DAIMI-PB-78*, (1977).
- MISRA J. [78], "Some aspects of the verification of loop computations", *IEEE Trans. on Soft. Eng.*, SE-4, 6 (1978), 478-486.
- MORRIS J.H., WEGBREIT B. [77], "Subgoal induction", *CACM* 20, 4 (1977), 209-222.
- NAUR P. [66], "Proof of algorithms by general snapshots", *BIT* 6, (1966), 310-316.

NEWTON G. [75], "Proving properties of interacting processes", Acta Informatica, 4(1975), 117-126.

OWICKI S., GRIES D. [76a], "An axiomatic proof technique for parallel programs I", Acta Informatica, 6(1976), 319-340.

OWICKI S., GRIES D. [76b], "Verifying properties of parallel programs: an axiomatic approach", CACM 19, 5(1976), 279-285.

RICART G., AGRAWALA A.K. [81], "An optimal algorithm for mutual exclusion in computer networks", CACM 24, 1(1981), 9-17.

TARSKI A. [55], "A lattice theoretical fixpoint theorem and its applications", Pacific Journal of Math., 5(1955), 285-310.



The first part of Radhia Cousot's thesis is available at  
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences.pdf>

The second part of Radhia Cousot's thesis is available at  
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences2.pdf>

The third part of Radhia Cousot's thesis is available at  
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences3.pdf>